

# Análisis de datos industriales usando Excel y Python

Notas para el master de FP en tecnología quesera

Juan Riera

27/10/25



# Tabla de contenidos

|                                                                                                                                          |           |
|------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                                                                                   | <b>1</b>  |
| <b>I Introducción</b>                                                                                                                    | <b>5</b>  |
| <b>2. El análisis de datos de producción en el entorno industrial.</b>                                                                   | <b>7</b>  |
| 2.1. Introducción . . . . .                                                                                                              | 8         |
| 2.2. El pensamiento estadístico y el método científico. . . . .                                                                          | 9         |
| 2.3. Los datos industriales . . . . .                                                                                                    | 11        |
| 2.3.1. Estudios retrospectivos o históricos . . . . .                                                                                    | 12        |
| 2.3.2. Estudios observacionales . . . . .                                                                                                | 13        |
| 2.3.3. Experimentos diseñados . . . . .                                                                                                  | 14        |
| <b>3. Herramientas para el análisis. La reproducibilidad.</b>                                                                            | <b>15</b> |
| 3.1. Introducción . . . . .                                                                                                              | 15        |
| 3.2. La hoja de cálculo . . . . .                                                                                                        | 15        |
| 3.2.1. Aplicaciones de la hoja de cálculo . . . . .                                                                                      | 16        |
| 3.3. Los lenguajes de programación . . . . .                                                                                             | 17        |
| 3.3.1. <b>Python</b> en el análisis de datos industriales: una herramien-<br>ta versátil . . . . .                                       | 17        |
| 3.4. Otras herramientas: R, Julia . . . . .                                                                                              | 19        |
| 3.4.1. El software estadístico R . . . . .                                                                                               | 19        |
| 3.4.2. El lenguaje Julia . . . . .                                                                                                       | 20        |
| 3.5. El concepto de reproducibilidad . . . . .                                                                                           | 21        |
| 3.5.1. Importancia en la Ciencia . . . . .                                                                                               | 21        |
| 3.5.2. Reproducibilidad en metrología . . . . .                                                                                          | 21        |
| 3.6. Ventajas de los lenguajes de programación en scripts frente a la<br>hoja de cálculo en la reproducibilidad de un análisis . . . . . | 22        |
| <b>4. La organización de los datos y el flujo de trabajo. Los datos<br/>ordenados o <i>arreglados</i> (<i>tidy data</i>)</b>             | <b>25</b> |
| 4.1. Definiciones y términos útiles . . . . .                                                                                            | 25        |
| 4.1.1. Población y muestra . . . . .                                                                                                     | 25        |

|           |                                                                                                 |           |
|-----------|-------------------------------------------------------------------------------------------------|-----------|
| 4.1.2.    | Parámetro y estadístico . . . . .                                                               | 26        |
| 4.1.3.    | Variables y casos . . . . .                                                                     | 27        |
| 4.1.4.    | Tipos de variables . . . . .                                                                    | 27        |
| 4.1.5.    | Reglas básicas para establecer los nombres de las variables. . . . .                            | 29        |
| 4.2.      | El flujo de trabajo . . . . .                                                                   | 30        |
| 4.3.      | Etapas en un flujo de trabajo estructurado. . . . .                                             | 31        |
| 4.3.1.    | Recolección de datos . . . . .                                                                  | 31        |
| 4.3.2.    | Inspección de los datos . . . . .                                                               | 31        |
| 4.3.3.    | Limpieza de los datos . . . . .                                                                 | 32        |
| 4.3.4.    | Transformación de los datos . . . . .                                                           | 32        |
| 4.3.5.    | Análisis exploratorio de datos (EDA) . . . . .                                                  | 32        |
| 4.3.6.    | Modelado de datos . . . . .                                                                     | 32        |
| 4.3.7.    | Comunicación de resultados . . . . .                                                            | 32        |
| 4.4.      | Razones para seguir un flujo de trabajo . . . . .                                               | 33        |
| 4.5.      | Problemas de no seguir un flujo de trabajo estructurado . . . . .                               | 33        |
| 4.6.      | Un ejemplo: revisando los datos existentes. . . . .                                             | 34        |
| 4.7.      | Los datos ordenados o arreglados ( <i>tidy data</i> ) . . . . .                                 | 36        |
| 4.8.      | Qué es un fichero plano y un fichero CSV . . . . .                                              | 37        |
| 4.8.1.    | Cómo exportar un fichero CSV desde Excel. . . . .                                               | 37        |
| 4.9.      | Practica con el CSV . . . . .                                                                   | 39        |
| 4.10.     | Para resolver . . . . .                                                                         | 39        |
| <b>5.</b> | <b>Introducción a Python en el entorno Google Colab</b> . . . . .                               | <b>41</b> |
| 5.1.      | Cómo usar este cuaderno . . . . .                                                               | 42        |
| 5.2.      | Primeros pasos . . . . .                                                                        | 42        |
| 5.3.      | Cálculos simples con python . . . . .                                                           | 43        |
| 5.4.      | Creación y asignación de variables . . . . .                                                    | 43        |
| 5.5.      | Cómo establecer los nombres de variables . . . . .                                              | 44        |
| 5.5.1.    | Algunos ejercicios . . . . .                                                                    | 47        |
| 5.6.      | Los tipos de datos . . . . .                                                                    | 48        |
| 5.7.      | Las Funciones en Python: ¿Qué son y por qué las usamos? . . . . .                               | 49        |
| 5.7.1.    | <code>print()</code> : Mostrar información . . . . .                                            | 49        |
| 5.7.2.    | <code>len()</code> : Obtener la longitud . . . . .                                              | 50        |
| 5.7.3.    | <code>type()</code> : Conocer el tipo de dato . . . . .                                         | 50        |
| 5.7.4.    | ¿Cómo funcionan las funciones? Argumentos y Retorno . . . . .                                   | 50        |
| 5.8.      | Importando Herramientas: Bibliotecas y Módulos . . . . .                                        | 51        |
| 5.8.1.    | Sintaxis de <code>import</code> . . . . .                                                       | 51        |
| 5.9.      | Las principales estructuras de datos en Python para el análisis . . . . .                       | 52        |
| 5.9.1.    | Series de <code>pandas</code> (El “vector” en <code>Python</code> para datos) . . . . .         | 52        |
| 5.9.2.    | DataFrames de <code>Pandas</code> (La “Tabla” o “Hoja de Cálculo”) . . . . .                    | 53        |
| 5.10.     | El <i>data frame</i> (continuación con ejemplo real) . . . . .                                  | 56        |
| 5.11.     | Visualización de Datos con Seaborn y Matplotlib . . . . .                                       | 59        |
| 5.11.1.   | Ejemplo: Histograma de la longitud del sépalo (Distribución de una variable numérica) . . . . . | 59        |
| 5.12.     | Cómo leer nuestros datos en <code>Python</code> desde Google Colaborate . . . . .               | 61        |



|                                                                                            |            |
|--------------------------------------------------------------------------------------------|------------|
| 5.12.1. Opción 1: Subir los datos al espacio de trabajo de Google Colaborate . . . . .     | 62         |
| 5.12.2. Opción 2: Leer directamente los datos de nuestra carpeta de Google Drive . . . . . | 62         |
| 5.12.3. Explicación de los pasos: . . . . .                                                | 63         |
| 5.13. Resumen . . . . .                                                                    | 64         |
| <b>6. La exploración de los datos mediante gráficos.</b>                                   | <b>67</b>  |
| 6.1. Explorando los datos . . . . .                                                        | 67         |
| 6.1.1. El diagrama de tallo y hojas ( <i>stem and leaf plot</i> o <i>stemplot</i> )        | 67         |
| 6.1.2. Interpretación del diagrama de tallo y hojas . . . . .                              | 70         |
| 6.1.3. Resumen . . . . .                                                                   | 70         |
| 6.2. Distribuciones de frecuencias . . . . .                                               | 70         |
| 6.2.1. Construcción en Excel . . . . .                                                     | 71         |
| 6.3. Instrucciones paso a paso en Excel . . . . .                                          | 71         |
| 6.3.1. Construcción de una tabla de frecuencias en <b>Python</b> . . .                     | 73         |
| 6.4. Ejercicio propuesto . . . . .                                                         | 76         |
| 6.5. Diagrama de barras . . . . .                                                          | 77         |
| 6.6. Histograma . . . . .                                                                  | 77         |
| 6.6.1. Construcción de un histograma en Excel . . . . .                                    | 78         |
| 6.6.2. Análisis de un caso real . . . . .                                                  | 80         |
| 6.7. Visualización de Datos en <b>Python</b> con Seaborn y Matplotlib . . .                | 82         |
| 6.7.1. Otros ejemplos . . . . .                                                            | 85         |
| 6.8. Diagrama de caja o <i>boxplot</i> . . . . .                                           | 86         |
| 6.8.1. Relación entre el <i>boxplot</i> y el histograma . . . . .                          | 86         |
| 6.9. Alternativas al boxplot . . . . .                                                     | 87         |
| 6.10. Gráficos de densidad . . . . .                                                       | 90         |
| 6.10.1. Otra alternativa al <i>boxplot</i> : el <i>violin plot</i> . . . . .               | 91         |
| 6.11. Gráficos de dispersión . . . . .                                                     | 93         |
| 6.12. Gráficos de series temporales . . . . .                                              | 95         |
| 6.13. Algunos ejemplos . . . . .                                                           | 95         |
| 6.13.1. Cómo hacer los gráficos de series temporales en Excel . .                          | 95         |
| 6.13.2. Gráficos de series temporales en <b>Python</b> utilizando <b>pandas</b>            | 96         |
| 6.13.3. Resumen . . . . .                                                                  | 102        |
| 6.13.4. Algunas sugerencias para la creación de gráficos . . . . .                         | 103        |
| <b>7. Medidas centrales y medidas de dispersión: la media y la varian-</b>                 | <b>105</b> |
| <b>anza.</b>                                                                               |            |
| 7.1. Introducción . . . . .                                                                | 105        |
| 7.2. Recursos utilizados . . . . .                                                         | 106        |
| 7.3. La media aritmética: un ejemplo de cálculo. . . . .                                   | 106        |
| 7.3.1. Definición . . . . .                                                                | 106        |
| 7.3.2. Deducción de la Fórmula: . . . . .                                                  | 106        |
| 7.3.3. Ejemplo: . . . . .                                                                  | 107        |
| 7.4. Las medidas de dispersión: la varianza y la desviación típica . . .                   | 113        |
| 7.5. Las limitaciones de la media y la desviación típica . . . . .                         | 117        |

|                                                                                                                 |            |
|-----------------------------------------------------------------------------------------------------------------|------------|
| 7.5.1. Ventajas e inconvenientes de las medidas paramétricas . . .                                              | 121        |
| 7.5.2. Ventajas e inconvenientes de las medidas no paramétricas . . .                                           | 121        |
| 7.6. Resumen . . . . .                                                                                          | 122        |
| <b>8. Práctica 1. Análisis exploratorio de una fabricación de queso camembert usando Python y Google Colab.</b> | <b>123</b> |
| 8.1. Flujo de trabajo . . . . .                                                                                 | 123        |
| 8.1.1. 1. Importar los datos . . . . .                                                                          | 124        |
| 8.1.2. 2. Ordenar/limpiar los datos . . . . .                                                                   | 126        |
| 8.1.3. 3. Visualizar: Análisis gráfico . . . . .                                                                | 127        |
| 8.1.4. 4. Comunicar: discusión de la información en equipo. . . .                                               | 146        |
| 8.2. Resumen del capítulo . . . . .                                                                             | 147        |
| 8.3. Ejercicios propuestos . . . . .                                                                            | 147        |
| 8.3.1. Ejercicio 1 . . . . .                                                                                    | 147        |
| 8.3.2. Ejercicio 2 (avanzado) . . . . .                                                                         | 148        |
| <b>II Análisis cuantitativo de la producción quesera</b>                                                        | <b>149</b> |
| <b>9. El balance de materia y los rendimientos queseros.</b>                                                    | <b>151</b> |
| 9.1. Etapas generales en el proceso de fabricación de queso. . . . .                                            | 152        |
| 9.2. Algunas causas de las pérdidas de materia en el proceso quesero .                                          | 154        |
| 9.3. Algunas causas de las pérdidas de eficiencia en la transformación quesera . . . . .                        | 155        |
| 9.4. Análisis de datos en producción con Python: más allá del balance contable . . . . .                        | 155        |
| <b>10.El balance de materia</b>                                                                                 | <b>157</b> |
| 10.0.1. Descripción de las columnas (variables) del fichero de datos                                            | 158        |
| <b>11.Los rendimientos queseros</b>                                                                             | <b>167</b> |
| 11.1. Un ejemplo de cálculo de rendimientos . . . . .                                                           | 167        |
| 11.2. Lectura y exploración de los datos . . . . .                                                              | 168        |
| 11.3. Rendimiento simple . . . . .                                                                              | 169        |
| 11.4. Cálculo de la cantidad de leche necesaria para fabricar 1 kg de queso . . . . .                           | 170        |
| 11.5. La materia grasa . . . . .                                                                                | 175        |
| 11.6. El estándar de fabricación o estándar técnico . . . . .                                                   | 176        |
| 11.7. La importancia de la estandarización de la leche . . . . .                                                | 177        |
| <b>12.La valoración económica del balance de materia y los rendimientos.</b>                                    | <b>183</b> |
| 12.1. El pago de la leche por calidad en España . . . . .                                                       | 183        |
| 12.1.1. Conceptos clave del pago por calidad . . . . .                                                          | 183        |
| 12.1.2. Aplicación práctica: ejemplo de cálculo . . . . .                                                       | 184        |
| <b>13.Práctica 2. Análisis de los rendimientos en fabricación de queso</b>                                      |            |

|                                                                                                                              |            |
|------------------------------------------------------------------------------------------------------------------------------|------------|
| <b>mediante Python</b>                                                                                                       | <b>187</b> |
| 13.1. Exploración gráfica de los datos . . . . .                                                                             | 189        |
| 13.2. Análisis previo de los datos . . . . .                                                                                 | 191        |
| <b>Anexos</b>                                                                                                                | <b>213</b> |
| <b>A. La media aritmética como centro de gravedad de un grupo de datos</b>                                                   | <b>213</b> |
| A.0.1. Ejemplo . . . . .                                                                                                     | 214        |
| A.0.2. Paso a Paso . . . . .                                                                                                 | 214        |
| A.0.3. Ejemplo Práctico . . . . .                                                                                            | 215        |
| A.0.4. Media Ponderada . . . . .                                                                                             | 215        |
| A.0.5. Centro de Gravedad como Media Ponderada . . . . .                                                                     | 216        |
| A.0.6. Ejemplo Numérico . . . . .                                                                                            | 216        |
| A.0.7. Media Aritmética y Centro de Gravedad con Masas Idén-<br>ticas . . . . .                                              | 217        |
| A.0.8. Conclusión . . . . .                                                                                                  | 218        |
| A.1. Importancia del concepto de la media como centro de gravedad<br>o punto de equilibrio de un conjunto de datos . . . . . | 218        |
| <b>Bibliografía</b>                                                                                                          | <b>219</b> |
| <b>B. Índice de DataFrames utilizados</b>                                                                                    | <b>221</b> |
| B.1. Análisis de producto terminado de una fabricación diaria de ca-<br>membert . . . . .                                    | 221        |
| B.2. Datos de fabricación de diferentes recetas de queso ibérico para<br>balance de materia . . . . .                        | 221        |
| B.3. Lecturas de humedad relativa y temperatura en una cueva para<br>maduración de queso . . . . .                           | 222        |
| B.4. Datos de análisis de quesos tipo ibérico a la salida de la salmuera                                                     | 222        |
| B.5. Datos a salida de saladero de quesos ibéricos . . . . .                                                                 | 222        |
| B.6. Datos de un proceso de envasado de queso en unidades de 250 g .                                                         | 222        |
| B.7. Datos de entrada de leche en una fábrica de queso ibérico de mezcla.                                                    | 223        |
| B.8. Datos de análisis de producto terminado de diferentes productos,<br>en una tabla Excel . . . . .                        | 223        |



# Capítulo 1

## Introducción

Este documento trata sobre el análisis de datos industriales utilizando tanto la hoja de cálculo como un lenguaje de programación moderno como es **Python**. El uso de las técnicas estadísticas se ha reducido al mínimo, orientando el análisis al uso de métodos gráficos descriptivos sencillos.

¿Por qué usar **Python** como herramienta? Hay varias razones que parecen oportunas:

- **Python** es actualmente el lenguaje de programación más extendido y el más utilizado en los nuevos campos de la inteligencia artificial (IA). Su conocimiento, aunque sea de forma básica, aportará un plus a los alumnos en su *curriculum*.
- El Principado de Asturias ha establecido una línea prioritaria de desarrollo de conocimientos en las áreas relacionadas con la IA para los estudiantes de Formación Profesional. Esto, unido al hecho de que algunas empresas de la región ya están implantando algunas experiencias en esta dirección, parece indicar la conveniencia de que los alumnos empiecen a familiarizarse con estos ámbitos de conocimiento.
- **Python** posee los elementos necesarios para construir los gráficos que se necesitan para una explicación sencilla de un conjunto de datos de producción alimentaria, y su sintaxis resulta fácil de comprender, aunque otras herramientas, como el lenguaje R, sean más utilizadas en el análisis estadístico y de datos.
- Tener un contacto práctico con el lenguaje facilitará la adquisición de conocimientos suplementarios, ya sea en el ámbito del análisis estadístico de datos, de los métodos cuantitativos de mejora de procesos (*Six Sigma*), en la programación informática, o en la introducción a la IA.
- **Python** es una herramienta gratuita, por lo que tanto el lenguaje como

las herramientas para trabajar con él están a disposición libre en la web. En nuestro caso, utilizaremos una aplicación de Google, llamada Google Colaboratory conocido coloquialmente como **Colab**, que permite utilizar scripts **Python** mediante la interfase jupyter. Colab se usa en el navegador web, Google Chrome u otro, y no es necesario hacer ninguna instalación de software. Para los objetivos de la enseñanza, Google Colab resulta conveniente y suficiente (ver uso de jupyter en educación).

En la industria alimentaria actual se produce un constante flujo de datos como resultado tanto de la implantación de sistemas de captura automáticos como de la tecnificación de los puestos de trabajo; se requiere por parte de los profesionales industriales que sean capaces de analizar esta enorme cantidad de datos para transformarlos en información para la decisión. En la empresa industrial media actual, son los ingenieros y técnicos de planta, y no estadísticos o ingenieros informáticos, quienes participan diariamente en la presentación y discusión de los datos y en la toma de decisiones operativas, tanto en los equipos de trabajo como ante la Dirección. Por esta razón, es necesario proporcionar a los estudiantes de la Formación Profesional un conocimiento básico de los conceptos, herramientas y métodos del análisis de datos, así como de algunas técnicas de presentación y comunicación de la información.

La experiencia industrial muestra que en las situaciones reales, sobre el terreno, la comprensión práctica de los conceptos es más importante que su rigurosa formulación matemática. Por esta razón, en el desarrollo del contenido de este documento insistiré más en la forma de aplicar las herramientas y entender los análisis que en el conocimiento formal de las fórmulas estadísticas y su deducción matemática, o de la programación informática en sí. He hecho especial hincapié en la utilización de herramientas sencillas, casi siempre gráficas, que son una ayuda valiosa para comprender la información contenida en un conjunto de datos. El objetivo es proporcionar al estudiante de industrias alimentarias las bases de la metodología para análisis de sus datos y su aplicación a la resolución de problemas técnicos concretos, más que el conocimiento de la teoría matemática de la estadística o de la programación informática.

Se evitan las explicaciones formales sobre temas estadísticos. Así, por ejemplo, al explicar la media de un conjunto de datos considero más importante entender el concepto físico de “centro de gravedad” que los conceptos estadísticos de esperanza matemática, que no se tocan en este texto. En este sentido, se ha intentado que el alumno aprenda bien la diferencia entre “en qué consiste” un estadístico, y “cómo se calcula”. Comprender la diferencia entre el concepto y su fórmula de cálculo es fundamental para entender cómo aplicarlo.

Por ejemplo, veremos cómo el modo de cálculo de la media aritmética hace que su valor sea equivalente al centro de gravedad de un conjunto de datos; si aproximamos esta idea a la de un sistema de palanca en equilibrio sobre un fulcro, será muy fácil entender y hacer entender por qué los valores anómalos extremos desvían notablemente la media y deben analizarse con cuidado: una pequeña desviación en un ítem muy lejos del centro de gravedad tiene un gran efecto sobre

el punto de equilibrio del sistema, debido a la longitud del brazo de palanca; de la misma manera, los valores extremos pueden introducir desviaciones en el valor de la media aritmética que siempre hay que considerar.

También será fácil comprender que otros estadísticos, como la mediana, que se calculan de forma diferente (la mediana es, simplemente, el valor central del conjunto de datos) resultan menos o nada afectados por los valores extremos;; veremos en la práctica las ventajas e inconvenientes de cada uno.

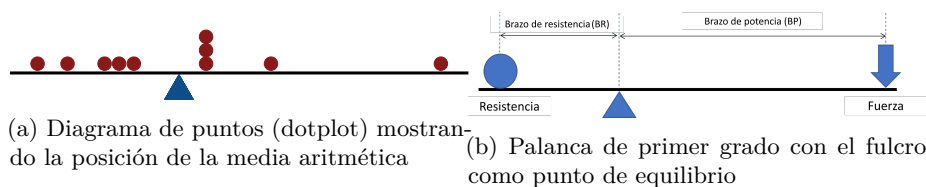


Figura 1.1: Comparación gráfica del significado de la media en un diagrama de puntos (*dotplot*) con una palanca de primer grado.

Un curso de introducción al análisis de datos industriales debe ser, ante todo, práctico y orientado a su aplicación en el entorno industrial real. Los principales temas de trabajo de análisis de los datos en la industria tienen que ver con su captura, su almacenamiento y su depuración, su descripción utilizando gráficos, la inferencia (intervalos de confianza y tests), la construcción de modelos explicativos, el diseño de los experimentos industriales, el control estadístico de la calidad y la exposición y presentación de resultados. Dado el alcance limitado de este documento, no se entrará en los métodos estadísticos en sí, y necesitarán de un estudio posterior si el alumno tiene interés en profundizar en ellos.

A pesar de que los temas más especializados puedan ser importantes en algunas aplicaciones específicas, no preparan al estudiante para lo que se va a encontrar en el terreno en la mayor parte de las ocasiones. En cambio, la resolución de problemas en equipo en un entorno de aprendizaje dinámico enfrentándose a problemas exigentes, y el desarrollo de las habilidades de análisis, de síntesis y de comunicación, tendrán un impacto mucho más positivo.

He intentado mostrar la necesidad de que los estudiantes comprendan y apliquen el método científico en el entorno industrial, y no sólo apliquen un recetario de procedimientos de manera automática. Son mucho mas importantes la comprensión y la utilización adecuada del método científico y de las herramientas y gráficos básicos, antes que la aplicación rutinaria y mecánica de determinadas fórmulas matemáticas o métodos sofisticados y complejos (como la IA) que el alumno puede no comprender en toda su profundidad.

Las industrias líderes destacan por la aplicación intensiva de métodos tales como *Six Sigma*, *Lean Manufacturing*, diseño robusto de productos, y otros que hacen un uso intensivo de los datos, tanto de los obtenidos en producción como de los obtenidos en la realización de experimentos diseñados. Pero la mejora de

la competitividad en estas empresas no se debe tanto a la aplicación de unos u otros métodos, como al desarrollo del juicio analítico de sus equipos humanos y a la aplicación de lo aprendido a la mejora continua de los procesos industriales. Veremos que la experiencia y el conocimiento tecnológico de estos procesos son fundamentales para el desarrollo del buen juicio analítico, y, en consecuencia, para la buena interpretación de los resultados que se obtienen con las herramientas estadísticas y de análisis.

Tratándose de un libro para el uso en la Formación Profesional, considero prioritario que su estudio se oriente al desarrollo de habilidades que sean de aplicación práctica directa en el puesto de trabajo y además faciliten la empleabilidad del estudiante, y no a la obtención de conocimiento abstracto. En la última edición del *Informe sobre el futuro del empleo* disponible cuando se edita este libro, publicada por el Foro Económico Mundial (WEF) en junio de 2025 (World Economic Forum 2025), el WEF considera que:

...al igual que en las dos ediciones anteriores de este informe, el **pensamiento analítico** sigue siendo la principal habilidad básica para los empleadores, ya que siete de cada 10 empresas lo consideran esencial. A esto le siguen la resiliencia, la flexibilidad y la agilidad, junto con el liderazgo y la influencia social, lo que subraya el papel fundamental de la adaptabilidad y la colaboración junto con las habilidades cognitivas. El pensamiento creativo y la motivación y la autoconciencia ocupan el cuarto y quinto lugar, respectivamente. Esta combinación de habilidades cognitivas, de autoeficacia e interpersonales dentro de los cinco primeros enfatiza la importancia atribuida por los encuestados a tener una fuerza laboral ágil, innovadora y colaborativa, donde tanto la capacidad de resolución de problemas como la resiliencia personal son críticas para el éxito.

A estas habilidades se suman las dos que más crecen en estos dos últimos años: la **inteligencia artificial (IA)** y el **análisis de datos**. El objetivo de este documento es proporcionar conocimientos que ayuden al estudiante a desarrollarse en esta dirección, y servir de apoyo a los docentes para conseguir ese objetivo. En este sentido, se incluyen casos de análisis con ejemplos prácticos de la metodología presentada aplicada al análisis, y con propuestas de ejercicios a resolver, apoyándose en las herramientas de IA.



## Parte I

# Introducción



## Capítulo 2

# El análisis de datos de producción en el entorno industrial.



Figura 2.1: Un grupo de trabajo en planta ante un panel de variables indicadoras

## 2.1. Introducción

¿Alguna vez te has preguntado cómo podemos resolver problemas en el trabajo de forma sistemática y ordenada? La estadística y el método científico nos ayudan a hacerlo, y son más sencillos de lo que parece.

El método científico es como una receta que seguimos para resolver problemas: primero observamos qué está pasando, luego pensamos qué puede estar causando el problema, después hacemos pruebas para comprobarlo, y finalmente sacamos conclusiones y tomamos acciones orientadas a eliminar el problema o conseguir una mejora, aunque sea parcial, y repetimos el ciclo. La estadística nos ayuda a entender si nuestras pruebas funcionaron o no, usando números y datos reales.

Pensar de forma estadística significa entender que todo lo que hacemos en el trabajo está conectado, y que es normal que las cosas cambien o varíen un poco. Lo importante es saber cuándo estos cambios son normales y cuándo indican un problema real. Es como cuando cocinamos: sabemos que cada plato puede salir un poco diferente, pero reconocemos cuando algo ha salido realmente mal.

En las fábricas y talleres, estas herramientas son muy útiles. Por ejemplo, si una máquina empieza a producir piezas defectuosas, podemos:

1. Observar qué está pasando
2. Pensar en posibles causas
3. Hacer pruebas controladas para encontrar el problema
4. Usar números y datos para confirmar si lo hemos solucionado

La experimentación es muy importante en la industria. En vez de cambiar las cosas al azar, hacemos pruebas organizadas para ver qué funciona mejor. Es como cuando ajustas la temperatura del horno y el tiempo de cocción para que un pastel salga perfecto: pruebas diferentes combinaciones y anotas los resultados.

Lo mejor de todo es que cada vez que hacemos estos experimentos, aprendemos algo nuevo. Incluso si algo no funciona como esperábamos, esa información nos ayuda a mejorar la próxima vez. Es un proceso continuo de aprendizaje y mejora.

Esta forma de trabajar nos ayuda a:

- Resolver problemas de forma ordenada
- Tomar mejores decisiones basadas en datos reales
- Mejorar la calidad de nuestro trabajo
- Ahorrar tiempo y dinero evitando errores

Recuerda: no necesitas ser un genio de las matemáticas para usar estas herramientas. Lo importante es ser organizado, observar bien lo que pasa y anotar los resultados de lo que hacemos. Así, poco a poco, podemos mejorar nuestro trabajo y resolver problemas de forma más eficiente.

Los métodos estadísticos nos ayudan a describir y comprender la **variabilidad**. Cuando hablamos de variabilidad queremos decir que sucesivas observaciones

de un mismo proceso o sistema no dan exactamente los mismos resultados. Por ejemplo, el consumo de gasolina de un coche no es siempre igual, sino que varía de manera considerable. Esta variación depende de muchos factores, como la forma de conducir, el tipo de carretera, la situación del propio vehículo (presión de neumáticos, compresión del motor, ...), la marca de la gasolina, el octanaje, o incluso las condiciones meteorológicas. Todos estos factores son causas de **variabilidad** en el consumo de gasolina. La estadística nos permite analizar estos **factores** y determinar cuáles son los más importantes o tienen mayor impacto en el consumo; una vez conocidos, podemos actuar sobre ellos.

En este libro aprenderemos a utilizar herramientas diversas, tanto estadísticas como de la ciencia de datos, para realizar nuestro análisis. Para aprender de los datos necesitamos más que los simples números; para interpretarlos necesitaremos siempre el conocimiento del proceso industrial que estamos analizando. En un análisis de la producción de un producto lácteo, por ejemplo, los números significan poco sin un conocimiento del proceso; los valores de pH, temperatura o concentración de lactosa influyen en el resultado del proceso de forma diferente. Los datos son números dentro de un contexto, y necesitamos conocer este contexto para dar sentido a los números.

**! Importante**

El objetivo principal de la **mejora industrial** es la **reducción de la variabilidad**.

## 2.2. El pensamiento estadístico y el método científico.

La estadística y el método científico mantienen una relación fundamental que impulsa el avance del conocimiento y la innovación. El método científico, con sus pasos de observación, hipótesis, experimentación y conclusiones, encuentra en la estadística las herramientas necesarias para validar o refutar hipótesis de manera objetiva y cuantificable.

El pensamiento estadístico es una filosofía de aprendizaje y acción basada en tres principios fundamentales:

- que todo proceso industrial está compuesto a su vez por procesos interconectados,
- que la variabilidad existe y es inherente a estos procesos, y
- que entender y reducir la variación es clave para la mejora y el éxito

En el entorno industrial, esta integración entre estadística y método científico ha revolucionado los procesos de mejora continua. Por ejemplo, cuando una línea de producción enfrenta problemas de calidad, el método científico guía la investigación sistemática: se observa el proceso, se formulan hipótesis sobre las causas

del problema, se diseñan experimentos controlados, y se analizan los resultados estadísticamente para determinar la validez de las soluciones propuestas.

La experimentación industrial, particularmente a través del diseño de experimentos (DOE), se ha convertido en una herramienta fundamental para la mejora continua. En lugar de modificar procesos basándose en intuiciones o experiencias pasadas, las empresas pueden realizar experimentos controlados que:

- Optimizan múltiples variables simultáneamente
- Identifican interacciones entre factores que afectan el proceso
- Reducen el tiempo y costo de las mejoras
- Proporcionan conclusiones respaldadas por evidencia estadística

Al realizar experimentos controlados, las empresas pueden probar hipótesis que permitan evaluar el impacto de diferentes variables en los procesos y productos y determinar los valores óptimos de los parámetros de los procesos para maximizar la calidad y la eficiencia. Esto genera conocimiento valioso sobre los procesos y productos, permite tomar decisiones más informadas, y ayuda a crear y mantener una ventaja competitiva.

La experimentación, combinada con el análisis estadístico, permite a las empresas implementar ciclos de mejora continua, donde los resultados de cada experimento se utilizan para refinar los procesos y productos.

Al integrar el pensamiento estadístico con el método científico en el entorno industrial, las organizaciones pueden desarrollar una cultura de mejora continua basada en la evidencia de los datos y no en suposiciones o intuiciones. En una cultura industrial basada en el método científico, cada problema es una oportunidad para experimentar, aprender y optimizar. Esta aproximación sistemática no solo mejora la calidad y eficiencia de los procesos, sino que también fomenta la innovación y el aprendizaje organizacional continuo.

La clave del éxito radica en entender que la experimentación no es un evento aislado, sino un proceso continuo de aprendizaje y mejora. Cada experimento, exitoso o no, aporta información valiosa que, analizada correctamente mediante métodos estadísticos, contribuye al conocimiento colectivo de la organización y sienta las bases para futuras mejoras. La estadística y el método científico no solo son pilares fundamentales para la investigación académica, sino que también desempeñan un papel crucial en el entorno industrial. La aplicación sistemática del método científico, respaldada por herramientas estadísticas, permite a las empresas optimizar procesos, mejorar la calidad de sus productos y servicios, y fomentar la innovación.

### ! El pensamiento estadístico en la toma de decisiones industriales

En el entorno industrial, el pensamiento estadístico es esencial para:

- **Tomar decisiones basadas en datos**, evitando decisiones basadas en intuiciones o suposiciones.

- **Evaluar riesgos**, cuantificando la incertidumbre y permitiendo tomar decisiones que minimicen los riesgos.
- **Mejorar el conocimiento** de los procesos para una toma de decisiones más eficaz.
- **Comunicar resultados**, presentando los resultados de los análisis de manera clara y concisa.

La integración del método científico y el pensamiento estadístico en la cultura empresarial impulsa la innovación, la eficiencia y la competitividad.

## 2.3. Los datos industriales

En el entorno industrial, podemos organizar los datos en varias categorías, considerando tanto su naturaleza como la forma en que se obtienen:

**Según su estructura:**

- **Datos estructurados:**
  - Son datos organizados en un formato definido, como tablas de bases de datos, hojas de cálculo o archivos CSV. Su estructura facilita el análisis y la consulta.
  - Ejemplos: lecturas de sensores, registros de producción, datos de control de calidad, información de inventario.
- **Datos no estructurados:**
  - Son datos que no tienen un formato predefinido, como texto, imágenes, audio o video. Su análisis requiere técnicas más avanzadas, como el procesamiento de lenguaje natural o la visión artificial.
  - Ejemplos: registros de mantenimiento, informes de incidentes, imágenes de inspección visual, grabaciones de audio de maquinaria.
- **Datos semiestructurados:**
  - Son datos que tienen cierta estructura, pero no tan rígida como los datos estructurados, y permiten una mayor flexibilidad en el almacenamiento y el intercambio de información.
  - Ejemplos: archivos XML o JSON, registros de eventos.

**Según su origen y método de obtención:**

- **Datos históricos (Estudios retrospectivos):**
  - Son datos recopilados en el pasado, que se utilizan para analizar tendencias, identificar patrones y predecir el comportamiento futuro. Permiten comprender la evolución de los procesos.
  - Ejemplos: registros de producción de años anteriores, datos de fallos de maquinaria, históricos de ventas.
- **Datos observacionales (Estudios observacionales):**
  - Son datos recopilados mediante la observación de procesos o sistemas, sin intervenir en ellos. Permiten identificar relaciones entre variables y

comprender el comportamiento de los procesos en condiciones reales.

- Ejemplos: mediciones de temperatura, vibración o presión de maquinaria, registros de tiempo de ciclo de producción.

▪ **Datos experimentales (Experimentos diseñados):**

- Son datos recopilados mediante la realización de experimentos controlados, en los que se manipulan variables para evaluar su efecto. Permiten establecer relaciones de causa y efecto, estudiar las interacciones entre las variables y optimizar el rendimiento de los procesos.
- Ejemplos: datos de pruebas de rendimiento de nuevos materiales, resultados de experimentos de optimización de procesos.

▪ **Datos de monitorización o control de procesos en tiempo real:**

- son datos que se recaban de manera instantánea, y permiten actuar de manera casi inmediata en los procesos; permiten implementar el mantenimiento predictivo, y evitar pérdidas de producción.
- Ejemplos: datos de sensores que detectan fallos en maquinaria, alarmas de procesos fuera de control.

En el contexto de la Industria 4.0, la cantidad y variedad de datos industriales está aumentando exponencialmente.

### 2.3.1. Estudios retrospectivos o históricos

Un **estudio retrospectivo o histórico** es el que utiliza una muestra o todos los datos históricos de un proceso, recogidos en el pasado durante un período determinado de tiempo. El objetivo de un estudio de este tipo puede ser la investigación sobre la relación entre algunas variables, o explorar la calidad de la información disponible, o construir un modelo que permita explicar el proceso tal como es actualmente, o saber si se ha desviado. Estos modelos del proceso se denominan **modelos empíricos**, porque están basados en los propios datos del proceso y no en una formulación teórica sobre el mismo.

Un estudio retrospectivo tiene la ventaja de tener a su disposición un gran número de datos que ya han sido recogidos, minimizando el esfuerzo de obtenerlos. Sin embargo, tiene varios problemas potenciales:

1. Si no disponemos de detalles suficientes, es posible que no podamos determinar si las condiciones de variación de los valores obtenidos responden a las mismas causas que en la situación actual.
2. Es posible que nos falte algún valor clave que no haya sido recogido o que lo haya sido de manera defectuosa
3. Algunas veces, la fiabilidad y validez de los datos de proceso históricos son dudosas, o al menos, cuestionables.
4. Los datos históricos no siempre se han recogido con la perspectiva actual del proceso, y es posible que no nos proporcionen explicaciones adecuadas del proceso en su situación actual.
5. A veces queremos utilizar los datos históricos de proceso para fines que no estaban previstos cuando se recogieron



6. Las notas sobre los valores del proceso, incluyendo los valores anormales, pueden ser insuficientes o inexistentes, y no tenemos ninguna explicación sobre los posibles valores anómalos que detectamos en el análisis.

Usar datos históricos siempre tiene el riesgo de que, por la razón que sea, no se hayan recogido datos importantes, o que estos datos se hayan perdido, o se hayan transcrito de forma inadecuada o incorrecta. Es decir, los datos históricos pueden tener problemas de calidad de datos.

El hecho de que algunos datos se hayan recogido históricamente no siempre quiere decir que estos datos sean relevantes o útiles. Cuando el grado de conocimiento del proceso no es suficiente, o no se basa en un análisis metódico y riguroso de los datos, es posible que no se hayan recogido algunos datos que pueden ser importantes para el proceso, a veces simplemente porque son complejos o difíciles de analizar. Los datos históricos no pueden proporcionar la información que buscamos si la información de las variables clave nunca se ha recogido o se ha hecho sin una buena base experimental.

El propósito del análisis de los datos industriales es aislar las causas que están detrás de los sucesos que afectan e influyen en los procesos. En los datos históricos, estos sucesos pueden haber ocurrido semanas, meses o incluso años antes, sin que haya registros ni notas que hayan intentado explicar estas causas, y los recuerdos de las personas que han participado en ellos se pierden con el tiempo, o se alteran involuntariamente, proporcionando explicaciones supuestamente válidas pero que en realidad son incorrectas. Por eso, con frecuencia, el análisis de los datos históricos puede poner de manifiesto hechos interesantes, pero sus causas quedan sin explicar.

Los estudios históricos pueden requerir una fase previa de preparación y depuración de datos que puede llegar a ser muy larga y tediosa. Se estima que en muchos estudios de ciencia de datos, el tiempo de preparación de los datos puede llegar al 60 % del tiempo total empleado en el estudio. Las herramientas de análisis de datos son de gran ayuda en esta fase del proceso, aunque en muchas ocasiones será necesario un trabajo manual de recolección de datos en papel, hojas de cálculo diversas y otras fuentes. Esta fase es muy útil no sólo para la preparación de datos para el estudio, sino para mejorar el conocimiento de los datos, cómo se originan y cómo se almacenan. Este conocimiento siempre es de gran utilidad para mejorar los procedimientos actuales de captura de datos, facilitando la fiabilidad de los análisis futuros.

### 2.3.2. Estudios observacionales

Como su nombre indica, un **estudio observacional** simplemente observa un proceso durante un tiempo de operación en rutina. Normalmente, el ingeniero o técnico interfiere lo mínimo posible en el proceso; sólo lo suficiente para recoger la información que necesita, si piensa que esa información puede ser relevante. En muchas ocasiones, el estudio no forma parte de los controles de rutina, y representa un trabajo adicional.

Si se planifican adecuadamente, los estudios observacionales proporcionan datos fiables, precisos y completos para documentar un proceso. Por otra parte, estos estudios proporcionan una información limitada sobre las relaciones entre las variables del proceso, porque es posible que durante el tiempo limitado de observación, el rango de variación de las variables no recoja todas las situaciones posibles; por ejemplo, las situaciones extraordinarias.

### 2.3.3. Experimentos diseñados

La tercera forma de recoger información de un proceso son los **experimentos diseñados**. En un experimento de este tipo, el ingeniero o técnico hace un cambio deliberado en las variables que controla (llamadas **factores**), observa el resultado, y toma una decisión respecto a qué variable o variables son responsables de los cambios que observa en el proceso.

Una diferencia importante respecto a los estudios históricos y los observacionales es que las diferentes combinaciones de factores se aplican al azar sobre un conjunto de unidades experimentales. Esto permite establecer con precisión las relaciones causa-efecto, cosa que no suele ser posible ni en los estudios históricos ni en los observacionales.

## Capítulo 3

# Herramientas para el análisis. La reproducibilidad.

### 3.1. Introducción

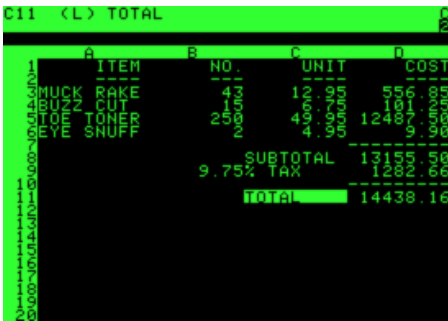
En este capítulo veremos las herramientas que utilizaremos en el análisis de datos industriales. Nos enfocaremos en dos de ellas: la hoja de cálculo Microsoft Excel, y el lenguaje de programación **Python**. Ambas herramientas están ampliamente extendidas en las empresas y en las instituciones docentes de todo el mundo. **Python** es, además del lenguaje de programación más utilizado en el mundo, un software libre, y por lo tanto, con un coste de adquisición cero, lo que facilita su utilización. Microsoft Excel tiene versiones web que se pueden utilizar para un uso básico también sin coste. No son las únicas opciones: hay otros programas estadísticos y de análisis muy utilizados y potentes, tales como Matlab o Minitab, que son de gran interés en ingeniería, aunque tienen un coste bastante elevado, y hojas de cálculo como Google Docs o LibreOffice, casi totalmente compatibles con Microsoft Excel.

### 3.2. La hoja de cálculo

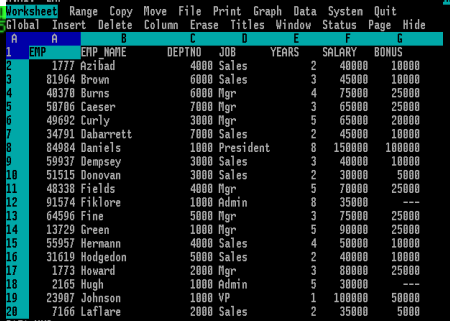
La hoja de cálculo es una herramienta presente hoy día en todos los ámbitos de trabajo y educativos. Desde la aparición de Visicalc, en 1978, ha contribuido a la gestión de miles de empresas, se ha utilizado de manera general en análisis de datos y sus gráficos se han utilizado y se utilizan en publicaciones e informes de todas clases. En la década de los años 80 del pasado siglo, la hoja de cálculo Lotus 1-2-3 fue la aplicación más utilizada en los ordenadores IBM-PC y compatibles, y

## 16CAPÍTULO 3. HERRAMIENTAS PARA EL ANÁLISIS. LA REPRODUCIBILIDAD.

consiguió facturaciones millonarias para la empresa matriz. Lotus 1-2-3 dominó el mercado hasta la aparición de Microsoft Windows a finales de los años 80; este nuevo sistema operativo favoreció la implantación de Microsoft Excel, que desde entonces se convirtió en la hoja de cálculo dominante.



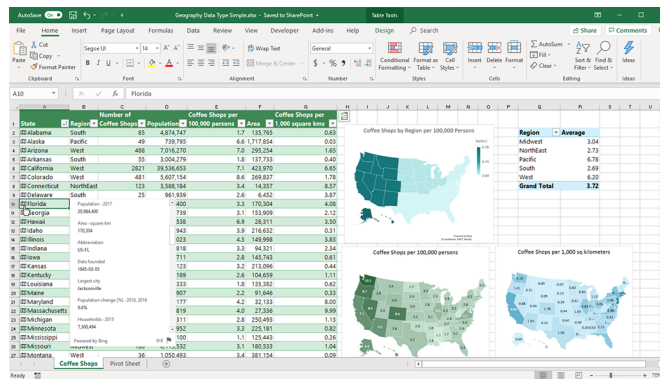
| ITEM      | NO. | UNIT | COST     |
|-----------|-----|------|----------|
| MUCK      | 4   | 12   | 5        |
| RAKE      | 1   | 5    | 1        |
| CUT       | 1   | 1    | 1        |
| SHUT      | 1   | 1    | 1        |
| TEB       | 1   | 1    | 1        |
| SUBTOTAL  |     |      | 1315     |
| 9.75% TAX |     |      | 128      |
| TOTAL     |     |      | 14438.16 |

| EMP   | EMP_NAME  | DEPTNO | JOB       | YEARS | SALARY | BONUS  |
|-------|-----------|--------|-----------|-------|--------|--------|
| 1777  | Azibad    | 4000   | Sales     | 2     | 40000  | 10000  |
| 81964 | Brown     | 6000   | Sales     | 3     | 45000  | 10000  |
| 48370 | Burns     | 6000   | Mgr       | 4     | 75000  | 25000  |
| 50706 | Caeser    | 7000   | Mgr       | 3     | 65000  | 25000  |
| 49692 | Curly     | 3000   | Mgr       | 5     | 65000  | 20000  |
| 34701 | Dabavrett | 7000   | Sales     | 2     | 45000  | 10000  |
| 84984 | Daniels   | 1000   | President | 8     | 150000 | 100000 |
| 59937 | Dempsey   | 3000   | Sales     | 3     | 40000  | 10000  |
| 51515 | Donovan   | 3000   | Sales     | 2     | 30000  | 5000   |
| 48338 | Fields    | 4000   | Mgr       | 5     | 70000  | 25000  |
| 91574 | Fiklore   | 1000   | Admin     | 8     | 35000  | ---    |
| 64586 | Fine      | 5000   | Mgr       | 3     | 75000  | 25000  |
| 13729 | Green     | 1000   | Mgr       | 5     | 90000  | 25000  |
| 55957 | Hermann   | 4000   | Sales     | 4     | 50000  | 10000  |
| 31619 | Hodgedon  | 5000   | Sales     | 2     | 40000  | 10000  |
| 1773  | Howard    | 2000   | Mgr       | 3     | 80000  | 25000  |
| 2165  | Hugh      | 1000   | Admin     | 5     | 30000  | ---    |
| 23987 | Johnson   | 1000   | VP        | 1     | 100000 | 50000  |
| 7166  | Laflare   | 2000   | Sales     | 2     | 35000  | 5000   |

(a) Visicalc, primera hoja de cálculo para el ordenador *Apple II* (1979)

(a) Hoja de cálculo Lotus 1-2-3 para MS-DOS (1983)



(a) Microsoft Excel (2023)

### 3.2.1. Aplicaciones de la hoja de cálculo

Las **hojas de cálculo** son muy útiles para recoger la información de un conjunto de observaciones. Entre sus principales usos, están:

- La introducción, edición y almacenamiento de datos.
- El filtrado y corrección de errores.
- La manipulación básica, por ejemplo, mediante tablas dinámicas
- La preparación y edición de gráficos, incluyendo gráficos dinámicos
- La presentación de la información, con el apoyo opcional de herramientas adicionales como Microsoft PowerPoint.

Los datos se pueden recoger y guardar de múltiples formas. Cuando la recogida de datos se hace de forma manual en papel, es necesario registrar en el ordenador los datos recogidos. Lo más frecuente es que este registro se haga en hojas de cálculo, como Microsoft Excel o Google Sheets. En algunos casos, el almacenamiento se hace sobre bases de datos, genéricas o desarrolladas a medida.

Actualmente, la tendencia es recoger los datos o bien de forma automática, o bien de forma manual sobre sistemas informatizados (pantallas), lo que permite eliminar el papel y disponer directamente de los datos en un formato digitalizado.

En la actualidad, la mayoría de los equipos y líneas de producción se interconectan con los sistemas de información (ver IoT) y almacenan en tiempo real todos los datos necesarios, lo que libera al operario de la pesada tarea de reintroducirlos manualmente, a la vez que reduce los errores debidos a la imputación incorrecta.

En todos los casos, es imprescindible asegurar que los sistemas de información pueden exportar sus datos a ficheros de texto tipo *fichero plano* o tipo CSV, de forma que podamos importarlos tanto a Excel como a R, como veremos más adelante. Estos sistemas de exportación de datos deben diseñarse de forma flexible y abierta, para que tanto la captura como la exportación puedan modificarse y adaptar la recogida de la información a las necesidades de cada momento.

En este libro trataremos exclusivamente de lo que llamaremos **datos rectangulares**: grupos de valores que están asociados a una o más variables, y a varias observaciones. Hay muchos más datos que no se ajustan a esta organización tabular, es el caso de imágenes, sonidos o archivos documentales de texto. Pero la forma más común de almacenar datos industriales es la de las tablas rectangulares, organizadas según el principio de los **datos arreglados**, que detallaremos en el siguiente capítulo.

### 3.3. Los lenguajes de programación

#### 3.3.1. Python en el análisis de datos industriales: una herramienta versátil

Python se ha convertido en el lenguaje de referencia para el análisis de datos en entornos industriales por su combinación de simplicidad, potencia y ecosistema científico. En industrias alimentarias, donde se manejan grandes volúmenes de datos de producción, calidad, trazabilidad y consumo energético, Python ofrece ventajas clave:

### Ventajas técnicas

- **Sintaxis clara y accesible:** ideal para estudiantes sin experiencia previa en programación.
- **Bibliotecas especializadas:**
  - **pandas** para manipulación de datos tabulares (ej. registros de producción, análisis de lotes).
  - **matplotlib** y **seaborn** para gráficos de control, tendencias y correlaciones.
  - **numpy** para cálculos numéricos precisos (temperaturas, tiempos, concentraciones).
- **Automatización de tareas repetitivas:** generación de informes, limpieza de datos, detección de anomalías.

### Aplicaciones concretas en industrias alimentarias

- **Control de calidad:** análisis de desviaciones en parámetros críticos (pH, humedad, temperatura).
- **Optimización de procesos:** visualización de curvas de cocción, enfriamiento, fermentación.
- **Trazabilidad:** seguimiento de lotes desde materia prima hasta producto final.
- **Seguridad alimentaria:** detección de patrones en alertas sanitarias o fallos de producción.

### ¿Por qué incluir Python en la Formación Profesional?

Un módulo básico de Python y gráficos en programas de formación profesional en industrias alimentarias permite:

### Desarrollo de competencias clave

- **Alfabetización digital aplicada:** no solo usar software, sino entender cómo se procesan los datos.
- **Pensamiento lógico y estructurado:** útil para resolver problemas técnicos y documentar procesos.
- **Autonomía en el análisis:** no depender exclusivamente de hojas de cálculo o software cerrado.

### Enfoque pedagógico sugerido

- **Aprendizaje basado en proyectos:** por ejemplo, analizar datos reales de producción o simular un sistema de trazabilidad.
- **Visualización como herramienta de comprensión:** enseñar a interpretar gráficos de dispersión, histogramas, series temporales.
- **Modularidad y reproducibilidad:** fomentar el uso de scripts anotados y reutilizables, alineado con tu enfoque docente.

## 3.4. Otras herramientas: R, Julia

### 3.4.1. El software estadístico R

R es un lenguaje de programación y un entorno de software utilizado para el análisis estadístico, la visualización de datos y la modelización. Algunas de las características clave de R son:

1. **Amplio espectro de funcionalidades:** R abarca una amplia gama de herramientas y paquetes diseñados para realizar diversos análisis estadísticos, exploración de datos y modelización.
2. **Herramientas gráficas:** R dispone de algunas de las bibliotecas gráficas más potentes para la exploración y descripción de datos.
3. **Estadística descriptiva:** R ofrece funciones para calcular estadísticas descriptivas básicas, como la media, mediana, desviación estándar, varianza, rango, cuartiles y percentiles. Estas funciones son esenciales para explorar y resumir datos.
4. **Contrastes de hipótesis:** R proporciona funciones para realizar pruebas estadísticas, como t-tests, test chi-cuadrado, ANOVA y pruebas no paramétricas. Estas pruebas permiten evaluar hipótesis y comparar grupos de datos.
5. **Distribuciones de probabilidad:** R incluye una amplia variedad de funciones para trabajar con distribuciones de probabilidad (por ejemplo, normal, uniforme, binomial, Poisson). Esto es útil para generar números aleatorios, calcular probabilidades y cuantiles.
6. **Bibliotecas de funciones (librerías):** Además de las amplias funciones básicas de las que dispone, R es capaz de utilizar bibliotecas de funciones (llamadas *librerías*) que han sido desarrolladas por los propios usuarios, y que amplían sus funcionalidades a todos los campos imaginables, desde el análisis genético al análisis de riesgos bancarios o el control estadístico de procesos.

### Usos y aplicaciones de R en la estadística industrial

En el contexto de la estadística industrial, R se utiliza para:

- **Control de calidad:** R permite analizar datos de procesos industriales, identificar desviaciones y controlar la calidad de los productos.
- **Optimización de procesos:** Mediante técnicas estadísticas avanzadas, R ayuda a optimizar procesos industriales, reducir costos y mejorar la eficiencia.
- **Análisis de fiabilidad:** R se utiliza para evaluar la confiabilidad de sistemas y componentes en la industria.

### Utilización práctica de R en el entorno industrial

[..a desarrollar..]

- Importación de datos y exportación de datos
- Manipulación de datos: depuración, corrección, filtrado de datos.
- Exploración gráfica de los datos
- Análisis estadísticos específicos.
- Gráficos de control
- Informes automatizados

### 3.4.2. El lenguaje Julia

**Julia** es un lenguaje de programación más reciente, diseñado específicamente para el cálculo numérico y la ciencia de datos. Ofrece un rendimiento cercano al de lenguajes de bajo nivel como C, pero con la sintaxis y facilidad de uso de lenguajes de alto nivel como Python y R. Julia es especialmente útil en aplicaciones donde el rendimiento es crítico, como en simulaciones industriales y análisis de grandes volúmenes de datos.

- **Ventajas**
  - **Rendimiento:** Diseñado específicamente para computación numérica de alto rendimiento, Julia ofrece velocidades comparables a lenguajes compilados como C o Fortran, pero con una sintaxis más cercana a los lenguajes de scripting. Es excelente para cálculos numéricos intensivos y proporciona soporte nativo para paralelismo y computación distribuida.
  - **Sintaxis expresiva:** Su sintaxis es intuitiva y similar a la de lenguajes matemáticos, lo que facilita la escritura de código conciso y legible.
  - **Interoperabilidad:** Permite llamar a código escrito en otros lenguajes como C, Python o R, lo que facilita la integración con herramientas existentes.
  - **En crecimiento:** Aunque más joven que Python y R, Julia está ganando rápidamente popularidad en la comunidad científica y de datos.

En resumen, mientras que R sigue siendo una opción poderosa y preferida para la estadística industrial, Python y Julia se presentan como alternativas viables y, en algunos casos, superiores, dependiendo de los requisitos específicos del proyecto.



### 3.5. El concepto de reproducibilidad

La **reproducibilidad** de un ensayo o experimento es la capacidad de ser reproducido o replicado por otros, en particular, por la comunidad científica. La reproducibilidad se refiere a la capacidad de obtener resultados consistentes al replicar un estudio o experimento *utilizando los mismos datos*, metodología original y, en su caso, el mismo código informático empleado para los análisis. En otras palabras, cuando se replica un análisis de datos o un experimento, los resultados deben ser alcanzados nuevamente con un alto grado de confiabilidad.

La **repetibilidad** o **replicabilidad** se refiere a la posibilidad de obtener resultados consistentes al replicar un estudio con un *conjunto distinto de datos*, pero obtenidos siguiendo el mismo diseño experimental. Implica obtener resultados consistentes utilizando nuevos datos o nuevos resultados computacionales para responder a la misma pregunta científica.

El químico irlandés **Robert Boyle**, en el siglo XVII, subrayó la importancia de la reproducibilidad en la ciencia. Boyle sostenía que los fundamentos del conocimiento debían basarse en hechos producidos experimentalmente, que pudieran volverse creíbles para la comunidad científica por su reproducibilidad. La **bomba de aire de Boyle**, un aparato científico complicado y costoso en ese momento, condujo a una de las primeras disputas documentadas sobre la reproducibilidad de un fenómeno científico.

#### 3.5.1. Importancia en la Ciencia

La reproducibilidad es esencial para la investigación científica, ya que permite validar y verificar los resultados obtenidos. En las últimas décadas, ha habido una creciente preocupación por la falta de reproducibilidad en muchos resultados científicos publicados, lo que ha llevado a una crisis de reproducibilidad o replicación. La reproducibilidad garantiza que los resultados científicos sean confiables y puedan ser validados por otros investigadores. Es un pilar fundamental para el avance del conocimiento en todas las disciplinas.

#### 3.5.2. Reproducibilidad en metrología

En metrología, la reproducibilidad es la capacidad de un instrumento de dar el mismo resultado en mediciones diferentes, realizadas en las mismas condiciones a lo largo de periodos dilatados de tiempo. Esta cualidad debe evaluarse a largo plazo.

### 3.6. Ventajas de los lenguajes de programación en scripts frente a la hoja de cálculo en la reproducibilidad de un análisis

Las ventajas de utilizar un lenguaje de scripts en lugar de hojas de cálculo tradicionales (como Microsoft Excel) incluyen:

1. **Código abierto (*scripts*):** los lenguajes permiten crear flujos de trabajo basados en código, lo que mejora la reproducibilidad de los análisis y facilita la colaboración entre investigadores. Puedes escribir scripts en **Python** para automatizar tareas y asegurar la reproducibilidad. Los scripts son transparentes y pueden ser compartidos y verificados por otros investigadores.
2. **Flexibilidad estadística:** **Python** es especialmente útil para técnicas avanzadas de análisis, lo que lo convierte en una excelente opción para investigadores que buscan análisis de vanguardia, como es el caso de los modelos de inteligencia artificial (IA)
3. **Gráficos muy potentes y completos:** Los gráficos disponibles en estos lenguajes son una de sus fortalezas, no sólo por su variedad sino también por su flexibilidad.
4. **Capacidad para tratar grandes cantidades de datos:** pueden manejar grandes conjuntos de datos sin problemas, lo que es fundamental en la estadística industrial.
5. **Precisión:** Aunque no está diseñado específicamente para el análisis estadístico, como sí es el caso de **R**, **Python** puede realizar todos los análisis que se necesitan en el entorno industrial, y es más preciso que **Excel** en ciertos casos, como en análisis de regresión lineal.
6. **Capacidad avanzada:** **Python** ofrece una amplia gama de librerías y funciones para realizar análisis estadísticos avanzados, como modelos lineales, series temporales, análisis multivariante y más.

Por su parte, la hoja de cálculo tiene una *curva de aprendizaje* más sencilla y es en general más fácil de usar, pero tiene algunos inconvenientes:

7. **Inexactitudes:** Estudios han demostrado que **Excel** puede mostrar ciertas inexactitudes en análisis de regresión lineal y otros métodos estadísticos.
8. **Limitaciones gráficas:** Los gráficos de **Excel** son bastante limitados cuando se trata de presentar información sobre un análisis de datos.
9. **Limitaciones estadísticas:** **Excel** no está diseñado específicamente para análisis estadístico avanzado, por lo que puede carecer de algunas capacidades necesarias para investigaciones más complejas.
10. **Falta de transparencia para la auditoría:** Las fórmulas y cálculos en **Excel** pueden ser difíciles de rastrear y verificar, lo que afecta la reproducibilidad.

Por estas razones usaremos Excel para el almacenamiento de datos y el análisis

### *3.6. VENTAJAS DE LOS LENGUAJES DE PROGRAMACIÓN EN SCRIPTS FRENTE A LA HOJA DE CÁLCULO*

básico, y usaremos `Python` para el análisis gráfico más detallado y el análisis numérico y estadístico.



## Capítulo 4

# La organización de los datos y el flujo de trabajo. Los datos ordenados o *arreglados (tidy data)*

### 4.1. Definiciones y términos útiles

#### 4.1.1. Población y muestra

Una **población** es un conjunto de personas, cosas o, en general, objetos en estudio. A veces, una población es demasiado grande para que podamos abarcarla completa; para poder estudiarla, obtenemos una **muestra**, que consiste en un subconjunto de la población que hemos seleccionado para su estudio. El proceso de obtener una muestra se llama **muestreo**, y se realiza de acuerdo con normas y procedimientos específicos.

En muchas ocasiones, cuando se recogen los datos como resultado de una experimentación, definimos la población como *todos los resultados que podríamos haber obtenido*. Llamamos a este conjunto de posibles resultados una **población conceptual**. Por ejemplo, cuando medimos el *pH* de varias muestras de leche, la población es el conjunto de todos los resultados posibles que podríamos haber tenido. Muchos problemas de ingeniería y tecnología se refieren a poblaciones conceptuales.

**! Recuerda**

En la mayoría de las ocasiones, nuestros datos provienen de una **muestra** obtenida de una **población**,

Cuando tomamos una muestra, debemos estar seguros de que contiene las propiedades que queremos estudiar en la población. En ese caso, decimos que la muestra es **representativa**: los individuos de la muestra son representativos de la población. Para que la muestra sea representativa, debe ser obtenida mediante un **muestreo aleatorio**. Una **muestra aleatoria simple** de tamaño  $n$  consiste en  $n$  individuos de una población, elegidos de forma que cada conjunto posible de  $n$  individuos tiene la misma **probabilidad** de ser elegido.

**Ejemplo 1: Muestreando una cámara de maduración de queso**

Imagina que debes analizar el contenido de extracto seco de una producción de quesos en fase de maduración en una cámara frigorífica. Dado que la cámara está muy llena y es complicado acceder a su interior, decides tomar tu muestra de los quesos que están más a tu alcance, situados cerca de la puerta y a la altura de la vista.

¿Consideras que esta es una forma correcta de obtener una muestra representativa?

¿Cómo definirías la población en este caso?

**Respuesta al ejemplo 1: Muestreando una cámara de maduración de queso**

La forma de tomar la muestra no es correcta, ya que al seleccionar únicamente los quesos más accesibles introduces un *sesgo* en la muestra. Las condiciones de humedad, temperatura y circulación de aire podrían variar dentro de la cámara, afectando el contenido de extracto seco según la ubicación de los quesos.

Para obtener resultados fiables, es fundamental que la muestra sea representativa de toda la producción. Para ello, se debe aplicar un método de **muestreo aleatorio**, donde todos los quesos tengan la misma *probabilidad* de ser seleccionados.

En este caso, la **población** está formada por el conjunto total de quesos en proceso de maduración dentro de la cámara.

**4.1.2. Parámetro y estadístico**

Un **parámetro** es una característica de una población. Podemos estimar su valor mediante la extracción de una **muestra**, que utilizaremos para calcular un **estadístico muestral**. Llamamos **estadístico** a un número que representa una propiedad o característica de la muestra, y constituye una estimación del

valor de un parámetro de la población que estamos estudiando.

4.1.3. Variables y casos

A los objetos descritos en un conjunto de datos los llamamos **casos**, de forma genérica. A veces, estos casos pueden corresponder a personas; en ese caso podemos llamarlos **individuos**. Cuando los objetos que estudiamos no son personas, como es lo habitual en el entorno industrial, utilizamos la nomenclatura genérica.

Un **atributo** es una característica que define una propiedad de un objeto, persona o cosa. Por ejemplo, edad, peso, altura, sexo, color de ojos, son atributos de una persona. Llamamos **variable** a una característica cualquiera de un individuo o caso que puede ser medida u observada. Una variable puede tomar diferentes valores en diferentes individuos o casos.

Según estas definiciones que acabamos de ver, una muestra está formada por un conjunto de casos, y cada caso contiene un determinado número de variables, que contienen los valores que hemos analizado o medido.

4.1.4. Tipos de variables

Algunas variables, como el color, sirven para clasificar los individuos en categorías. Otras, como la altura o el peso de un individuo, pueden tomar valores numéricos con los que podemos hacer cálculos. Por ejemplo, podemos sumar la altura de varias personas, pero no tiene sentido *sumar* los colores del arco-iris (aunque sí podemos *contarlos*, y hacer cálculos con estos recuentos). También podemos *categorizar* variables continuas: podemos clasificar nuestro grupo de personas en *altas* o *bajas*, y podemos contar cuántas personas entran en cada categoría.

| Variables cualitativas o categóricas             |                                                | Variables cuantitativas o métricas                      |                                                          |
|--------------------------------------------------|------------------------------------------------|---------------------------------------------------------|----------------------------------------------------------|
| Nominales                                        | Ordinales                                      | Discretas                                               | Continuas                                                |
| Valores en categorías arbitrarias (sin unidades) | Valores en categorías ordenadas (sin unidades) | Valores enteros en escala numérica<br>Unidades contadas | Valores continuos en escala numérica<br>Unidades medidas |

Una **variable categórica** coloca a un individuo en uno o más grupos o categorías

Una **variable métrica** toma valores numéricos con los que tiene sentido realizar cálculos aritméticos como sumar, restar, etc.

Las variables categóricas se conocen también como **variables cualitativas** porque indican *cualidades*.

Las variables métricas se conocen también como **variables cuantitativas** porque indican *cantidades*.

 **Comentario:** ¿Cualitativo quiere decir “que tiene calidad”?

Algunas veces se usa el término *cualitativo* de forma incorrecta para referirse a la *calidad* de algo, por ejemplo, cuando alguien dice: “Este envase es muy cualitativo”. La forma correcta sería: “Este envase es de gran calidad”. En realidad, *cualitativo* no proviene *calidad* sino de *cualidad*, y hace referencia a las características o propiedades de un objeto o fenómeno, sin implicar juicio de valor. Como hemos visto, este término se usa en estadística para describir variables que representan categorías o atributos, como el color, el tipo de material o la marca, sin medir cantidades.

Para resolver

**Ejemplo 1.** Tiramos un dado al aire. Describe a qué corresponde la variable y el caso.

**Ejemplo 2.** Durante un proceso de envasado de un producto que dura una hora, controlamos el peso de cada envase cada minuto. Describe la variable y el caso. ¿Puede haber más de una variable?

Respuestas: Para resolver

**Ejemplo 1:** La **variable** es el resultado que obtenemos cada vez; podríamos denominarla, por ejemplo, *resultado\_obtenido*. Colocaríamos este nombre en el encabezado de una columna en una hoja de cálculo. Cada tirada que hacemos es un **caso**; iríamos colocando el resultado que obtenemos cada vez en una nueva fila de nuestra hoja de cálculo.

**Ejemplo 2.** En este caso, la **variable** es el *peso\_obtenido*, y cada pesada constituye un **caso**. Si registrásemos, además, la hora y el minuto en el que hemos hecho cada control de peso, podríamos definir una nueva variable, que podríamos llamar *hora*, y que colocaríamos en una columna al lado del *peso\_obtenido*. Incluso podríamos definir otra variable adicional, el *numero\_de\_pesada*, que sería un número secuencial empezando en 1 y que se incrementaría en cada pesada, de forma que al final esta variable nos daría el número de pesadas realizadas, y nos indicaría además el orden en el que las hemos realizado. Puesto que hemos realizado una pesada cada minuto, tendríamos tres variables y 61 líneas (un encabezado y 60 líneas correspondientes una a cada minuto)



#### 4.1.5. Reglas básicas para establecer los nombres de las variables.

Según hemos visto, existen diferentes tipos de variables, **cualitativas** (*categorías*) y **cuantitativas** (*métricas*). Normalmente, los valores de las variables categóricas se describen mediante textos del tipo “color blanco”, “hombre”, “mujer”, “alto”, “bajo”, etc. Suelen corresponder con características descriptivas, y por lo tanto, no puede hacerse cálculos directamente con ellos, a menos que se hayan resumido, por ejemplo, mediante un conteo. Las variables métricas consisten en valores numéricos, que pueden ser **enteros** o **continuos**, y pueden utilizarse directamente para hacer cálculos tales como sumas, etc.

Una **variable** está descrita siempre por **un nombre**, que designa la variable, y **un valor o conjunto de valores**, que corresponden a los casos. Este conjunto de valores, como acabamos de ver, pueden ser textos o números.

Ejemplos de valores de texto: “Carlos”, “fruta”, “Lluvia fuerte”, “muy ácido”, “sabor a fresa”

Ejemplos de valores numéricos: 1; 7; 10, 65

Siempre que sea posible, utilizaremos el nombre del atributo o característica que estamos midiendo o analizando, o su abreviatura, para designar una variable; por ejemplo, si estamos recogiendo la altura de una serie de personas, llamaremos **altura** a la variable; si estamos recogiendo el peso, usaremos el nombre **peso**, etc.

A veces, asignar un nombre a una variable no es todo lo fácil que podría parecer a simple vista. Por ejemplo, ¿qué nombre daríamos a una variable que va a recoger los valores de *pH* de la leche en una cuba de queso en el momento de añadir el cuajo? Está claro que *pH* no es suficiente, porque en el proceso hay varias medidas de *pH* y sería bueno que pudiésemos diferenciarlas con facilidad. En un caso como éste, es probable que necesitemos utilizar varias palabras o abreviaturas que describan mejor el nombre de la variable.

Para la construcción correcta de estos nombres, se han establecido un conjunto de reglas, con el objetivo de evitar errores y facilitar el intercambio de los datos entre diferentes programas de análisis. Son éstas:

1. Un nombre válido consiste en una combinación de letras, números y signo de subrayado (  )
2. Un nombre de variable no puede empezar por un número, un punto o un signo de subrayado (  ); debe empezar siempre por una letra.
3. Los nombres de variables irán siempre en minúsculas. Según esta regla, *Peso* no es un nombre válido, pero *peso* si lo es.
4. No utilizaremos espacios en blanco, acentos ni caracteres especiales como ñ, %, guiones o paréntesis.

A partir de aquí utilizaremos una nomenclatura compatible con las hojas de cálculo en formato europeo para escribir los números; usaremos la **coma** para la separación decimal y el **punto y coma** para la separación de los números cuando los escribamos de forma seriada. Existen también otros tipos de variables que veremos más tarde, como variables lógicas o fechas, según el tipo de dato que almacenemos en esa variable.

5. Hay veces en que nos interesa unir varias palabras para construir un nombre de variable. Se utilizan diferentes formas de unir palabras, por ejemplo:

- un punto, como en *peso.en.cm*,
- lo que se ha llamado *escritura de camello* (*camelCase*), que se llama así por el uso de mayúsculas y minúsculas mezcladas (*PesoEnCm*)
- el signo de subrayado `_`, como en *peso\_en\_cm*

Algunas de estas opciones son utilizadas en distintas comunidades de usuarios, por ejemplo la opción 1 es utilizada en la guía de estilo de Google, y la opción 2 es muy utilizada por los programadores del entorno de los lenguajes de Microsoft. Nosotros utilizaremos el signo de subrayado (`_`), que es la forma más usada en el entorno de programación de **Python**.

6. Siempre se separarán las palabras mediante el signo de subrayado (`_`) para facilitar la lectura. Así, aunque *temperatura1* es un nombre válido, preferiremos *temp\_1*; es más corto y de lectura más clara. Igualmente, preferiremos *peso\_empaquetado* a *pesoempaquetado*
7. Mantendremos los nombres razonablemente cortos para facilitar la lectura. Aunque podemos hacer los nombres todo lo largos que queramos, es más cómodo utilizar nombres cortos. Por ejemplo, podríamos utilizar *temperatura\_de\_la\_leche\_al\_cuaajar*, pero preferiremos abreviarlo como *temp\_cuaajo*.

Nombres no válidos:

- *peso en gramos* (contiene espacios)
- *pH\_de\_la\_leche\_en\_Recepción* (demasiado largo, tiene un acento, tiene mayúsculas)
- *extracto\_seco\_total\_a\_la\_salida\_de\_la\_salmuera* (demasiado largo)

Alternativas válidas:

- *peso\_g*
- *pH\_leche\_rec* (en este caso, de manera excepcional, podemos mantener el uso de la mayúscula por corrección formal)
- *est\_salida\_sal*

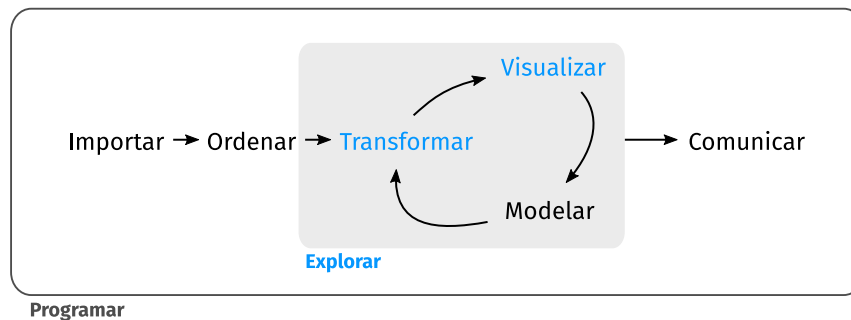
Un caso particular es el uso de la *ñ*, ya que no hay una alternativa fácil para el uso en las fechas (*año*). **Python** admite el uso de la *ñ* en los nombres de variables, por lo que podremos usarlo con cuidado, poniendo atención a los posibles errores que se pudiesen producir en algunas librerías.

## 4.2. El flujo de trabajo

Un **flujo de trabajo en análisis de datos** es un proceso sistemático y estructurado que guía la manipulación, exploración y análisis de datos desde su

recolección hasta la obtención de resultados finales y su comunicación. Es una hoja de ruta que asegura que cada paso se realice de manera ordenada, eficiente y reproducible, facilitando la comprensión y utilización de los datos.

Hadley Wickham (Garret Golemund Hadley Wickham Mine Çetinkaya-Rundel 2023) ha propuesto un método de flujo de trabajo que se ha convertido en estándar en la ciencia de datos (hay versión en español: (Garret Golemund Hadley Wickham 2023))



Este flujo de trabajo abarca diversas actividades como la importación de datos, su limpieza y transformación, el análisis exploratorio, y el modelado, culminando en la interpretación y presentación de los resultados. Todo esto se hace siguiendo metodologías específicas para asegurar la calidad y precisión del análisis.

La importancia de seguir un flujo de trabajo bien definido radica en la capacidad de replicar estudios, minimizar errores y fomentar la transparencia, permitiendo que cualquier persona pueda entender y validar las decisiones tomadas durante el análisis. Además, mejora la eficiencia al estandarizar procedimientos y facilita la colaboración entre diferentes analistas o equipos de trabajo.

## 4.3. Etapas en un flujo de trabajo estructurado.

### 4.3.1. Recolección de datos

La primera etapa es la **recolección de datos**. Esto implica obtener datos desde diversas fuentes como archivos CSV, bases de datos, APIs, etc. La recolección de datos es fundamental porque la calidad del análisis depende de la calidad de los datos recolectados.

### 4.3.2. Inspección de los datos

Una vez recolectados, se procede a **inspeccionar los datos** para entender su estructura y contenido. Esto incluye examinar los tipos de datos, la presencia de valores faltantes, duplicados y la distribución de las variables.

### 4.3.3. Limpieza de los datos

La **limpieza de datos** es crucial para asegurar que la información sea precisa y esté en el formato adecuado. Esta etapa incluye:

- Manejo de valores faltantes.
- Eliminación de duplicados.
- Corrección de inconsistencias.
- Transformación de datos a un formato adecuado para el análisis.

### 4.3.4. Transformación de los datos

Transformar los datos a un formato **ordenado o arreglado** (*tidy*) es esencial. Según Wickham, los datos **arreglados** tienen una estructura clara: cada variable es una columna, cada observación es una fila, y cada valor tiene su propia celda. Este formato facilita el análisis y la visualización de datos.

### 4.3.5. Análisis exploratorio de datos (EDA)

El **Análisis Exploratorio de Datos (EDA)** busca entender los patrones y relaciones en los datos mediante estadísticas descriptivas y visualizaciones. Durante esta etapa se realizan:

- Estadísticas básicas (media, mediana, desviación estándar).
- Gráficos y diagramas para visualizar la distribución de los datos y las relaciones entre variables.

### 4.3.6. Modelado de datos

Dependiendo del objetivo del análisis, se pueden aplicar diversos modelos estadísticos para extraer información y hacer predicciones. Esto puede incluir:

- Modelos de regresión.
- Análisis de clasificación.
- Modelos de series temporales, entre otros.

### 4.3.7. Comunicación de resultados

Finalmente, es fundamental **comunicar los resultados** de manera clara y efectiva. Esto se hace a través de:

- Tablas y resúmenes interpretativos.
- Gráficos y visualizaciones.
- Informes y presentaciones que expliquen los hallazgos y sus implicaciones.

Siguiendo estos pasos, puedes manejar y analizar datos de manera organizada y reproducible, facilitando la colaboración y la toma de decisiones informadas. Este flujo de trabajo asegura que los datos se traten de manera sistemática, desde su recolección hasta la comunicación de los resultados.

## 4.4. Razones para seguir un flujo de trabajo

1. **Reproducibilidad:** Un flujo de trabajo organizado permite que los análisis sean reproducibles. Otros pueden seguir los mismos pasos para obtener resultados similares, lo que es crucial en la investigación y en la toma de decisiones basadas en datos.
2. **Consistencia:** Ayuda a asegurar que los pasos se realizan de manera consistente cada vez que se ejecuta el análisis, reduciendo la posibilidad de errores humanos.
3. **Transparencia:** Proporciona un registro claro de los pasos tomados durante el análisis, facilitando la revisión y validación de los resultados.
4. **Eficiencia:** Mejora la eficiencia al estandarizar el proceso, permitiendo a los analistas concentrarse en el análisis y la interpretación de los datos en lugar de tareas repetitivas.
5. **Colaboración:** Facilita la colaboración entre equipos, ya que los flujos de trabajo bien documentados permiten que otros comprendan fácilmente los métodos y pasos utilizados.
6. **Adaptabilidad:** Permite adaptar y ajustar el análisis de manera más fácil cuando se presentan nuevos datos o cuando cambian los objetivos del análisis.

## 4.5. Problemas de no seguir un flujo de trabajo estructurado

1. **Errores y Sesgos:** La falta de un enfoque estructurado puede resultar en errores y sesgos inadvertidos en el análisis, lo que puede llevar a conclusiones incorrectas.
2. **Dificultad para Replicar Resultados:** Sin un flujo de trabajo claro, replicar resultados se vuelve complicado, lo que puede afectar la credibilidad del análisis y la capacidad de validación por otros.
3. **Falta de Documentación:** La ausencia de una documentación adecuada dificulta entender los pasos y las decisiones tomadas durante el análisis, lo que puede ser un obstáculo en auditorías y revisiones.
4. **Ineficiencia:** Sin una estructura clara, los analistas pueden gastar tiempo valioso realizando tareas repetitivas y resolviendo problemas que podrían haberse evitado con un enfoque más organizado.
5. **Problemas de Colaboración:** La colaboración se vuelve más difícil si los miembros del equipo no pueden seguir o entender los pasos tomados por otros, lo que puede llevar a malentendidos y duplicación de esfuerzos.

6. **Dificultad para Adaptarse a Cambios:** Sin un flujo de trabajo definido, adaptar el análisis a nuevos datos o cambios en los objetivos puede ser más complejo y propenso a errores.

En resumen, seguir un flujo de trabajo estructurado es esencial para garantizar la precisión, eficiencia, y reproducibilidad del análisis de datos, evitando problemas que puedan comprometer la integridad y utilidad de los resultados. .

## 4.6. Un ejemplo: revisando los datos existentes.

Cuando nos incorporamos a un equipo de trabajo existente, lo más seguro es que ya se disponga de un sistema de archivo de los datos, de acuerdo con los métodos habituales del equipo. En muchos casos, el diseño de la captura de datos sigue aproximadamente el modelo manual en papel; se introducen los datos en la hoja de cálculo y una vez completados, se imprime el documento para su archivo.

El error más común que se suele cometer es, precisamente, tratar la hoja de cálculo como un bloc de notas, es decir, hacer anotaciones de forma libre, colocar los datos y el resultado de los análisis al lado y en cualquier parte de la hoja, y apoyarnos en el contexto para interpretar lo que hemos guardado. Pero para que el ordenador sea capaz de analizar nuestros datos de manera eficiente, debemos estructurarlos de tal forma que el programa use la información tal como nosotros queremos.

Es común utilizar una hoja para guardar múltiples tablas de datos, tal como vemos en la Figura 4.1. Esta estructura, sin embargo, resulta enormemente confusa para su análisis, o lo imposibilita completamente.

|    | A                     | B       | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | AA | AB | AC | AD | AE | AF | AG |
|----|-----------------------|---------|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|
| 1  |                       |         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 2  | lake site May 29 2012 |         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 3  |                       |         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 4  | 1                     | T1      | 1 | 1 | 2 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 5  | 2                     | T1      | 1 | 1 | 2 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 6  | 3                     | T1      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 7  | 4                     | T1      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 8  | 5                     | T1      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 9  | 6                     | T2      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 10 | 7                     | T2      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 11 | 8                     | T2      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 12 | 9                     | T2      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 13 | 10                    | T2      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 14 | 11                    | control | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 15 | 12                    | control | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 16 | 13                    | control | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 17 | 14                    | control | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 18 | 15                    | control | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 19 |                       |         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 20 |                       |         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 21 | lake site May 29 2012 |         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 22 |                       |         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 23 | 1                     | T1      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 24 | 2                     | T1      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 25 | 3                     | T1      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 26 | 4                     | T1      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 27 | 5                     | T1      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 28 | 6                     | T2      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 29 | 7                     | T2      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 30 | 8                     | T2      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 31 | 9                     | T2      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 32 | 10                    | T2      | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 33 | 11                    | control | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 34 | 12                    | control | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 35 | 13                    | control | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 36 | 14                    | control | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 37 | 15                    | control | 1 | 1 | 3 |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 38 |                       |         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |
| 39 |                       |         |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |    |    |    |    |    |    |    |

Figura 4.1: Hoja Excel desordenada: ¡No hagas esto!

En otros casos, los datos se guardan en hojas de cálculo que se componen de diferentes pestañas para cada semana, cada mes o cada año, como vemos en la Figura 4.2. Sin embargo, esta forma de almacenar los datos tampoco es la óptima para su análisis.

| Enero               |       | Día    |        |       |       |       |       |       |
|---------------------|-------|--------|--------|-------|-------|-------|-------|-------|
|                     | Media | Máximo | Mínimo | 1     | 2     | 3     | 4     | 5     |
| Extracto seco total | 46,35 | 48,09  | 44,83  |       |       |       |       |       |
| Materia Grasa       | 24,41 | 28,00  | 23,00  |       |       |       |       |       |
| pH                  | 4,78  | 4,99   | 4,67   |       |       |       |       |       |
| Cloruros            | 1,49  | 1,69   | 1,24   |       |       |       |       |       |
| Coliformes totales  | 609   | 9200   | 0      |       |       |       |       |       |
| Febrero             |       | Día    |        |       |       |       |       |       |
|                     | Media | Máximo | Mínimo | 1     | 2     | 3     | 4     | 5     |
| Extracto seco total | 45,25 | 47,13  | 43,59  | 46,22 | 45,65 | 43,84 | 44,76 | 44,11 |
| Materia Grasa       | 23,34 | 25,50  | 22,00  | 23,00 | 23,50 | 22,00 | 23,50 | 23,50 |
| pH                  | 4,67  | 4,79   | 4,56   | 4,61  | 4,67  | 4,79  | 4,66  | 4,69  |
| Cloruros            | 1,45  | 1,88   | -1,00  | 1,88  | 1,67  | 1,57  | 1,76  | 1,82  |
| Coliformes totales  | 7     | 100    | 0      | 0     | 0     | 0     | 0     | 0     |
| Marzo               |       | Día    |        |       |       |       |       |       |
|                     | Media | Máximo | Mínimo | 1     | 2     |       |       |       |
| Extracto seco total | 45,31 | 47,43  | 42,92  | 45,28 | 46,13 |       |       |       |

Figura 4.2: Hoja Excel con una estructura no ordenada

Si las diferentes tablas presentan situaciones diferentes, o datos que no están relacionados, podemos utilizar diferentes pestañas. Pero si los datos están vinculados, por ejemplo, se corresponden con las mismas medidas, hechas en fechas diferentes (meses, años), la respuesta es que las pestañas no son la forma correcta de almacenarlos datos; lo mejor es añadir una variable que nos permita diferenciar los datos por fecha; nuestro programa de análisis nos permitirá *filtrar* los datos según la fecha que deseemos, y todos estarán en una única tabla, facilitando la coherencia y el análisis posterior.

Hay muchas formas de almacenar la información en una hoja de cálculo, pero sólo la estructura de **datos ordenados o arreglados** facilita la utilización de los datos tanto por la hoja de cálculo como por otros programas de análisis.

## 4.7. Los datos ordenados o arreglados (*tidy data*)

De la misma manera que la gramática permite ordenar y estructurar un escrito de acuerdo a reglas comunes, hay reglas para que el almacenamiento de los datos sea lo más homogéneo posible y se reduzcan los errores al mínimo.

Las reglas principales al almacenar nuestros datos en una hoja de cálculo son tres:

- columnas=variables,
- filas=observaciones,
- celdas=valores.

Cada variable debe tener su propia columna, cada observación debe tener su propia fila, y cada valor debe tener su propia celda o casilla .

Estas tres reglas básicas son las que hacen que nuestro conjunto de datos sea **ordenado (o arreglado)**(Garret Grolemund Hadley Wickham Mine Çetinkaya-Rundel 2023) (hay edición *online* en español: (Garret Grolemund Hadley Wickham 2023):

La Figura 4.3 muestra estas reglas de forma visual.

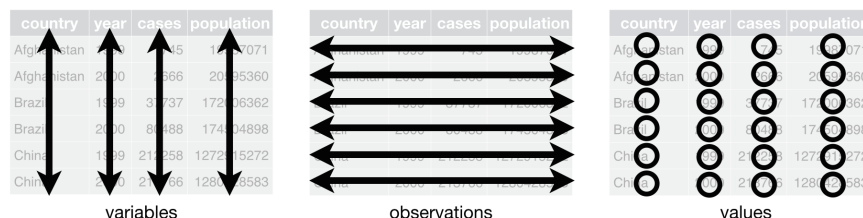


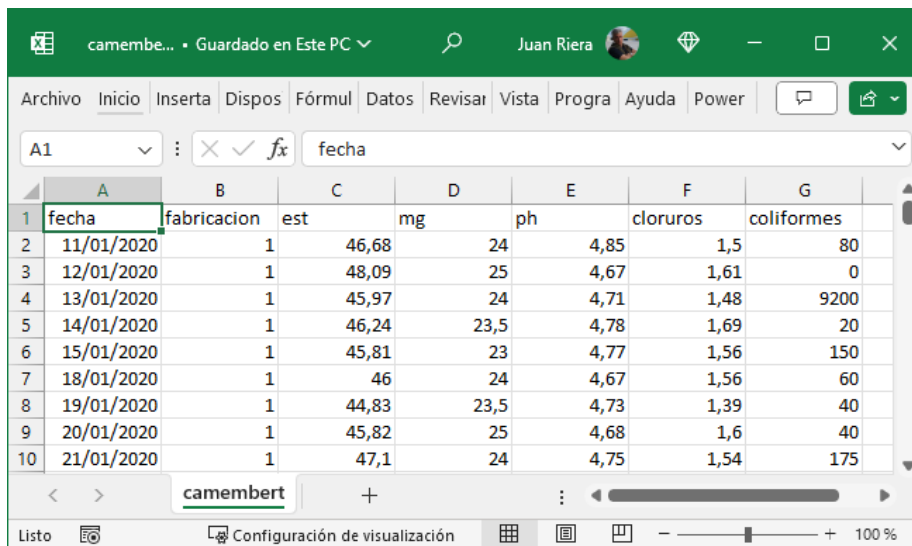
Figura 4.3: Sigue estas tres reglas para tener un conjunto de datos ordenado: las variables están en columnas, las observaciones están en filas, y los valores están en celdas. Fuente de la imagen: Garret Grolemund Hadley Wickham Mine Çetinkaya-Rundel (2023)

Estas tres reglas están interrelacionadas porque es imposible satisfacer sólo dos de tres.

En una hoja de cálculo, una tabla de datos arreglada tendría este aspecto:

Datos rectangulares: formato tabla





|    | A          | B           | C     | D    | E    | F        | G          |
|----|------------|-------------|-------|------|------|----------|------------|
| 1  | fecha      | fabricacion | est   | mg   | ph   | cloruros | coliformes |
| 2  | 11/01/2020 | 1           | 46,68 | 24   | 4,85 | 1,5      | 80         |
| 3  | 12/01/2020 | 1           | 48,09 | 25   | 4,67 | 1,61     | 0          |
| 4  | 13/01/2020 | 1           | 45,97 | 24   | 4,71 | 1,48     | 9200       |
| 5  | 14/01/2020 | 1           | 46,24 | 23,5 | 4,78 | 1,69     | 20         |
| 6  | 15/01/2020 | 1           | 45,81 | 23   | 4,77 | 1,56     | 150        |
| 7  | 18/01/2020 | 1           | 46    | 24   | 4,67 | 1,56     | 60         |
| 8  | 19/01/2020 | 1           | 44,83 | 23,5 | 4,73 | 1,39     | 40         |
| 9  | 20/01/2020 | 1           | 45,82 | 25   | 4,68 | 1,6      | 40         |
| 10 | 21/01/2020 | 1           | 47,1  | 24   | 4,75 | 1,54     | 175        |

Figura 4.4: Hoja Excel con estructura rectangular de datos ordenados

## 4.8. Qué es un fichero plano y un fichero CSV

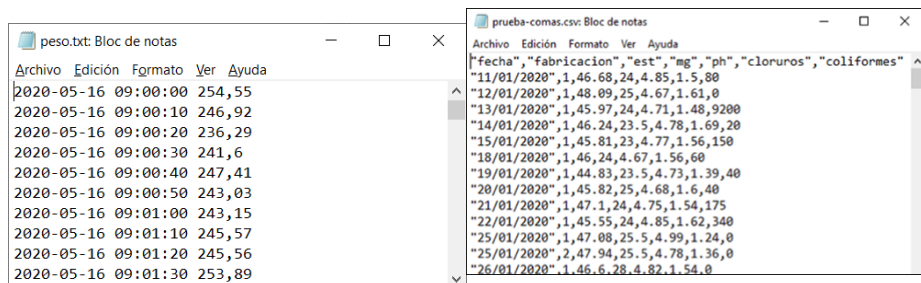
Se suele llamar **fichero plano** a un fichero de datos de texto sin ningún tipo de formato, donde los datos están separados por espacios o tabulaciones. Muchos equipos automáticos, como balanzas de laboratorio o básculas de proceso, producen ficheros planos de texto, que se pueden importar a Excel o **Python**. Un fichero CSV es un fichero plano en el que los valores están separados por un carácter especial, llamado **separador**. Este separador puede ser una coma , (cuando los decimales se separan mediante un punto, como en EEUU) o un punto y coma ; (cuando los decimales se separan mediante una coma, como en España)

En un fichero plano o en un fichero CSV, la primera fila puede contener los nombres de las columnas. En algunos casos, los elementos de texto pueden estar entre comillas. En estos casos, los programas de importación se ocupan de la conversión de formatos.

La importación de un fichero CSV en Excel en español es directa si se ha generado con puntos y comas como separador y comas para los decimales; si no es así, nos aparecerá como un fichero plano de texto sin formato, y tendremos que realizar una conversión.

### 4.8.1. Cómo exportar un fichero CSV desde Excel.

Una vez que tenemos nuestros datos en Excel, hay dos formas en las que podemos poner los datos a disposición de **Python** para su análisis: exportarlos a

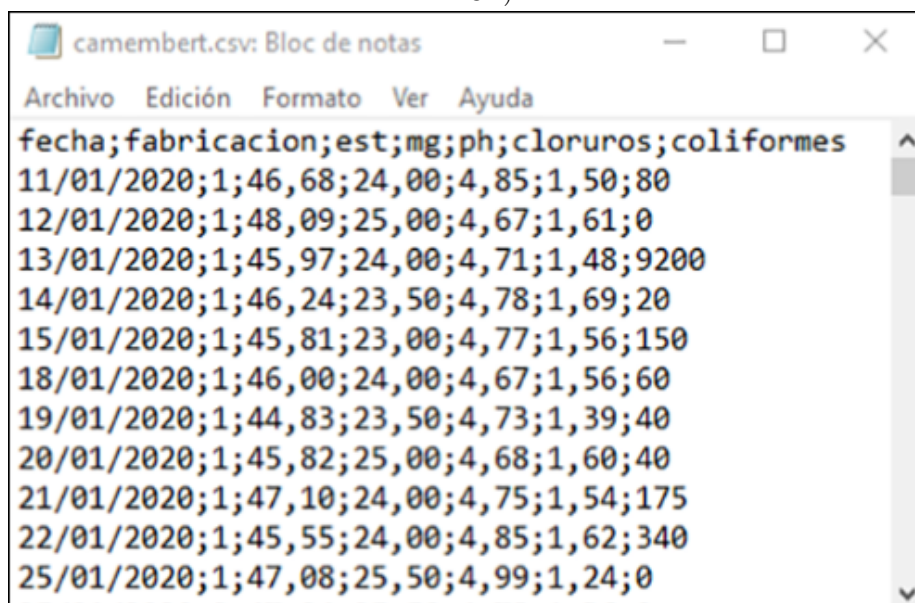


| Archivo    | Edición  | Formato | Ver | Ayuda |
|------------|----------|---------|-----|-------|
| 2020-05-16 | 09:00:00 | 254,55  |     |       |
| 2020-05-16 | 09:00:10 | 246,92  |     |       |
| 2020-05-16 | 09:00:20 | 236,29  |     |       |
| 2020-05-16 | 09:00:30 | 241,6   |     |       |
| 2020-05-16 | 09:00:40 | 247,41  |     |       |
| 2020-05-16 | 09:00:50 | 243,03  |     |       |
| 2020-05-16 | 09:01:00 | 243,15  |     |       |
| 2020-05-16 | 09:01:10 | 245,57  |     |       |
| 2020-05-16 | 09:01:20 | 245,56  |     |       |
| 2020-05-16 | 09:01:30 | 253,89  |     |       |

| fecha        | fabricacion | est       | mg        | ph | cloruros | coliformes |
|--------------|-------------|-----------|-----------|----|----------|------------|
| "11/01/2020" | 1,46.68     | 24,4.85   | 1,5.80    |    |          |            |
| "12/01/2020" | 1,48.09     | 25,4.67   | 1,61.0    |    |          |            |
| "13/01/2020" | 1,45.97     | 24,4.71   | 1,48.9200 |    |          |            |
| "14/01/2020" | 1,46.24     | 23,5.4.78 | 1,69.20   |    |          |            |
| "15/01/2020" | 1,45.81     | 23,4.77   | 1,56.150  |    |          |            |
| "18/01/2020" | 1,46.24     | 4.67      | 1,56.60   |    |          |            |
| "19/01/2020" | 1,44.83     | 23,5.4.73 | 1,39.40   |    |          |            |
| "20/01/2020" | 1,45.82     | 25,4.68   | 1,6.40    |    |          |            |
| "21/01/2020" | 1,47.1.24   | 4.75      | 1,54.175  |    |          |            |
| "22/01/2020" | 1,45.55     | 24,4.85   | 1,62.340  |    |          |            |
| "25/01/2020" | 1,47.08     | 25,5.4.99 | 1,24.0    |    |          |            |
| "25/01/2020" | 2,47.94     | 25,5.4.78 | 1,36.0    |    |          |            |
| "26/01/2020" | 1.46.6.28   | 4.82      | 1,54.0    |    |          |            |

(a) Fichero plano separado por espacios (b) Fichero CSV separado por comas (USA, GB)



| fecha      | fabricacion | est   | mg   | ph   | cloruros | coliformes |
|------------|-------------|-------|------|------|----------|------------|
| 11/01/2020 | 1;46,68     | 24,00 | 4,85 | 1,50 | 80       |            |
| 12/01/2020 | 1;48,09     | 25,00 | 4,67 | 1,61 | 0        |            |
| 13/01/2020 | 1;45,97     | 24,00 | 4,71 | 1,48 | 9200     |            |
| 14/01/2020 | 1;46,24     | 23,50 | 4,78 | 1,69 | 20       |            |
| 15/01/2020 | 1;45,81     | 23,00 | 4,77 | 1,56 | 150      |            |
| 18/01/2020 | 1;46,00     | 24,00 | 4,67 | 1,56 | 60       |            |
| 19/01/2020 | 1;44,83     | 23,50 | 4,73 | 1,39 | 40       |            |
| 20/01/2020 | 1;45,82     | 25,00 | 4,68 | 1,60 | 40       |            |
| 21/01/2020 | 1;47,10     | 24,00 | 4,75 | 1,54 | 175      |            |
| 22/01/2020 | 1;45,55     | 24,00 | 4,85 | 1,62 | 340      |            |
| 25/01/2020 | 1;47,08     | 25,50 | 4,99 | 1,24 | 0        |            |

(c) Fichero CSV separado por puntos y comas (Europa, España)

Figura 4.5: Tres tipos de ficheros planos de texto.

un archivo de texto con **formato CSV**, o leer directamente los datos de Excel desde **Python** utilizando las funciones adecuadas. En ambos casos, el resultado es un **DataFrame** o **cuadro de datos**, que es una estructura equivalente a la de nuestra tabla de datos en Excel. En el capítulo siguiente veremos cómo realizar esta importación de datos **Python** desde nuestro fichero **CSV**

En el ejemplo, crearemos un fichero CSV desde Excel y haremos un manejo básico de los datos.

#### Paso 1: Guardar el Fichero CSV desde Excel

1. Abre tu fichero en Excel.
2. Selecciona **Archivo > Guardar como**.
3. Elige el formato **CSV (delimitado por comas) (\*.csv)**.
4. Guarda el archivo.

## 4.9. Practica con el CSV

- En Excel, prueba a guardar y recuperar tus datos en formato CSV.

Una vez cargados los datos en Excel,

- selecciona el menú **Datos > Filtro**

y prueba a filtrar y seleccionar los datos según diferentes criterios

## 4.10. Para resolver

Poner aquí distintos ejemplos de nombres de variables para ver si son válidos o no. Describir medidas y preguntar cómo llamaríamos a esa variable (por ejemplo, temperatura de la leche que acabamos de descargar de una cisterna) :::



## Capítulo 5

# Introducción a Python en el entorno Google Colab

Python es un lenguaje de programación. Para trabajar con el lenguaje, se han desarrollado diferentes interfaces que mejoran el manejo del programa y facilitan el trabajo.

Nosotros utilizaremos Google Colaborate, conocido como Colaboratory o simplemente, Google Colab, basado en Jupyter, entorno originalmente desarrollado para Python y que se ha convertido en universal para muchos lenguajes de programación. En este entorno, podemos escribir código y ejecutarlo. También podemos escribir en celdas de texto, lo que nos permite intercalar explicaciones a nuestros cálculos o cualquier otro contenido escrito.

En este cuaderno vamos a ver algunas características de **Python** que resultan importantes para nuestro objetivo de análisis de datos industriales. Usaremos para ello fuentes de datos disponibles en la web y veremos diferentes aplicaciones del lenguaje sobre dichas fuentes de datos. Aunque el código resulte complejo a primera vista, el objetivo es simplemente una toma de contacto, en cuadernos sucesivos veremos datos reales y aplicación a la industria láctea.

Con el fin de ayudar al desarrollo de la sesión de estudio, este cuaderno se puede seguir en el texto a continuación o puede ser abierto en Colab para trabajar directamente sobre él. Una vez abierto, es conveniente guardarlo en el espacio de Colab en el Google Drive de cada uno.

Abrir este cuaderno en Google Colab/

Descargar los datos de ejemplo del archivo `camembert.csv`

## 5.1. Cómo usar este cuaderno

- Si has abierto el cuaderno en Colab por primera vez, sigue estos pasos:
  1. Guarda una copia en tu carpeta de Google Drive. Para ello, vete a **Archivo/Guardar una copia en Drive** y guárdalo.
  2. Una vez guardada la copia, puedes cambiar el nombre en 'Archivo/Cambiar nombre', o bien directamente haciendo *click* sobre el nombre en la parte superior de la hoja.
  3. SI usas la versión web de Drive (abriendo el Drive en el navegador), posiblemente tendrás activada la opción de abrir los cuadernos que tengas en tu carpeta usando el botón derecho de tu ratón.
- Si vas a crear una hoja nueva (en blanco) mediante el enlace a Google Colaborate, sigue estos pasos:
  1. Haz click en el enlace a continuación <https://colab.research.google.com/>, o copia y pega el texto en la barra de navegación de tu navegador, o simplemente introduce "Google Colab" en la barra de búsqueda y haz click sobre el resultado correcto de la búsqueda, irás directamente al sitio.
  2. Abre un cuaderno nuevo.
  3. Guarda una copia en **Archivo/Guardar una copia como**
  4. Renombra la hoja en **Archivo/Cambiar nombre** usando el nombre que quieras

En este cuaderno usaremos varios archivos de datos de demostración, ya existentes en diversas fuentes de Python, que nos servirán para hacer diferentes ejercicios numéricos y gráficos.

## 5.2. Primeros pasos

Colab funciona en un entorno **Jupyter**. En este entorno, el texto y el código Python se introducen en **celdas**.

Hay dos tipos de **celdas**: texto y código. En la parte superior de la ventana, puedes elegir el tipo que quieres insertar haciendo *click* en **+ Código** o bien en **+ Texto**.

Para editar una celda de texto existente, haz doble click sobre ella. Ahora puedes escribir usando las convenciones de código **markdown**. Cuando hayas terminado, ejecuta la celda para que Colab formatee y presente el texto.

Para ejecutar una celda de código, haz click sobre ella.

- **Shift-Enter** ejecuta el código y pasa a la siguiente celda.
- **Ctrl-Enter** ejecuta el código y permanece en la misma celda.

Recuerda también que las celdas de código deben ejecutarse en orden secuencial desde el principio.

Si la ejecución de una celda de código produce un error, no te preocupes, no tiene consecuencias graves. Simplemente, edita el código y corrige el error. En muchos casos, Google Gemini te puede ayudar a detectar los errores en el código y reescribirlo correctamente.

### 5.3. Cálculos simples con python

En su forma más básica, python se puede utilizar como una simple calculadora, utilizando los siguientes operadores aritméticos:

Adición: + (Ejemplo:  $2 + 2$ )

Resta: - (Ejemplo:  $2 - 2$ )

Multiplicación: \* (Ejemplo:  $2 * 2$ )

División: / (Ejemplo:  $2/2$ )

Exponenciación: \*\* (Ejemplo:  $2^{**}2$ )

Prueba a ejecutar las siguientes celdas:

```
2+2
```

```
4
```

```
2-2
```

```
0
```

```
2*2
```

```
4
```

```
2/2
```

```
1.0
```

```
2**2
```

```
4
```

```
(3*4)/2
```

```
6.0
```

### 5.4. Creación y asignación de variables

Una variable permite almacenar un valor (por ejemplo, 4) o un objeto (por ejemplo, una descripción de función). Más tarde se puede usar el nombre de esta variable para acceder fácilmente al valor o al objeto que está almacenado dentro de ella.

Podemos imaginar este concepto como una estantería llena de celdas vacías, en las cuales podemos colocar diferentes objetos: números, letras, frases, etc. La variable es el espacio que almacena un valor, y que podemos *llamar* u obtener simplemente escribiendo su nombre.

En python, una variable es un *objeto*. Podemos asignar *valores* a ese objeto, como si colocásemos libros en nuestra estantería. Esta asignación se hace siempre con el operador de asignación `=`, en la forma

```
nombre_de_objeto = valor
```

Por ejemplo, asignaremos un valor 4 a una variable que se llame `mi_var` con el operador de asignación de la siguiente forma:

```
mi_var = 4
```

Recuerda que ahora estamos en una **celda de texto**; prueba a hacer la asignación en la **celda de código** a continuación, añadiendo el operador de asignación y ejecutando la celda con **Mayus-Intro**:

```
mi_var 4
```

Una vez que asignamos un valor a una variable, python recuerda su valor hasta que lo cambiemos mediante una nueva asignación, borremos la variable, o finalicemos nuestra sesión.

Para ver el contenido del objeto(lo que hemos almacenado en `mi_var`), escribimos el nombre y ejecutamos la celda (sólo si hemos editado y ejecutado correctamente la celda anterior, en caso contrario obtendremos un error porque `mi_var` no está definida):

```
mi_var
```

## 5.5. Cómo establecer los nombres de variables

Para facilitar la legibilidad del código, los nombres de variables en python deben cumplir algunas normas generales. Buscaremos siempre que los nombres de los objetos sean descriptivos, por lo que necesitamos una forma de unir varias palabras.

Las convenciones más aceptadas para la designación de nombres de variables en Python se basan principalmente en la **PEP 8**, la guía de estilo oficial para el código Python. Aquí te resumo los puntos clave:

### 1. Snake Case (Recomendado para variables y funciones):

- Para nombres de variables y funciones que constan de varias palabras, se utiliza **snake\_case**, donde las palabras se escriben en minúscula y se separan por guiones bajos.
- Ejemplos: `mi_variable`, `nombre_completo`, `calcular_total`.

### 2. Camel Case (Para clases):



- Para nombres de **clases**, se utiliza **CamelCase** (también conocido como PascalCase), donde cada palabra comienza con mayúscula y no hay separadores.
  - Ejemplos: `MiClase`, `GestionDeUsuarios`, `HttpRequest`.
3. **Constantes (Mayúsculas con guiones bajos):**
- Para constantes (variables cuyo valor no debería cambiar durante la ejecución del programa), se utiliza **ALL\_CAPS** con guiones bajos para separar las palabras.
  - Ejemplos: `MAX_VALOR`, `PI`, `TASA_INTERES`.
4. **Reglas generales (obligatorias por el lenguaje):**
- **Longitud:** Las variables pueden tener cualquier longitud.
  - **Caracteres permitidos:** Pueden contener letras (mayúsculas y minúsculas), números y guiones bajos (`_`).
  - **No pueden empezar con un número:** Un nombre de variable no puede comenzar con un dígito.
  - **Sensibilidad a mayúsculas y minúsculas:** Python distingue entre mayúsculas y minúsculas. `mi_variable` y `Mi_Variable` son nombres de variables diferentes.
  - **Evitar palabras reservadas:** No se pueden usar palabras reservadas de Python (como `if`, `else`, `for`, `while`, etc.) como nombres de variables.
  - **No usar caracteres especiales:** No se permiten espacios, signos de puntuación u otros símbolos especiales.
5. **Recomendaciones de estilo:**
- **Nombres descriptivos:** Elige nombres de variables que sean claros y descriptivos, que reflejen su propósito en el programa. Esto mejora la legibilidad del código.
  - **Evitar nombres de una sola letra:** A menos que sean contadores simples (`i`, `j`, `k` en bucles) o iteradores, es mejor evitar nombres de variables de una sola letra.
  - **Evitar nombres que se confundan:** Ten cuidado con letras que pueden confundirse fácilmente, como la `l` minúscula y el número `1`, o la `O` mayúscula y el número `0`.
  - **Guiones bajos al principio (`_nombre`):** Un solo guion bajo al principio (`_`) suele indicar que una variable o función es “interna” o “no pública” dentro de un módulo o clase. Es una convención, no una restricción estricta.
  - **Dobles guiones bajos al principio (`__nombre`):** Los dobles guiones bajos al principio de un nombre de atributo de clase (`__nombre`) activan el “name mangling” (mutilación de nombre) de Python, que hace que el atributo sea más difícil de acceder directamente desde fuera de la clase, actuando como una forma de “privacidad”.

Seguir estas convenciones, especialmente las de la PEP 8, ayuda a que tu código sea más consistente, legible y fácil de entender por otros desarrolladores (y por ti mismo en el futuro).

Usemos un nombre realmente largo para designar a una variable:

```
este_es_un_nombre_realmente_largo = 5.3
```

Ahora recuperemos el valor que hemos almacenado

```
este_es_un_nombre_realmente_largo
```

5.3

Recuerda que python utiliza la escritura anglosajona para la separación de decimales, usando un punto, y no una coma como en España. ¿Qué pasaría si utilizamos una coma para separar los decimales?

```
mi_var = 2,5
```

Python acepta la asignación, pero cuando investigamos la variable, el resultado es diferente de lo que esperábamos, no obtenemos un valor numérico:

```
mi_var
```

(2, 5)

Python ha entendido que estábamos creando una *tupla* de dos valores (2 y 5) (en python, una *tupla* es una colección ordenada e inmutable de elementos)

Para python las mayúsculas y minúsculas son diferentes: `mi_var` y `mi_Var` son variables diferentes.

En una variable python podemos almacenar también texto:

```
mi_var = "Esto es una frase 12345"
```

Python almacena esta *cadena alfanumérica* exactamente igual que antes hizo con los valores numéricos:

```
mi_var
```

```
'Esto es una frase 12345'
```

```
mi_var = 123.45
```

```
mi_var
```

123.45

```
mi_var*5
```

617.25

Para almacenar una *cadena alfanumérica* necesitamos encerrarla entre comillas, ya sean sencillas o dobles. De hecho, si almacenamos un número entre comillas, python no va a identificarlo como un número, sino como un texto:

```
mi_var = "123.45"
```

```
mi_var
```

```
'123.45'
```

```
mi_var*5
```

```
'123.45123.45123.45123.45123.45'
```

Como ahora `mi_var` es una cadena de letras, python interpreta `mi_var*5` como una multiplicación, sino como una instrucción para repetir esa cadena de caracteres cinco veces.

En el ejemplo que acabamos de hacer, seguro que te has dado cuenta de que python permite **reutilizar** los objetos simplemente reasignándoles el valor correspondiente. Al hacerlo, perdemos el valor original y lo sustituimos por el nuevo valor.

Hay que tener atención con las reglas de escritura: para asignar un texto a un objeto debemos tener cuidado de *cerrar* las comillas, si no lo hacemos, python nos advertirá del error:

```
mi_var = "Esto es una cadena alfanumerica que no hemos cerrado
```

### 5.5.1. Algunos ejercicios

Intenta detectar los errores en las siguientes celdas de código:

```
mi_variable = 10
```

```
mi_variable
```

```
mi_var = 5
```

```
Mi_Var
```

Podemos asignar un objeto a otro objeto con el operador de asignación:

```
x = 5
```

```
y = 3
```

```
x + y
```

```
8
```

```
y = x
```

```
x + y
```

```
10
```

Supongamos que tenemos 5 peras y 4 manzanas. Crea una variable que se llame *fruta* que sume el total de unidades de fruta que tenemos, insertando debajo

de esta celda todas las celdas de código que necesites (Pista: ¡En Python sí podemos sumar **peras** con **manzanas**!)

```
# Celda para el ejercicio de las frutas
peras = 5
manzanas = 4
fruta = peras + manzanas
print(fruta)
```

9

## 5.6. Los tipos de datos

Hemos visto que python puede trabajar con diferentes tipos de datos, tales como números y textos.

Los valores decimales como 4.5 se llaman **flotantes**(float). Los números enteros como 4 se llaman **enteros**(int). Los valores booleanos (VERDADERO o FALSO) se denominan **booleanos**(bool o logical). Los valores de texto (o cadena alfanumérica) se denominan **cadena de caracteres**. También se les llama simplemente **cadena**. Las comillas en el editor indican que “un texto entre comillas” es una *cadena de caracteres*. En python, una cadena de caracteres puede escribirse entre comillas dobles, como "pera" o simples, como 'pera'

Algunos ejemplos:

```
mi_var_numero = 4.5
mi_var_texto = "esto es un texto"
mi_var_texto_2 = 'esto también es un texto'
mi_var_logica = False
```

Las variables **lógicas** pueden tomar los valores **True** o **False** (recuerda que las mayúsculas son significativas en python: **False** no es lo mismo que **false** o **FALSE**). Python responde también con un valor lógico cuando hacemos una *evaluación lógica*. Por ejemplo,

```
mi_var_numero == 4.5
```

True

```
mi_var_texto == 4.5
```

False

Como en python se usa = como operador de asignación, el operador lógico que prueba una igualdad es ==, un doble igual, y no un =, un igual sencillo. La desigualdad se verifica con el operador != que significa “no es igual a”.

Veamos otros dos ejemplos que proporcionan una respuesta de valor lógico:

## 5.7. LAS FUNCIONES EN PYTHON: ¿QUÉ SON Y POR QUÉ LAS USAMOS?49

```
mi_var_numero == 10
```

False

```
mi_var_numero > 5
```

False

```
mi_var_numero != 5 # el operador != significa "no es igual a"
```

True

### 5.7. Las Funciones en Python: ¿Qué son y por qué las usamos?

Hasta ahora hemos usado operadores aritméticos para realizar cálculos básicos y hemos asignado valores a variables. Pero, ¿qué pasa si queremos realizar tareas más complejas o repetir una misma operación muchas veces? Aquí es donde entran las **funciones**.

Una función es un bloque de código organizado y reutilizable que se utiliza para realizar una única acción relacionada. Piensa en una función como una “receta” o un “mini-programa” que encapsula una serie de pasos. En lugar de escribir esos pasos una y otra vez, simplemente “llamamos” a la función por su nombre.

#### ¿Por qué son útiles las funciones?

- **Reutilización de Código:** Evitan tener que escribir el mismo código varias veces.
- **Modularidad:** Dividen el programa en partes más pequeñas y manejables.
- **Claridad:** Hacen el código más fácil de leer y entender.

Python ya trae consigo muchas **funciones integradas** (*built-in functions*) que podemos usar directamente. Veamos algunos ejemplos muy comunes:

#### 5.7.1. `print()`: Mostrar información

Esta función ya la hemos usado. Nos permite mostrar valores, texto o el contenido de variables en la consola

```
print("¡Hola, esto es una función!")
mi_nombre = "Juan"
print("Mi nombre es:", mi_nombre)
```

```
¡Hola, esto es una función!
Mi nombre es: Juan
```

### 5.7.2. `len()`: Obtener la longitud

Esta función nos devuelve el número de elementos de un objeto, como el número de caracteres en una cadena de texto o el número de elementos en una lista.

```
frase = "Programar en Python es divertido"
longitud_frase = len(frase)
print("La frase tiene", longitud_frase, "caracteres.")

mi_lista_numeros = [10, 20, 30, 40]
print("Mi lista tiene", len(mi_lista_numeros), "elementos.")
```

La frase tiene 32 caracteres.  
Mi lista tiene 4 elementos.

### 5.7.3. `type()`: Conocer el tipo de dato

Útil para verificar qué tipo de dato tiene una variable.

```
mi_numero = 100
mi_texto = "Cien"
mi_booleano = True

print(type(mi_numero))
print(type(mi_texto))
print(type(mi_booleano))
```

```
<class 'int'>
<class 'str'>
<class 'bool'>
```

### 5.7.4. ¿Cómo funcionan las funciones? Argumentos y Retorno

- **Argumentos:** Muchas funciones necesitan información para poder trabajar. Esta información se les pasa entre paréntesis, y se llaman **argumentos**. Por ejemplo, a `print()` le pasamos lo que queremos mostrar.
- **Retorno de Valores:** Algunas funciones realizan un cálculo o una operación y nos devuelven un resultado. Por ejemplo, `len()` nos devuelve un número.

Entender las funciones es clave, porque las librerías que usaremos a continuación son, en esencia, grandes colecciones de funciones especializadas.

## 5.8. Importando Herramientas: Bibliotecas y Módulos

Python es un lenguaje muy potente, pero su verdadera fuerza reside en su vasto ecosistema de **bibliotecas** (también llamadas módulos o paquetes).

Una **biblioteca** es una colección de código (funciones, clases, etc.) escrito por otros programadores y disponible para que tú lo uses en tus propios proyectos. Imagina que son “cajas de herramientas” especializadas:

- ¿Necesitas hacer cálculos matemáticos complejos? Hay una librería para eso.
- ¿Necesitas manipular datos en tablas? Hay una librería para eso.
- ¿Necesitas crear gráficos? ¡También hay librerías para eso!

Usar librerías nos permite evitar “reinventar la rueda”, ahorrando mucho tiempo y esfuerzo, y aprovechando código que ya ha sido probado y optimizado.

Para poder usar las funciones y herramientas que están dentro de una librería, primero debemos **importarla** a nuestro entorno de trabajo. La forma más común de hacerlo es con la instrucción `import`.

### 5.8.1. Sintaxis de `import`

La sintaxis básica es:

```
import nombre_de_la_libreria
```

Sin embargo, para las librerías de análisis de datos, es muy común y recomendado usar un **alias** (un nombre corto) para hacer el código más conciso y fácil de leer. Esto se hace con la palabra clave `as`:

```
import nombre_de_la_libreria as alias
```

Ahora vamos a importar las librerías que serán esenciales para trabajar con datos, realizar cálculos estadísticos y crear gráficos en Python.

```
# Importando las librerías fundamentales para el análisis de datos

# Pandas: Es la librería principal para trabajar
#         con estructuras de datos tabulares
#         como Series (vectores) y DataFrames (tablas).
import pandas as pd

# NumPy: Proporciona soporte para arreglos (arrays)
#        y matrices multidimensionales,
#        además de funciones matemáticas de alto nivel.
#        Pandas se construye sobre NumPy.
import numpy as np
```

```
# Matplotlib.pyplot: Es la librería base
#                               para crear gráficos en Python.
#                               Seaborn la usa "por debajo" para dibujar.
import matplotlib.pyplot as plt

# Seaborn: Es una librería de visualización de datos de alto nivel
#          basada en Matplotlib. Facilita la creación
#          de gráficos estadísticos atractivos.
import seaborn as sns

print("¡Librerías principales para el análisis de datos importadas correctamente!")
print("Ahora podemos acceder a sus funciones usando sus alias (pd, np, plt, sns).")
```

```
¡Librerías principales para el análisis de datos importadas correctamente!
Ahora podemos acceder a sus funciones usando sus alias (pd, np, plt, sns).
```

Una vez que hemos importado una librería con un alias (por ejemplo, `pandas` as `pd`), para usar una función de esa librería, escribimos el alias seguido de un punto y el nombre de la función: `pd.nombre_de_la_funcion()`.

## 5.9. Las principales estructuras de datos en Python para el análisis

Ahora que entendemos las funciones y cómo importar librerías, podemos introducir las estructuras de datos que nos permitirán organizar y manipular nuestros datos de manera eficiente. Las dos más importantes en el contexto del análisis de datos con Python (y con la librería `Pandas`) son las **Series** y los **DataFrames**.

### 5.9.1. Series de `pandas` (El “vector” en Python para datos)

Si en otros lenguajes de programación o herramientas estadísticas has trabajado con el concepto de “vector” (una secuencia de valores de un mismo tipo), el equivalente más directo en Python, usando `pandas`, es una *serie*.

- **Concepto:** Una *serie* de `pandas` es un arreglo unidimensional (como una columna de una hoja de cálculo o un vector). Puede contener cualquier tipo de dato (números enteros, flotantes, texto, booleanos, etc.) y tiene un índice asociado para acceder a sus elementos.

**Creación de una serie:** Podemos crear una Serie a partir de una lista de Python.

```
# Crear una lista de edades
lista_edades = [25, 30, 22, 35, 28, 40, 33, 29]

# Convertir la lista a una Serie de Pandas
```



```
# Usamos 'pd.' porque la función Series() viene
# de la librería Pandas.
# Atención: la función se escribe con mayúsculas.
edades_serie = pd.Series(lista_edades)

print("Mi Serie de Edades:")
print(edades_serie)
```

Mi Serie de Edades:

```
0    25
1    30
2    22
3    35
4    28
5    40
6    33
7    29
```

dtype: int64

**Operaciones básicas con series:** Las series tienen muchas funciones y métodos útiles. Por ejemplo, podemos calcular la media:

```
# Usando el método .mean() de la Serie
print("\nEdad promedio:", edades_serie.mean())
print("Edad máxima:", edades_serie.max())
print("Edad mínima:", edades_serie.min())
```

Edad promedio: 30.25

Edad máxima: 40

Edad mínima: 22

### 5.9.2. DataFrames de Pandas (La “Tabla” o “Hoja de Cálculo”)

El **DataFrame** es la estructura de datos más importante y utilizada en **pandas** para el análisis de datos. Es el equivalente a una tabla en una base de datos, una hoja de cálculo de Excel, o un “data frame” en R.

- **Concepto:** Un DataFrame es una estructura bidimensional (filas y columnas). Piensa en ella como una colección de series (columnas) que comparten el mismo índice (las filas). Cada columna en un DataFrame es una serie de **pandas**.

**Creación de un DataFrame (a partir de un diccionario):** Una forma sencilla de crear un DataFrame pequeño es a partir de un diccionario de **Python**, donde las claves del diccionario se convierten en los nombres de las columnas y los valores son listas que se convierten en las columnas.

```
# Crear un diccionario con datos
datos_alumnos = {
    'Nombre': ['Ana', 'Luis', 'Marta', 'Pedro', 'Sofía'],
    'Edad': [24, 30, 28, 35, 26],
    'Ciudad': ['Madrid', 'Barcelona', 'Valencia',
               'Sevilla', 'Málaga'],
    'Calificación': [8.5, 7.2, 9.1, 6.8, 8.9]
}

# Crear un DataFrame a partir del diccionario
df_alumnos = pd.DataFrame(datos_alumnos)

print("\nDataFrame de Alumnos:")
print(df_alumnos)
```

DataFrame de Alumnos:

|   | Nombre | Edad | Ciudad    | Calificación |
|---|--------|------|-----------|--------------|
| 0 | Ana    | 24   | Madrid    | 8.5          |
| 1 | Luis   | 30   | Barcelona | 7.2          |
| 2 | Marta  | 28   | Valencia  | 9.1          |
| 3 | Pedro  | 35   | Sevilla   | 6.8          |
| 4 | Sofía  | 26   | Málaga    | 8.9          |

### Visualización y exploración básica de DataFrames:

Es fundamental poder echar un vistazo rápido a los datos una vez que están cargados en un DataFrame.

- `df.head()`: Muestra las primeras 5 filas (útil para ver la estructura).
- `df.tail()`: Muestra las últimas 5 filas.
- `df.info()`: Proporciona un resumen conciso del DataFrame, incluyendo el número de entradas, columnas, tipos de datos no nulos y uso de memoria.
- `df.describe()`: Genera estadísticas descriptivas (conteo, media, desviación estándar, etc.) de las columnas numéricas.

```
print("\nPrimeras 3 filas del DataFrame de alumnos:")
print(df_alumnos.head(3)) # Podemos especificar número de filas

print("\nInformación del DataFrame de alumnos:")
df_alumnos.info()

print("\nEstadísticas descriptivas del DataFrame de alumnos:")
print(df_alumnos.describe())
```

Primeras 3 filas del DataFrame de alumnos:

|   | Nombre | Edad | Ciudad    | Calificación |
|---|--------|------|-----------|--------------|
| 0 | Ana    | 24   | Madrid    | 8.5          |
| 1 | Luis   | 30   | Barcelona | 7.2          |

## 5.9. LAS PRINCIPALES ESTRUCTURAS DE DATOS EN PYTHON PARA EL ANÁLISIS

```
2  Marta      28  Valencia      9.1
```

Información del DataFrame de alumnos:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 5 entries, 0 to 4
```

```
Data columns (total 4 columns):
```

| # | Column       | Non-Null Count | Dtype   |
|---|--------------|----------------|---------|
| 0 | Nombre       | 5 non-null     | object  |
| 1 | Edad         | 5 non-null     | int64   |
| 2 | Ciudad       | 5 non-null     | object  |
| 3 | Calificación | 5 non-null     | float64 |

```
dtypes: float64(1), int64(1), object(2)
```

```
memory usage: 292.0+ bytes
```

Estadísticas descriptivas del DataFrame de alumnos:

|       | Edad      | Calificación |
|-------|-----------|--------------|
| count | 5.000000  | 5.000000     |
| mean  | 28.600000 | 8.100000     |
| std   | 4.219005  | 1.036822     |
| min   | 24.000000 | 6.800000     |
| 25%   | 26.000000 | 7.200000     |
| 50%   | 28.000000 | 8.500000     |
| 75%   | 30.000000 | 8.900000     |
| max   | 35.000000 | 9.100000     |

### Selección de Columnas en un DataFrame:

Para trabajar con una columna específica de un DataFrame, la seleccionamos usando corchetes [] y el nombre de la columna entre comillas. El resultado será una Serie de Pandas.

```
# Seleccionar la columna 'Edad'
edades_alumnos = df_alumnos['Edad']
print("\nColumna 'Edad' (como Serie):")
print(edades_alumnos)

# Calcular la media de la columna 'Calificación'
media_calificacion = df_alumnos['Calificación'].mean()
print(f"\nLa calificación promedio de los alumnos es: {media_calificacion:.2f}")
```

Columna 'Edad' (como Serie):

```
0    24
1    30
2    28
3    35
4    26
```

Name: Edad, dtype: int64

La calificación promedio de los alumnos es: 8.10

## 5.10. El *data frame* (continuación con ejemplo real)

Ahora que conocemos los `DataFrames`, podemos entender mejor cómo leer datos reales. El marco de datos o *DataFrame* es el objeto más útil y más usado en el análisis de datos. Consiste en una estructura de dos dimensiones, formada por una serie de vectores de igual longitud, igual que una tabla de una hoja de cálculo, en la que cada columna es una variable y cada fila, un caso, observación o individuo. Algunas bibliotecas Python incluyen *data frames* de muestra, que son útiles para entender cómo están formados.

Veamos uno de ellos, el famoso conjunto de datos **iris**, creada por el investigador y estadístico Ronald Fisher, que contiene un conjunto de medidas realizadas sobre flores del género *Iris* realizadas por este investigador en los años 30 del siglo XX.

Para cargar este conjunto de datos de ejemplo, vamos a utilizar una función de la librería `seaborn` que ya importamos:

```
# Cargamos el dataset 'iris' usando una función de Seaborn
df_iris_sns = sns.load_dataset('iris')

print("Primeras 5 filas del dataset Iris:")
print(df_iris_sns.head())
```

Primeras 5 filas del dataset Iris:

|   | sepal_length | sepal_width | petal_length | petal_width | species |
|---|--------------|-------------|--------------|-------------|---------|
| 0 | 5.1          | 3.5         | 1.4          | 0.2         | setosa  |
| 1 | 4.9          | 3.0         | 1.4          | 0.2         | setosa  |
| 2 | 4.7          | 3.2         | 1.3          | 0.2         | setosa  |
| 3 | 4.6          | 3.1         | 1.5          | 0.2         | setosa  |
| 4 | 5.0          | 3.6         | 1.4          | 0.2         | setosa  |

La función `head()` es útil para presentarnos sólo el *encabezado* del *data frame*. Veamos la estructura de este *data frame* más a fondo:

```
print("\nInformación general del DataFrame Iris:")
df_iris_sns.info()
```

Información general del DataFrame Iris:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
#   ...          ...
#   sepal_length    150 non-null    float64
#   sepal_width     150 non-null    float64
#   petal_length    150 non-null    float64
#   petal_width     150 non-null    float64
#   species         150 non-null    object
dtypes: float64(4), object(1)
memory usage: 11.6 KB
```

```

-----
0  sepal_length  150 non-null  float64
1  sepal_width  150 non-null  float64
2  petal_length  150 non-null  float64
3  petal_width  150 non-null  float64
4  species      150 non-null  object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

```

```

print(
    "\nEstadísticas descriptivas "
    "de las columnas numéricas del DataFrame Iris:"
)
print(df_iris_sns.describe())

```

Estadísticas descriptivas de las columnas numéricas del DataFrame Iris:

|       | sepal_length | sepal_width | petal_length | petal_width |
|-------|--------------|-------------|--------------|-------------|
| count | 150.000000   | 150.000000  | 150.000000   | 150.000000  |
| mean  | 5.843333     | 3.057333    | 3.758000     | 1.199333    |
| std   | 0.828066     | 0.435866    | 1.765298     | 0.762238    |
| min   | 4.300000     | 2.000000    | 1.000000     | 0.100000    |
| 25%   | 5.100000     | 2.800000    | 1.600000     | 0.300000    |
| 50%   | 5.800000     | 3.000000    | 4.350000     | 1.300000    |
| 75%   | 6.400000     | 3.300000    | 5.100000     | 1.800000    |
| max   | 7.900000     | 4.400000    | 6.900000     | 2.500000    |

Para utilizar una columna de un dataframe, la seleccionamos por su nombre:

```

print("\nContenido de la columna 'sepal_length':")

# podemos usar también la doble comilla
# para designar las columnas
print(df_iris_sns["sepal_length"])

```

Contenido de la columna 'sepal\_length':

```

0      5.1
1      4.9
2      4.7
3      4.6
4      5.0
...
145    6.7
146    6.3
147    6.5
148    6.2
149    5.9
Name: sepal_length, Length: 150, dtype: float64

```

Ahora que tenemos una columna seleccionada (que es una serie de `pandas`), podemos aplicar directamente funciones estadísticas sobre ella:

```
# Calculamos la media de la columna 'sepal_length'
media_sepal_length = df_iris_sns["sepal_length"].mean()

# esta es una forma de dar formato al número
# para imprimirlo con dos decimales fijos
print(f"\nLa longitud media del sépalos es: {media_sepal_length:.2f}")
```

La longitud media del sépalos es: 5.84

```
# También podemos calcular la desviación estándar
desviacion_sepal_length = df_iris_sns["sepal_length"].std()
print(f"La desviación estándar "
      f"de la longitud del sépalos es: {desviacion_sepal_length:.2f}"
      )
```

La desviación estándar de la longitud del sépalos es: 0.83

Con nuestros DataFrames cargados y la comprensión de las Series, podemos realizar rápidamente cálculos estadísticos básicos sobre nuestras columnas numéricas. `Pandas` hace que esto sea muy sencillo.

#### ■ Estadísticas Descriptivas Comunes:

- `.mean()`: Media
- `.median()`: Mediana
- `.std()`: Desviación estándar
- `.min()`: Valor mínimo
- `.max()`: Valor máximo
- `.sum()`: Suma de todos los valores
- `.count()`: Número de valores no nulos

Vamos a aplicar algunas de estas operaciones al DataFrame `df_iris_sns`:

```
print("Estadísticas de 'petal_width' en el DataFrame Iris:")
print(f"Media: {df_iris_sns['petal_width'].mean():.2f}")
print(f"Mediana: {df_iris_sns['petal_width'].median():.2f}")
print(f"Desviación estándar: {df_iris_sns['petal_width'].std():.2f}")
print(f"Valor mínimo: {df_iris_sns['petal_width'].min():.2f}")
print(f"Valor máximo: {df_iris_sns['petal_width'].max():.2f}")
print(f"Número de observaciones: {df_iris_sns['petal_width'].count()}")
```

Estadísticas de 'petal\_width' en el DataFrame Iris:

Media: 1.20

Mediana: 1.30

Desviación estándar: 0.76

Valor mínimo: 0.10

Valor máximo: 2.50

Número de observaciones: 150

## 5.11. Visualización de Datos con Seaborn y Matplotlib

Una imagen vale más que mil palabras, y en el análisis de datos, la visualización es fundamental para entender patrones, distribuciones y relaciones en nuestros datos. En Python, las librerías **Matplotlib** y **Seaborn** son las herramientas estándar para crear gráficos.

- **Matplotlib.pyplot (alias plt):** Es la base, una librería de bajo nivel que da un control muy granular sobre cada aspecto del gráfico.
- **Seaborn (alias sns):** Construye sobre Matplotlib, ofreciendo una interfaz de alto nivel para crear gráficos estadísticos complejos y estéticamente agradables con menos código. Es ideal para explorar distribuciones, relaciones entre variables y comparaciones entre grupos.

Siempre importamos ambas, ya que Seaborn a menudo usa funciones de Matplotlib para mostrar y personalizar los gráficos (como `plt.title()` para el título o `plt.show()` para mostrar el gráfico).

El texto siguiente incluye el código para crear el gráfico, no te preocupes si no lo entiendes a la primera, iremos explicándolo línea a línea.

### 5.11.1. Ejemplo: Histograma de la longitud del sépalo (Distribución de una variable numérica)

Un histograma nos permite ver cómo se distribuyen los valores de una variable numérica.

```
import matplotlib.pyplot as plt
import seaborn as sns

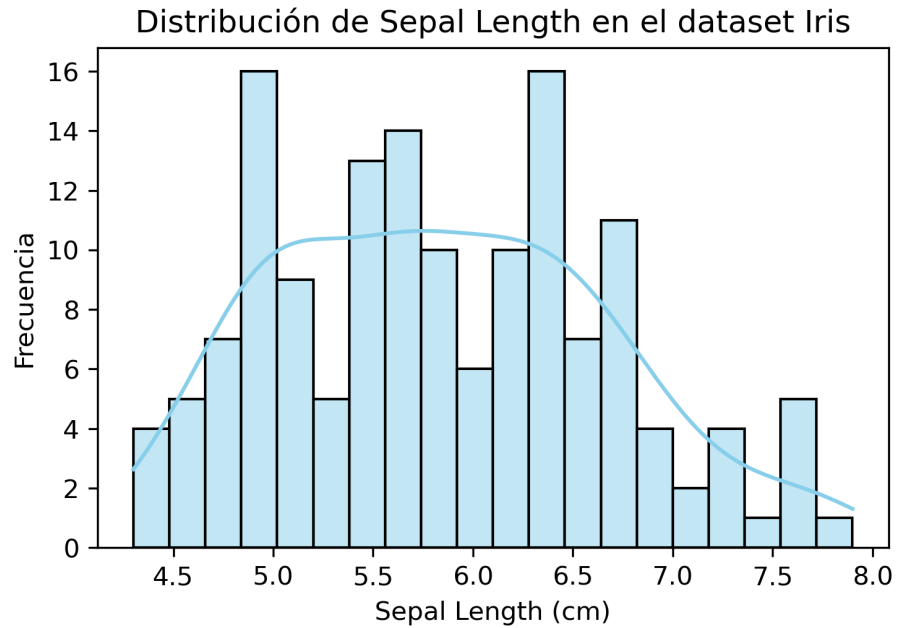
# Cargar el dataset iris
iris = sns.load_dataset("iris")

# Crear el histograma
sns.histplot(data=iris,
              x="sepal_length",
              bins=20,
              kde=True,
              color="skyblue",
              edgecolor="black")

# Añadir título y etiquetas
plt.title("Distribución de Sepal Length en el dataset Iris")
```

```
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Frecuencia")

# Mostrar el gráfico
plt.show()
```



### Explicación línea por línea

1. **import seaborn as sns** - Importa la biblioteca Seaborn con el alias **sns**. - Seaborn es una capa de alto nivel sobre Matplotlib, ideal para visualizaciones estadísticas.
2. **import matplotlib.pyplot as plt** - Importa Matplotlib para controlar aspectos del gráfico como título, ejes y visualización.
3. **iris = sns.load\_dataset("iris")** - Carga el conjunto de datos **iris** directamente desde Seaborn. - Este dataset contiene medidas de sépalos y pétalos de tres especies de iris.
4. **sns.histplot(...)** Este es el núcleo del histograma. Vamos a desglosar cada argumento:



| Parámetro                      | Descripción                                                                                                  |
|--------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>data=iris</code>         | Indica que usaremos el DataFrame <code>iris</code> como fuente de datos.                                     |
| <code>x="sepal_length"</code>  | Especifica la variable que queremos graficar en el eje X.                                                    |
| <code>bins=20</code>           | Divide el rango de valores en 20 intervalos (barras). Puedes ajustar este número para más o menos detalle.   |
| <code>kde=True</code>          | Añade una curva de densidad suavizada sobre el histograma. Útil para visualizar la forma de la distribución. |
| <code>color="skyblue"</code>   | Cambia el color de las barras del histograma. Puedes usar nombres de colores o códigos hexadecimales.        |
| <code>edgecolor="black"</code> | Añade bordes negros a cada barra para mejorar la legibilidad.                                                |

5. `plt.title(...)` - Añade un título descriptivo al gráfico.

6. `plt.xlabel(...)` y `plt.ylabel(...)` - Etiquetan los ejes X e Y respectivamente, lo que es esencial para la interpretación del gráfico.

7. `plt.show()` - Muestra el gráfico en pantalla. Sin esta línea, el gráfico no se renderiza en algunos entornos.

En el capítulo siguiente veremos con detalle distintos tipos de gráficos y su construcción en Excel y en Python.

## 5.12. Cómo leer nuestros datos en Python desde Google Colaborate

Hasta ahora hemos utilizado datos que hemos creado sobre la marcha, o hemos usado un conjunto de datos preexistente, como es el caso de los datos *Iris*.

Ahora vamos a ver cómo podemos leer o utilizar nuestros propios datos, a partir de ficheros CSV. También podemos leer directamente nuestras hojas Excel.

### 5.12.1. Opción 1: Subir los datos al espacio de trabajo de Google Colaborate

Los pasos son los mismos tanto si queremos trabajar con un CSV o directamente con una hoja Excel: 1. Dentro del entorno Colab, seleccionamos el icono de carpeta a la izquierda, que nos abre la barra lateral de **Archivos** 2. Hacemos click sobre el icono de la hoja con la flecha vertical, lo que nos abre una ventana de selección de archivos. 3. Seleccionamos la hoja de cálculo o CSV con la que vamos a trabajar y la subimos al espacio de trabajo de Google Colaborate

Una vez la hoja de cálculo o el CSV en nuestro espacio de trabajo, procedemos a leer los datos.

El mayor inconveniente de esta forma de trabajo es que cada vez que salimos de la sesión, Google Colab borra todos nuestros archivos del espacio de trabajo; cada vez que iniciemos una sesión, tendremos que repetir el proceso de subir nuestros datos al espacio de trabajo. Por eso, la mejor opción es tener nuestros datos en una carpeta de Google Drive, que puede utilizarse desde Colab tal como vamos a ver a continuación.

### 5.12.2. Opción 2: Leer directamente los datos de nuestra carpeta de Google Drive

Esta es una opción mucho más cómoda que nos evita los pasos intermedios, ya que no necesitamos subir la hoja de cálculo al espacio de trabajo.

Empezamos, como siempre, cargando las bibliotecas que nos proporcionan las funciones que vamos a utilizar durante el ejercicio. Aunque algunas no se usarán en la lectura de datos, en buenas prácticas es conveniente agrupar al principio del cuaderno todas las instrucciones de carga de bibliotecas en una misma celda.

```
import os

# manejo de dataframes y series
import pandas as pd

# funciones gráficas específicas de `matplotlib.pyplot`
import matplotlib.pyplot as plt

import seaborn as sns

# estilo de `seaborn` con fondo blanco
sns.set_style("whitegrid")
```

A continuación, la instrucciones para acceder a nuestro Google Drive y leer el archivo de datos.

```
# 1. Ejecuta esta celda para montar tu Google Drive.
from google.colab import drive
```

```

drive.mount('/content/drive')

# 2. Verifica y ajusta esta ruta de archivo si es necesario:
google_drive_path_folder = '/content/drive/MyDrive/'
nombre_archivo_base = 'camembert.csv'
nombre_archivo_completo = os.path.join(google_drive_path_folder,
                                         nombre_archivo_base)

try:
    df = pd.read_csv(nombre_archivo_completo,
                     decimal = ",",
                     sep=';',
                     encoding='ISO-8859-1')
    print("¡Archivo ", nombre_archivo_base, " cargado correctamente!")
    print(f"Dimensiones de los datos: {df.shape}")
except Exception as e:
    print(f"Error al cargar el archivo: {e}")

```

### 5.12.3. Explicación de los pasos:

#### 1. Montar Google Drive

| Línea                                      | Explicación                                                                                           |
|--------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>from google.colab</code>             | Importa el módulo <code>drive</code> de Google Colab para                                             |
| <code>import drive</code>                  | acceder a Google Drive.                                                                               |
| <code>drive.mount('/content/drive')</code> | Monta tu Google Drive en la ruta <code>/content/drive</code> . Te pedirá autorización la primera vez. |

#### 2. Definir ruta del archivo CSV

| Línea                                   | Explicación                                                                                                       |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>google_drive_path_folder =</code> | Ruta donde está el archivo dentro de Google Drive.                                                                |
| <code>'/content/drive/MyDrive/'</code>  |                                                                                                                   |
| <code>nombre_archivo_base =</code>      | Nombre del archivo CSV que se quiere cargar.                                                                      |
| <code>'camembert.csv'</code>            |                                                                                                                   |
| <code>os.path.join(...)</code>          | Une la ruta y el nombre del archivo para formar la ruta completa. Evita errores en distintos sistemas operativos. |

### 3. Cargar el archivo CSV con pandas

| Línea                               | Explicación                                                                                         |
|-------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>try:</code>                   | Inicia un bloque que intenta ejecutar el código. Si hay error, pasa al bloque <code>except</code> . |
| <code>pd.read_csv(...)</code>       | Carga el archivo CSV como un DataFrame de pandas. Parámetros:                                       |
| <code>decimal=","</code>            | Indica que los decimales usan coma (formato europeo).                                               |
| <code>sep=";"</code>                | Indica que los campos están separados por punto y coma.                                             |
| <code>encoding="ISO-8859-1"</code>  | Codificación para leer caracteres especiales como tildes.                                           |
| <code>print(...)</code>             | Muestra mensaje de éxito si el archivo se carga correctamente.                                      |
| <code>df.shape</code>               | Devuelve número de filas y columnas del DataFrame.                                                  |
| <code>except Exception as e:</code> | Captura cualquier error que ocurra al cargar el archivo.                                            |
| <code>print(f"Error...")</code>     | Muestra el mensaje de error en pantalla.                                                            |

En Python siempre se prefiere encapsular la instrucción en una estructura `try...except`, que maneja los posibles errores o **excepciones** que pueden producirse en la lectura, por ejemplo, que nuestro *path* sea incorrecto, o que hayamos escrito mal el nombre del fichero, o cualquier otro error que haga que la instrucción de lectura falle. Añadimos también algunas instrucciones de `print()` que nos ayuden a saber que todo ha ido bien (o mal).

Una vez cargados los datos, podemos revisarlos y ver la estructura del DataFrame usando las funciones adecuadas: `df.dtypes`, `df.head()`, `df.info()`, o `df.describe()`

## 5.13. Resumen

En este capítulo hemos tomado contacto con las funciones básicas de Python y su utilización a través de un interface Jupyter, en este caso **Google Colaborate**, y hemos visto algunas funciones gráficas básicas.

Hemos visto los principales tipos de datos, particularmente el *data frame*, y hemos aprendido a leer datos que previamente se habían introducido en una hoja de cálculo.

Hemos aprendido a leer nuestros datos partiendo de ficheros CSV exportados desde Excel o desde otros programas.

En el capítulo siguiente veremos con más detalle algunos gráficos que nos permitirán describir y entender mejor nuestros conjuntos de datos.

A modo de ejemplo, el siguiente enlace muestra varios ejemplos de cuadernos Jupyter de todos los niveles de complejidad, desde introducción a su uso hasta aplicaciones complejas. Explorarlos ayudará a comprender mejor la interfaz.

Ver ejemplos de cuadernos Jupyter



## Capítulo 6

# La exploración de los datos mediante gráficos.

Una imagen vale más que mil palabras, y en el análisis de datos, la visualización es fundamental para entender patrones, distribuciones y relaciones en nuestros datos.

En este capítulo estudiaremos cómo describir conjuntos de datos de forma visual, utilizando varios tipos de gráficos distintos:

- *Stemplot*
- Histograma
- Diagrama de caja o *boxplot*
- Gráfico de dispersión
- Gráfico de series temporales

Veremos la relación visual entre un histograma y un diagrama de caja, y aprenderemos también a construir tablas de frecuencias en Excel y en **Python**. Finalmente, veremos algunos otros tipos de gráficos que son útiles para aplicaciones concretas, como los gráficos de densidad.

### 6.1. Explorando los datos

#### 6.1.1. El diagrama de tallo y hojas (*stem and leaf plot* o *stemplot*)

El diagrama de tallo y hojas, también conocido como *stemplot*, es una herramienta gráfica utilizada en estadística para representar la distribución de un conjunto de datos. Es especialmente útil para conjuntos de datos pequeños y proporciona una forma rápida y efectiva de visualizar la forma de los datos y

su dispersión. El *stemplot* recibe este nombre porque el dibujo que resulta se asemeja a un tallo el que le salen las hojas que son los datos individuales.

Los componentes de un *stemplot* son:

- **Tallo:** Representa el grupo principal de los valores de los datos. Generalmente, se usa la parte más significativa del número. Por ejemplo, en el número 43, el tallo podría ser 4.
- **Hojas:** Representan los dígitos finales o menos significativos de los valores de los datos. Siguiendo el ejemplo anterior, la hoja sería 3.

### Construcción del diagrama en la hoja de cálculo

Supongamos que queremos medir la altura de un grupo de alumnos de nuestra clase. Éste es nuestro grupo:



Realizamos la medida de altura de cada persona y registramos los valores en una hoja de cálculo, siguiendo las buenas prácticas que hemos visto al estudiar los *datos ordenados*.



|    | A       | B         | C | D |
|----|---------|-----------|---|---|
| 1  | nombre  | altura_cm |   |   |
| 2  | Luis    | 153       |   |   |
| 3  | Ana     | 135       |   |   |
| 4  | Iván    | 140       |   |   |
| 5  | Lucía   | 140       |   |   |
| 6  | Jessica | 175       |   |   |
| 7  | Antonio | 138       |   |   |
| 8  | Mikel   | 145       |   |   |
| 9  | Marta   | 154       |   |   |
| 10 | Carmen  | 152       |   |   |
| 11 | Javier  | 159       |   |   |
| 12 | María   | 154       |   |   |
| 13 |         |           |   |   |
| 14 |         |           |   |   |
| 15 |         |           |   |   |

Vamos a utilizar los datos de medidas de altura de nuestro grupo de alumnos. Quitamos el último dígito a la derecha de nuestros valores y colocamos verticalmente los valores resultantes ordenándolos de menor a mayor, y evitando las repeticiones. Para evitar errores en la escala, debemos incluir los valores intermedios aunque no haya ninguno en nuestros datos (en el ejemplo, el valor 16 que correspondería a los 160). Esto forma el “tallo” de nuestro diagrama:

| Stem plot |
|-----------|
| 13        |
| 14        |
| 15        |
| 16        |
| 17        |

A continuación añadimos las “hojas” en la celda a la derecha, que consisten en los valores que hemos “cortado” de nuestro árbol, uno al lado de otro, incluyendo esta vez los valores repetidos, en orden de menor a mayor. Por ejemplo, para el valor 135, descartamos 13 y utilizamos 5; para el valor 138, descartamos 13 y utilizamos 8, y así sucesivamente para todos los valores.

| Stem plot |       |
|-----------|-------|
| 13        | 58    |
| 14        | 005   |
| 15        | 23449 |
| 16        |       |
| 17        | 5     |

### 6.1.2. Interpretación del diagrama de tallo y hojas

- **Tallo:** Los números a la izquierda del símbolo | representan los valores base (o tallos), en este caso, las decenas de las alturas.
- **Hojas:** Los números a la derecha del símbolo | representan los dígitos adicionales (o hojas). Por ejemplo, en la línea 13 | 58, el tallo es 13 (130), y las hojas son 5 y 8, que corresponden a los datos 135 y 138.

El diagrama nos dice que los valores en torno a 150 cm son los más frecuentes, y que hay un valor alto (175) que se separa un poco del resto.

### 6.1.3. Resumen

El *stemplot* es muy sencillo de hacer y nos da una visión rápida y compacta de la *distribución* de nuestros valores, así como de la posible existencia de valores que se separan del conjunto. Estos valores alejados, que se conocen en inglés como *outliers*, tienen mucha importancia en el análisis e interpretación de los datos, como veremos más adelante.

La ventaja principal del *stemplot* es que mantiene los valores originales de las observaciones, y puede hacerse fácilmente con bolígrafo y papel, sin necesidad de más herramientas.

Su principal inconveniente es la elaboración manual (aunque lenguajes como R tienen funciones que lo contruyen de forma automática), y por lo tanto, la dificultad de aplicarlo a volúmenes de datos medios o grandes. El uso generalizado de los ordenadores ha hecho que actualmente esta herramienta tenga muy poco uso, y se utilicen en su lugar otras más gráficas y de construcción automática, como el **histograma**.

## 6.2. Distribuciones de frecuencias

Una **distribución de frecuencias** es una tabla que muestra la frecuencia con la que ocurren los valores diferentes en un conjunto de datos. Esta herramienta es fundamental en la estadística descriptiva y permite resumir y visualizar cómo se

distribuyen los datos de manera clara y comprensible. A partir de una tabla de frecuencias se pueden construir diagramas de barra o histogramas para visualizar la tabla de forma gráfica.

Para construir una distribución de frecuencias, agrupamos nuestros valores por intervalos, y contamos el número de observaciones que aparecen en cada intervalo. Los componentes de una distribución de frecuencias son:

- las **categorías o clases** son los intervalos o valores específicos de los datos que se están analizando. Cada categoría representa un rango de valores en caso de datos continuos, o valores específicos en caso de datos discretos.
- la **frecuencia absoluta** es un *recuento simple* de cuántas veces aparece cada valor en un conjunto de datos.
- la **frecuencia relativa** nos muestra la *proporción* de cada valor frente al total. Puede expresarse como fracción (entre 0 y 1) o como porcentaje (respecto a 100), y se calcula como:

$$\text{Frecuencia Relativa} = \frac{\text{Frecuencia Absoluta}}{\text{Número Total de Observaciones}}$$

- la **frecuencia acumulada** nos dice cuántas observaciones están por debajo de un cierto valor.
- la **frecuencia relativa acumulada** es la proporción de valores que están por debajo de un cierto valor

### 6.2.1. Construcción en Excel

La tabla a continuación muestra una distribución de frecuencias de las alturas de nuestro grupo de alumnos, calculada mediante una tabla dinámica de Excel.

## 6.3. Instrucciones paso a paso en Excel

Para crear una tabla de frecuencias de la variable `altura_cm` mediante tablas dinámicas en Excel, sigue estos pasos:

### 1. Selecciona los Datos

1. Abre tu archivo de Excel y selecciona toda la tabla que incluye los encabezados (`nombre` y `altura_cm`).

### 2. Inserta una tabla dinámica

1. Ve a la pestaña **Insertar** en la barra de herramientas de Excel.
2. Haz clic en **Tabla Dinámica**.
3. En el cuadro de diálogo que aparece, asegúrate de que el rango de datos seleccionado es correcto y elige dónde deseas colocar la tabla dinámica (en una nueva hoja de cálculo o en la hoja actual).

### 3. Añade la frecuencia absoluta

1. En el panel de campos de la tabla dinámica, arrastra el campo `altura_cm` a la sección **Filas**.
2. Arrastra nuevamente el campo `altura_cm` a la sección **Valores**.

#### 4. Ajusta la configuración de valores

1. Haz clic en el campo `altura_cm` en la sección **Valores**.
2. Selecciona **Configuración de campo de valor**.
3. En el cuadro de diálogo que aparece, asegúrate de que esté seleccionada la opción **Recuento**.
4. Acepta todo hasta volver a Excel.

#### 5. Añade la frecuencia relativa

1. En el panel de campos de la tabla dinámica, arrastra de nuevo el campo `altura_cm` a la sección **Valores**. Ahora la variable aparecerá como `altura_cm2`.

#### 6. Ajusta de nuevo la configuración de valores

1. Haz clic en el campo `altura_cm2` en la sección **Valores**.
2. Selecciona **Configuración de campo de valor**.
3. En el cuadro de diálogo que aparece, asegúrate de que esté seleccionada la opción **Recuento**.
4. En ese mismo cuadro, haz clic en el botón **Formato de número**, selecciona **Número y 2 decimales**, y acepta.
5. En ese mismo cuadro, selecciona la pestaña que dice **Mostrar valores como**.
6. En el menú desplegable, escoge la opción **% del total de columnas**.
7. Acepta todo hasta volver a Excel.

#### 7. Ordena y formatea

1. Puedes ordenar las alturas en orden ascendente o descendente haciendo clic en la flecha junto a `altura_cm` en la tabla dinámica.
2. También puedes cambiar el formato de la tabla dinámica para que sea más fácil de leer.
3. Puedes renombrar los encabezados de la tabla para que sea más fácil de leer, rotulando las columnas, por ejemplo, como `frec_absy` `frec_rel`, o cualquier otro encabezado que te resulte claro y útil.

|    | A                    | B         | C             | D |
|----|----------------------|-----------|---------------|---|
| 1  |                      |           |               |   |
| 2  |                      |           |               |   |
| 3  | Etiquetas de fila ▾  | frec abs  | frec rel      |   |
| 4  | 135-144              | 4         | 36,4%         |   |
| 5  | 145-154              | 5         | 45,5%         |   |
| 6  | 155-164              | 1         | 9,1%          |   |
| 7  | 165-175              | 1         | 9,1%          |   |
| 8  | <b>Total general</b> | <b>11</b> | <b>100,0%</b> |   |
| 9  |                      |           |               |   |
| 10 |                      |           |               |   |
| 11 |                      |           |               |   |

### 6.3.1. Construcción de una tabla de frecuencias en Python

```
import pandas as pd

altura_cm = [153,135,140,140,175,138,145,154,152,159,154]
frecuencias = pd.Series(altura_cm).value_counts()
print(frecuencias)
```

```
140    2
154    2
153    1
135    1
175    1
138    1
145    1
152    1
159    1
Name: count, dtype: int64
```

Con un poco más de código podemos hacer la tabla agrupando los valores en clases de amplitud 10, con las frecuencias absolutas y relativas. Es un poco más complicado, tómate tu tiempo para entender cada paso de las instrucciones.

```
import pandas as pd
import numpy as np

altura_cm = [153,135,140,140,175,138,145,154,152,159,154]
```

```

serie = pd.Series(altura_cm)

# Crear intervalos automáticamente con amplitud 10
bins = np.arange(min(altura_cm)//10*10,
                  max(altura_cm)//10*10 + 20, 10)

# Agrupar y contar
tabla = pd.cut(serie, bins=bins).value_counts(sort=False)

# Convertir a DataFrame con acumulados y relativos
df = tabla.to_frame(name='Frecuencia')
df['Frecuencia acumulada'] = df['Frecuencia'].cumsum()

# escribimos la instrucción en dos líneas usando los paréntesis
df['Frecuencia relativa (%)'] = (
    (df['Frecuencia'] / df['Frecuencia'].sum() * 100).round(2)
)
df['Acumulada (%)'] = df['Frecuencia relativa (%)'].cumsum()

print(df)

```

|            | Frecuencia | Frecuencia acumulada | Frecuencia relativa (%) \ |
|------------|------------|----------------------|---------------------------|
| (130, 140] | 4          | 4                    | 36.36                     |
| (140, 150] | 1          | 5                    | 9.09                      |
| (150, 160] | 5          | 10                   | 45.45                     |
| (160, 170] | 0          | 10                   | 0.00                      |
| (170, 180] | 1          | 11                   | 9.09                      |

|            | Acumulada (%) |
|------------|---------------|
| (130, 140] | 36.36         |
| (140, 150] | 45.45         |
| (150, 160] | 90.90         |
| (160, 170] | 90.90         |
| (170, 180] | 99.99         |

Explicación del código línea a línea.

Este script en Python construye una tabla de frecuencias agrupadas a partir de un conjunto de datos numéricos (`altura_cm`), usando `pandas` y `numpy`.

```

import pandas as pd
import numpy as np

```

Importa las bibliotecas necesarias: - `pandas`: para manipulación tabular. - `numpy`: para cálculos numéricos y generación de rangos.

```
altura_cm = [153,135,140,140,175,138,145,154,152,159,154]
```

Define el conjunto de datos como una lista de alturas en centímetros.

```
serie = pd.Series(altura_cm)
```

Convierte la lista en una `Series` de pandas, lo que permite aplicar funciones estadísticas fácilmente.

```
bins = np.arange(min(altura_cm)//10*10, max(altura_cm)//10*10 + 20, 10)
```

Genera los límites de clase con amplitud 10: - `min(altura_cm)//10*10`: redondea hacia abajo el mínimo a la decena más cercana. - `max(altura_cm)//10*10 + 20`: redondea hacia arriba el máximo y añade 10 extra para incluir el último dato. - `np.arange(...)`: crea un array de límites desde el mínimo hasta el máximo, en pasos de 10.

Ejemplo de salida: [130, 140, 150, 160, 170, 180]

```
tabla = pd.cut(serie, bins=bins).value_counts(sort=False)
```

Agrupar los datos en los intervalos definidos: - `pd.cut(...)`: asigna cada dato a un intervalo. - `.value_counts(sort=False)`: cuenta cuántos datos hay en cada intervalo, sin reordenarlos.

```
df = tabla.to_frame(name='Frecuencia')
```

Convierte la serie de frecuencias en un `DataFrame` con una columna llamada 'Frecuencia'.

```
df['Frecuencia acumulada'] = df['Frecuencia'].cumsum()
```

Calcula la frecuencia acumulada sumando progresivamente los valores de la columna 'Frecuencia'.

```
df['Frecuencia relativa (%)'] = (df['Frecuencia'] / df['Frecuencia'].sum() * 100).round(2)
```

Calcula la frecuencia relativa en porcentaje: - Divide cada frecuencia por el total. - Multiplica por 100. - Redondea a 2 decimales.

```
df['Acumulada (%)'] = df['Frecuencia relativa (%)'].cumsum()
```

Calcula el porcentaje acumulado, útil para análisis tipo Pareto.

```
print(df)
```

Muestra la tabla final con: - Intervalos - Frecuencia absoluta - Frecuencia acumulada - Frecuencia relativa (%) - Porcentaje acumulado

Como ves en el resultado, a veces se utilizan los símbolos ( y [ para definir los intervalos, tal como se hace en matemáticas.

- Intervalo **abierto**: El símbolo  $($  se utiliza para denotar un intervalo abierto. El límite correspondiente **no** está incluido en el intervalo.
- Intervalo **cerrado o semiabierto**: El símbolo  $[$  se utiliza para denotar un intervalo cerrado o semiabierto. El límite correspondiente **sí** está incluido en el intervalo.

Ejemplos:

- $(a, b)$  representa todos los números reales **mayores** que  $a$  y **menores** que  $b$  (excluye los valores  $a$  y  $b$ ).
- $[a, b]$  representa todos los números reales **mayores o iguales** que  $a$  y **menores o iguales** que  $b$  (incluye  $a$  y  $b$ ).
- $[a, b)$  representa todos los números reales **mayores o iguales** que  $a$  y **menores** que  $b$  (incluye  $a$ , pero excluye  $b$ ).
- $(a, b]$  representa todos los números reales **mayores** que  $a$  y **menores o iguales** que  $b$  (excluye  $a$ , pero incluye  $b$ ). ::

Si comparamos los dos métodos que hemos utilizado para construir la tabla de frecuencias, vemos que:

- en Excel los pasos que hemos dado no están registrados y a la vista, y, por lo tanto, no son fácilmente revisables
- en **Python**, todos los pasos y opciones que hemos utilizado están a la vista en el código del script

Si otra persona quisiera modificar la tabla, le sería fácil editar el código y relanzar el script, mientras que en Excel no sería fácil asegurarse de todos y cada uno de los pasos y clicks de ratón que hemos dado para construir y formatear la tabla.

Es el caso, por ejemplo, de que enviásemos la tabla a otra persona y ésta tuviese que editarla en nuestra ausencia. En Excel, tendríamos que enviar a esa persona una explicación con las instrucciones oportunas; en cambio, en **Python**, una vez que se comprende el código, no hacen falta más explicaciones adicionales. Incluso sin una comprensión total del código, se podría duplicar exactamente la tabla copiando y ejecutando el código.

Esta es una de las principales razones de la conveniencia del aprendizaje de **Python** incluso para las actividades más sencillas.

## 6.4. Ejercicio propuesto

En la tabla de frecuencias anterior, calcular **frecuencia absoluta acumulada** y **frecuencia relativa acumulada** en Excel, e incluirlas en la tabla como dos columnas adicionales. La **frecuencia relativa acumulada** nos permitirá crear **diagramas de Pareto**, muy útiles en los procesos de mejora de calidad.



## 6.5. Diagrama de barras

Cuando nuestra variable es *discreta*, podemos representar las frecuencias de cada valor de forma gráfica utilizando un diagrama de barras. Este diagrama utiliza barras rectangulares para representar la frecuencia de cada categoría.

Este gráfico es muy utilizado para representar, por ejemplo, resultados de encuestas, como el número de votos que han obtenido los diferentes partidos políticos en unas elecciones, o tablas discretas, como los kilos fabricados por meses en una fábrica.



## 6.6. Histograma

Para visualizar las variables continuas se utiliza el **histograma**, que es un diagrama que utiliza las barras rectangulares para hacer un gráfico de la distribución

de valores continuos, previamente agrupados en intervalos (*bins* en inglés), tal como se ha hecho en la tabla de frecuencias.

### Componentes de un histograma

1. **Eje horizontal (X)**: Representa los intervalos de valores de la variable. Cada intervalo abarca un rango específico de valores.
2. **Eje vertical (Y)**: Muestra la frecuencia o el número de veces que los valores caen dentro de cada intervalo.
3. **Barras**: Cada barra en el histograma representa un intervalo. La altura de la barra indica la frecuencia de los valores dentro de ese intervalo.

### Cómo interpretar un histograma

- Si las barras son altas en un intervalo específico, significa que muchos valores del conjunto de datos caen dentro de ese rango.
- Un histograma puede ayudar a identificar patrones, como si los datos están distribuidos de manera simétrica, sesgada, o si hay picos y valles significativos.

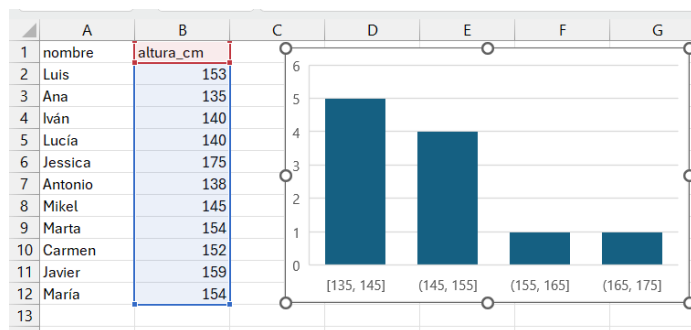
#### 6.6.1. Construcción de un histograma en Excel

Usaremos el ejemplo de la altura en cm. de un grupo de alumnos, cuyos datos hemos guardado en el fichero csv `aula1.csv`.

Podemos utilizar dos métodos para hacer un histograma en Excel

##### Método 1: histograma directo

- Seleccionamos el rango de datos. Podemos hacerlo mediante un click en el encabezado de la columna de la variable `altura_cm`, en este caso, la columna B.
- En la opción de menú **Insertar**, seleccionamos el icono **Seleccionar gráficos de estadística**
- Seleccionamos **Histograma**



Excel calcula automáticamente la amplitud de los intervalos y el número de columnas; estas opciones pueden modificarse seleccionando con el botón derecho del ratón el elemento a modificar. En este caso, utilizamos estas opciones:

- Eliminamos título del gráfico
- Modificamos anchura de las barras seleccionando **Dar formato a serie de datos**, **Opciones de serie**, **Ancho del rango**= 50 %
- Modificamos los intervalos seleccionando el eje X: **Dar formato al eje**, **Opciones de eje**, **Ancho del rango** = 10

La descripción de los intervalos utiliza los mismos símbolos que hemos visto en las tablas de frecuencias de **Python**.

En el momento de escribir este manual, Excel no permite hacer histogramas múltiples ni agrupados por otra variable, por lo que para diseños más complejos, no hay más remedio que recurrir a otras opciones como las tablas dinámicas de Excel o, mejor aún, **Python** o **R**.

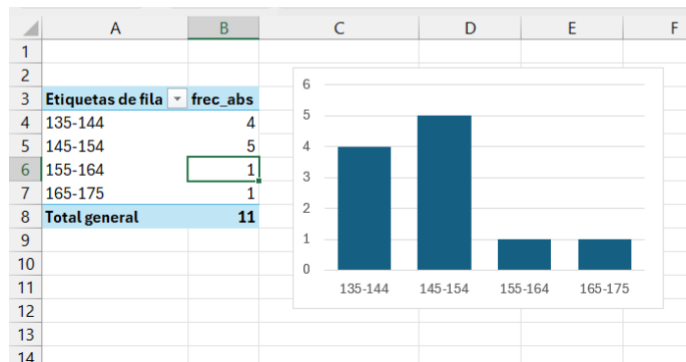
### Método 2: utilizar una tabla dinámica

La tabla dinámica que hemos construido en Excel ha convertido nuestra variable continua, **altura\_cm**, en una tabla de valores discretos, al agrupar los valores en intervalos. En Excel podemos representar las frecuencias absolutas de nuestra tabla gráficamente, insertando un gráfico de barras a partir de la tabla:

- Con el cursor dentro de la tabla, seleccionamos la opción de menú **Insertar**
- Insertamos un gráfico de barras

Opcionalmente, aplicamos las siguientes opciones de formato:

- Hacemos click sobre uno de los botones del gráfico dinámico con el botón derecho del ratón, y seleccionamos la opción ocultar todos los botones del gráfico.
- Eliminamos el título del gráfico
- Abrimos el formato de la serie de datos, e introducimos en la opción **Ancho del rango** el valor 50 % para ensanchar las barras.



Al utilizar la tabla dinámica para construir el gráfico, Excel utiliza las categorías de la tabla dinámica. Dado que estas categorías (los intervalos que ha formado la tabla dinámica) son discretas, Excel utiliza el resultado de la tabla dinámica para hacer el gráfico con un diagrama de barras. No es posible insertar un histograma a partir de una tabla dinámica.

### 6.6.2. Análisis de un caso real

El histograma muestra su utilidad cuando representamos la distribución de un conjunto de valores más grande que nuestros once alumnos. Veamos su aplicación a los datos diarios de una fabricación de queso camembert a lo largo de un año. Los datos de esta fabricación están en el fichero `camembert.csv`.

La tabla de datos tiene esta estructura:

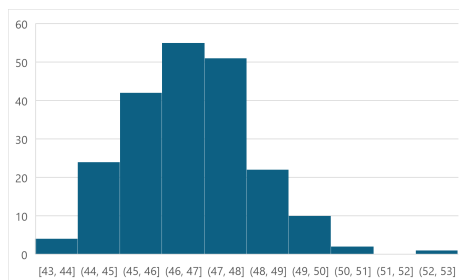
|    | A          | B           | C     | D    | E    | F        | G          |  |
|----|------------|-------------|-------|------|------|----------|------------|--|
| 1  | fecha      | fabricacion | est   | mg   | ph   | cloruros | coliformes |  |
| 2  | 11/01/2020 | 1           | 46,68 | 24   | 4,85 | 1,5      | 80         |  |
| 3  | 12/01/2020 | 1           | 48,09 | 25   | 4,67 | 1,61     | 0          |  |
| 4  | 13/01/2020 | 1           | 45,97 | 24   | 4,71 | 1,48     | 9200       |  |
| 5  | 14/01/2020 | 1           | 46,24 | 23,5 | 4,78 | 1,69     | 20         |  |
| 6  | 15/01/2020 | 1           | 45,81 | 23   | 4,77 | 1,56     | 150        |  |
| 7  | 18/01/2020 | 1           | 46    | 24   | 4,67 | 1,56     | 60         |  |
| 8  | 19/01/2020 | 1           | 44,83 | 23,5 | 4,73 | 1,39     | 40         |  |
| 9  | 20/01/2020 | 1           | 45,82 | 25   | 4,68 | 1,6      | 40         |  |
| 10 | 21/01/2020 | 1           | 47,1  | 24   | 4,75 | 1,54     | 175        |  |
| 11 | 22/01/2020 | 1           | 45,55 | 24   | 4,85 | 1,62     | 340        |  |
| 12 | 25/01/2020 | 1           | 47,08 | 25,5 | 4,99 | 1,24     | 0          |  |

La tabla está formada por 211 casos.

### Construcción del histograma con Excel

#### Método directo

Seleccionamos el rango del extracto seco (columna `est`) e insertamos el histograma, ajustando el ancho de intervalo a 1 en las opciones de gráfico:



#### Método 2: mediante una tabla dinámica

Los pasos a seguir son:

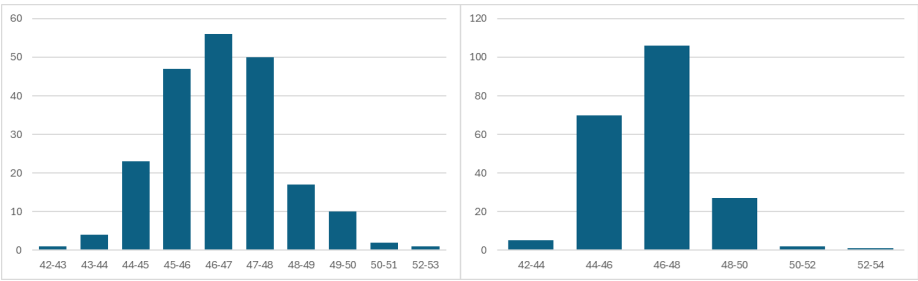
1. Construir la tabla dinámica
2. Agrupar los datos

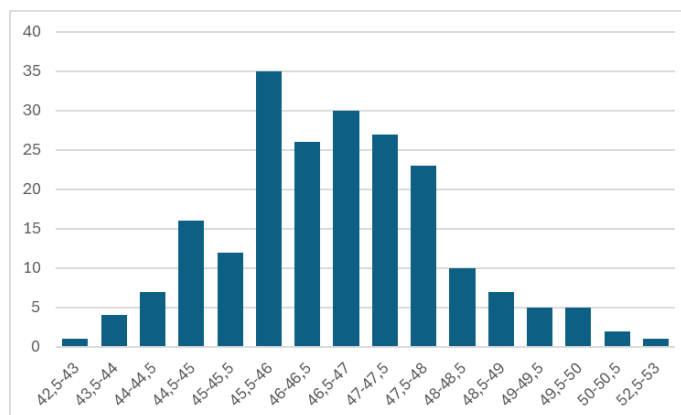
3. Insertar el gráfico a partir de la tabla

Con una agrupación de datos en intervalos de 1, esta es nuestra tabla dinámica:

|    | A                 | B        |
|----|-------------------|----------|
| 1  |                   |          |
| 2  |                   |          |
| 3  | Etiquetas de fila | frec_abs |
| 4  | 42-43             | 1        |
| 5  | 43-44             | 4        |
| 6  | 44-45             | 23       |
| 7  | 45-46             | 47       |
| 8  | 46-47             | 56       |
| 9  | 47-48             | 50       |
| 10 | 48-49             | 17       |
| 11 | 49-50             | 10       |
| 12 | 50-51             | 2        |
| 13 | 52-53             | 1        |
| 14 | Total general     | 211      |
| 15 |                   |          |

En la figura siguiente vemos el histograma correspondiente a la tabla anterior, con un intervalo de clase de 1 punto de extracto seco total, y otras dos alternativas si el intervalo de clase fuese de 2 puntos o de de 0,5 puntos.





La decisión de cambiar la amplitud del intervalo en un histograma influye en el aspecto del gráfico y es una opción personal; lo mejor es utilizar la que en nuestra opinión refleje mejor el aspecto de la distribución de datos, ni demasiado grande ni demasiado pequeña. En todo caso, debemos ser capaces de interpretar que la distribución de los valores es la misma en los tres casos: hay una mayoría de casos con valores entre 46 y 48, y muy pocos casos con valores muy bajos o muy altos. En este caso, la distribución de los valores es aproximadamente simétrica, y se reparten alrededor de una mayoría de valores centrales.

## 6.7. Visualización de Datos en Python con Seaborn y Matplotlib

En Python, las librerías **Matplotlib** y **Seaborn** son las herramientas estándar para crear gráficos.

- **Matplotlib.pyplot (alias plt):** Es la base, una librería de bajo nivel que da un control muy granular sobre cada aspecto del gráfico.
- **Seaborn (alias sns):** Construye sobre Matplotlib, ofreciendo una interfaz de alto nivel para crear gráficos estadísticos complejos y estéticamente agradables con menos código. Es ideal para explorar distribuciones, relaciones entre variables y comparaciones entre grupos.

Siempre importamos ambas, ya que Seaborn a menudo usa funciones de Matplotlib para mostrar y personalizar los gráficos (como `plt.title()` para el título o `plt.show()` para mostrar el gráfico).

El texto siguiente incluye el código para crear el gráfico, no te preocupes si no lo entiendes a la primera, iremos explicándolo línea a línea.

### Creación del histograma en Python

Las funciones de histograma de Python permiten construir el histograma directamente sin necesidad de hacer previamente una tabla de frecuencias (en realidad,

## 6.7. VISUALIZACIÓN DE DATOS EN PYTHON CON SEABORN Y MATPLOTLIB<sup>83</sup>

la tabla de frecuencias se calcula internamente). Es mucho más sencillo utilizar estas funciones, ya que el código se simplifica mucho.

Vemos que los dos gráficos no son idénticos a pesar de provenir de los mismos datos, porque la construcción de los intervalos subyacente es ligeramente diferente. Esto no debe preocuparnos, porque el aspecto general de la distribución de los datos es el mismo.

En la fase de exploración nos importa más entender estas propiedades de los datos (aspecto, forma de la distribución) que la precisión en la construcción del gráfico. La función básica `.hist()`, por su parte, es más sencilla de usar.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

url_datos = (
    "https://raw.githubusercontent.com/"
    "juanriera/master-queseria/master/"
    "datos/camembert.csv"
)

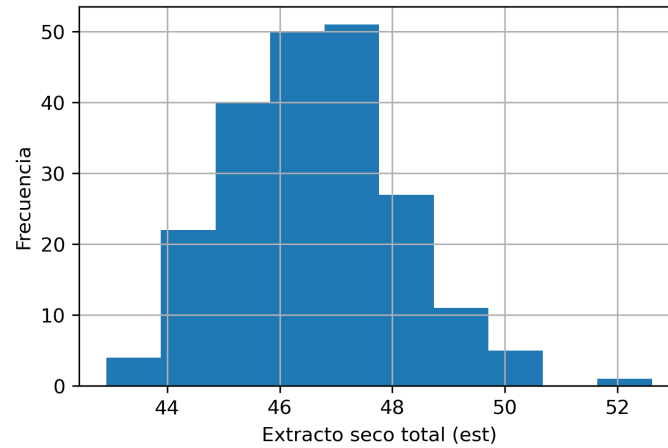
try:
    df = pd.read_csv(url_datos,
                     decimal = ",",
                     sep=';',
                     encoding='ISO-8859-1')
except Exception as e:
    print(f"Error al cargar el archivo: {e}")

df["est"].hist() # histograma básico de matplotlib

# Añadir título y etiquetas
plt.title("Distribución del extracto seco total"
          " en una fabricación anual de camembert")
plt.xlabel("Extracto seco total (est)")
plt.ylabel("Frecuencia")

# Mostrar el gráfico
plt.show()
```

Distribución del extracto seco total en una fabricación anual de camembert



Podemos utilizar también las funciones de **seaborn** para el histograma:

```
sns.displot(df["est"], kde = True)

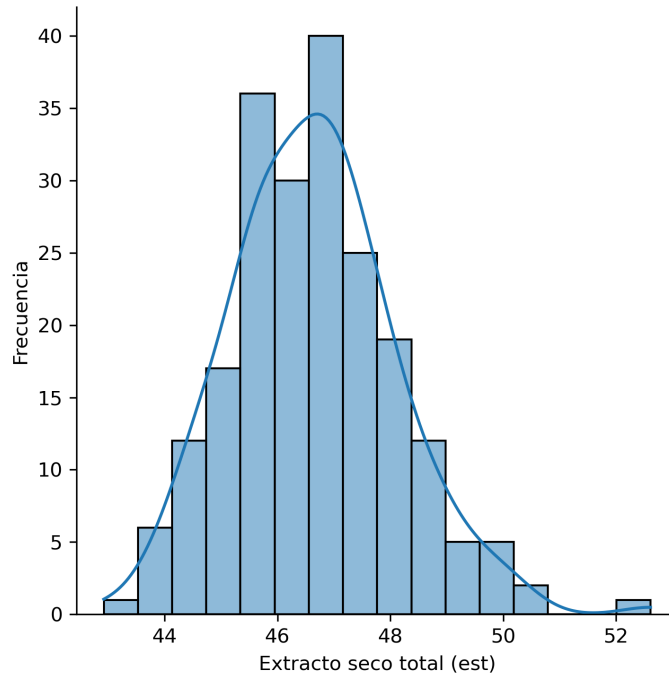
# Añadir título y etiquetas
plt.title("Distribución del extracto seco total"
          "en una fabricación anual de camembert")
plt.xlabel("Extracto seco total (est)")
plt.ylabel("Frecuencia")

# Mostrar el gráfico
plt.show()
```



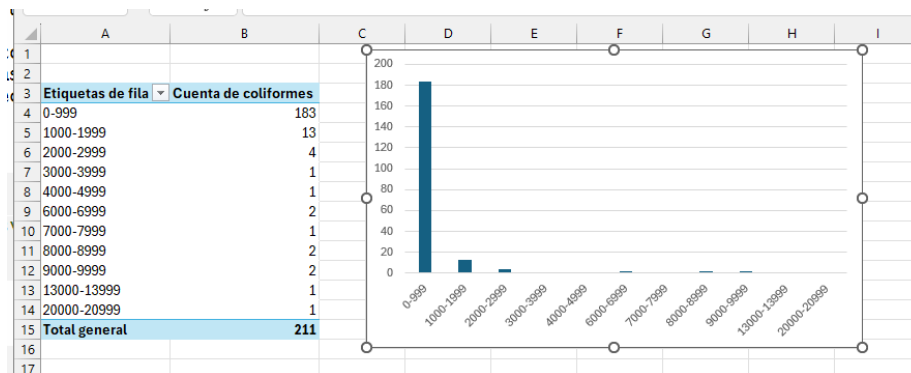
## 6.7. VISUALIZACIÓN DE DATOS EN PYTHON CON SEABORN Y MATPLOTLIB<sup>85</sup>

Distribución del extracto seco total en una fabricación anual de camembert



### 6.7.1. Otros ejemplos

En ocasiones, nos encontramos con datos que son asimétricos: hay una mayoría de valores bajos o bien de valores altos. Veamos un caso: los recuentos bacterianos de bacterias coliformes, que tenemos en la última columna a la derecha de nuestra tabla, en la variable 'coliformes'.



En este caso, vemos que la mayoría de los casos tienen valor cero. Es el caso de los recuentos de bacterias contaminantes, en el que la mayoría de los análisis tienen recuentos cero o muy bajos, y sólo en pocos casos tienen valores altos.

Veremos con más detalle cómo tratar estas distribuciones más adelante.

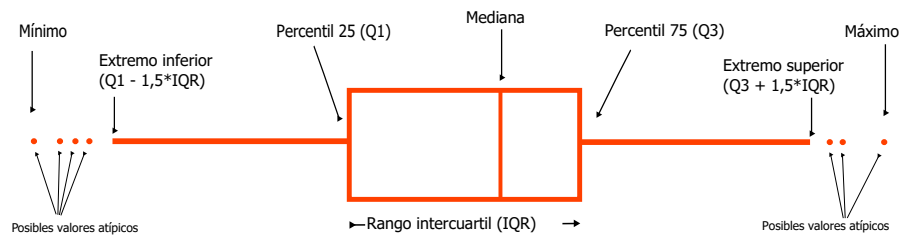
## 6.8. Diagrama de caja o *boxplot*

Este gráfico fue creado por el estadístico John Tukey en 1977, y es una herramienta fundamental en la exploración de datos. Se basa en un grupo de medidas que se utiliza ampliamente en la descripción de conjuntos de datos, el conjunto de **cuartiles**. Si dividimos un grupo de datos **ordenados** en cuatro partes iguales, mediante **tres** puntos de corte, llamamos **primer cuartil** o  $Q1$  al valor que se sitúa en el 25 %; **segundo cuartil**, o  $Q2$ , al valor que se sitúa en el centro (50 %), y **tercer cuartil**, o  $Q3$ , al punto que se sitúa en el 75 % de los datos. A estos tres valores añadimos el **mínimo** y el **máximo**, y tenemos un conjunto de **cinco números** que nos permiten describir la forma de la distribución de datos con cierta precisión. El segundo cuartil ( $Q2$ ), que corresponde al 50 % de los datos, se conoce habitualmente como **mediana**. El valor resultante de restar  $Q3 - Q1$  es lo que se conoce como **rango intercuartil** o *IQR*, y es una medida de la dispersión de la distribución de datos (mide la amplitud de la distribución).

El **diagrama de caja**, también conocido como *boxplot*, es un gráfico que permite resumir las características principales de un conjunto de datos utilizando estos cinco números, tal como se explica a continuación. Sus ventajas son:

- Muestra la mediana y los cuartiles ( $Q1$  y  $Q3$ ) de los datos.
- Permite identificar la simetría de la distribución: si la mediana no está en el centro, la distribución no es simétrica.
- Permite detectar posibles valores atípicos, representando los valores atípicos (*outliers*) que están lejos del resto de los datos (un valor es atípico si está más allá de  $(Q3 + 1.5 \cdot IQR)$  o  $(Q1 - 1.5 \cdot IQR)$ ).

La construcción de un diagrama de caja es como sigue:

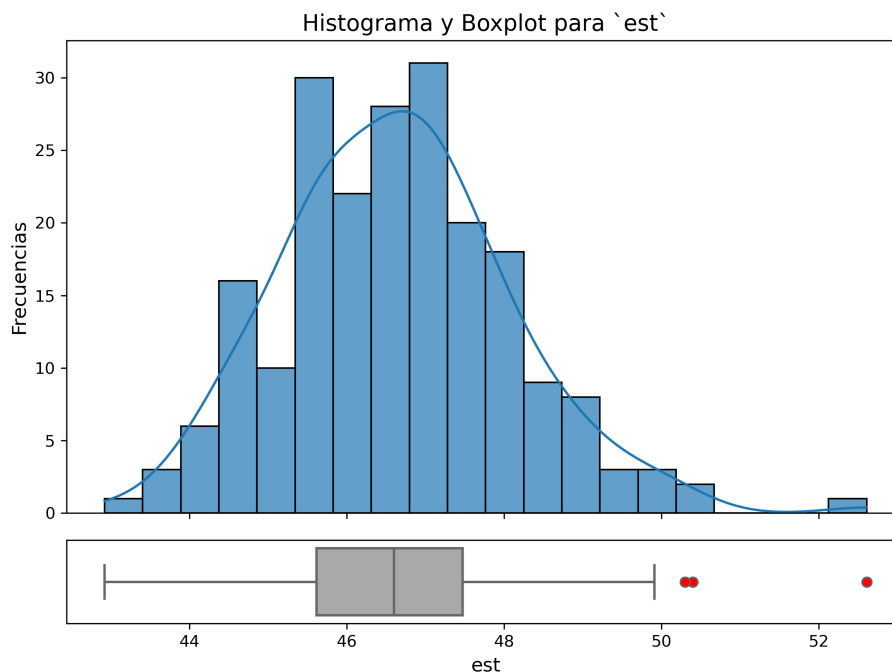


Microsoft Excel no dispone de un diseño de gráficos de caja que sea práctico, por lo que recurriremos siempre a Python para realizarlos.

### 6.8.1. Relación entre el *boxplot* y el histograma

Resulta muy útil comprender visualmente la relación entre el *boxplot* y el histograma para entender la distribución de los datos. En la gráfica siguiente analizamos el extracto seco total de nuestra fabricación usando ambos gráficos simul-

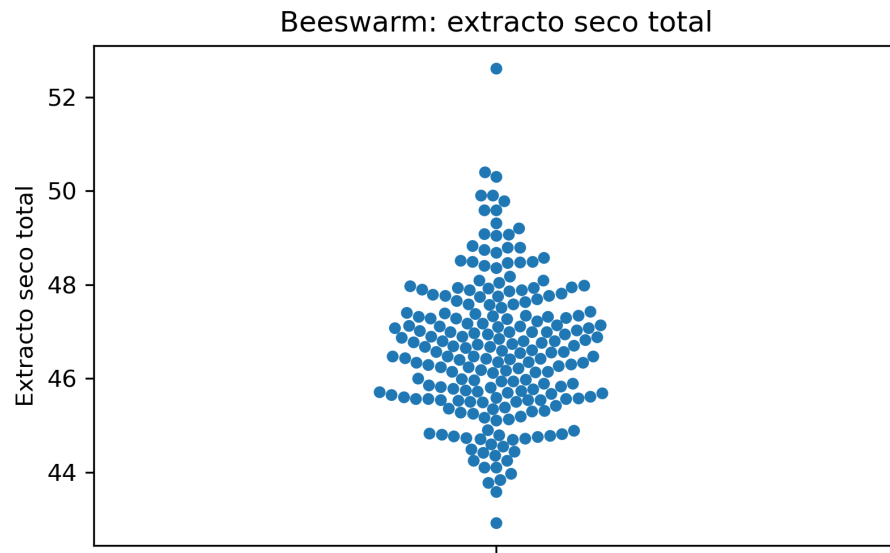
táneamente. La caja del `boxplot` se corresponde con el 50 % central de los datos, y la línea media nos representa la mediana, como hemos visto. Vemos también que hemos utilizado una opción de formato para destacar las observaciones que se separan de forma anormal del conjunto de datos (valores atípicos o *outliers*)



## 6.9. Alternativas al boxplot

El *boxplot* es muy útil para resumir grandes cantidades de información, pero cuando el número de puntos es muy bajo, puede resultarnos más útil la visualización mediante otro tipo de gráficos que nos muestren los puntos individuales agrupados. Entre estas visualizaciones, la biblioteca **seaborn** nos ofrece el gráfico llamado *beeswarm* mediante la función `swarmplot()`, y también el *dotplot*, mediante la función `stripplot()`.

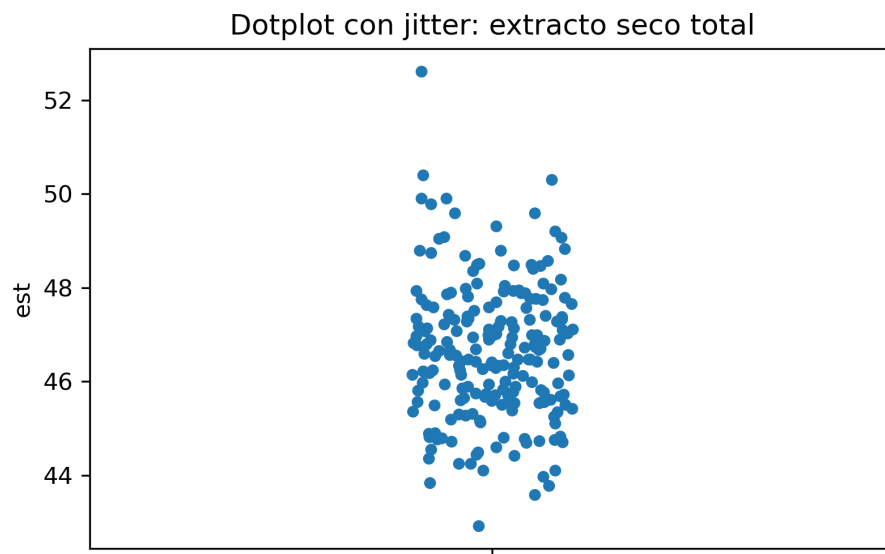
```
sns.swarmplot(data=df, y='est', size=5)
plt.xticks(rotation=0) # es opcional rotar lo rótulos para mejorar la visibilidad cuando los titulos son largos
plt.title('Beeswarm: extracto seco total')
plt.ylabel('Extracto seco total')
plt.tight_layout()
plt.show()
```



El *beeswarm* nos da una distribución aproximada de los valores individuales, y los coloca al lado para evitar solapamientos.

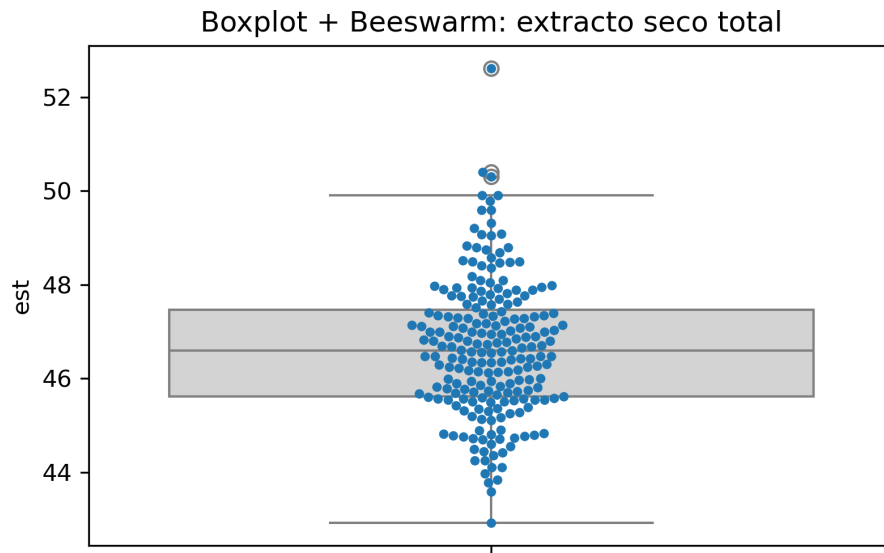
El *stripplot* muestra los puntos en distribución vertical sobre las categorías; para evitar que los puntos con las mismas coordenada se solapen, utilizamos la opción *jitter*, que introduce un pequeño *ruido* aleatorio horizontal para desplazar las coordenadas de los puntos evitando su solapamiento.

```
sns.stripplot(data=df, y='est', jitter=True, size=5)
plt.xticks(rotation=0)
plt.title('Dotplot con jitter: extracto seco total')
plt.tight_layout()
plt.show()
```



Este diagrama puede superponerse al *boxplot*, lo que nos permite completar la visualización de los puntos y la interpretación que hace el *boxplot* de su distribución.

```
sns.boxplot(data=df, y='est', color='lightgray')
sns.swarmplot(data=df, y='est', size=4)
plt.xticks(rotation=0)
plt.title('Boxplot + Beeswarm: extracto seco total')
plt.tight_layout()
plt.show()
```



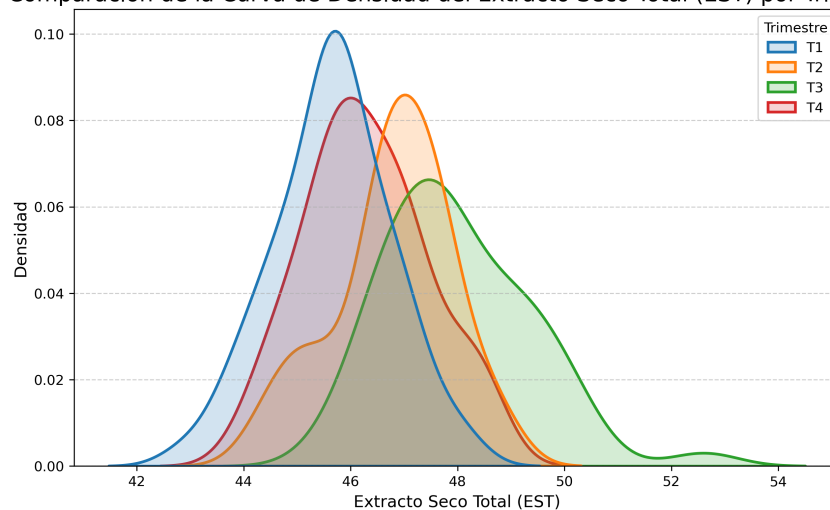
## 6.10. Gráficos de densidad

Un gráfico de densidad es una representación visual suavizada de la distribución de un conjunto de datos. A diferencia de los histogramas, que dividen los datos en intervalos y cuentan las frecuencias, los gráficos de densidad utilizan técnicas estadísticas no paramétricas para estimar la función de densidad de probabilidad.

Excel no permite la representación de los gráficos de densidad; en **Python** lo hemos hecho arriba hacerlo con **seaborn**, añadiendo la función de densidad a un histograma añadiendo la opción `kde = True`.

Los gráficos de densidad son especialmente útiles cuando estamos comparando diferentes grupos de datos entre sí, por ejemplo en este caso comparamos el extracto seco total de nuestra fabricación de camembert trimestre a trimestre:

Comparación de la Curva de Densidad del Extracto Seco Total (EST) por Trimestre



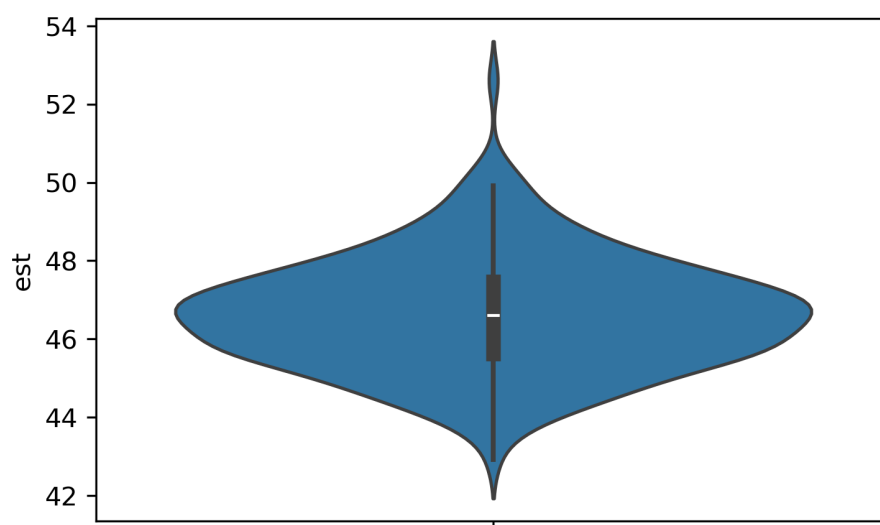
La curva de densidad nos permite visualizar la desviación que se produce por el verano (T3); son los valores que el *boxplot* nos ha mostrado como *outliers*.

### 6.10.1. Otra alternativa al *boxplot*: el *violin plot*

El *violin plot* es una alternativa reciente al *boxplot*, tiene la ventaja de que muestra la *densidad* de la distribución superpuesta al *boxplot*, lo que permite visualizar con más claridad la distribución de los datos:

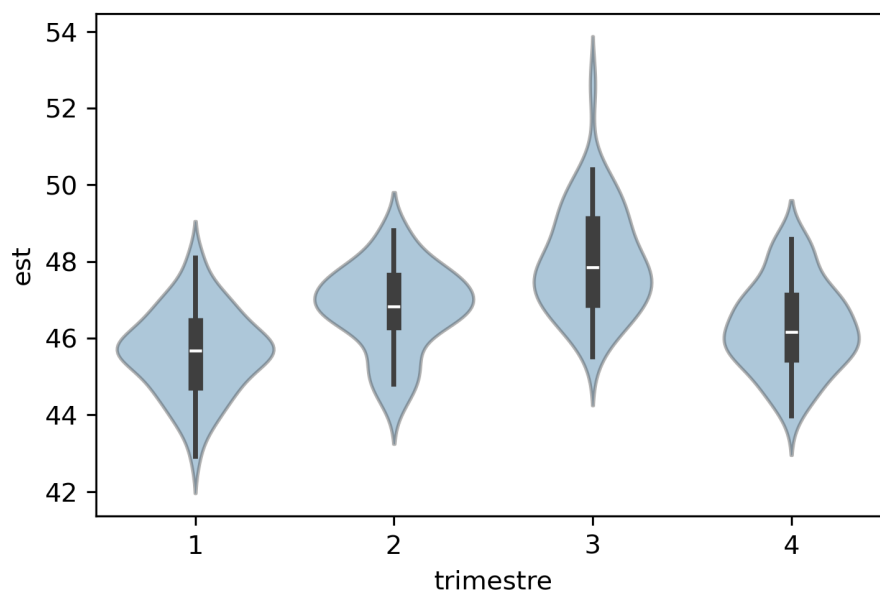
```
sns.violinplot(y='est', data=df)

plt.show()
```



El *violin plot*, como el *boxplot*, se usan muy a menudo para comparar distribuciones de datos. Puedes comparar el gráfico de densidades por trimestres que hemos hecho antes con este *violin plot* múltiple por trimestres.

```
sns.violinplot(x= 'trimestre', y='est', data=df, alpha=0.4)
plt.show()
```





## 6.11. Gráficos de dispersión

Un gráfico de dispersión, también conocido como diagrama de dispersión o *scatter plot*, es una representación gráfica que utiliza puntos para mostrar la relación entre dos variables numéricas. Cada punto en el gráfico representa una observación del conjunto de datos y se coloca en el plano cartesiano de acuerdo con sus valores en las dos variables que se están comparando.

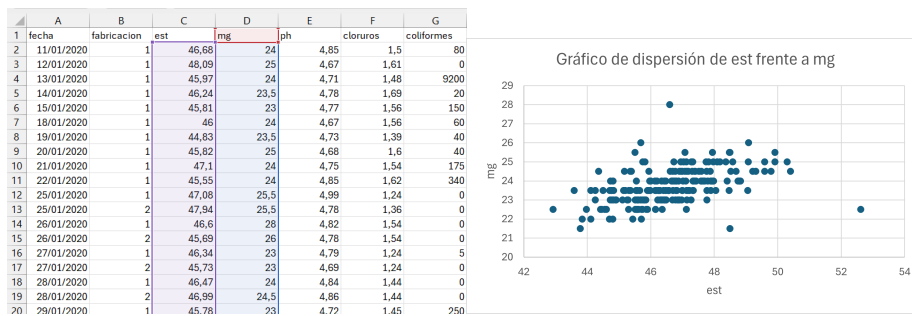
Un gráfico de dispersión se compone mediante puntos:

- Cada punto en el gráfico representa una observación.
- La posición del punto en el gráfico está determinada por los valores de las dos variables para esa observación.

Los gráficos de dispersión son útiles para identificar varios aspectos de la relación entre las dos variables:

- Si los puntos tienden a agruparse a lo largo de una línea recta ascendente, esto indica una correlación positiva (a medida que una variable aumenta, la otra también lo hace).
- Si los puntos se agrupan a lo largo de una línea descendente, esto indica una correlación negativa (a medida que una variable aumenta, la otra disminuye).
- Si los puntos forman una curva en lugar de una línea recta, esto sugiere una relación no lineal entre las variables.
- La dispersión de los puntos puede indicar la variabilidad de los datos. Puntos que están muy lejos del patrón general pueden ser valores atípicos.

Representamos las variables **esty** **mg** de nuestros datos de la fabricación de camembert.



(a) Tabla de origen mostrando una parte de los datos seleccionados

(b) Gráfico de dispersión

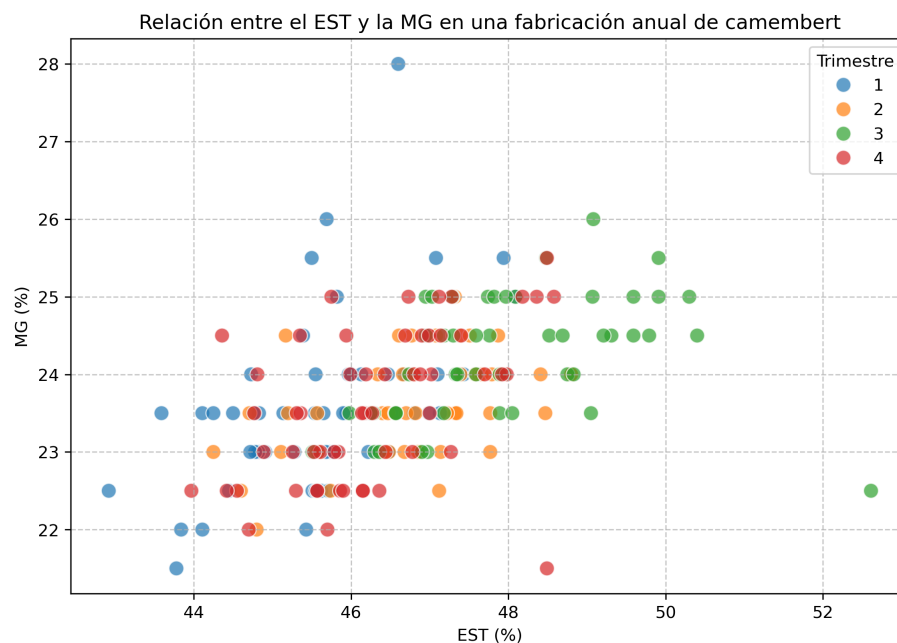
Figura 6.1: Gráfico de dispersión en Excel

Lo esperable en nuestros datos es que el valor de **est** y el de **mg** estén asociados, y a valores altos de la primera variable correspondan valores altos de la segunda (para el mismo producto y sin cambios de tecnología). Sin embargo, algunos

valores parecen no encajar en este modelo; es el caso de un valor de `est` por encima de 52 % con un valor de `mg` inferior a 23 %, o el caso de un valor de `mg` en torno al 28 % con un valor de `est` por debajo de 47 %. Debemos revisar estos valores aparentemente anormales para verificar si ha habido un error en la toma de muestras o un error analítico. En fábricas con productos diferentes cuyos resultados analíticos se recogen en una tabla de datos común, a veces puede haber errores en la introducción de los códigos de producto, de forma que la toma de muestras y los análisis pueden ser correctos, pero estar mal asignados al guardar los datos.

Python nos permite separar el diagrama de dispersión en función de una tercera variable, en este caso, usaremos el trimestre para ver lo que ha sucedido por el verano.

```
plt.figure(figsize=(9, 6))
sns.scatterplot(data=df, x='est', y='mg', hue='trimestre', s=80, alpha=0.7)
plt.title('Relación entre el EST y la MG en una fabricación anual de camembert')
plt.xlabel('EST (%)')
plt.ylabel('MG (%)')
plt.legend(title='Trimestre') # Mostrar leyenda para la especie
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



Como vemos, los gráficos de dispersión son una herramienta esencial en el análisis exploratorio de datos, ya que permiten visualizar relaciones y patrones en los datos, identificar correlaciones y detectar posibles anomalías. Esta información

es crucial para realizar análisis estadísticos más profundos y tomar decisiones basadas en datos.

## 6.12. Gráficos de series temporales

Hasta ahora hemos utilizado gráficos y tablas que describen la estructura y forma de una variable, o las relaciones entre dos variables. Hay otros gráficos que tienen en cuenta la forma en la que esos datos cambian con el tiempo. En este caso, será necesario que hayamos recogido en una variable de nuestra tabla los intervalos de tiempo en los que se han producido nuestros valores.

## 6.13. Algunos ejemplos

- proceso de llenado de envases de queso crema: se llena una tarrina cada 15 segundos. Nuestros datos deben recoger el tiempo y el peso.
- nuestro fichero de fabricación de queso camembert recoge los valores analíticos medios diarios del producto fabricado.
- Una fábrica recoge leche diariamente y analiza cada día la composición de la leche que entra en la fábrica.

En un gráfico de series temporales,

- el eje horizontal (X) representa el tiempo. Los puntos de tiempo pueden ser minutos, horas, días, meses, años, etc.
- el eje vertical (Y) representa los valores de la variable que se está estudiando. Estos valores pueden ser medidas como temperatura, ventas, precios, etc.
- cada valor individual corresponde a un punto
- los valores se conectan mediante una línea que conecta los puntos de datos, mostrando cómo cambian los valores de la variable a lo largo del tiempo.
- normalmente, en un gráfico de series temporales no suelen representarse los puntos individuales para facilitar la legibilidad del gráfico.

En nuestro conjunto de datos de fabricación de queso camembert, la primera columna de la tabla recoge la variable **fecha**, lo que nos permite ordenar nuestros valores en el tiempo.

Cuando representamos valores en el tiempo, nunca usaremos el diagrama de barras, sino el gráfico de líneas.

### 6.13.1. Cómo hacer los gráficos de series temporales en Excel

Para hacer el gráfico en Excel, seleccionamos la columna **este** insertamos un gráfico de líneas. A continuación, con el cursor sobre el gráfico, pulsamos el

boton derecho y seleccionamos la opción **Seleccionar datos**. Una vez abierto el cuadro de opciones, editamos las etiquetas del eje X y seleccionamos el rango de la variable **fecha** desde la fila 2 hasta la última. Aceptamos, y a continuación editamos el formato del eje Y para sustituir el valor mínimo de 0 por 42, que es el valor que queremos como mínimo para nuestro gráfico.

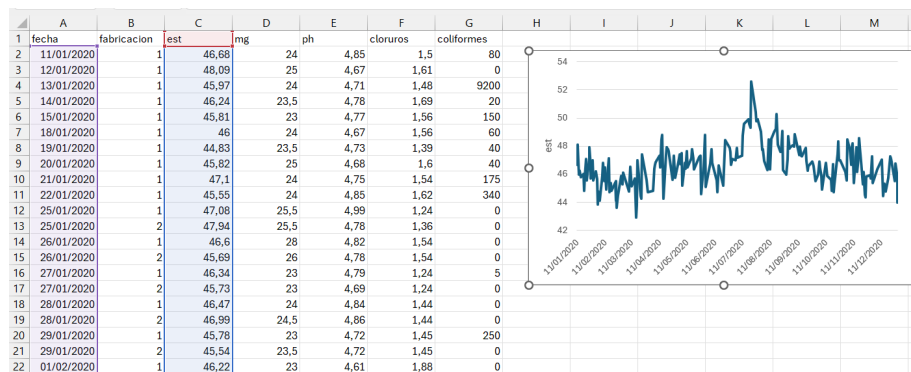


Figura 6.2: Tabla y gráfico de series

### 6.13.2. Gráficos de series temporales en Python utilizando pandas

En los procesos industriales, el manejo de datos que cambian con el tiempo (**series temporales**) es fundamental. Hablamos de registrar **temperaturas** en un proceso de cocción, el **pH** de una fermentación a lo largo de las horas o días, o la **evolución de inventarios** de productos perecederos.

Para abordar este análisis de manera **eficiente y sencilla**, la biblioteca **pandas** de Python es la herramienta de referencia. Su principal estructura, el **DataFrame**, está especialmente diseñada para entender y manipular datos etiquetados con el tiempo. Esto nos ofrece una serie de ventajas clave:

1. **Fácil Indexación Temporal:** Pandas permite que la fecha y hora sean el “índice” principal de los datos. Esto simplifica enormemente el **filtrado** (ej. “dame todos los datos del último turno”) y la **selección por rangos** (ej. “analiza solo el periodo de 9:00 a 11:00”).
2. **Remuestreo (Resampling) Sencillo:** Es habitual tener datos muy detallados (ej. una medición por minuto) y querer analizarlos a una escala mayor (ej. la media por hora). Pandas permite este cambio de frecuencia con una única función, lo que es vital para suavizar el “ruido” y ver la **tendencia real** del proceso.
3. **Ventanas Móviles (rolling):** Es la función estrella para la industria. Permite calcular **promedios móviles** o desviaciones estándar sobre un periodo de tiempo definido (ej. el promedio de temperatura de los últimos

15 minutos, o cada 15 minutos). Esto es crucial para el **control de calidad** y la identificación de anomalías o desviaciones en tiempo real.

En resumen, Pandas elimina las tareas de manipulación de datos tediosas, permitiendo a los profesionales de la alimentación centrarse en lo realmente importante: la **interpretación de los datos** para optimizar procesos, garantizar la **seguridad alimentaria** y mejorar la eficiencia.

### Construyendo la serie temporal con pandas

Vamos a incluir de nuevo el código de lectura de los datos completo, para mayor claridad. Además, vamos a incluir una opción en la función de lectura del csv que formatea el campo `fecha` al formato que `pandas` puede reconocer.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

url_datos = (
    "https://raw.githubusercontent.com/"
    "juanriera/master-queseria/master/"
    "datos/camembert.csv"
)

try:
    df = pd.read_csv(url_datos,
                     decimal = ",",
                     sep=';',
                     encoding='ISO-8859-1')
except Exception as e:
    print(f"Error al cargar el archivo: {e}")

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 211 entries, 0 to 210
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   fecha       211 non-null   object
1   fabricacion 211 non-null   int64
2   est         211 non-null   float64
3   mg          211 non-null   float64
4   ph          211 non-null   float64
5   cloruros    211 non-null   float64
6   coliformes  210 non-null   float64
dtypes: float64(5), int64(1), object(1)
```

memory usage: 11.7+ KB

Como `fecha` ha leído como object, la convertimos en fecha

```
df['fecha'] = pd.to_datetime(
    df['fecha'],
    format='%d/%m/%Y',
    errors='coerce' # Si hay una fecha inválida convierte a 'NaT'
                   # (Not a Time) en lugar de fallar.
)
```

Verificamos que el contenido del dataframe es correcto

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 211 entries, 0 to 210
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   fecha           211 non-null   datetime64[ns]
1   fabricacion     211 non-null   int64
2   est             211 non-null   float64
3   mg             211 non-null   float64
4   ph             211 non-null   float64
5   cloruros        211 non-null   float64
6   coliformes      210 non-null   float64
dtypes: datetime64[ns](1), float64(5), int64(1)
memory usage: 11.7 KB
```

Ahora `fecha` es `datetime64` y `pandas` lo reconoce como campo de fecha. Verificamos el dataframe:

```
df.head()
```

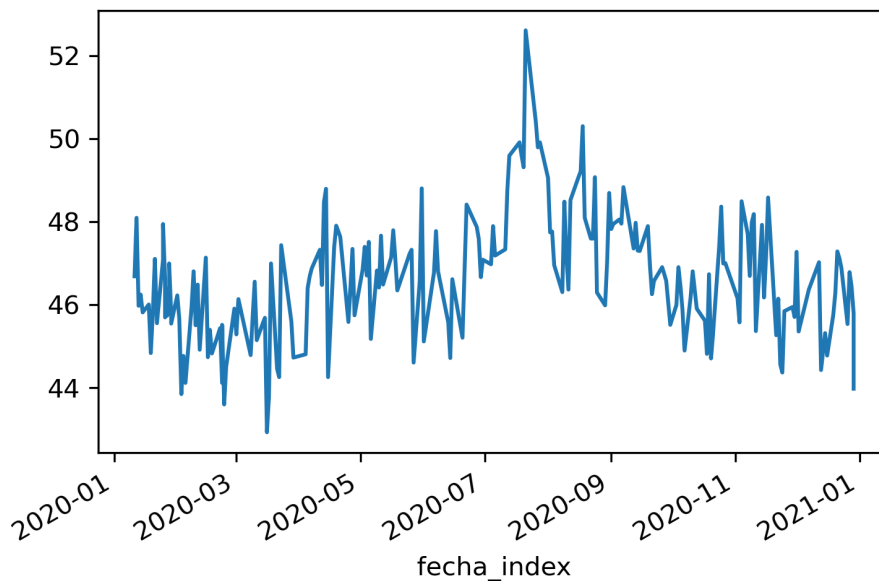
|   | fecha      | fabricacion | est   | mg   | ph   | cloruros | coliformes |
|---|------------|-------------|-------|------|------|----------|------------|
| 0 | 2020-01-11 | 1           | 46.68 | 24.0 | 4.85 | 1.50     | 80.0       |
| 1 | 2020-01-12 | 1           | 48.09 | 25.0 | 4.67 | 1.61     | 0.0        |
| 2 | 2020-01-13 | 1           | 45.97 | 24.0 | 4.71 | 1.48     | 9200.0     |
| 3 | 2020-01-14 | 1           | 46.24 | 23.5 | 4.78 | 1.69     | 20.0       |
| 4 | 2020-01-15 | 1           | 45.81 | 23.0 | 4.77 | 1.56     | 150.0      |

Para utilizar las funciones de series de `pandas` es necesario crear una columna `índice` y asignarla como `index` del dataframe; `pandas` utilizará el `index` para toda la gestión de las fechas de manera rápida y flexible.

```
df['fecha_index'] = pd.DatetimeIndex(df.fecha).normalize()
df.set_index('fecha_index', inplace=True)
df.sort_index(inplace=True)
```

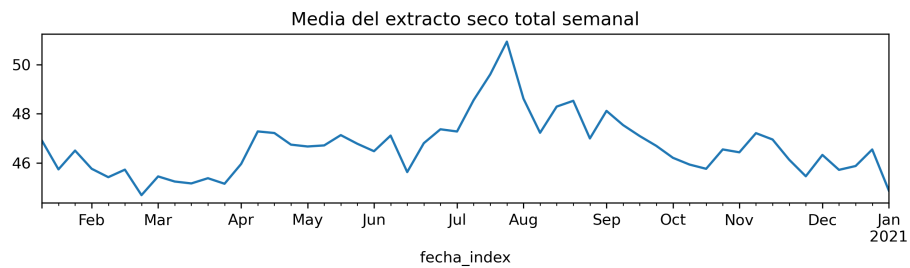
La función `.plot()` nos representa los valores de la columna en orden secuencial, y toma las fechas del `index` de forma automática, sin necesidad de explicitarlo:

```
df["est"].plot()
plt.show()
```



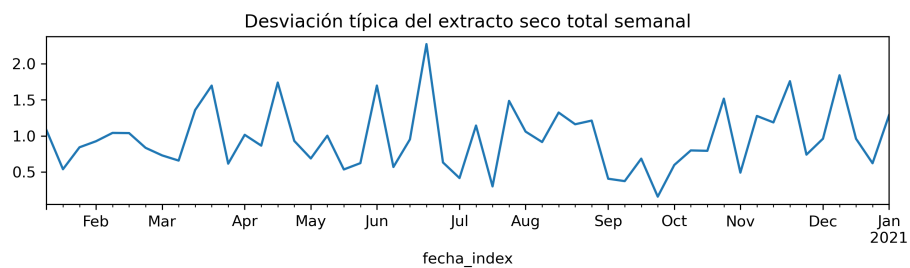
Mediante las funciones de **pandas**, podemos reformatear las fechas remuestreando para hacer las medias semanales del extracto seco total. La función `resample('W-MON')` formatea las fechas para que las semanas empiecen en lunes, como es el caso en Europa (en USA la semana empieza el domingo). Aprovechamos para mostrar una de las potencias de **Python**: podemos hacer que varios cálculos se hagan a continuación de los otros, simplemente enlazando las funciones, hasta el `.plot()`

```
plt.figure(figsize=(10.0, 2.0))
df["est"].resample('W-MON').mean().plot(
    title="Media del extracto seco total semanal")
plt.show()
```



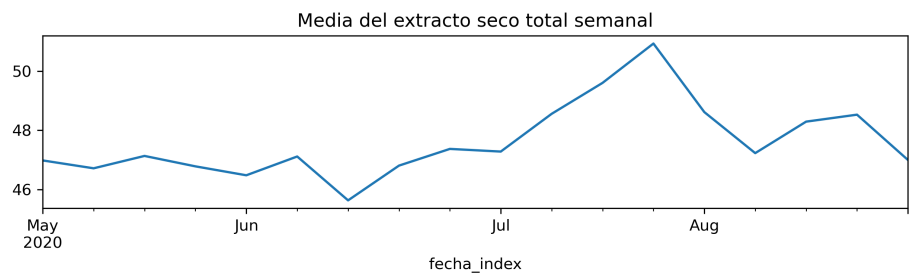
Veamos a continuación otras formas de formatear la serie sobre la marcha, pero esta vez representando la **desviación típica** en vez de la **media**.

```
plt.figure(figsize=(10.0, 2.0))
df["est"].resample('W-MON').std().plot(
    title="Desviación típica del extracto seco total semanal")
plt.show()
```



O podemos representar un período específico de la serie indicando a **pandas** los límites inferior y superior de las fechas que queremos.

```
plt.figure(figsize=(10.0, 2.0))
df["est"]["2020-05":"2020-08"].resample('W-MON').mean().plot(
    title="Media del extracto seco total semanal")
plt.show()
```



Por último, combinamos el gráfico de media y el de la desviación típica semanal en un único gráfico. Queda para el trabajo en clase ir buscando la explicación y comprensión de cada línea (sugerencia: copiar y pegar en Gemini el código y



solicitar explicación detallada)

```
import pandas as pd
import matplotlib.pyplot as plt

# --- 1. Preparación de los datos
#       (Calculando la Media y la Desviación Típica Semanal)
media_semanal = df["est"].resample('W-MON').mean()
dt_semanal = df["est"].resample('W-MON').std()

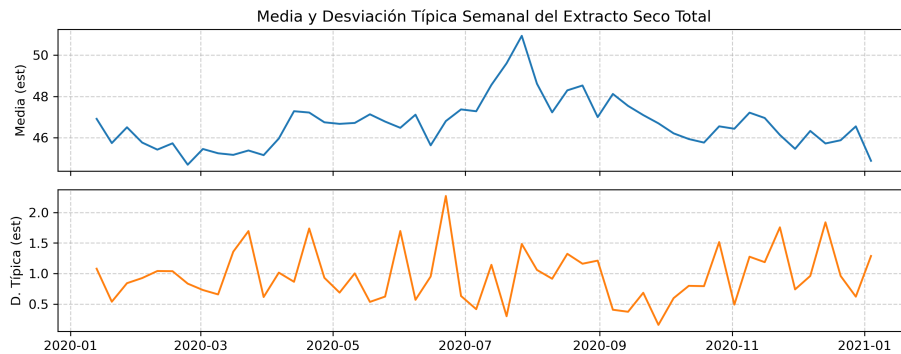
# --- 2. Creación de los Subgráficos (El paso clave)
# Creamos una figura (conjunto de gráficos) con 2 filas y 1 columna.
# El parámetro 'figsize=(10, 4)' controla el tamaño:
# 10 de ancho y 4 de alto total (como un gráfico normal).
# 'sharex=True' asegura que ambos gráficos compartan
# el mismo eje temporal.
fig, (ax1, ax2) = plt.subplots(
    nrows=2,
    ncols=1,
    figsize=(10, 4),
    sharex=True
)

# --- 3. Graficar la Media (Gráfico Superior) ---
ax1.plot(media_semanal, label="Media Semanal", color='tab:blue')
ax1.set_title("Media y Desviación Típica Semanal"
              " del Extracto Seco Total")
ax1.set_ylabel("Media (est)")
ax1.grid(True, linestyle='--', alpha=0.6)

# Quita las etiquetas de fecha/hora del gráfico superior
ax1.tick_params(labelbottom=False)

# --- 4. Graficar la Desviación Típica (Gráfico Inferior) ---
ax2.plot(dt_semanal,
         label="Desviación Típica Semanal",
         color='tab:orange')
ax2.set_ylabel("D. Típica (est)")
ax2.grid(True, linestyle='--', alpha=0.6)

# Ajusta el espacio entre los gráficos para que no se superpongan
fig.tight_layout()
plt.show()
```



### 6.13.3. Resumen

Los gráficos de series temporales son útiles para:

1. **Identificar Tendencias:**

- Una tendencia es una dirección general en la que los datos se mueven a lo largo del tiempo. Puede ser creciente, decreciente o constante.

2. **Detección de Estacionalidad:**

- La estacionalidad se refiere a patrones que se repiten en intervalos regulares de tiempo, como las ventas de productos estacionales.

3. **Identificar Ciclos:**

- Los ciclos son fluctuaciones que ocurren en intervalos no regulares y pueden deberse a factores económicos o de otra índole.

4. **Detección de Anomalías:**

- Los picos y caídas repentinas pueden indicar eventos inusuales o errores en los datos.

Los gráficos de series temporales son cruciales en diversas áreas:

1. **Economía y Finanzas:**

- Seguimiento de precios de acciones, tasas de interés y otros indicadores económicos.

2. **Ciencia y Tecnología:**

- Monitoreo de variables ambientales, datos meteorológicos y medidas científicas.

3. **Negocios:**

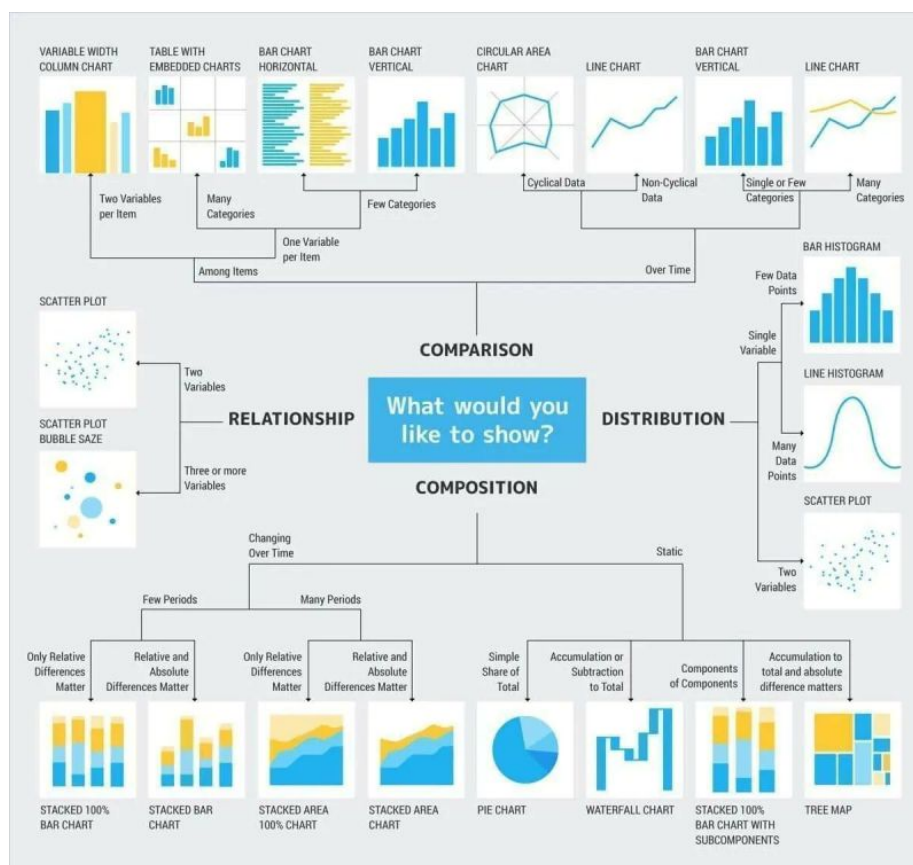
- Análisis de ventas, demanda de productos y desempeño empresarial a lo largo del tiempo.

Los gráficos de series temporales proporcionan una visión clara y concisa de cómo cambian los datos a lo largo del tiempo. Esta visualización es fundamental para el análisis predictivo, la toma de decisiones y la identificación de patrones y anomalías en los datos.

#### 6.13.4. Algunas sugerencias para la creación de gráficos

El enlace <https://tidyplots.org/> tiene muchos gráficos útiles que puede resultar interesante y formativo ver con atención.

También esta guía proporciona información sobre el destino adecuado de cada tipo de gráfico, facilitando la elección según el objetivo.



### INTUITIVENESS

Use intuitive colors. When choosing them, consider what associations they evoke. If possible, use colors that audience will associate with your data anyway.

### MODERATION

Use colors in moderation. For a simple dataset, a simple color is preferable. Use color as a strategic tool to highlight the important parts of your visual.

### CONSISTENCY

Use colors consistently. Change colors if you want your audience to feel the change for the specific reason, but never simply for the sake of novelty.

### CLARITY

Use colors to make the data easier to read. Make sure your audience will be able to distinguish between the items shown in the visualization.

### CLASSIFICATION

Don't use a gradient color palette for categories. And the other way round - different colors for same measurement.

### EXPLAINABILITY

Make sure to explain to your audience what exactly used colors mean. Remember to create a color key.

## COLOR SCHEMES

**Monochromatic** - the simplest formula for harmony is monochromatic. Consists of different shades of one hue. Not a good choice if we want to highlight something.

**Analogous** - this scheme is composed of colors that are next to each other on the wheel. Usually they match up pretty well, making elegant and clear look.

**Complementary** - uses two colors which are opposite on the color wheel. With saturated colors makes very vibrant look. Try to tone down colors to avoid overabundance, by adjusting saturation and lightness/darkness. Do not use with with saturated colors.

**Triadic** - uses three colors that evenly spaced on the color wheel. Makes that none of colors is dominant and quite vibrant look.

**Split Complementary** - variation of complementary scheme. Uses base color and two adjacent to its complementary color. Often this scheme is more pleasant to the eye than usual complementary scheme.

**Tetradic** - this scheme consists of four colors, two of them are complementary to other two. Choosing one color as dominant and the rest as accents, gives the best result.

**Remember!** Do not stick strictly to colors imposed by a scheme. These patterns are just starting points, you can create your own variations based on schemes above. Check also [palette.com](#)

## PRO TIPS

the color grey is the most important color in data visualization.

The use of color should always be an intentional decision. Never let your tool make this important decision for you!

After creating your visualization, close your eyes and then look back at it, taking note of where your eyes are drawn first. Is it where you want your audience to focus?

When picking colors consider the neurodivergent colors issue (color blindness). You can check: <https://projects.sussex.ac.uk/vis/visualizations/colors-in-cultures>

Remember about color deficiency issues (color blindness). You can check: <https://projects.sussex.ac.uk/vis/visualizations/colors-in-cultures>

## COLOR PALETTES

**QUANTITATIVE DATA - SEQUENTIAL OR DIVERGING COLORS**

Color is used to show variations in the data. The palette contains a sequence of colors that clearly indicate which values are larger or smaller than which other ones (sequential scale). It can also visualize the deviation of data values in one of two directions relative to a neutral midpoint (diverging scale). Diverging scale can be viewed as two merged sequential scales.

**Sequential scales**

Blues

Viridis

YlOrRd

**Diverging scales**

PiYG

RdYlGn

BrBG

**CATEGORICAL DATA - QUALITATIVE COLORS**

Color is used to separate areas into distinct categories. The palette should consist of colors as distinct from one another as possible. The maximum number of categories that can be displayed is about 12 (practically speaking, probability lower).

Colorblind

Dark

Accent

Pastels

All examples are available in Seaborn library. Check also <https://seaborn.pydata.org/>

**SCALES**

**Sequential**

Categorical

Ordinal

Interval

Ratio

**USAGE GUIDELINES**

**Sequential**

**Diverging**

**Categorical**

**Authors:**

Mateusz Kizynski

Paweł Wojciechowski

Artur Jędrasinski

Warsaw University of Technology  
Faculty of Mathematics and Information Science

## Capítulo 7

# Medidas centrales y medidas de dispersión: la media y la varianza.

### 7.1. Introducción

Al estudiar el diagrama de caja (*boxplot*) hemos visto que este gráfico se describe mediante los **cinco números**, que son:

- mínimo y máximo,
- primer y tercer cuartil (25 % y 75 %), y
- segundo cuartil (50 %).

De estos valores, el segundo cuartil, que se corresponde con la mitad de los valores, representa una estimación del centro de la distribución, y por eso lo llamamos **mediana**. La distancia entre el primer y tercer cuartil es lo que se conoce como **rango intercuartil** (se suele representar por sus siglas en inglés, **IQR**), y nos da una indicación de la **dispersión**: cuanto mayor mayor es la dispersión de nuestros valores, más alejados estarán del centro, y por lo tanto habrá mayor distancia entre el primer y el tercer cuartil.

Una de las mayores ventajas de la mediana y del rango intercuartil es que son estadísticos robustos, es decir, tiene una alta robustez a los valores atípicos. Como en el cálculo del rango intercuartil no se tienen en cuenta los valores extremos, su valor variará muy poco si aparecen nuevas observaciones atípicas (outliers). Como sus valores no dependen de la distribución de los datos, a estos estadísticos se los conoce como **no paramétricos**.

Existen otras estadísticos, llamados **paramétricos**, que, en determinadas con-

diciones, tienen ventajas frente a los no paramétricos. Los principales son la **media aritmética**, o simplemente, **media**, y la **varianza**.

## 7.2. Recursos utilizados

En este ejercicio usaremos diversos datos y gráficas realizados en Excel. Para facilitar el análisis, puede descargarse esta hoja Excel, con muchas tablas y gráficos, en el enlace a continuación.

Descargar hoja Excel

## 7.3. La media aritmética: un ejemplo de cálculo.

### 7.3.1. Definición

La media aritmética de un conjunto de valores es el valor central que se obtiene al dividir la suma de todos los valores por la cantidad de valores. Es una medida de tendencia central que proporciona un punto de referencia para el conjunto de datos.

### 7.3.2. Deducción de la Fórmula:

Supongamos que tenemos un conjunto de  $n$  valores numéricos:  $x_1, x_2, x_3, \dots, x_n$ .

1. **Suma de Todos los Valores:** Primero, sumamos todos los valores del conjunto. Matemáticamente, esto se expresa como:

$$S = x_1 + x_2 + x_3 + \dots + x_n$$

2. **Cantidad de Valores:** Luego, contamos cuántos valores hay en el conjunto. Este número es  $n$ .
3. **División de la Suma por la Cantidad de Valores:** Finalmente, dividimos la suma total  $S$  por la cantidad de valores  $n$  para obtener la media aritmética:

$$\text{Media Aritmética} = \frac{S}{n}$$

4. **Expresión General:** Sustituyendo la suma  $S$  en la fórmula, tenemos:

$$\text{Media Aritmética} = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n}$$

**7.3.3. Ejemplo:**

Supongamos que tenemos los siguientes valores: 5, 7, 9.

**1. Suma de los Valores:**

$$S = 5 + 7 + 9 = 21$$

**2. Cantidad de Valores:**

$$n = 3$$

**3. Cálculo de la Media Aritmética:**

$$\text{Media Aritmética} = \frac{21}{3} = 7$$

Entonces, la media aritmética de los valores 5, 7, 9 es 7.

La media de una muestra se representa habitualmente mediante el símbolo

$$\bar{x}$$

y, de una manera más formal, su valor se obtiene mediante la fórmula siguiente:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

El signo  $\sum$  se conoce como *sumatorio*, e indica que ese término consiste en la suma de los  $x$  valores desde el primero hasta el valor  $n$ . Expresado mediante una formulación matemática,

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

lo quiere decir: “la suma de todos los valores observados dividido entre el número de estos valores”.

La **media** es lo que conocemos como un *valor central*, ya que representa el centro de nuestro conjunto de números. Como es el centro de nuestro conjunto de datos, la suma de las distancias de todos los valores a este valor central es *cero*. Más adelante veremos la importancia de este hecho, al hablar de la dispersión y las formas de cálculo de la misma. Como hemos visto, la media de una **muestra** se representa como  $\bar{x}$ , mientras que la media de una **población** se representa con la letra griega *mu*:  $\mu$ . En ambos casos, el cálculo se realiza de forma idéntica.

Volvamos a nuestro ejemplo de la altura de un grupo de alumnos, para realizar los cálculos según el modelo que hemos descrito. En nuestro caso, la *altura media* de nuestros alumnos (la *media* de nuestro conjunto de números) se calcula como:

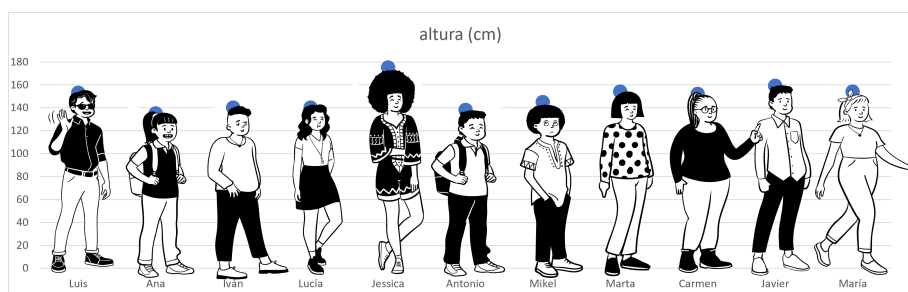
$$\bar{x} = \frac{153 + 135 + 140 + 140 + 175 + 138 + 145 + 154 + 152 + 159 + 154}{11} = 149,54$$

Utilicemos una hoja de cálculo para guardar nuestros valores.

|    | A       | B         | C | D |
|----|---------|-----------|---|---|
| 1  | nombre  | altura_cm |   |   |
| 2  | Luis    | 153       |   |   |
| 3  | Ana     | 135       |   |   |
| 4  | Iván    | 140       |   |   |
| 5  | Lucía   | 140       |   |   |
| 6  | Jessica | 175       |   |   |
| 7  | Antonio | 138       |   |   |
| 8  | Mikel   | 145       |   |   |
| 9  | Marta   | 154       |   |   |
| 10 | Carmen  | 152       |   |   |
| 11 | Javier  | 159       |   |   |
| 12 | María   | 154       |   |   |
| 13 |         |           |   |   |
| 14 |         |           |   |   |
| 15 |         |           |   |   |

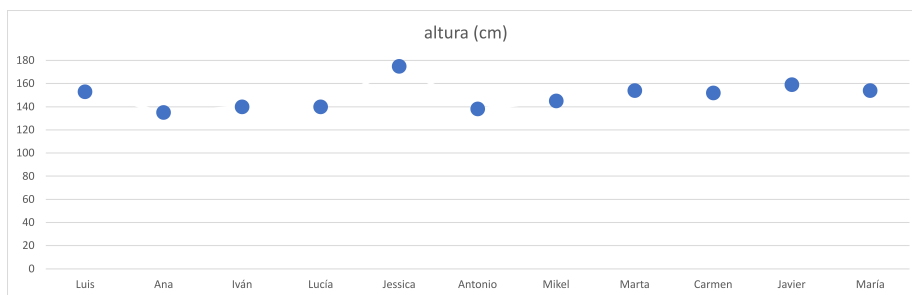
La fórmula para obtener la media en la hoja de cálculo, por ejemplo en la versión en español de *Microsoft Excel*, es `=PROMEDIO(. . .)`, donde los puntos suspensivos deben sustituirse por el rango a calcular. En nuestro ejemplo, introduciríamos la fórmula en la celda B13 como `=PROMEDIO(B2..B12)` (Para más detalles, verificar la hoja Excel adjunta).

Para representar más cómodamente nuestros valores, dibujamos un punto a la altura de cada alumno,

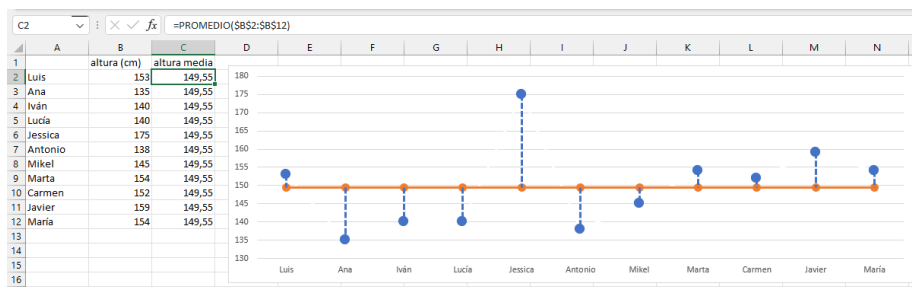


y eliminamos del gráfico los dibujos de nuestros alumnos; así hemos convertido nuestro dibujo en un *diagrama de puntos*:





Para representar la media, aunque la media es un valor único, necesitamos añadir una columna a la derecha de nuestros datos, que rotulamos en la fila 1, celda C como **altura media**, e introducimos en cada una de las celdas desde C2 hasta C12 la fórmula del promedio, con el valor de nuestro rango de datos (Verificar hoja de cálculo). A continuación, designamos nuestro rango de datos para hacer un gráfico de puntos, y hacemos un *zoom* en los valores de manera que el eje Y se escale mejor entre los valores mínimo y máximo. Por último, hacemos unos ajustes en el formato para dibujar las líneas verticales que nos representan la distancia de cada valor a la media.



Si verificamos el eje Y, veremos que en este gráfico hemos ajustado la escala respecto al gráfico anterior, situando el mínimo en 130. Esto permite visualizar las diferencias con mucha más claridad. Hemos representado la media  $\bar{x}$  como una línea, y hemos dibujado unas líneas que unen cada valor individual con la media, que se sitúa en el valor 149,55, tal como calculamos más arriba.

Hemos representado la media como una serie de puntos unidos por una línea amarilla. Representamos en azul nuestros valores, uniendo cada valor con la línea media mediante una línea de puntos vertical. A partir de ahora, por conveniencia, eliminaremos los puntos en la línea media, dejando sólo la línea.

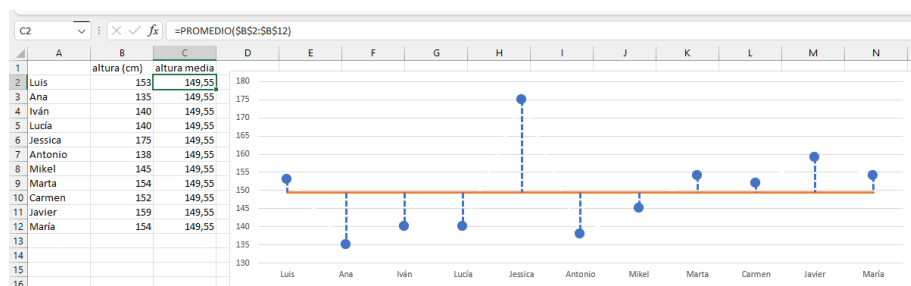


Figura 7.1: Hoja de cálculo con los valores y el gráfico de puntos

Esta línea azul de puntos representa la *distancia* de cada valor a la media. Usaremos esta distancia para calcular una **distancia media**, que será una medida de la dispersión de nuestros valores.

Hemos visto que para describir un conjunto de números, en nuestro ejemplo, las medidas de la altura de un grupo de estudiantes, existe un valor, la *media* de este conjunto, que nos describe el centro de los valores. En nuestro ejemplo, si nuestro grupo tuviese un solo niño, éste tendría 149,55 *cm* de altura.

¿Es suficiente con este valor para describir el conjunto de valores? Vamos a ver que no: diferentes conjuntos de valores pueden proporcionar el mismo *valor medio*, y sin embargo los grupos pueden ser muy diferentes.

Veamos un caso extremo. Comparemos dos grupos, uno formado por individuos iguales y otro formado por diez individuos iguales y uno distinto. Para ello usaremos nuestra hoja de cálculo:

| B13 <span>⌵</span> <span>✕</span> <span>✓</span> <i>fx</i> =PROMEDIO(B2:B12) |             |                        |                        |   |
|------------------------------------------------------------------------------|-------------|------------------------|------------------------|---|
|                                                                              | A           | B                      | C                      | D |
| 1                                                                            |             | altura media<br>caso 1 | altura media<br>caso 2 |   |
| 2                                                                            | 1           | 149,55                 | 143,5                  |   |
| 3                                                                            | 2           | 149,55                 | 143,5                  |   |
| 4                                                                            | 3           | 149,55                 | 143,5                  |   |
| 5                                                                            | 4           | 149,55                 | 143,5                  |   |
| 6                                                                            | 5           | 149,55                 | 143,5                  |   |
| 7                                                                            | 6           | 149,55                 | 143,5                  |   |
| 8                                                                            | 7           | 149,55                 | 143,5                  |   |
| 9                                                                            | 8           | 149,55                 | 143,5                  |   |
| 10                                                                           | 9           | 149,55                 | 143,5                  |   |
| 11                                                                           | 10          | 149,55                 | 143,5                  |   |
| 12                                                                           | 11          | 149,55                 | 210                    |   |
| 13                                                                           | Valor medio | 149,55                 | 149,55                 |   |
| 14                                                                           |             |                        |                        |   |

Figura 7.2: Dos grupos de valores con la misma media

¿Podemos describir adecuadamente los valores de la altura de cada uno de los grupos utilizando el valor medio? Parece evidente que no, ya que a partir de diferentes valores de altura estamos obteniendo el mismo valor medio. Sin embargo, uno de los grupos *es más alto* que el otro, si no fuera por un sólo individuo que aparentemente distorsiona el cálculo. Podríamos incluir nuestro grupo original, y veremos que los tres grupos son diferentes, aunque su valor medio es idéntico.

D13

✕

✓

$f_x$

=PROMEDIO(D2:D12)

|    | A           | B                      | C                      | D                               |
|----|-------------|------------------------|------------------------|---------------------------------|
| 1  |             | altura media<br>caso 1 | altura media<br>caso 2 | altura<br>(datos<br>originales) |
| 2  | 1           | 149,55                 | 143,5                  | 153                             |
| 3  | 2           | 149,55                 | 143,5                  | 135                             |
| 4  | 3           | 149,55                 | 143,5                  | 140                             |
| 5  | 4           | 149,55                 | 143,5                  | 140                             |
| 6  | 5           | 149,55                 | 143,5                  | 175                             |
| 7  | 6           | 149,55                 | 143,5                  | 138                             |
| 8  | 7           | 149,55                 | 143,5                  | 145                             |
| 9  | 8           | 149,55                 | 143,5                  | 154                             |
| 10 | 9           | 149,55                 | 143,5                  | 152                             |
| 11 | 10          | 149,55                 | 143,5                  | 159                             |
| 12 | 11          | 149,55                 | 210                    | 154                             |
| 13 | Valor medio | 149,55                 | 149,55                 | 149,55                          |

Figura 7.3: Tres grupos de valores con la misma media

Si nos ayudamos de un gráfico equivalente al que hemos utilizado antes, vemos estas diferencias con claridad:

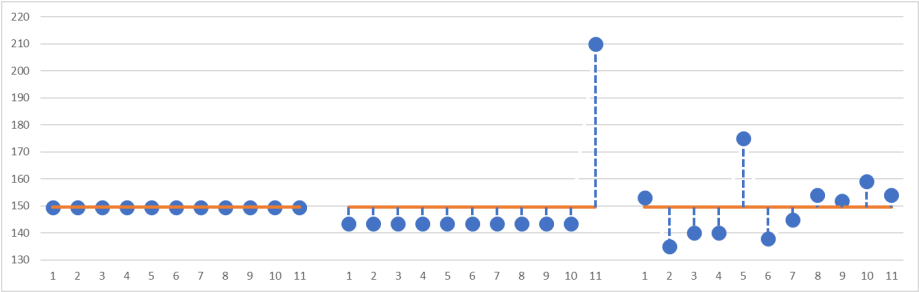


Figura 7.4: Gráfico de tres grupos de valores

Aunque el valor medio de estos tres grupos de datos es idéntico, parece claro que los tres grupos son muy distintos en su composición, y por lo tanto la *media* no es suficiente para describir con suficiente precisión cada uno de los grupos. Necesitamos un valor adicional, que nos indique de qué forma los valores se alejan del valor medio. Para ello, vamos a introducir un concepto nuevo: la *medida de la dispersión*.

## 7.4. Las medidas de dispersión: la varianza y la desviación típica

Como hemos visto en el apartado anterior, diferentes conjuntos de datos pueden tener el mismo valor medio y sin embargo ser muy diferentes. En la última gráfica que hemos visto, el primer grupo se caracteriza por tener todos sus valores idénticos; el segundo tiene todos sus valores idénticos menos uno, que está muy apartado del resto, y el tercero tiene todos sus valores diferentes.

Ahora que conocemos cómo calcular un valor resumen de un conjunto de datos, podríamos utilizar una medida semejante para describir de qué forma en cada caso los valores se separan de la media. Podríamos utilizar una *distancia media*: calculamos las diferencias entre cada valor y la media, y hacemos su promedio: esto debería darnos una indicación de la magnitud de la separación de los valores en cada uno de los tres grupos.

Usemos la hoja de cálculo para ello:

|     |                   |                     |                 |            |   |                     |                 |            |   |                           |                 |            |
|-----|-------------------|---------------------|-----------------|------------|---|---------------------|-----------------|------------|---|---------------------------|-----------------|------------|
| D14 | =PROMEDIO(D2:D12) |                     |                 |            |   |                     |                 |            |   |                           |                 |            |
|     | A                 | B                   | C               | D          | E | F                   | G               | H          | I | J                         | K               | L          |
|     |                   | altura media caso 1 | media del grupo | diferencia |   | altura media caso 2 | media del grupo | diferencia |   | altura (datos originales) | media del grupo | diferencia |
| 1   |                   |                     |                 |            |   |                     |                 |            |   |                           |                 |            |
| 2   | 1                 | 149,5454545         | 149,55          | 0,00       |   | 143,5               | 149,55          | -6,05      |   | 153                       | 149,55          | 3,45       |
| 3   | 2                 | 149,5454545         | 149,55          | 0,00       |   | 143,5               | 149,55          | -6,05      |   | 135                       | 149,55          | -14,55     |
| 4   | 3                 | 149,5454545         | 149,55          | 0,00       |   | 143,5               | 149,55          | -6,05      |   | 140                       | 149,55          | -9,55      |
| 5   | 4                 | 149,5454545         | 149,55          | 0,00       |   | 143,5               | 149,55          | -6,05      |   | 140                       | 149,55          | -9,55      |
| 6   | 5                 | 149,5454545         | 149,55          | 0,00       |   | 143,5               | 149,55          | -6,05      |   | 175                       | 149,55          | 25,45      |
| 7   | 6                 | 149,5454545         | 149,55          | 0,00       |   | 143,5               | 149,55          | -6,05      |   | 138                       | 149,55          | -11,55     |
| 8   | 7                 | 149,5454545         | 149,55          | 0,00       |   | 143,5               | 149,55          | -6,05      |   | 145                       | 149,55          | -4,55      |
| 9   | 8                 | 149,5454545         | 149,55          | 0,00       |   | 143,5               | 149,55          | -6,05      |   | 154                       | 149,55          | 4,45       |
| 10  | 9                 | 149,5454545         | 149,55          | 0,00       |   | 143,5               | 149,55          | -6,05      |   | 152                       | 149,55          | 2,45       |
| 11  | 10                | 149,5454545         | 149,55          | 0,00       |   | 143,5               | 149,55          | -6,05      |   | 159                       | 149,55          | 9,45       |
| 12  | 11                | 149,5454545         | 149,55          | 0,00       |   | 210                 | 149,55          | 60,45      |   | 154                       | 149,55          | 4,45       |
| 13  |                   |                     |                 |            |   |                     |                 |            |   |                           |                 |            |
| 14  |                   |                     | Promedio        | 0,00       |   |                     | Promedio        | 0,00       |   |                           | Promedio        | 0,00       |

Figura 7.5: Tres grupos de valores en la hoja de cálculo

Algo parece que no está funcionando aquí: el promedio de las diferencias es cero en los tres casos; no podemos usar este cálculo para calcular la dispersión. Pero esto es esperable: ya que la media es un valor central, como hemos visto antes, la suma de las diferencias de todos los valores respecto de su media debe ser forzosamente cero, y esto es lo que estamos obteniendo.

Para encontrar una solución, vamos a recurrir al viejo teorema de Pitágoras, que si recuerdas, nos dice que, en un triángulo rectángulo, el cuadrado de la hipotenusa es igual a la suma de los cuadrados de los catetos (una explicación gráfica muy divertida en el anexo ...):

$$h^2 = a^2 + b^2$$

Esta fórmula es la base del cálculo de la distancia entre dos puntos:

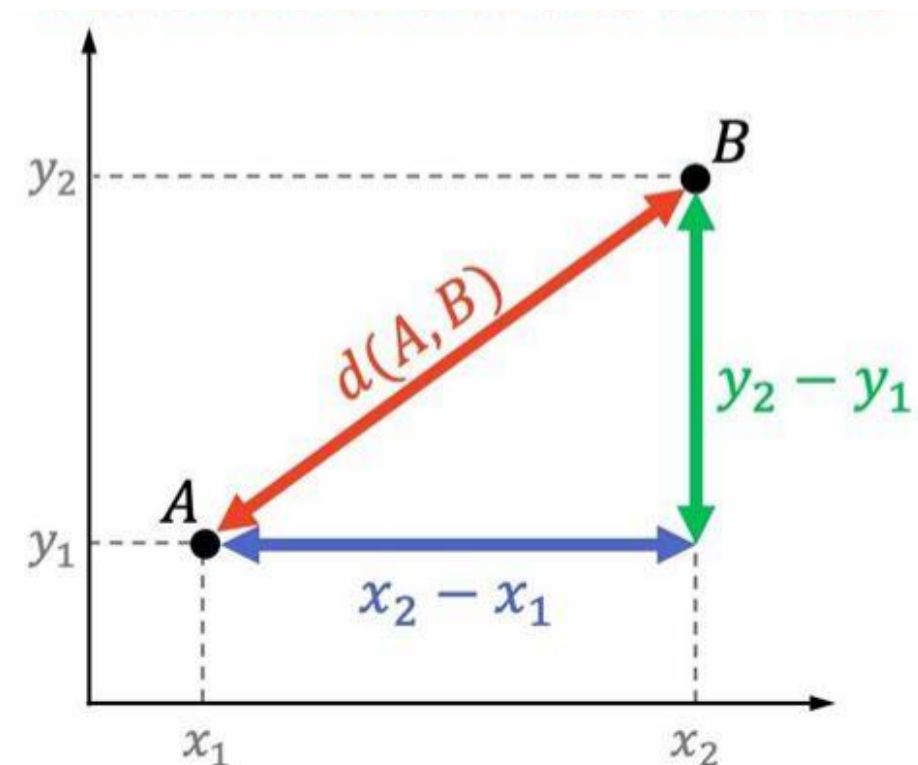


Figura 7.6: Distancia entre dos puntos

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Podemos adaptar esta fórmula al cálculo de nuestra distancia media. Como estamos calculando la distancia en una dimensión, sólo necesitamos la coordenada  $X$ . Si tenemos en cuenta un solo punto, esta distancia  $d$  sería:

$$(d \text{ del valor } 1 \text{ a la media})^2 = (x_1 - \bar{x})^2$$



Vamos a analizar con detalle esta tabla.

En la columna J tenemos nuestra población original de 11 alumnos, con las alturas que hemos medido. En la columna B hemos supuesto que todos los alumnos fuesen iguales, con la misma altura del valor medio de los datos originales. En la columna F hemos simulado otro grupo, con todos los valores iguales excepto uno, y con la misma media que los otros dos grupos.

A la derecha de cada columna de medias, tenemos la columna de diferencias (columnas D, H y L), y en la fila 13, nuestro primer intento de calcular una dispersión media; intento fallido, puesto que obteníamos el valor 0 para los tres grupos.

En la siguiente columna a la derecha, para los tres grupos (columnas E, I y M), hemos elevado al cuadrado la distancia de cada valor a la media, siguiendo los hallazgos que nos ha proporcionado el teorema de Pitágoras y la fórmula de la distancia entre dos puntos. En la fila 13 de estas columnas, calculamos el promedio de la distancia a la media al cuadrado: esta vez el resultado ya no es cero, sino que obtenemos el valor de la **varianza**, de acuerdo con la fórmula que hemos deducido más arriba. En la fila 14 (columnas B, F y J) utilizamos la fórmula de la hoja de cálculo para la **varianza poblacional** (más detalles posteriormente), y vemos que coincide exactamente con el promedio de las diferencias al cuadrado, tal como debe ser, ya que en eso consiste la fórmula que hemos deducido.

Por último, en la fila 15 calculamos la desviación típica de ambas formas, con la fórmula de la hoja de cálculo para la **desviación típica poblacional** (columnas B, F y J), que Excel llama **desviación estándar**, y como la raíz cuadrada del promedio calculado antes (columnas E, I y M). De nuevo, ambos valores coinciden exactamente, como esperamos.

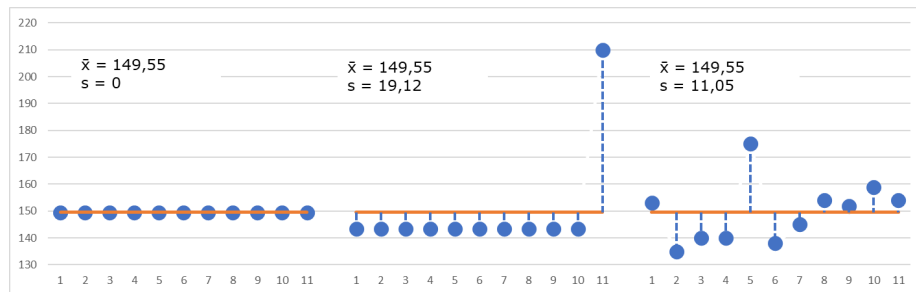


Figura 7.8: Gráfico con tres conjuntos de datos con la misma media y diferente desviación típica

Ahora sí tenemos una forma más completa de describir nuestro conjunto de valores. Aunque el valor medio es el mismo en los tres casos, la *dispersión* de los valores es muy distinta.



¿Son suficientes estos dos parámetros que hemos calculado para describir un conjunto de datos? La respuesta a esta pregunta es sí y no. La explicación es que, más allá de los valores numéricos que hemos obtenido, la visualización gráfica de los valores nos debe hacer reflexionar.

En el primer grupo, todos los valores son iguales a la media. La variación es cero. Son valores que hemos simulado en nuestra hoja de cálculo, pero difícilmente en el mundo real encontraremos una población en la que todos sus valores, en este caso, la altura de un grupo de alumnos, sean idénticos.

En el segundo grupo, todos los valores son idénticos, salvo uno, que se distancia mucho. ¿Debemos aceptar esto como bueno? En realidad, ¿es cierto que el valor medio de este grupo sea el mismo que el del primero? Para responder a esta pregunta debemos recurrir a nuestra experiencia, la estadística no nos da *fórmulas mágicas*. Pero, con un poco de sentido común, parece que el caso extremo que aparece en este grupo no es coherente con el resto de valores. Es lo que se llama un *valor anormal* o *extraño* (en inglés, *outlier*), y debe hacernos reflexionar sobre si el valor es correcto y realmente pertenece a esta población, o es un error de medida. O, simplemente, un valor que corresponde a otro grupo y que por error hemos situado en éste. La decisión de eliminar o no un valor anormal es una de las decisiones más complejas en estadística, que pueden tener una influencia enorme en la interpretación de los datos, y por lo tanto, hay que hacer con sumo cuidado. En este caso, extremo y artificial, el valor anormal debería ser eliminado, ya que, en realidad, todos los valores restantes son idénticos y más bajos que los del grupo 1. No tiene sentido lógico decir que sus medias son idénticas.

En el tercer grupo todos los valores son diferentes, y no podemos decir nada especial sobre sus valores individuales. Hay un valor que se destaca del resto, pero ¿podemos afirmar que es anormal? Seguramente, no con seguridad. De nuevo la experiencia debe indicarnos cómo proceder, aunque en este caso no tendría sentido eliminar este valor. En la situación real, todos conocemos a niños que han *pegado el estirón* antes que sus compañeros, y en algunos casos, pueden llegar a ser mucho más altos (o más bajos, si han tenido un retraso en este *estirón*) La experiencia nos dice que no es seguro que este valor sea realmente anormal, y por lo tanto, deberíamos conservarlo.

## 7.5. Las limitaciones de la media y la desviación típica

En ocasiones nos enfrentamos a conjuntos de datos con valores de media y desviación típica idénticos o muy parecidos, pero que en realidad son muy diferentes. Veamos un ejemplo, semejante a los que hemos visto hasta ahora.

| B14 $\downarrow$ : $\times$ $\checkmark$ $f_x$ =DESVEST.P(B2:B12) |             |                        |                                 |   |
|-------------------------------------------------------------------|-------------|------------------------|---------------------------------|---|
|                                                                   | A           | B                      | C                               | D |
| 1                                                                 |             | altura media<br>caso 2 | altura<br>(datos<br>originales) |   |
| 2                                                                 | 1           | 146,05                 | 153                             |   |
| 3                                                                 | 2           | 146,05                 | 135                             |   |
| 4                                                                 | 3           | 146,05                 | 140                             |   |
| 5                                                                 | 4           | 146,05                 | 140                             |   |
| 6                                                                 | 5           | 146,05                 | 175                             |   |
| 7                                                                 | 6           | 146,05                 | 138                             |   |
| 8                                                                 | 7           | 146,05                 | 145                             |   |
| 9                                                                 | 8           | 146,05                 | 154                             |   |
| 10                                                                | 9           | 146,05                 | 152                             |   |
| 11                                                                | 10          | 146,05                 | 159                             |   |
| 12                                                                | 11          | 184,48                 | 154                             |   |
| 13                                                                | Valor medio | 149,55                 | 149,55                          |   |
| 14                                                                | DesvT       | 11,05                  | 11,05                           |   |

Figura 7.9: Hoja de cálculo con dos conjuntos de datos diferentes, con la misma media y desviación típica

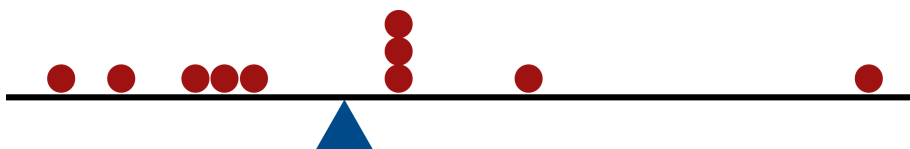


Figura 7.10: Diagrama de puntos de las alturas de los alumnos, con indicación del valor medio

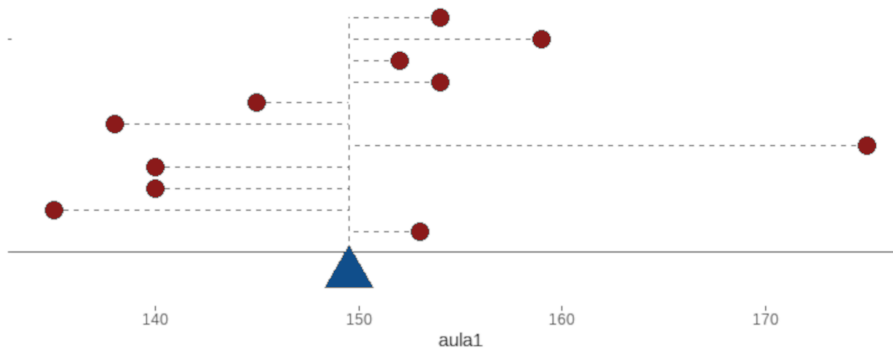


Figura 7.11: Diagrama de puntos indicando la variabilidad

En este caso, vemos que tanto la media como la desviación típica son idénticos, y sin embargo los datos son muy diferentes, tal como nos muestra el gráfico de dispersión que hemos estado utilizando:

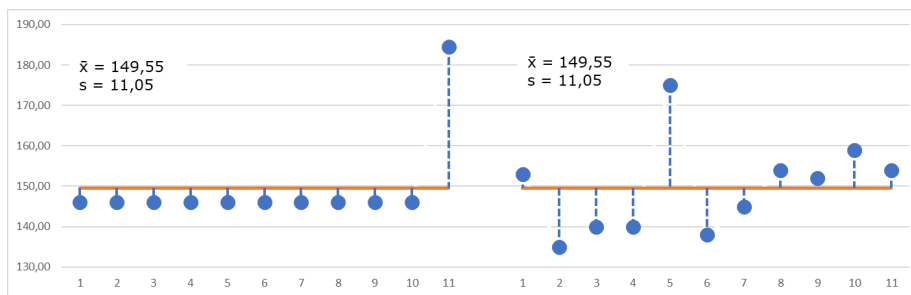


Figura 7.12: Gráfico con dos conjuntos de datos diferentes, con la misma media y desviación típica

La existencia de valores anormales o extremos muestra una de las debilidades de la media y la desviación típica como descriptores de un conjunto de datos: **ambos son muy sensibles a los casos extremos.**

Por eso, **solo deberíamos usar la media y la desviación típica cuando estamos razonablemente seguros de que los datos tienen una forma aproximadamente simétrica y equilibrada**, como la famosa **campana de Gauss** (distribución normal), que podemos relacionar con el boxplot, tal como vemos en el gráfico a continuación,

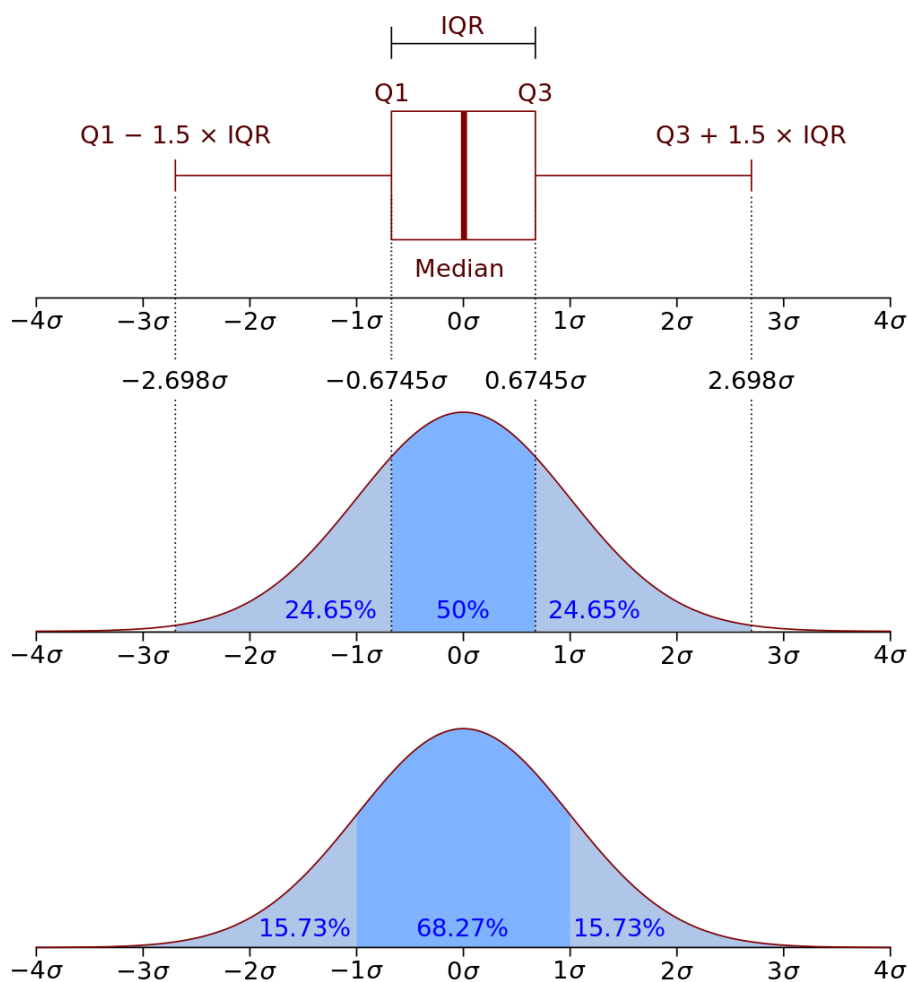


Figura 7.13: Campana de Gauss y boxplot mostrando las relaciones entre los elementos no paramétricos del boxplot y los elementos paramétricos de la distribución normal

En otros casos, especialmente cuando hay valores muy altos o muy bajos, **la mediana y el rango intercuartílico** (que ya vimos en el boxplot) son **estadísticos más robustos**, es decir, **más resistentes a los extremos y más seguros para describir el conjunto de datos**.

Esta diferencia entre estadísticos sensibles y robustos es clave para elegir bien qué medidas usar según el tipo de datos.

### 7.5.1. Ventajas e inconvenientes de las medidas paramétricas

#### Ventajas

1. Precisión y Sensibilidad: La media y la varianza son muy precisas y sensibles a todos los valores del conjunto de datos.
2. Propiedades Matemáticas: La media y la varianza tienen propiedades matemáticas deseables, como la facilidad para realizar operaciones algebraicas.
3. Distribución Normal: Son especialmente útiles si los datos siguen una distribución normal, ya que permiten aprovechar las propiedades de esta distribución.

#### Inconvenientes

1. Sensibilidad a Valores Atípicos: La media y la varianza pueden ser distorsionadas significativamente por valores atípicos.
2. Requieren Suposiciones: Su uso eficaz a menudo requiere que los datos sigan ciertas suposiciones, como la normalidad y la homogeneidad de la varianza.

### 7.5.2. Ventajas e inconvenientes de las medidas no paramétricas

#### Ventajas

1. Robustez: La mediana y el rango intercuartil son menos sensibles a valores atípicos y distribuciones asimétricas.
2. Flexibilidad: No requieren suposiciones fuertes sobre la distribución de los datos, lo que las hace útiles para una amplia variedad de distribuciones.
3. Resumir Datos: Son excelentes para resumir datos en situaciones en las que los valores extremos podrían distorsionar la interpretación.

#### Inconvenientes

1. Menor Sensibilidad: La mediana y el rango intercuartil no utilizan toda la información de los datos y pueden ser menos sensibles a cambios en los datos.
2. Menor Precisión en Ciertos Contextos: En situaciones donde los datos siguen una distribución normal, las medidas no paramétricas pueden ser menos precisas.

Las medidas paramétricas son útiles para datos que siguen suposiciones específicas, como la normalidad, y son precisas y sensibles, pero pueden ser distorsionadas por valores atípicos. Las medidas no paramétricas son robustas y flexibles, ideales para distribuciones no normales y resistentes a valores atípicos, aunque pueden perder sensibilidad y precisión en ciertos contextos.

## 7.6. Resumen

Las medidas paramétricas, como la media y la varianza, y las no paramétricas, como la mediana y el rango intercuartil, proporcionan información sobre el centro y la dispersión de la distribución de los datos, y tienen diferentes ventajas e inconvenientes según el contexto y los datos con los que se trabaja.

En el capítulo siguiente aplicaremos estos conceptos al análisis de un conjunto de datos de fabricación de queso. Veremos también algunas formas de detectar visualmente los valores extremos que pueden afectar a la media, usando histogramas y boxplots, sin necesidad de cálculos complicados.

## Capítulo 8

# Práctica 1. Análisis exploratorio de una fabricación de queso camembert usando Python y Google Colab.

En este ejercicio usaremos un fichero o archivo de datos en formato CSV que habremos exportado previamente desde nuestra hoja de cálculo. Más adelante aprenderemos cómo leer directamente los datos de nuestras hojas Excel.

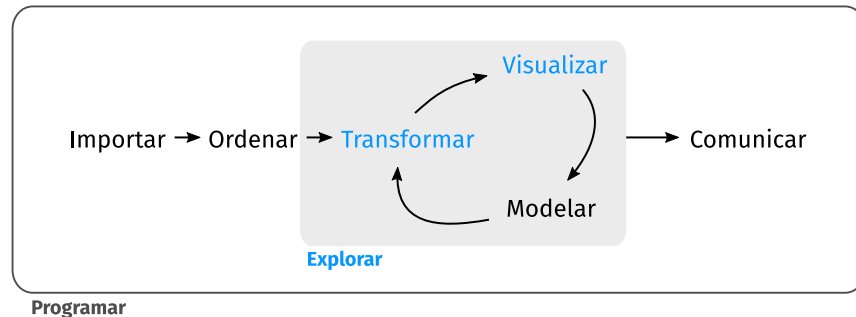
Abrir este cuaderno en Google Colab

Descargar los datos de ejemplo `camembert.csv` usados en este cuaderno

### 8.1. Flujo de trabajo

Coo vimos en el capítulo 2, es muy conveniente establecer un flujo de trabajo regular, de forma que cada etapa del análisis sea un paso que se apoye en los anteriores.

Este flujo de trabajo debe cubrir las etapas que veíamos en ese capítulo:



Vamos a ir desarrollando este flujo paso a paso.

### 8.1.1. 1. Importar los datos

En este cuaderno vamos a cargar los datos desde GitHub; estos datos pueden ser descargados en el ordenador o Google Drive de cada uno, junto con el cuaderno, en los enlaces facilitados al principio. Una vez en Drive, puede leerse la tabla mediante el método que vimos al final del capítulo anterior.

```

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

url_datos = (
    "https://raw.githubusercontent.com/"
    "juanriera/master-queseria/master/"
    "datos/camembert.csv"
)

try:
    df = pd.read_csv(url_datos,
                     decimal = ",",
                     sep=';',
                     encoding='ISO-8859-1')
    print("¡Archivo ", url_datos, " cargado correctamente!")
    print(f"Dimensiones de los datos: {df.shape}")
except Exception as e:
    print(f"Error al cargar el archivo: {e}")

```

```

¡Archivo https://raw.githubusercontent.com/juanriera/master-queseria/master/datos/camembert.csv cargado correctamente!
Dimensiones de los datos: (211, 7)

```

Vemos el `dataframe` que hemos leído, mediante la función `.head()`, que nos muestra las cinco primeras líneas.



```
df.head()
```

|   | fecha      | fabricacion | est   | mg   | ph   | cloruros | coliformes |
|---|------------|-------------|-------|------|------|----------|------------|
| 0 | 11/01/2020 | 1           | 46.68 | 24.0 | 4.85 | 1.50     | 80.0       |
| 1 | 12/01/2020 | 1           | 48.09 | 25.0 | 4.67 | 1.61     | 0.0        |
| 2 | 13/01/2020 | 1           | 45.97 | 24.0 | 4.71 | 1.48     | 9200.0     |
| 3 | 14/01/2020 | 1           | 46.24 | 23.5 | 4.78 | 1.69     | 20.0       |
| 4 | 15/01/2020 | 1           | 45.81 | 23.0 | 4.77 | 1.56     | 150.0      |

Verificamos la estructura interna de nuestro `dataframe` y el tipo de los datos (entero, numérico, carácter...) mediante la función `.info()`

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 211 entries, 0 to 210
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   fecha           211 non-null   object
1   fabricacion     211 non-null   int64
2   est             211 non-null   float64
3   mg              211 non-null   float64
4   ph              211 non-null   float64
5   cloruros        211 non-null   float64
6   coliformes      210 non-null   float64
dtypes: float64(5), int64(1), object(1)
memory usage: 11.7+ KB
```

Como resultado de la importación el csv, la `fecha` aparece como columna `object`, que es un tipo genérico de datos que incluye el tipo `character`, que es en realidad como `python` considera esta columna; nos interesa convertirla en un formato de fecha que `pandas` pueda reconocer.

```
df['fecha'] = pd.to_datetime(
    df['fecha'],
    format='%d/%m/%Y',
    errors='coerce' # Si hay una fecha inválida, la convierte a 'NaT'
                    # (Not a Time) en lugar de fallar.
)
```

Recuerda que si no comprendes bien el código, puedes copiarlo y pegarlo en Google Gemini y pedir a Gemini una explicación detallada de lo que hace. Puedes también preguntar a Gemini que te explique bien la función `.to_datetime()` con los diferentes formatos de conversión de fechas.

Verificamos la conversión:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 211 entries, 0 to 210
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   fecha            211 non-null   datetime64[ns]
1   fabricacion      211 non-null   int64
2   est              211 non-null   float64
3   mg               211 non-null   float64
4   ph               211 non-null   float64
5   cloruros         211 non-null   float64
6   coliformes       210 non-null   float64
dtypes: datetime64[ns](1), float64(5), int64(1)
memory usage: 11.7 KB
```

Ahora la fecha tiene formato `datetime64` que es el formato de fecha que reconoce `pandas`.

### 8.1.2. 2. Ordenar/limpiar los datos

La tabla de datos que hemos leído ya está en un formato *tidy*: una línea por observación y una columna por medida, como comprobamos inspeccionando las primeras líneas de código:

```
df.head()
```

|   | fecha      | fabricacion | est   | mg   | ph   | cloruros | coliformes |
|---|------------|-------------|-------|------|------|----------|------------|
| 0 | 2020-01-11 | 1           | 46.68 | 24.0 | 4.85 | 1.50     | 80.0       |
| 1 | 2020-01-12 | 1           | 48.09 | 25.0 | 4.67 | 1.61     | 0.0        |
| 2 | 2020-01-13 | 1           | 45.97 | 24.0 | 4.71 | 1.48     | 9200.0     |
| 3 | 2020-01-14 | 1           | 46.24 | 23.5 | 4.78 | 1.69     | 20.0       |
| 4 | 2020-01-15 | 1           | 45.81 | 23.0 | 4.77 | 1.56     | 150.0      |

Vamos entonces a verificar que los valores que hemos leído son correctos. ¿Qué tenemos que buscar?

- Valores mal introducidos que `Python` no tiene por qué saber que son erróneos (por ejemplo, un pH de 51,5 en vez de 5,15)
- Valores anormalmente altos o bajos, que pueden indicar errores de medida, de toma de muestras o de introducción de datos (por ejemplo, un pH de leche de 30, que puede indicar una lectura de temperatura introducida fuera de sitio)
- Valores erróneos: por ejemplo, valores negativos en cualquier medida

- Una distribución de los datos anormal:
  - por ejemplo, el pH de un queso debería tener la mayor frecuencia entorno al pH objetivo, y desviaciones con una frecuencia cada vez más baja a medida que se alejan del valor central
  - en cambio, los recuentos de bacterias coliformes deberían tener la mayor frecuencia en los valores cero o muy bajos, y la distribución no debería ser simétrica

En general, cualquier cosa que nos llame la atención en los datos y que debemos explicar.

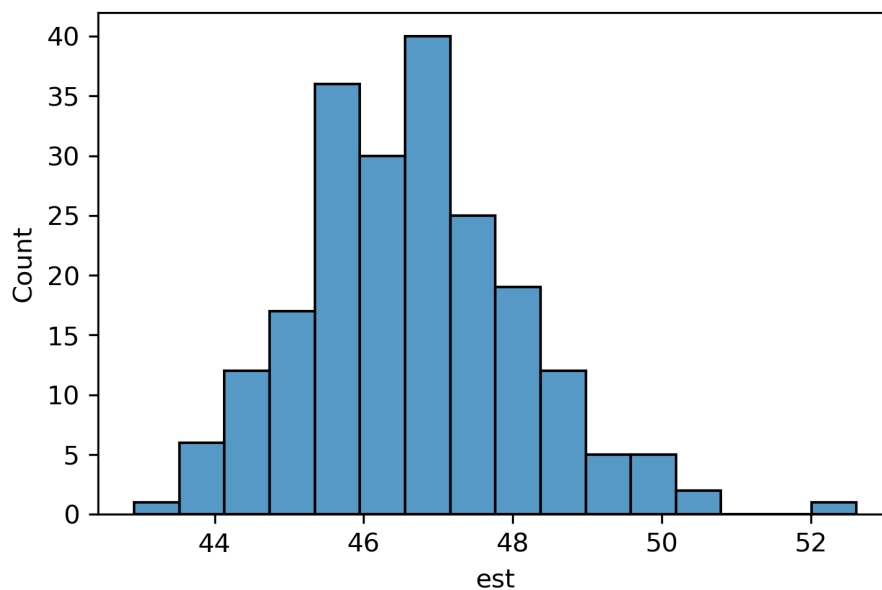
Para hacer esta verificación, analizamos cada variable (columna) por separado.

### 8.1.3. 3. Visualizar: Análisis gráfico

Comenzamos el análisis estudiando la variable o columna **est**, que almacena los valores de **extracto seco total**. Utilizamos las herramientas gráficas que conocemos: **histograma** y **diagrama de caja**. Por el momento no nos preocupamos de rotular el gráfico, estamos simplemente explorando de forma rápida.

#### El histograma

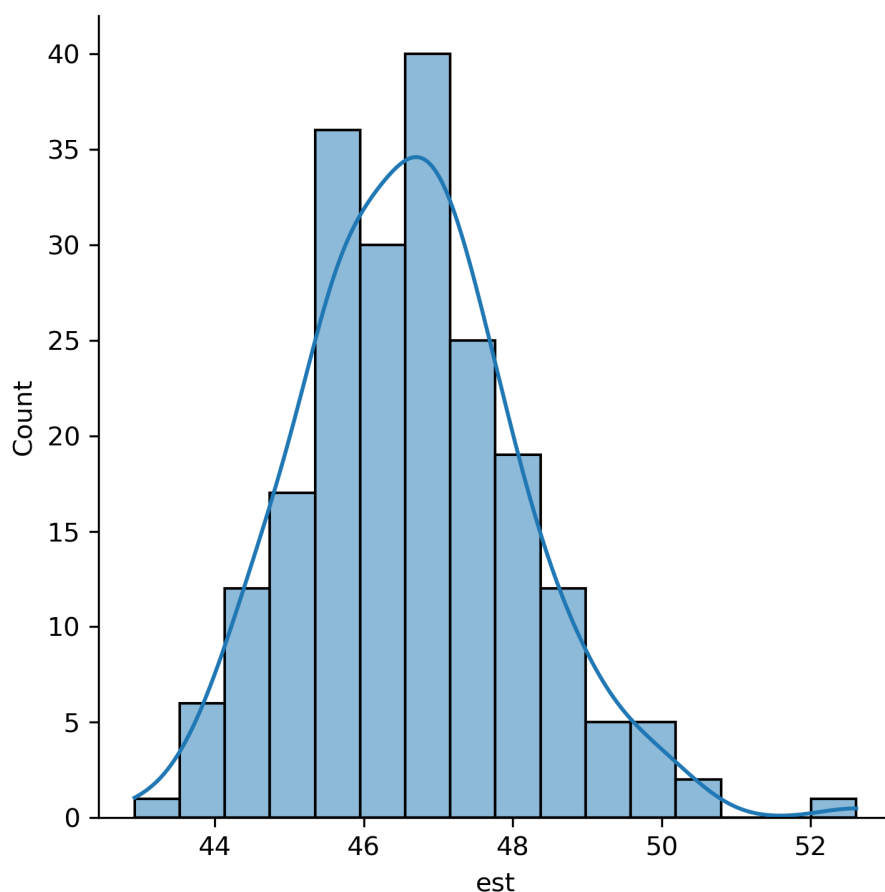
```
sns.histplot(df["est"])
plt.show()
```



También podemos utilizar **seaborn**, que nos permite incluir la curva de densidad de forma muy sencilla.

```
sns.displot(df["est"], kde = True)

plt.show()
```



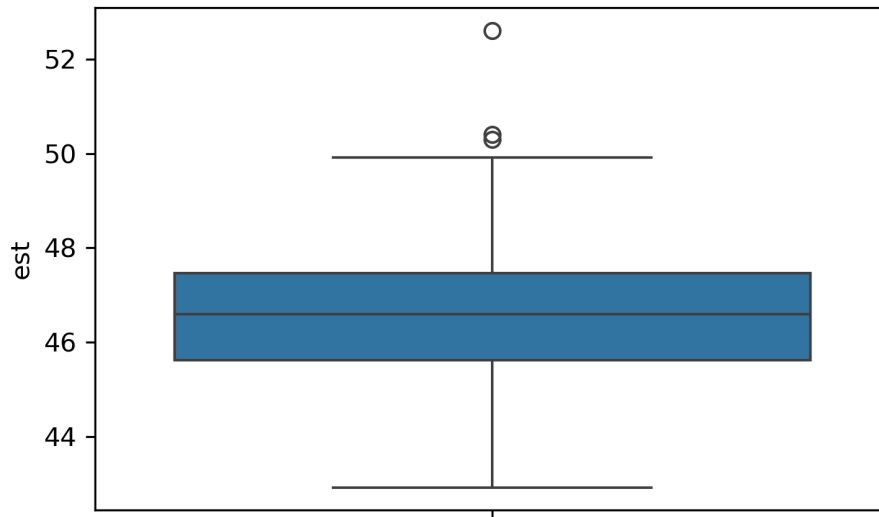
El histograma nos muestra la mayor frecuencia en torno a los valores centrales, la distribución es aproximadamente simétrica, y parece tener algún valor anormalmente alto (52). ¿Debería preocuparnos este valor? ¿Es razonable o puede ser un error?

La curva de densidad muestra algo que debe resultarnos sospechoso: hay una subida en torno a 52. Una curva de distribución que fuese semejante a una **campana de Gauss** (distribución *normal*) no debería mostrar uncambio en sus extremos, sino amortiguarse suavemente hasta cero.

Continuamos explorando con el *boxplot*.

El `boxplot`

```
sns.boxplot(df['est'])  
  
plt.show()
```



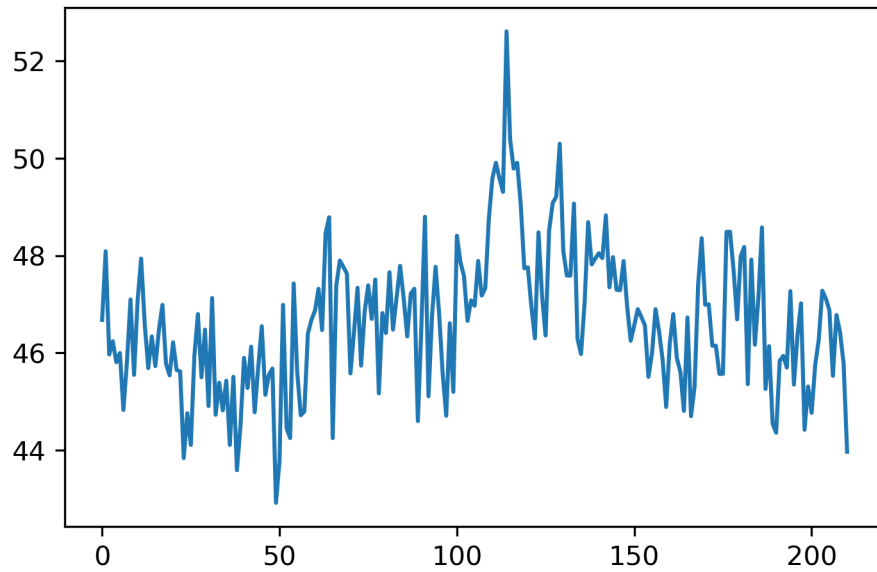
El *boxplot* nos indica que no sólo el valor ligeramente por encima de 52 parece anormal, también lo parecen algunos valores por encima de 50.

### Los gráficos de *series temporales*

Ya sabemos que parece haber algún valor atípico, anormalmente alto, en nuestros datos. ¿Cómo podemos profundizar un poco más y decidir si debemos excluir estos valores analíticos?

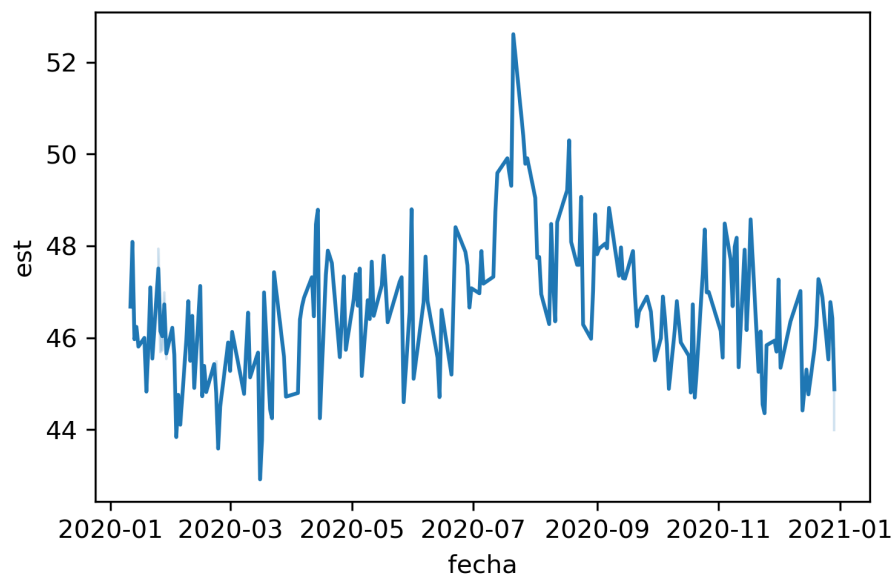
Sabemos que estos resultados se han obtenido a lo largo de un año. ¿Puede haber pasado algo en algún momento que haya hecho que algunos resultados se hayan desviado? Veamos cuál ha sido la evolución anual del extracto seco total de nuestra fabricación, descrito por la columna o variable `est`.

```
df['est'].plot()  
plt.show()
```



El gráfico representa los valores individuales de manera secuencial, y el eje X muestra el número de orden del valor. Sería mucho más útil si este eje estuviera rotulado con fechas. **Seaborn** puede ayudarnos con esto.

```
sns.lineplot(data= df, x='fecha', y='est')  
plt.plot()
```



Como hemos convertido la columna `fecha` a un formato de fecha, `seaborn` la reconoce y rotula el eje correctamente.

Los valores anormales se han producido en el verano, en torno a julio-agosto. En esta época ha habido una subida de los valores de extracto seco total. A la vista del gráfico, los valores por encima de 50, aunque extremos, parece que están dentro de un momento en el que el extracto seco se había desviado hacia arriba. Esto sitúa a esos valores en un contexto temporal en el que probablemente puede que no fueran tan anormales. ¿Qué quiere decir esto? Pues que si un valor de 52 es *atípico* frente al conjunto de datos completo, es posible que no lo sea si los valores que lo rodean también lo eran y se consideran sólo esos valores de contexto.

Vamos a explorar esto un poco más y mostrar los datos por meses, para ver si considerados mes a mes, el valor de 52 sigue pareciendo anormal, y vamos a utilizar también `pandas` para hacer los gráficos de series.

### El *boxplot* por categorías

Para representar los datos por mes con `pandas`, tenemos que crear una columna índice con la fecha:

```
df['fecha_index'] = pd.DatetimeIndex(df.fecha).normalize()
df.set_index('fecha_index', inplace=True)
df.sort_index(inplace=True)
```

La función `.normalize()` elimina de los campos de fecha/tiempo la parte referida a minutos y segundos, aquí se ha usado como ejemplo, porque nuestras fechas no tienen ese nivel de detalle.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 211 entries, 2020-01-11 to 2020-12-29
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   fecha            211 non-null   datetime64[ns]
1   fabricacion      211 non-null   int64
2   est              211 non-null   float64
3   mg               211 non-null   float64
4   ph               211 non-null   float64
5   cloruros         211 non-null   float64
6   coliformes       210 non-null   float64
dtypes: datetime64[ns](1), float64(5), int64(1)
memory usage: 13.2 KB
```

Aunque el índice `fecha_index` que hemos creado, que tiene tipo `DateTimeIndex`, no aparece entre las columnas, sí está reseñado en el encabezado de la des-

cripción, y muestra que existen 211 entradas desde el 11/1/2020 hasta el 29/12/2020. Usamos este índice para crear una columna con el mes, usando una de las muchas funciones de **pandas** asociadas a una columna de fecha. En este caso, `.month()` extrae el mes en formato numérico:

```
df['mes'] = df.index.month
```

Veamos la columna en nuestro *DataFrame*:

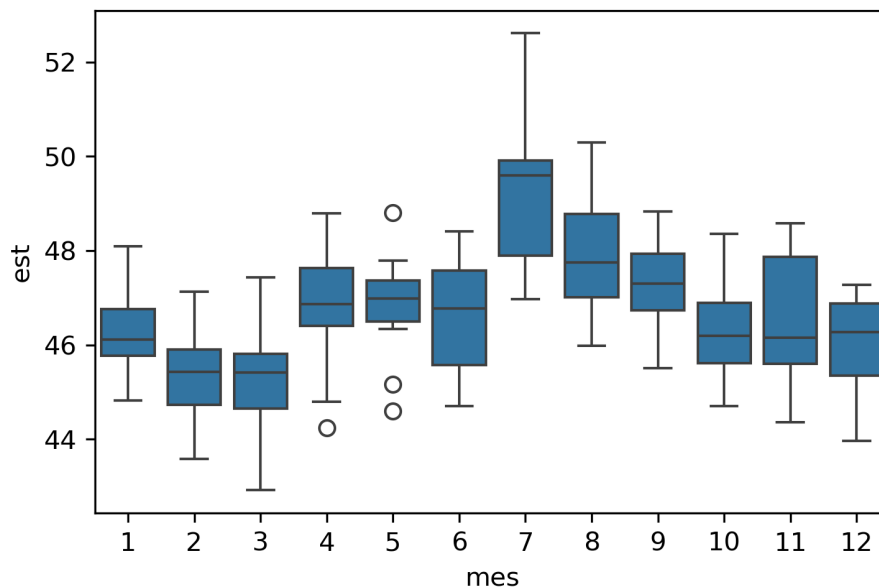
```
df.head()
```

|             | fecha      | fabricacion | est   | mg   | ph   | cloruros | coliformes | mes |
|-------------|------------|-------------|-------|------|------|----------|------------|-----|
| fecha_index |            |             |       |      |      |          |            |     |
| 2020-01-11  | 2020-01-11 | 1           | 46.68 | 24.0 | 4.85 | 1.50     | 80.0       | 1   |
| 2020-01-12  | 2020-01-12 | 1           | 48.09 | 25.0 | 4.67 | 1.61     | 0.0        | 1   |
| 2020-01-13  | 2020-01-13 | 1           | 45.97 | 24.0 | 4.71 | 1.48     | 9200.0     | 1   |
| 2020-01-14  | 2020-01-14 | 1           | 46.24 | 23.5 | 4.78 | 1.69     | 20.0       | 1   |
| 2020-01-15  | 2020-01-15 | 1           | 45.81 | 23.0 | 4.77 | 1.56     | 150.0      | 1   |

Ahora podemos agrupar nuestros valores por meses, y comprobar los valores anormales *dentro de cada mes*. Para ello, simplemente indicamos a **seaborn** la variable que va a estar en el eje X:

```
sns.boxplot(x='mes', y='est', data=df)
```

```
plt.show()
```

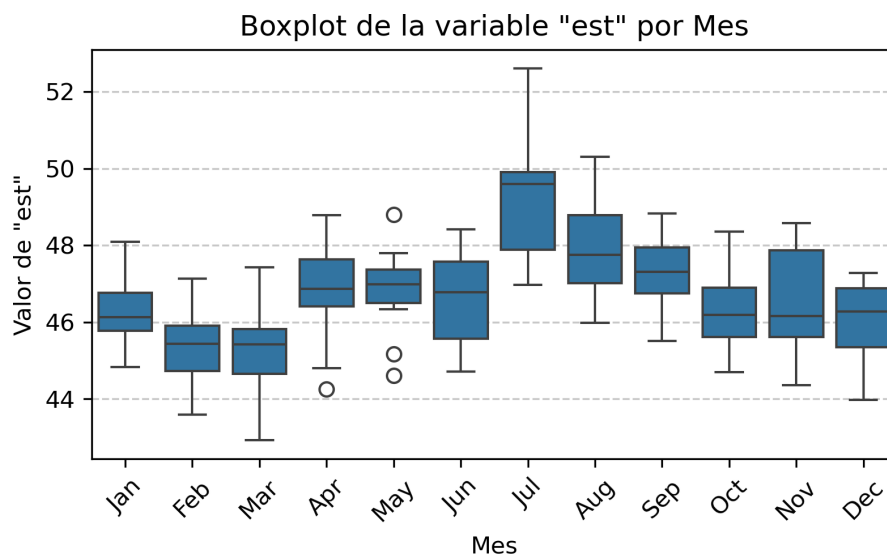




En vez de usar el número para el mes, podemos usar el código de letras abreviado. En este caso, **seaborn** utiliza la abreviatura en inglés, pero podríamos cambiar la codificación para que lo represente en español con la instrucción **locale** (queda como ejercicio personal explorar el uso y opciones de esta instrucción).

Aprovechamos para personalizar un poco el gráfico, y mostramos cómo pueden añadirse diferentes opciones de formato; cuando **Python** encuentra opciones de formato, el gráfico no se mostrará hasta que ejecutemos la instrucción **.show()**

```
df['mes_abreviado'] = df.index.strftime('%b')
sns.boxplot(x='mes_abreviado', y='est', data=df)
# Opcional: Personalizar el gráfico
plt.title('Boxplot de la variable "est" por Mes')
plt.xlabel('Mes')
plt.ylabel('Valor de "est"')
# Añadir una cuadrícula para mejor lectura
plt.grid(axis='y', linestyle='--', alpha=0.7)
# Rotar las etiquetas del eje X si son muchas
plt.xticks(rotation=45)
# Ajusta automáticamente los parámetros de la subtrama
# para un diseño ajustado
plt.tight_layout()
plt.show()
```

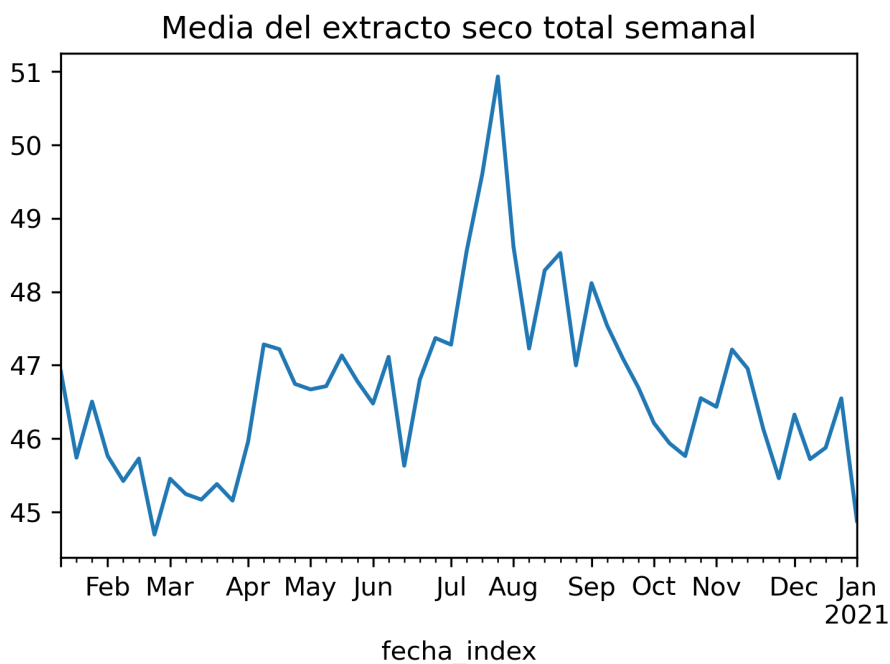


¡Ahora nuestra visión de los datos ha cambiado! Resulta que los valores analíticos de extracto seco han ido variando a lo largo del año, y el valor supuestamente anómalo de 52 ya no lo es considerando los datos mes a mes, porque *todos los valores de julio* han sido muy altos, y 52 no parece extremo en este contexto.

En cambio, vemos que en mayo sí ha habido valores anómalos en relación a los obtenidos en el mes. ¿Deben preocuparnos estos valores de mayo? Probablemente no, porque están dentro del conjunto de los valores anuales; sin embargo, es conveniente profundizar en la razón por la que la dispersión de los valores de mayo, mostrada por el rango intercuartil, es muy baja, y sin embargo en junio ha aumentado mucho. Los valores de julio sí parecen excesivamente altos casi durante todo el mes, la mediana de los valores ha sido la más alta del año.

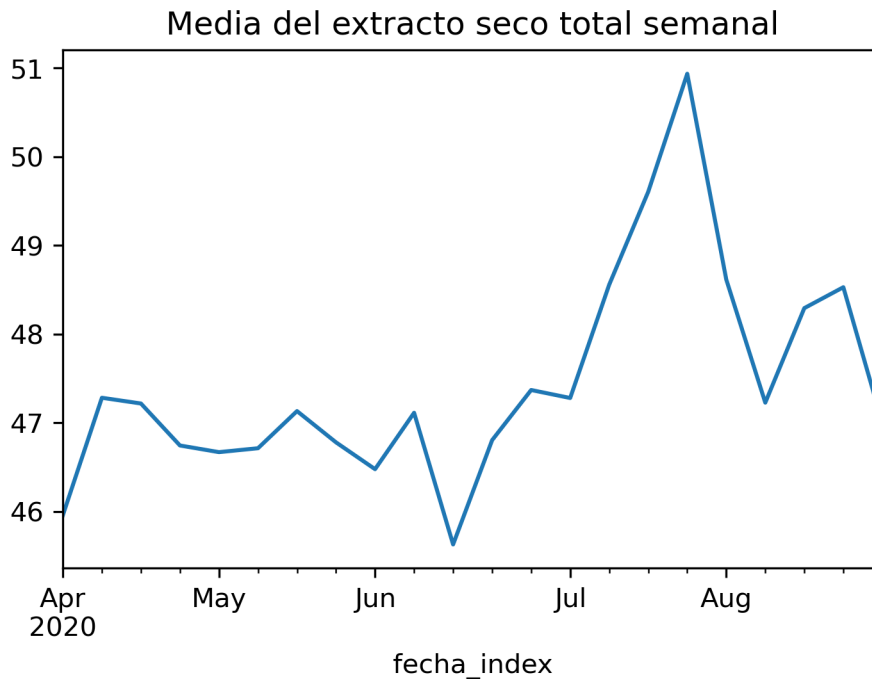
Detallemos un poco más, viendo los valores semanales:

```
df["est"].resample('W-MON').mean().plot(
    title="Media del extracto seco total semanal")
plt.show()
```



Hagamos un zoom en los valores de abril a agosto utilizando las opciones de filtro de fechas de **pandas**:

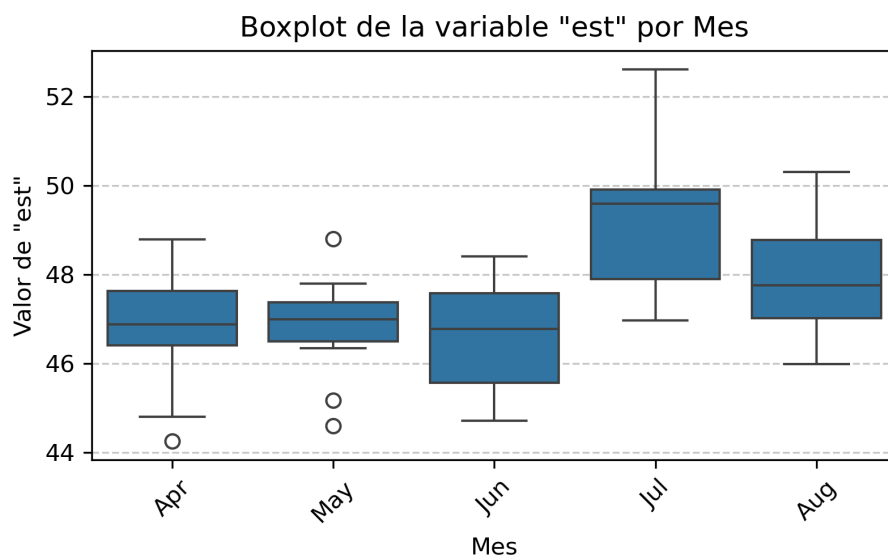
```
df["est"]["2020-04":"2020-08"].resample('W-MON').mean().plot(
    title="Media del extracto seco total semanal")
plt.show()
```



Efectivamente, las semanas de julio han tenido valores crecientes de extracto seco, y esto no se ha corregido hasta el mes de agosto. En realidad, a partir de octubre se ha bajado el extracto seco de nuevo a valores más cercanos a los del primer semestre.

La opción de filtro de fecha puede aplicarse también a las funciones de **seaborn**, veamos un ejemplo que muestra con claridad el aumento de extracto seco en julio frente a los meses anteriores.

```
df['mes_abreviado'] = df.index.strftime('%b')
sns.boxplot(x='mes_abreviado', y='est', data=df["2020-04":"2020-08"])
# Opcional: Personalizar el gráfico
plt.title('Boxplot de la variable "est" por Mes')
plt.xlabel('Mes')
plt.ylabel('Valor de "est"')
plt.grid(axis='y', linestyle='--', alpha=0.7) # Añadir una cuadrícula
plt.xticks(rotation=45) # Rotar las etiquetas del eje X si son muchas
plt.tight_layout() # Ajusta automáticamente los parámetros de la
                    # subtrama para un diseño ajustado
plt.show()
```

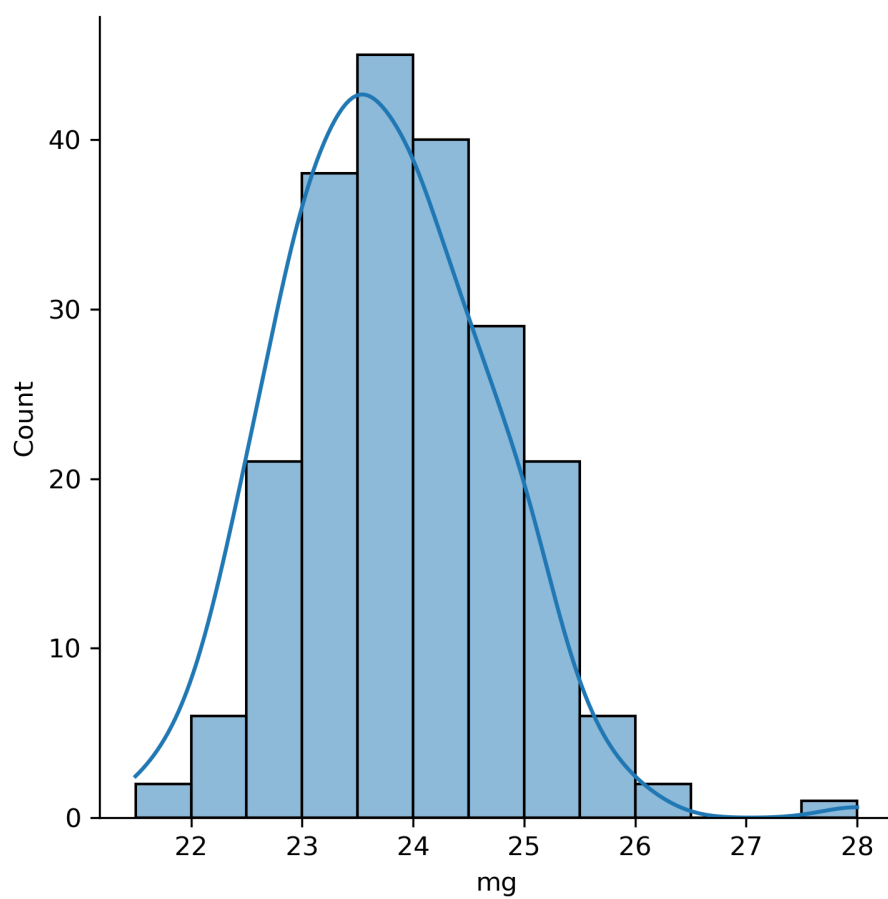


De manera visual, podemos aceptar que, en general, cuando la caja del boxplot se sitúa claramente por encima o por debajo de la caja de otro boxplot, las diferencias entre los dos conjuntos de valores son significativas; en este caso, podemos afirmar con claridad que los extractos secos de julio han sido significativamente superiores a los de los meses anteriores.

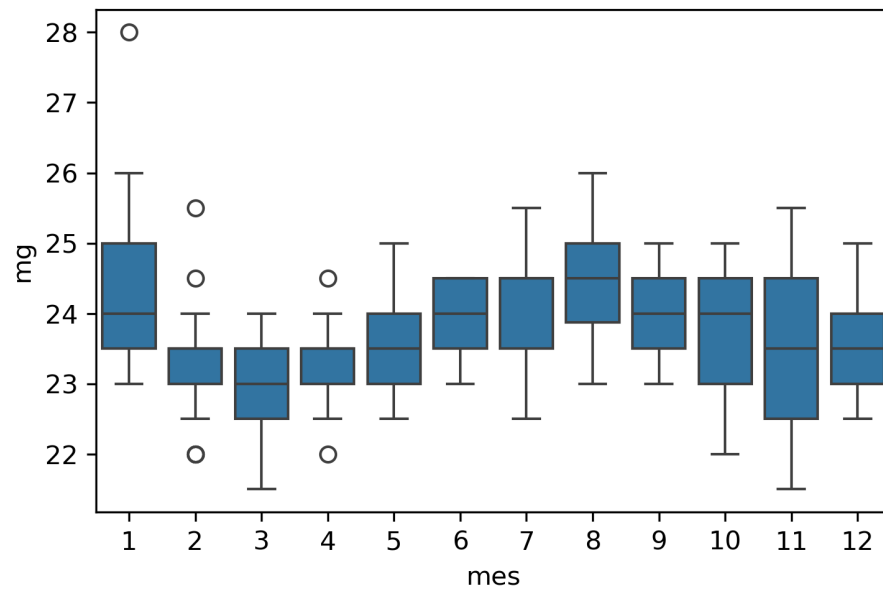
Antes de pensar en posibles causas de esta desviación, vamos a hacer el mismo análisis en la materia grasa, y después estableceremos una hipótesis de trabajo que explique esta desviación. Este es un punto importante del análisis en el que hay que insistir:

Nunca conviene precipitar el establecimiento de hipótesis sobre las causas de un problema, es imprescindible completar el proceso de análisis de los elementos que tengamos a nuestra disposición, sin perjuicio de que nuevos ciclos de análisis, transformación y visualización nos permitan seguir profundizando

```
sns.displot(df["mg"], kde = True)
plt.show()
```



```
sns.boxplot(x='mes', y='mg', data=df)
plt.show()
```

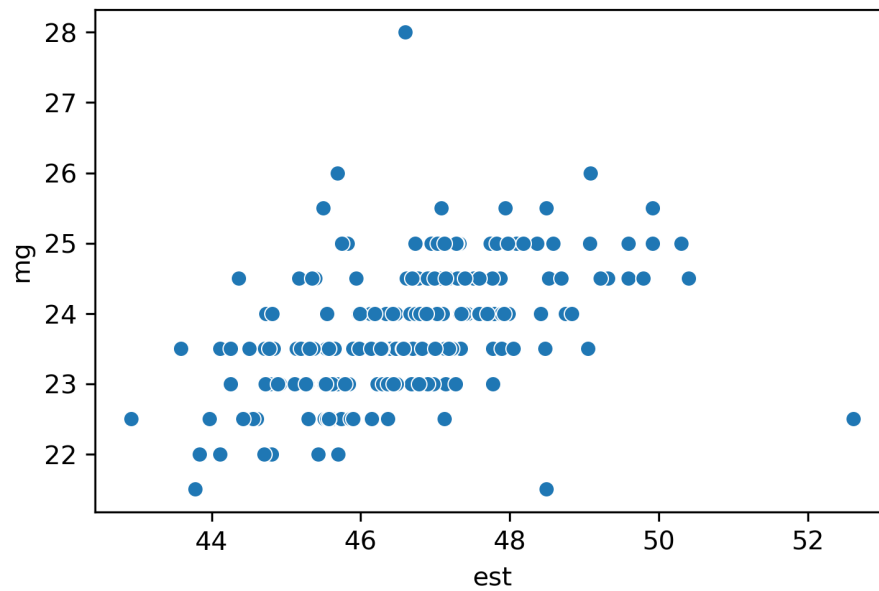


También en la materia grasa hay valores anormales, el único que parece fuera de contexto es el valor anormal de enero.

### El gráfico de dispersión

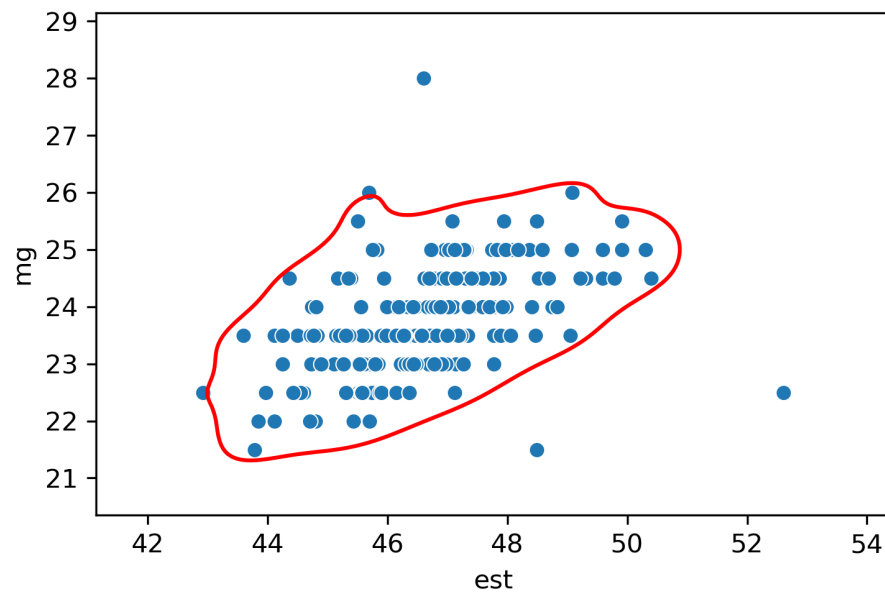
En general, cuando en una fabricación de queso aumenta el extracto seco, lo más común es que aumente también la materia grasa, que forma parte de ese extracto seco. ¿Cómo han variado la `mg` y el `est` durante el período que estamos considerando? Veámoslo con un diagrama de dispersión:

```
sns.scatterplot(data=df, x='est', y='mg')
plt.plot()
```



**Seaborn** Nos permite incluir una zona de confianza o *intervalo de confianza* para los valores que varían conjuntamente; utilizaremos la curva equivalente a la función de densidad que veíamos en el histograma:

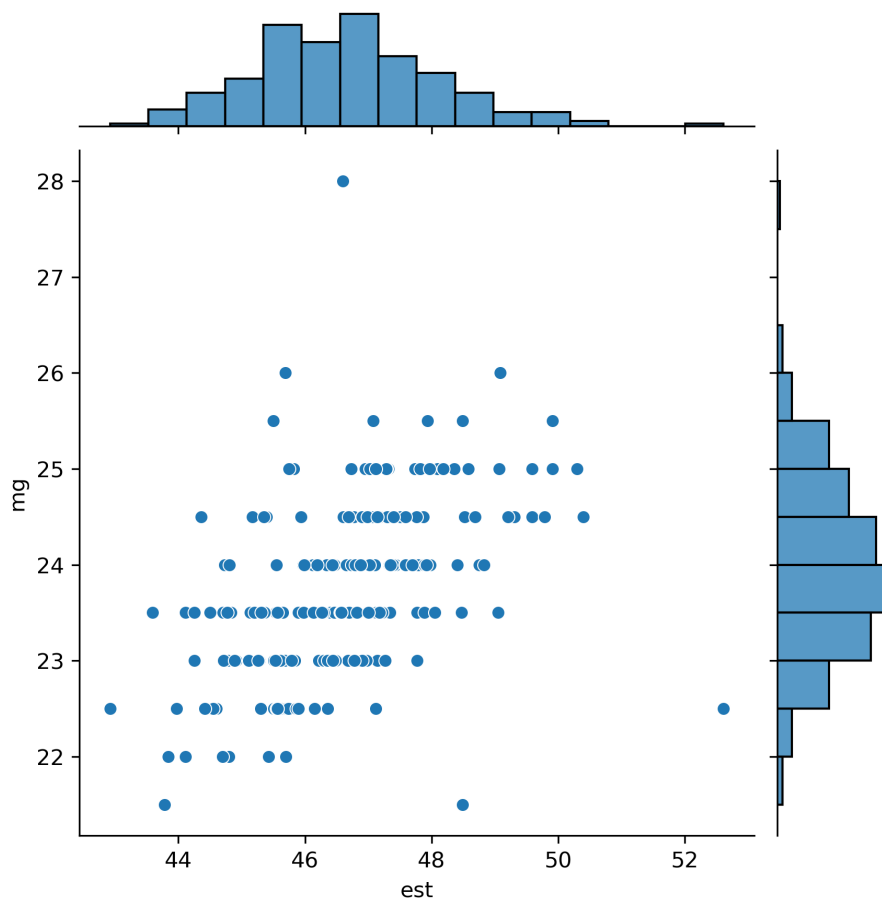
```
sns.scatterplot(data=df, x='est', y='mg')
sns.kdeplot(data=df,
             x='est',
             y='mg',
             levels=2,
             color='r',
             linewidths=1.5)
plt.plot()
```



**Seaborn** tiene un gráfico muy interesante que nos permite ver ambas variables en un gráfico conjunto, que nos muestra histogramas y un diagrama de dispersión.

```
sns.jointplot(
    x="est",
    y="mg",
    data = df
)
plt.show()
```





En estos dos gráficos, vemos que hay tres valores que podríamos considerar al menos como sospechosos:

1. El valor 52 de extracto seco total
2. El valor 28 de materia grasa
3. Un valor que los *boxplot* no nos habían detectado, por debajo de 22 de *mg* y por encima de 48 de *est*, lo que resulta anormal dentro de la composición habitual del queso.

El dibujo de la zona de confianza del gráfico de dispersión nos facilita el análisis de lo que debemos considerar anómalo y lo que no lo es; es muy útil para el análisis de variables que están relacionadas y que pueden (o deben) cambiar sus valores de forma conjunta.

### Completando el análisis

Antes de pasar al análisis y conclusiones, veamos dos gráficos complementarios de los anteriores, que muestran simultáneamente la media y la desviación típica semanales del extracto seco y la materia grasa,

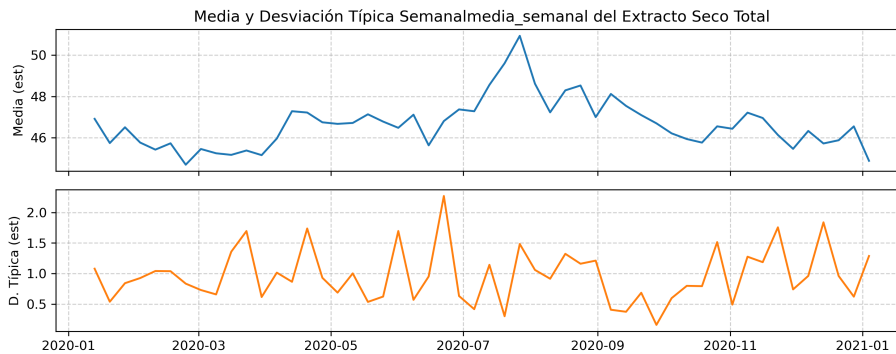
```
# --- 1. Preparación de los datos
#      (Calculando la Media y la Desviación Típica Semanal) ---
media_semanal = df["est"].resample('W-MON').mean()
dt_semanal = df["est"].resample('W-MON').std()

# --- 2. Creación de los Subgráficos (El paso clave) ---
# Creamos una figura (conjunto de gráficos) con 2 filas y 1 columna.
# El parámetro 'figsize=(10, 4)' controla el tamaño:
# 10 de ancho y 4 de alto total (como un gráfico normal).
# 'sharex=True' asegura que ambos gráficos compartan
# el mismo eje temporal.
fig, (ax1, ax2) = plt.subplots(
    nrows=2,
    ncols=1,
    figsize=(10, 4),
    sharex=True
)

# --- 3. Graficar la Media (Gráfico Superior) ---
ax1.plot(media_semanal, label="Media Semanal", color='tab:blue')
ax1.set_title("Media y Desviación Típica Semanal"
              "media_semanal del Extracto Seco Total")
ax1.set_ylabel("Media (est)")
ax1.grid(True, linestyle='--', alpha=0.6)
ax1.tick_params(labelbottom=False) # Quita las etiquetas de fecha
                                   # y hora del gráfico superior

# --- 4. Graficar la Desviación Típica (Gráfico Inferior) ---
ax2.plot(dt_semanal,
         label="Desviación Típica Semanal",
         color='tab:orange')
ax2.set_ylabel("D. Típica (est)")
ax2.grid(True, linestyle='--', alpha=0.6)

# Ajusta el espacio entre los gráficos para que no se superpongan
fig.tight_layout()
plt.show()
```



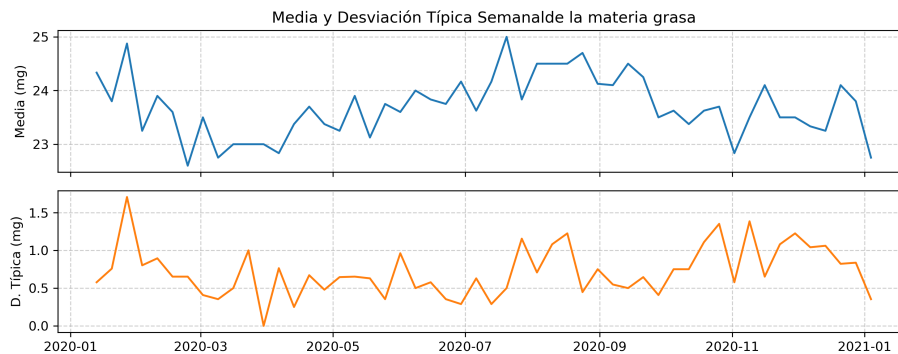
```
# --- 1. Preparación de los datos
# (Calculando la Media y la Desviación Típica Semanal) ---
media_semanal = df["mg"].resample('W-MON').mean()
dt_semanal = df["mg"].resample('W-MON').std()

# --- 2. Creación de los Subgráficos (El paso clave) ---
# Creamos una figura (conjunto de gráficos) con 2 filas y 1 columna.
# El parámetro 'figsize=(10, 4)' controla el tamaño:
# 10 de ancho y 4 de alto total (como un gráfico normal).
# 'sharex=True' asegura que ambos gráficos compartan
# el mismo eje temporal.
fig, (ax1, ax2) = plt.subplots(
    nrows=2,
    ncols=1,
    figsize=(10, 4),
    sharex=True
)

# --- 3. Graficar la Media (Gráfico Superior) ---
ax1.plot(media_semanal, label="Media Semanal", color='tab:blue')
ax1.set_title("Media y Desviación Típica Semanal"
              "de la materia grasa")
ax1.set_ylabel("Media (mg)")
ax1.grid(True, linestyle='--', alpha=0.6)
# Quita las etiquetas de fecha/hora del gráfico superior
ax1.tick_params(labelbottom=False)

# --- 4. Graficar la Desviación Típica (Gráfico Inferior) ---
ax2.plot(dt_semanal,
         label="Desviación Típica Semanal",
         color='tab:orange')
ax2.set_ylabel("D. Típica (mg)")
ax2.grid(True, linestyle='--', alpha=0.6)
```

```
# Ajusta el espacio entre los gráficos para que no se superpongan
fig.tight_layout()
plt.show()
```



Como tecnólogos queseros, sabemos que un factor crítico de la textura del queso, y en consecuencia, de su calidad, es la humedad del extracto seco desnatado, que calculamos a partir del extracto seco y la materia grasa como:

$$hqd = \frac{(100 - \text{est } \%) }{(100 - \text{mg } \%)} \times 100$$

Vamos a hacer un último gráfico, de la `hqd`, para completar nuestro análisis, creando una nueva columna en el `DataFrame`:

```
df['hqd'] = ((100 - df['est']) / (100 - df['mg'])) * 100

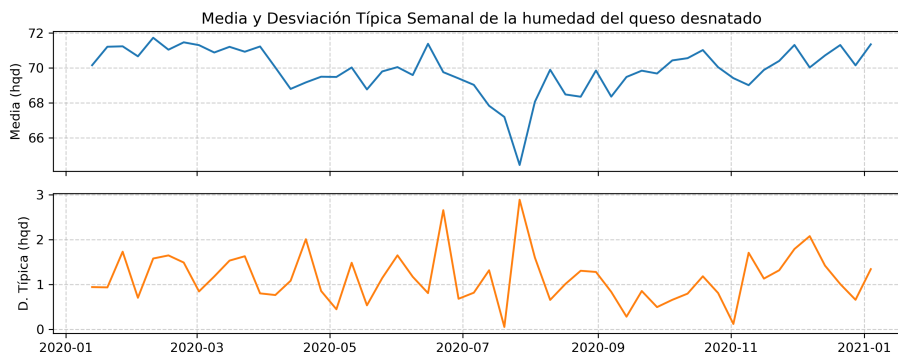
# --- 1. Preparación de los datos
# (Calculando la Media y la Desviación Típica Semanal) ---
media_semanal = df["hqd"].resample('W-MON').mean()
dt_semanal = df["hqd"].resample('W-MON').std()

# --- 2. Creación de los Subgráficos (El paso clave) ---
# Creamos una figura (conjunto de gráficos) con 2 filas y 1 columna.
# El parámetro 'figsize=(10, 4)' controla el tamaño:
# 10 de ancho y 4 de alto total (como un gráfico normal).
# 'sharex=True' asegura que ambos gráficos compartan
# el mismo eje temporal.
fig, (ax1, ax2) = plt.subplots(
    nrows=2,
    ncols=1,
    figsize=(10, 4),
    sharex=True
)
```

```
# --- 3. Graficar la Media (Gráfico Superior) ---
ax1.plot(media_semanal, label="Media Semanal", color='tab:blue')
ax1.set_title("Media y Desviación Típica Semanal "
              "de la humedad del queso desnatado")
ax1.set_ylabel("Media (hqd)")
ax1.grid(True, linestyle='--', alpha=0.6)
ax1.tick_params(labelbottom=False) # Quita las etiquetas de fecha/hora del gráfico superior

# --- 4. Graficar la Desviación Típica (Gráfico Inferior) ---
ax2.plot(dt_semanal,
         label="Desviación Típica Semanal",
         color='tab:orange')
ax2.set_ylabel("D. Típica (hqd)")
ax2.grid(True, linestyle='--', alpha=0.6)

# Ajusta el espacio entre los gráficos para que no se superpongan
fig.tight_layout()
plt.show()
```



### Análisis de los resultados

Una vez explorados los datos, siempre tenemos que formular hipótesis de trabajo para entender lo que nos dicen, y estudiar estas hipótesis con detalle hasta encontrar una explicación razonable. De estas conclusiones se deducirá una toma de decisiones para la mejora del proceso, que puede continuar con nuevos análisis e hipótesis.

A la vista de las series de datos, podemos establecer varias hipótesis:

1. A principio de año ha habido una cantidad de materia grasa anormalmente alta en el queso, que puede haberse debido a un error en la estandarización. De hecho, la composición de la leche en esta época es más rica en MG que el resto del año, lo que puede haber llevado a este error. Sin embargo esto no ha sido un problema porque también el extracto seco ha sido un poco

más alto, seguramente debido a una corrección del trabajo en la cuba. Como resultado, el responsable de fabricación ha mantenido la **hqd** constante en torno a 71 % hasta el mes de mayo.

2. A partir de febrero, se ha intentado corregir la estandarización aumentando la **mg** de forma constante hasta el verano.
3. En mayo se ha mantenido la composición en **mg**, pero se ha subido el **est**, lo que ha llevado a una reducción de **hqd**.
4. En verano se han obtenido valores de extracto seco muy altos que han producido una bajada brusca en **hqd** de hasta casi 7 puntos hasta un valor de 65 %, lo que es un gran problema para la textura de un queso camembert.
5. Además de esto, los tres valores anómalos que encontramos en el análisis del **jointplot** probablemente sean errores analíticos o de toma de muestras, ya que no se corresponden con el resto de los valores. ¿Sería más seguro eliminarlos del análisis?

#### 8.1.4. 4. Comunicar: discusión de la información en equipo.

Hemos ido a una reunión de trabajo con la quesera responsable de fabricación del queso, con el analista de laboratorio y con el responsable de producción, que es el que da las instrucciones para la modificación de la tecnología. Después de la discusión, hemos obtenido las siguientes conclusiones.

- Debido a las variaciones de composición, el responsable de fabricación nos confirma que, efectivamente, a lo largo del año hace pequeñas modificaciones en la estandarización y el trabajo en cuba, buscando mantener constante la **hqd**.
- El mayor consumo del queso camembert de este fabricante tiene lugar en las zonas costeras del sur y en Canarias, zonas en las que el queso es muy apreciado por los turistas que vienen a España. En verano, debido a que las temperaturas son más altas, suele bajar moderadamente la **hqd** para evitar problemas de proteolisis excesiva durante el ciclo de comercialización.
- Este año ha habido un problema adicional en el verano, y es que, durante las vacaciones, el quesero ha sido sustituido por una persona con menos experiencia, que ha forzado el extracto seco más de lo debido; también ha habido reclamaciones de consumidores por queso demasiado seco, lo que coincide con lo observado en los análisis.
- El laboratorio ha confirmado que los valores anómalos sospechosos pueden haber sido debidos a análisis incorrectos, y recomienda no tenerlos en cuenta. En consecuencia, deberíamos repetir el análisis gráfico eliminando estos valores, para que las conclusiones no se vean afectadas por ellos.

#### Posible plan de trabajo

A la vista de estos resultados, el equipo propone las siguientes acciones:

1. Revisar diariamente la composición de la leche para evitar errores de estandarización.
2. Mejorar la formación a las personas que son responsables de la toma de muestras para evitar errores.
3. Repasar los métodos analíticos con los analistas del laboratorio y posiblemente poner en marcha un estudio sobre la precisión de los análisis, incluyendo la dispersión por analista.
4. Actualizar con agilidad la información sobre la situación del queso en el punto de venta, con el fin de asegurar la calidad, evitando una sobrecorrección de la `hqd` en los meses de verano. Valorar si los problemas de proteolisis en el punto de venta durante estos meses de verano son generales o sólo se presentan en algunos clientes.
5. Cuidar las sustituciones en verano en los puntos críticos: quesero y analistas, para no introducir variabilidad por defectos de formación en las personas sustitutas.
6. Anticipar los ciclos de formación a las personas encargadas de sustituir puntos clave en situaciones de enfermedad o vacaciones, mejorando la polivalencia.

Dado que la capacidad de poner planes de acción del equipo es limitada, debido a la carga de trabajo de producción, en sucesivas reuniones se priorizarán los planes de acción que tengan mayor impacto en la calidad y en el coste del producto, y se establecerán calendarios de trabajo e indicadores que reflejen los impactos de las acciones en marcha. Estos indicadores se publicarán semanalmente en paneles situados en la planta, para que el equipo pueda verlos y discutirlos.

## 8.2. Resumen del capítulo

En este capítulo hemos analizado una fabricación anual de queso camembert y hemos aprendido a detectar posibles errores en los atos, así como a interpretar lo sucedido durante una temporada de fabricación, sacando conclusiones para establecer planes de mejora.

En el capítulo siguiente nos enfrentaremos al análisis de las entradas de leche en una gran fábrica de queso.

## 8.3. Ejercicios propuestos

### 8.3.1. Ejercicio 1

En el análisis de resultados, encontramos tres valores anormales de `esty mg`, y concluimos que deberíamos repetir el análisis sin estos valores.

Utilizando el filtro de Excel, busca los tres valores y elimínalos del `camembert.csv`. Guarda el archivo editado como `camembert_corregido.csv`. Modifica las instrucciones de lectura de datos del cuaderno para leer este nuevo

archivo y repite el análisis. ¿Llegas a las mismas conclusiones? Elabora un nuevo informe de conclusiones y prepara un PowerPoint para presentarlo al grupo. Puedes incluso usar los resultados del primer análisis y compararlos lado a lado con los del segundo.

Podrás comprobar que los scripts `python` nos permiten repetir el análisis sin ninguna modificación (salvo el nombre del archivo de datos) en escasos minutos, y que rápidamente podemos usar los resultados para escribir un informe.

Como sugerencia, puedes copiar los gráficos en cualquier otro programa (por ejemplo, en PowerPoint) simplemente copiando el gráfico del cuaderno y pegándolo donde quieras. Es un sistema manual pero muy flexible y suficientemente rápido.

### 8.3.2. Ejercicio 2 (avanzado)

En este ejercicio, vamos a añadir la fase **Transformar** a nuestro flujo de datos en `Python`.

En vez de editar el `csv` para modificar los datos, vamos a utilizar `Python` para hacer un filtro de los datos anómalos, y nos vamos a apoyar en Gemini o en ChatGPT para escribir el código.

Abre Gemini y haz esta pregunta:

Tengo un dataframe `df` con dos columnas, `est` y `mg`. Quiero filtrar algunos valores y convertirlos en 'NaN': los valores con `mg` mayor que 27, los valores con `est` mayor que 52 y los valores que sean simultáneamente con `mg` menor que 22 y `est` mayor que 48. Dame el código `python` para hacerlo de la manera más sencilla posible.

Copia la respuesta de Gemini e insértala en una casilla de código al final de este mismo cuaderno, después del último texto.

Ahora vete copiando y pegando a continuación las casillas de código del análisis que hemos realizado y que quieras repetir para verificar el filtrado de datos:

- el histograma
- el boxplot general
- el boxplot por meses
- el gráfico de dispersión
- el gráfico de series

Comprueba si las conclusiones que hemos hecho se mantienen o hay que modificarlas, y escribe las nuevas conclusiones si crees necesario modificarlas. Establece un plan de trabajo acorde con estas conclusiones y prepara un PowerPoint para comunicar al equipo el resultado del análisis y el nuevo plan de acción.



## Parte II

# Análisis cuantitativo de la producción quesera



## Capítulo 9

# El balance de materia y los rendimientos queseros.

La elaboración de queso tiene como objetivo principal **preservar y concentrar los componentes nutritivos de la leche**, transformándolos en un producto alimentario con **elevado valor sensorial, nutricional y comercial**. Este proceso da lugar a una amplia variedad de quesos, cada uno con características únicas que responden a factores como el tipo de leche, la tecnología aplicada y las condiciones de maduración.

El propósito de la tecnología quesera es **optimizar esta transformación**, guiando el proceso de forma que se obtenga el mejor producto posible, **garantizando la calidad sanitaria, tecnológica y organoléptica**, y todo ello de manera **eficiente y rentable**. Para lograrlo, los objetivos técnicos se centran en **minimizar las pérdidas de materia**, especialmente de **grasa y proteína**, que son los componentes clave en la formación del queso. Al mismo tiempo, se busca **preservar los atributos sensoriales** —como sabor, textura y aroma— y **cumplir con los requisitos higiénico-sanitarios** establecidos por la normativa vigente.

El análisis cuantitativo del proceso quesero se estructura en dos partes diferenciadas:

- **Balance de materia global:** comprende desde la recepción de la leche cruda hasta el final del proceso de elaboración, momento en que el queso está listo para entrar en la fase de maduración. Este balance permite evaluar el rendimiento general y detectar posibles pérdidas o desviaciones.
- **Transformación quesera propiamente dicha:** incluido en el balance de materia global, abarca el tramo desde la leche destinada a fabricación una vez en la cuba, hasta la obtención del queso fresco (antes o después del salado). Aquí se analizan los rendimientos específicos, la eficiencia de

coagulación, el corte, el desuerado, el moldeado y otros parámetros tecnológicos que inciden directamente en la calidad y cantidad del producto final.

Ambos análisis deben estar **valorados económicamente** para conocer el impacto de nuestras desviaciones y pérdidas. Este conocimiento permitirá establecer las prioridades del trabajo de mejora, que consisten fundamentalmente en:

- la reducción de las pérdidas de materia, y
- el aumento de la eficiencia de la transformación e la leche en queso.

En los siguientes capítulos abordaremos, pues:

- El balance de materia
- El análisis de rendimientos queseros
- La valoración del balance de materia, el rendimiento y las pérdidas a partir del precio de la leche.

### 9.1. Etapas generales en el proceso de fabricación de queso.

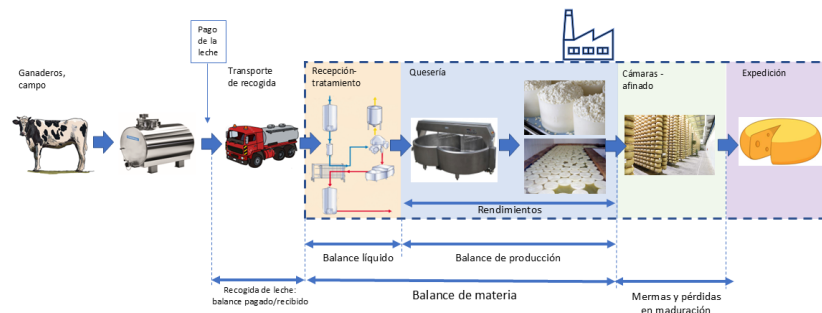


Figura 9.1: Las diferentes unidades de una fábrica de queso y las medidas del balance de materia

El proceso de la leche para elaborar queso empieza con el ordeño, y el almacenamiento en frío. La leche almacenada es cargada en el camión de transporte, que la lleva a la fábrica. En el momento de la carga, se toma una muestra representativa, que será analizada por el laboratorio interprofesional lechero de la zona, y que tiene como objetivo el pago por calidad, y se registran los litros cargados en la cisterna. En otras ocasiones, es el propio personal del laboratorio interprofesional el que toma la muestra, para evitar posibles conflictos con la industria, y el resultado es enviado por el laboratorio tanto al ganadero como a la industria.

A su llegada a fábrica, se toma una muestra representativa de la leche, después de agitarla adecuadamente; esta muestra será analizada en el laboratorio de fábrica, y este análisis establece las cantidades de materia que se dan de entrada en la unidad de producción.

Debido a los procesos de muestreo, y a los posibles arrastres con agua de los depósitos y tuberías, casi siempre se presentan diferencias entre las analíticas realizadas por el laboratorio interprofesional y las realizadas en fábrica a la recepción. Estas diferencias, tanto de analíticas como de cantidad medida (litros o kg), forman parte de lo que en el sector se conoce como **diferencia pagado-recibido**. Habitualmente, forman parte de la negociación en el proceso de compra y pueden partirse a medias, siempre que el promedio acumulado esté muy próximo a cero, indicando que no hay sesgos, o repartirlas entre comprador y vendedor con otros criterios, siempre de forma negociada y acordada.

Hay que tener en cuenta que en la aparición de diferencias analíticas puede influir incluso las diferencias en los métodos analíticos o sistemas automáticos utilizados; es fundamental realizar procesos de calibración inter-laboratorios y auditorías de proceso para asegurar que las analíticas de laboratorio interprofesional y de la fábrica proporcionan los mismos resultados para muestras de calibración idénticas.

Para el control del balance de materia de la fábrica, es fundamental que estas diferencias pagado-recibido no se incluyan en el balance, ya que corresponden a eventos o situaciones fuera de su control, y su resolución no puede hacerse por otros medios que no sean la negociación del reparto de diferencias.

El balance de materia propiamente dicho comienza con los volúmenes y analíticas que la unidad de producción ha dado de entrada en fábrica, teniendo cuidado de que no se intente llevar a diferencias pagado-recibido lo que realmente son pérdidas dentro de la fábrica.

El proceso de la leche puede tener varias etapas antes de llegar a la cuba:

- **Termización**
- **Estandarización** en grasa y/o proteína, que puede implicar **desnatado** y posterior **reconstitución**
- En algunos casos, como la elaboración de queso azul, una **homogeneización** de parte de la materia grasa
- El **trasvase** y movimiento de la leche y otras materias dentro de la sección de **recepción-tratamiento** entre tanques, desnatadoras, homogeneizadores y resto de sistemas.

Como resultado de todos estos movimientos y procesos, se producen una serie de pérdidas de materia por diversas razones, entre las que también hay que considerar la pérdida física de materia por valvulas averiadas, malos cierres, juntas estropeadas, etc. La consecuencia es que el total de la materia enviada a las cubas es menor que la recibida en fábrica. Estas diferencias se analizan mediante el **balance líquido**, que con mayor o menor detalle va contabilizando

las diferencias entre entradas y salidas de los diferentes procesos realizados sobre la materia líquida **hasta** su llegada a la cuba de fabricación.

Las cantidades analizadas en la cuba de fabricación constituyen el cierre (salida) del balance líquido y el inicio del **balance de producción**, que recoge la transformación quesera en sentido estricto, desde la leche en la cuba lista para fabricación, hasta el queso obtenido, antes o después de salado (opcional)

En el proceso de obtención del queso se produce una eliminación del suero de quesería, en el que se recogerá la mayor cantidad de la lactosa de la leche, proteínas solubles, y algo de grasa y proteína. Como resultado de la manipulación de la cuajada en la cuba y los procesos de corte y agitación, se producen pequeñas pérdidas de finos de quesería, que consisten en grasa y proteína coagulada que no se ha podido retener en la cuajada enviada a moldear. En el proceso de moldeo se produce también una pérdida de materia que puede ser significativa, según el proceso y según el control que el equipo de quesería tenga sobre el proceso. Finalmente, en los intercambios osmóticos en el proceso de salado, se produce una pérdida de suero y una ganancia de extracto seco magro en forma de aporte de sal.

El suero de quesería, que contiene una pequeña cantidad de grasa, en la mayor parte de las ocasiones, es desnatado para recuperar esta materia grasa, en forma de nata de suero. Esta nata de suero, si está producida y conservada en las condiciones adecuadas, puede adicionarse a la siguiente fabricación de queso en la estandarización, reduciendo las pérdidas de grasa, siempre que el producto lo admita y no haya problemas de calidad.

En resumen, todos los procesos que llevan a la elaboración de queso, constan a su vez de otros subprocesos, en los cuales se van a producir pérdidas de materia grasa y proteína en mayor o menor cantidad. Dado que en el pago por calidad de la leche, la grasa y la proteína son las materias que se valoran y son el objeto del pago, parece lógico que la optimización de la eficiencia de la elaboración de queso se enfoque en la reducción de las pérdidas de materia en todas las partes del proceso.

## 9.2. Algunas causas de las pérdidas de materia en el proceso quesero

Pérdidas de balance: sólo aquellas que son diferencias entre entradas y salidas. Por ejemplo, pérdidas de finos bien medidas en el suero pueden no ser una pérdida de balance si están bien contabilizadas, pero son pérdidas de rendimiento porque esos finos no se recuperan en el queso

- Mala calibración de la báscula de entrada o de los contadores de medida en la descarga
- Pérdidas en válvulas que cierran mal y no tienen un buen programa e mantenimiento preventivo de cierres y juntas

- Operaciones mal descritas y que dejan demasiado espacio a las decisiones individuales que pueden ser improvisadas
- Falta de auditorías de procesos que permitan evaluar la calidad de las operaciones y detectar los puntos críticos de mala operación
- Procedimientos de arrastres (fin de línea, depósitos) mal temporizados, lo que hace que sean demasiado cortos (pérdidas de materia) o demasiado largos (dilución excesiva)
- Mal estado de los útiles de corte en la cuba, lo que aumenta las pérdidas de finos en la cuba
- Información insuficiente (pocas muestras) para evaluar bien el balance y conocer los puntos de pérdida. Desconocimiento del error de medida (muestreo, variabilidad de analistas, repetibilidad de los análisis), lo que lleva a correcciones que pueden no corresponder a valores de pérdida real sino a errores analíticos.
- Insuficiente formación en la toma de muestras, que se realiza de manera incorrecta
- Uso de bombas centrífugas para el transporte de cuajada, lo que hace que aumente la cantidad de finos que se pierden en el suero (estrictamente, puede no ser una pérdida de balance sino de rendimiento quesero)

### 9.3. Algunas causas de las pérdidas de eficiencia en la transformación quesera

Recogida y almacenamiento de leche en la granja:

- Almacenamiento en frío durante tiempo prolongado: desarrollo de bacteria psicrotrofas que degradan la caseína

### 9.4. Análisis de datos en producción con Python: más allá del balance contable

Cuando realizamos un balance de materia en una fábrica o calculamos los rendimientos, no lo hacemos con un enfoque contable o notarial, cuyo objetivo principal es dejar constancia del resultado. Aunque registrar los datos es importante, lo que realmente necesita un técnico o ingeniero de producción es otra cosa: poder revisar esos resultados a lo largo del tiempo, analizar tendencias y cruzar la información con otras variables que ayuden a entender por qué se obtuvieron esos resultados.

En un entorno de mejora continua, el análisis de resultados no es un fin en sí mismo, sino una herramienta clave para avanzar. Su objetivo es reducir las pérdidas de materia y comprender mejor los procesos tecnológicos que las generan, con la intención de modificarlos y corregirlos.

Herramientas como Python permiten trabajar con los datos de forma progresiva

y creativa. Podemos agruparlos por fechas, aplicar distintos criterios de análisis y explorar dinámica y gráficamente nuevas perspectivas que quizás no habíamos considerado al inicio. Muchas veces, es durante el análisis cuando surgen las preguntas más interesantes.

Además, trabajar con scripts tiene una ventaja fundamental: separa claramente las instrucciones de análisis (el código) de los datos en sí, que suelen almacenarse en archivos .csv. Estos datos, por lo general, requieren una limpieza previa antes de analizarlos.

Tanto el proceso de limpieza como los pasos del análisis quedan documentados en el script. Esto facilita compartir el trabajo con otras personas, reproducir el análisis y, lo más importante, garantizar su trazabilidad. En cambio, en una hoja de cálculo es fácil mezclar datos y análisis: las fórmulas quedan ocultas en las celdas y el algoritmo utilizado suele estar disperso y poco visible entre fórmulas, datos y formatos de presentación.

Por esta razón, el análisis del balance de materia y de los rendimientos se hace aquí mediante python.



## Capítulo 10

# El balance de materia

En cualquier empresa quesera, con independencia de su tamaño, una de las primeras y más básicas cosas a realizar es un análisis del balance global de la elaboración quesera. Para un período determinado, que como primera aproximación, puede ser anual, se trata de conocer el balance entre la leche comprada y el queso producido, tanto en materia grasa como en proteína. Dado que toda la materia que no esté en el queso no puede ser vendida, no recuperaremos el coste de materia de las pérdidas (el suero de quesería ya no es un ingreso por venta; en la actualidad, en la mayor parte de los casos el suero es recogido a valor cero por las empresas que se encargan del secado de suero, con el valor del transporte como elemento de negociación).

En nuestro ejemplo, los datos que vamos a utilizar están registrados en un fichero de datos en formato `csv`, el cual incluye la analíticas de la leche y el queso, así como alguna analítica adicional del proceso y que de momento no tiene interés para nosotros.

Analizaremos el conjunto de datos con `python` Excel, siguiendo nuestra línea de trabajo.

En una fábrica en la que se elabora más de un producto; el fichero de datos de fabricación debe parecerse a éste, en el que se detallan los volúmenes y analíticas tanto de la leche enviada a fabricación como de los quesos elaborados. Caracterizaremos nuestro queso mediante un análisis de extracto seco total (EST) y una grasa butirométrica (MG), en un punto determinado de nuestro proceso (por ejemplo, a la salida de la salmuera, después de escurrido) y mediante un muestreo repetible, de forma que nuestro análisis sea representativo. A partir de estos dos análisis, calculamos otros parámetros técnicos necesarios, tales como la materia grasa sobre el extracto seco total (G/ES), el extracto seco magro o desnatado (ESM) y la humedad del queso desnatado (HQD), valores que se calculan a partir de los primeros.

|             | fecha      | receta  | litros_past | est_past | mg_past | mp_past | \ |
|-------------|------------|---------|-------------|----------|---------|---------|---|
| fecha_index |            |         |             |          |         |         |   |
| 2022-02-23  | 23/02/2022 | bajo mg | 12380       | 10.760   | 2.58    | 3.550   |   |
| 2022-02-24  | 24/02/2022 | mezcla  | 7440        | 13.290   | 5.15    | 4.090   |   |
| 2022-03-01  | 01/03/2022 | vaca    | 15350       | 12.542   | 4.20    | 3.076   |   |
| 2022-03-01  | 01/03/2022 | bajo mg | 13080       | 11.090   | 2.59    | 3.600   |   |
| 2022-03-04  | 04/03/2022 | mezcla  | 13420       | 14.220   | 5.45    | 4.310   |   |

|             | kg_cuba   | est_cuba | mg_cuba | mp_cuba | formato | \ |
|-------------|-----------|----------|---------|---------|---------|---|
| fecha_index |           |          |         |         |         |   |
| 2022-02-23  | 12860.510 | 10.28    | 2.46    | 3.38    | 3 kg    |   |
| 2022-02-24  | 7704.462  | 12.80    | 4.90    | 3.89    | barra   |   |
| 2022-03-01  | 15810.990 | 11.37    | 4.01    | 2.94    | barra   |   |
| 2022-03-01  | 13867.370 | 10.41    | 2.47    | 3.43    | barra   |   |
| 2022-03-04  | 14131.770 | 13.61    | 5.20    | 4.11    | 3 kg    |   |

|             | est_salida_salmuera | mg_salida_salmuera | ph_salida_salmuera | \ |
|-------------|---------------------|--------------------|--------------------|---|
| fecha_index |                     |                    |                    |   |
| 2022-02-23  | 50.85               | 20.34              | 5.44               |   |
| 2022-02-24  | 59.42               | 32.13              | 5.40               |   |
| 2022-03-01  | 55.94               | 29.50              | 5.41               |   |
| 2022-03-01  | 52.10               | 21.15              | 5.34               |   |
| 2022-03-04  | 52.65               | 28.80              | 5.44               |   |

|             | peso_queso_total | sal_queso |
|-------------|------------------|-----------|
| fecha_index |                  |           |
| 2022-02-23  | 1419.3           | 1.47      |
| 2022-02-24  | 1062.4           | 1.31      |
| 2022-03-01  | 1960.0           | 1.55      |
| 2022-03-01  | 1536.0           | 1.17      |
| 2022-03-04  | 2397.0           | 1.02      |

### 10.0.1. Descripción de las columnas (variables) del fichero de datos

En total, tenemos 45 entradas que corresponden a fabricaciones diarias, desde el 23/02/2022 hasta el 30/5/2022.

El fichero de datos incluye diferentes datos analíticos:

| Columna            | Descripción                                  |
|--------------------|----------------------------------------------|
| <b>fecha</b>       | día de fabricación                           |
| <b>receta</b>      | tipo de queso fabricado                      |
| <b>litros_past</b> | litros de leche enviados a pasteurización    |
| <b>kg_cuba</b>     | cantidad de leche recibida en cuba, en kilos |

| Columna                     | Descripción                                      |
|-----------------------------|--------------------------------------------------|
| <b>est_cuba</b>             | extracto seco de la leche en la cuba             |
| <b>mg_cuba</b>              | materia grasa de la leche en la cuba             |
| <b>mp_cuba</b>              | materia proteica de la leche en la cuba          |
| <b>lactosa_cuba</b>         | lactosa de la leche en la cuba                   |
| <b>ph_final_moldeo_cuba</b> | pH del queso al final del proceso de moldeo      |
| <b>formato</b>              | formato de moldeo                                |
| <b>est_salida_salmuera</b>  | extracto seco total del queso después del salado |
| <b>mg_salida_salmuera</b>   | materia grasa del queso después del salado       |
| <b>ph_salida_salmuera</b>   | pH del queso después del salado                  |
| <b>peso_queso_total</b>     | peso del queso después del salado                |
| <b>sal_queso</b>            | porcentaje de sal analizada en el queso          |

El queso se fabrica en varias recetas, y se envasa en diferentes formatos:

| Receta        | Formatos           | Comentario                     |
|---------------|--------------------|--------------------------------|
| i+D (ensayos) | 3 kg, 500 g, barra | Queso con bajo contenido en MG |
| bajo mg       | 3 kg, 500 g, barra |                                |
| mezcla        | 3 kg, 500 g, barra |                                |
| oveja         | barra              |                                |
| vaca          | barra              |                                |

El fichero de datos contiene varios ensayos del departamento de i+D que se han mantenido para mostrar que a veces las cosas no salen bien a la primera...

Podemos calcular el balance global para el período de producción registrado, totalizando las cantidades correspondientes:

```
# Agrupar y sumar
resumen = df.groupby('receta')[['litros_past', 'peso_queso_total']].sum()

# Calcular la relación litros/kilo
resumen['litros_por_kilo'] = resumen['litros_past']/ resumen['peso_queso_total']

# Estimación del balance global
resumen['balance_kilos'] = resumen['peso_queso_total']-(resumen['litros_past']*1.030)

# Estimación del% de suero producido
resumen['suero_estimado_%'] = (resumen['balance_kilos']/1.020)/resumen['litros_past']*100

# Mostrar la tabla resultante
print(resumen)
```

```
litros_past  peso_queso_total  litros_por_kilo  balance_kilos  \
```

|         |        |         |          |            |
|---------|--------|---------|----------|------------|
| receta  |        |         |          |            |
| bajo mg | 133820 | 15005.8 | 8.917885 | -122828.80 |
| i+D     | 59600  | 6763.0  | 8.812657 | -54625.00  |
| mezcla  | 135300 | 21628.7 | 6.255577 | -117730.30 |
| oveja   | 117970 | 21614.0 | 5.458036 | -99895.10  |
| vaca    | 129333 | 15814.0 | 8.178386 | -117398.99 |

|         |                  |  |
|---------|------------------|--|
|         | suero_estimado_% |  |
| receta  |                  |  |
| bajo mg | -89.986842       |  |
| i+D     | -89.855573       |  |
| mezcla  | -85.308103       |  |
| oveja   | -83.018032       |  |
| vaca    | -88.992793       |  |

Dado que nuestros valores analíticos están expresados en porcentaje en peso, calculamos las cantidades de materia como se indica a continuación:

```

### 1. Cálculos de Entradas
df['est_kg_cuba'] = df['kg_cuba'] * df['est_cuba'] / 100
df['mg_kg_cuba'] = df['kg_cuba'] * df['mg_cuba'] / 100
df['mp_kg_cuba'] = df['kg_cuba'] * df['mp_cuba'] / 100
df['esm_kg_cuba'] = df['est_kg_cuba'] - df['mg_kg_cuba']

### 2. Cálculos de Salidas
df['est_kg_queso'] = df['peso_queso_total'] * df['est_salida_salmuera'] / 100
df['mg_kg_queso'] = df['peso_queso_total'] * df['mg_salida_salmuera'] / 100
df['esm_kg_queso'] = df['est_kg_queso'] - df['mg_kg_queso']
df['sal_kg_queso'] = df['peso_queso_total'] * df['sal_queso']/100

### 3. Coeficientes de recuperación (expresado en porcentaje)
df['coef_recup_mg'] = df['mg_kg_queso']/df['mg_kg_cuba'] * 100
df['coef_recup_mp'] = df['esm_kg_queso']/df['mp_kg_cuba'] * 100 # atencion a la fórmula
df['coef_recup_esm'] = df['esm_kg_queso']/df['esm_kg_cuba'] * 100

### 4. Consumos de materia por kilo de queso (expresado en gramos de materia por kilo de queso)
df['consumo_mg_kg'] = df['mg_kg_cuba'] / df['peso_queso_total'] * 1000
df['consumo_mp_kg'] = df['mp_kg_cuba'] / df['peso_queso_total'] * 1000

### 5. Balance de Materia (Salida - Entrada)
# La falta de producto queda como negativo al hacer Salida - Entrada
df['balance_est'] = df['est_kg_queso'] - df['est_kg_cuba']
df['balance_mg'] = df['mg_kg_queso'] - df['mg_kg_cuba']
df['balance_esm'] = df['esm_kg_queso'] - df['esm_kg_cuba'] - df['sal_kg_queso']

```

```

### 6. Preparación y Exportación del DataFrame de Resultados
# Columnas finales a incluir en el reporte
columnas_balance = [
    'fecha', 'receta', 'formato',
    'litros_past', 'peso_queso_total',
    'est_kg_cuba', 'est_kg_queso',
    'mg_kg_cuba', 'mg_kg_queso', 'mp_kg_cuba',
    'esm_kg_cuba', 'esm_kg_queso',
    'coef_recup_mg', 'coef_recup_mp', 'coef_recup_esm',
    'consumo_mg_kg', 'consumo_mp_kg',
    'balance_est', 'balance_mg', 'balance_esm'
]

# Crear un nuevo DataFrame con las columnas de interés
df_balance = df[columnas_balance].copy()

# Guardar el DataFrame con punto decimal y separador ';'
nombre_archivo_salida = 'datos/informe_balance_materia.csv'
df_balance.to_csv(
    nombre_archivo_salida,
    sep=';',
    index=False,
    decimal=', '
)

```

Ahora vamos a crear un nuevo csv con el balance de materia semanal, aprovechando las funciones de **pandas** para crear una columna de semana ISO.

```

# 1. Crear la columna 'semana' usando el formato ISO (Lunes=Día 1)
# Usamos el índice del DataFrame (que debe ser DatetimeIndex)
df['semana_iso'] = df.index.strftime('%G-W%V')

print(" Columna 'semana_iso' creada con el estándar Lunes-Domingo.")

```

Columna 'semana\_iso' creada con el estándar Lunes-Domingo.

```

# Columnas que queremos sumarizar (Balances + Entradas + Salidas)
columnas_a_sumarizar = [
    # Entradas
    'est_kg_cuba', 'mg_kg_cuba', 'esm_kg_cuba',
    # Salidas
    'est_kg_queso', 'mg_kg_queso', 'esm_kg_queso',
    # Balances
    'balance_est', 'balance_mg', 'balance_esm'
]

```

```

# 2. Agrupar por 'semana_iso' y 'receta' y calcular la SUMA de todas las columnas

```

```
df_balance_semanal_completo = df.groupby(['semana_iso', 'balance_mg'])[columnas_a_sumarizar]

# Renombrar las columnas para indicar que son totales
df_balance_semanal_completo.columns = [f'total_{col}' for col in columnas_a_sumarizar]

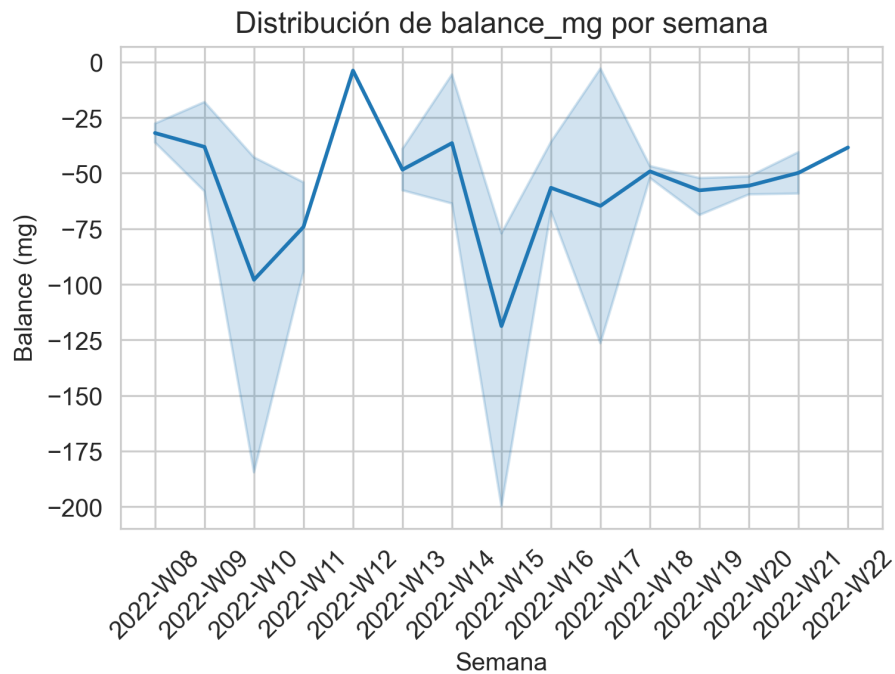
# 1. Definir el nombre del archivo de salida
nombre_archivo_final = 'datos/reporte_balance_completo_semanal_iso.csv'

# 2. Exportar el DataFrame a CSV
df_balance_semanal_completo.to_csv(
    nombre_archivo_final,
    sep=';',          # Usa punto y coma como separador de columnas
    index=True,       # Mantiene las columnas de agrupación ('semana_iso' y 'receta')
    decimal=',',      # Usa coma como separador decimal
)

df_sns = df_balance_semanal_completo.reset_index(drop=False) # False mantiene la columna

sns.lineplot(x='semana_iso', y='balance_mg', data=df_sns)

# Opcional: mejorar la visualización
plt.title('Distribución de balance_mg por semana')
plt.xlabel('Semana')
plt.ylabel('Balance (mg)')
plt.xticks(rotation=45) # Si los nombres de semana son largos
plt.show()
```

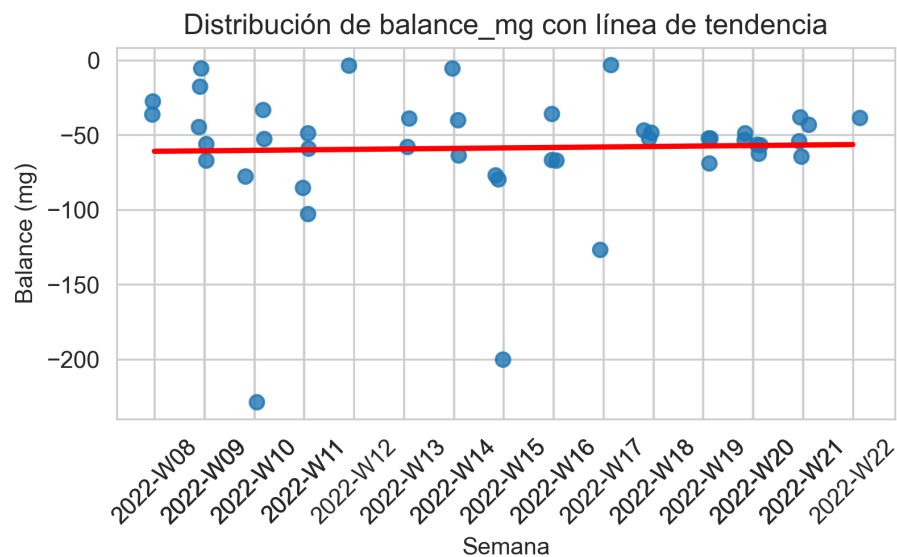


```
# Convertir semana_iso a números temporales
df_sns['semana_num'] = pd.factorize(df_sns['semana_iso'])[0]

# Dibujar la línea de tendencia con regplot
sns.regplot(
    x='semana_num',
    y='balance_mg',
    data=df_sns,
    scatter=True,
    x_jitter=0.2,      # añade jitter horizontal
    y_jitter=0.2,      # añade jitter vertical
    ci=None,
    line_kws={'color': 'red'},
)

# Reemplazar los ticks del eje X con las etiquetas originales
plt.xticks(
    ticks=df_sns['semana_num'],
    labels=df_sns['semana_iso'],
    rotation=45
)
```

```
# Mejorar visualización
plt.title('Distribución de balance_mg con línea de tendencia')
plt.xlabel('Semana')
plt.ylabel('Balance (mg)')
plt.tight_layout()
plt.show()
```

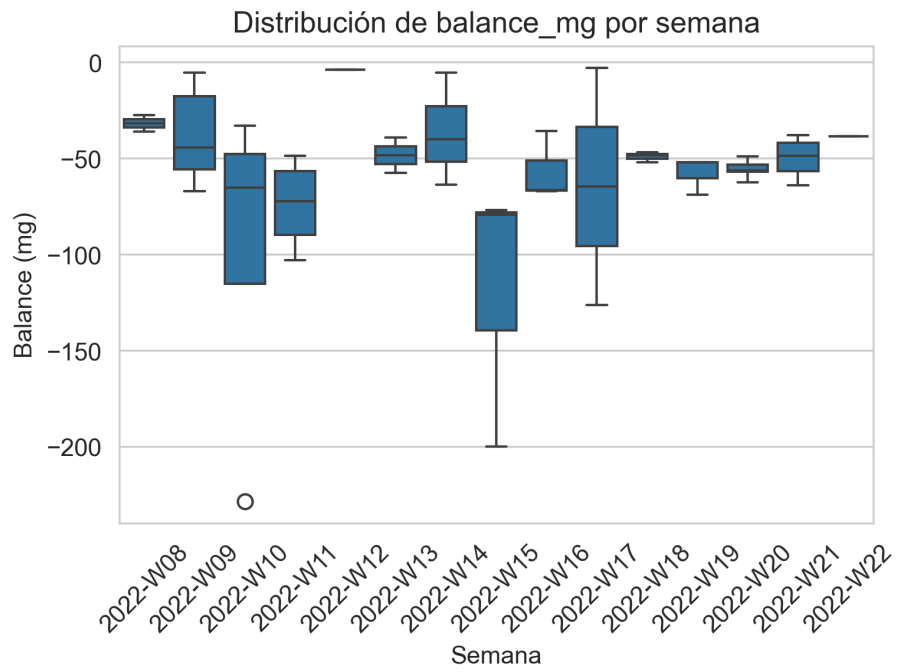


```
# Crear el boxplot
sns.boxplot(x='semana_iso', y='balance_mg', data=df_sns)

# Opcional: mejorar la visualización
plt.title('Distribución de balance_mg por semana')
plt.xlabel('Semana')
plt.ylabel('Balance (mg)')
plt.xticks(rotation=45) # Si los nombres de semana son largos

plt.show()
```







## Capítulo 11

# Los rendimientos queseros

En este capítulo abordaremos el análisis de la eficiencia de la transformación quesera, mediante los indicadores del rendimiento quesero, que calcularemos a partir de los valores analíticos de la leche de fabricación y del queso recién elaborado. Extraeremos diversas conclusiones:

- Estableceremos los parámetros de referencia de nuestro propio proceso, que nos servirán de punto de comparación para las fabricaciones que vayamos realizando.
- Evaluaremos las causas posibles de las desviaciones, y veremos cómo proponer planes de acción para la reducción o eliminación de estas desviaciones.

### 11.1. Un ejemplo de cálculo de rendimientos

En este ejemplo, vamos a suponer que elaboramos un queso de vaca, del que tenemos valores analíticos de producto terminado y de la leche utilizada en su elaboración. El proceso siempre tiene los mismos pasos tecnológicos, con la variabilidad que suponemos que es la habitual.

Utilizaremos `Python` para el cálculo, al final se proporciona también una hoja de cálculo con la información necesaria.

Como siempre, es opcional descargar los datos de ejemplo y abrir el cuaderno en Colab para ejecutar el código a medida que se avanza en el estudio.

Abrir este cuaderno en Google Colab

Descargar los datos de ejemplo utilizados en este cuaderno (archivo `fab_queso_bm.csv`)

## 11.2. Lectura y exploración de los datos

Antes de lanzarnos al análisis cuantitativo, siempre es conveniente una primera visualización de los datos, por si hubiese algún valor que nos pueda resultar cuestionable.

Es una mala práctica, aunque muy extendida, usar la hoja de cálculo para hacer la media aritmética de cada parámetro que vayamos a utilizar y trabajar con estos valores medios. En un conjunto de datos que recoja muchas fabricaciones, nunca podemos estar seguros de que el 100 % de los valores sean correctos, que no haya habido errores de muestreo, analíticos o problemas de fabricación que hayan producido valores anormales. Por eso vamos a utilizar siempre los valores *medianos* y no los valores medios; como sabemos, la **mediana** es resistente a los valores anormales y extremos, mientras que la media no lo es.

Procedemos a leer los datos, y visualizamos las primeras líneas. Se incluye el código `python` que permite visualizar las tablas, en algunos casos se han formateado mediante `HTML`.

|             | fecha      | receta | litros_past | est_past | mg_past | mp_past | kg_cuba  | est_c |
|-------------|------------|--------|-------------|----------|---------|---------|----------|-------|
| fecha_index |            |        |             |          |         |         |          |       |
| 2022-03-01  | 01/03/2022 | vaca   | 15350       | 12.542   | 4.200   | 3.076   | 15810.99 | 11.37 |
| 2022-03-04  | 04/03/2022 | vaca   | 14530       | 13.312   | 4.002   | 3.673   | 14989.74 | 13.25 |
| 2022-03-25  | 25/03/2022 | vaca   | 12633       | 12.856   | 3.889   | 3.372   | 13011.79 | 12.89 |
| 2022-04-01  | 01/04/2022 | vaca   | 15430       | 13.151   | 3.886   | 3.218   | 15946.80 | 11.88 |
| 2022-04-08  | 08/04/2022 | vaca   | 15930       | 13.103   | 4.022   | 3.396   | 16452.76 | 11.93 |

Los datos se corresponden con un conjunto de fabricaciones de queso de vaca, como hemos dicho. Las principales variables recogidas son las siguientes:

| Variable             | Descripción                                                   |
|----------------------|---------------------------------------------------------------|
| fecha                | Fecha de producción                                           |
| receta               | Tipo de receta utilizada                                      |
| litros_past          | Litros de leche pasteurizada                                  |
| est_past             | Extracto seco total de la leche pasteurizada (g/100 ml leche) |
| mg_past              | Materia grasa de la leche pasteurizada (g/100 ml leche)       |
| mp_past              | Materia proteica de la leche pasteurizada (g/100 ml leche)    |
| kg_cuba              | Kilos totales de leche en la cuba                             |
| est_cuba             | Extracto seco total en cuba (g/100 g leche)                   |
| mg_cuba              | Materia grasa en cuba (g/100 g leche)                         |
| mp_cuba              | Materia proteica en cuba (g/100 g leche)                      |
| lactosa_cuba         | Contenido de lactosa en cuba (g/100 g leche)                  |
| ph_final_moldeo_cuba | pH final antes del moldeo                                     |
| formato              | Formato del queso una vez moldeado                            |
| est_salida_salmuera  | Extracto seco total tras salida de salado (g/100 g queso)     |

| Variable           | Descripción                                                         |
|--------------------|---------------------------------------------------------------------|
| mg_salida_salmuera | Materia grasa tras salida de salado (g/100 g queso)                 |
| ph_salida_salmuera | pH tras salida de salado                                            |
| peso_queso_total   | Peso total del queso producido (medido a la salida de la salmuera)  |
| sal_queso          | Contenido de sal en el queso tras salida de salado (g/ 100 g queso) |

### 11.3. Rendimiento simple

Por esta razón, debemos calcular el rendimiento mediante parámetros que no se vean afectados por la humedad del queso, sino que reflejen con precisión la calidad del trabajo quesero. La mejor forma de hacerlo consiste en calcular el porcentaje de materia quesera que hemos sido capaces de retener en el queso. Dado que los elementos de valor de la leche, que son los que se utilizan en el pago por calidad, son la materia grasa y la proteína, calcularemos la recuperación de materia grasa, de materia proteica y de extracto seco magro.

Estos valores son propios de cada tecnología quesera, y cada uno sirve de referencia para comprender cuáles son los diferentes problemas en fabricación,

- Una mala retención de materia grasa
- Una mala retención de proteína
- Una mala retención de extracto seco magro

---

El laboratorio nos facilita la información analítica de la leche pasteurizada en porcentaje masa/volumen, mientras que la leche en cubas y el queso es masa/masa. Esto es así porque la cantidad de leche que pasa por el pasteurizador se mide con un contador volumétrico, mientras que en la cuba se mide con una célula de carga.

Si el volumen de leche en cuba se midiese también con contador volumétrico, necesitaríamos convertir los resultados analíticos a masa/volumen para obtener los kilos de cada materia, utilizando la densidad.

En la práctica, usar una densidad promedio fija o ajustada por estaciones funciona bastante bien, y evita los errores de medida de este parámetro que parecen sencillos pero siempre están sometidos al error de muestreo, además de la necesidad de corregir la temperatura.

Veamos los datos analíticos del queso

| Composición del queso  | Media (g/100 g) | Mediana (g/100 g) |
|------------------------|-----------------|-------------------|
| EST a salida de salado | 55.16 %         | 54.85 %           |
| MG a salida de salado  | 29.74 %         | 29.80 %           |

Para hacer este producto, estamos trabajando con una leche que hemos enriquecido en MG y MP, cuya composición es:

| Composición materia prima | Media (g/L) | Mediana (g/L) |
|---------------------------|-------------|---------------|
| Extracto seco total (ESM) | 123.17      | 119.30        |
| Materia grasa (MG)        | 38.48       | 38.10         |
| Proteínas totales (MP)    | 31.98       | 32.10         |

#### 11.4. Cálculo de la cantidad de leche necesaria para fabricar 1 kg de queso

Sabemos que nuestro proceso de fabricación quesera consiste en la coagulación de la caseína y expulsión de suero, y que la materia grasa queda retenida en la red de caseína.

Este concepto es la base fundamental de la comprensión de la tecnología quesera: es el rendimiento en la recuperación de la proteína el que nos va a definir todo el proceso, la materia grasa “acompañará” de forma estática (aunque tendrá influencia en el desuerado)

Por esta razón, el principal elemento del rendimiento en la definición de la tecnología es el porcentaje de recuperación de proteína en el extracto seco magro (aislamos el efecto la grasa)

Hemos definido nuestro producto mediante los análisis de EST y MG; necesitamos saber el resto de parámetros, en concreto el ESM que estamos definiendo para nuestro producto:

| Composición del queso     | Media (g/100 g) | Mediana (g/100 g) |
|---------------------------|-----------------|-------------------|
| EST a salida de salado    | 55.16 %         | 54.85 %           |
| MG a salida de salado     | 29.74 %         | 29.80 %           |
| ESM a salida de salado)   | 25.42 %         | 25.05 %           |
| MG/EST a salida de salado | 53.91 %         | 54.33 %           |

Ahora necesitamos conocer cuál es la tasa o ratio de conversión en ESM de queso del ESM, o bien de la MP, de la leche en cubas. Para ello calculamos estos ratios, que vamos a llamar **coeficiente de recuperación de ESM**, y el **coeficiente de recuperación de MP** junto con el **coeficiente de recuperación de MG**, que indicará la cantidad de MG que somos capaces de recuperar en el queso.

Si la recuperación de MG fuese perfecta, el coeficiente de recuperación sería 100 %. El coeficiente refleja que hay pérdidas de MG, bien en el suero, bien en los finos de moldeo o en cuajada que se ha caído, etc, en resumen, que no ha ido

#### 11.4. CÁLCULO DE LA CANTIDAD DE LECHE NECESARIA PARA FABRICAR 1 KG DE QUESO171

a parar al queso moldeado. Podemos argumentar, correctamente, que la parte de los finos recoge también la eficiencia de la coagulación, pero al final lo que nos interesa es recoger la conversión real en queso de nuestra materia prima, excluyendo todo lo que constituyen las **pérdidas de proceso**.

El **coeficiente de recuperación de ESM** tiene un valor mucho más bajo, porque hay una parte significativa del ESM de la leche que vamos a perder en el suero y durante el salado, como las proteínas solubles, la lactosas y las sales minerales; por otra parte, vamos a incorporar a ese ESM la sal añadida durante el salado.

Lo importante es que el valor de este coeficiente, si nuestro proceso es estable, debe ser siempre constante: en un proceso estable, tanto las pérdidas como las ganancias deben ser muy semejantes de un día a otro, si las condiciones del proceso se han mantenido. Por lo tanto, lo que nos interesa es este valor *mediano* del coeficiente, y debemos asegurarnos de que es regular.

EL último coeficiente, el **coeficiente de recuperación de MP** se basa en esta misma regularidad del proceso para establecer la relación entre la proteína puesta en cuba y el ESM del queso. La ventaja de este coeficiente es que con los medios analíticos habituales, es más fácil analizar la leche mediante un NIR (*near-infrared*) como el MilkoScan, que hacer un extraco seco convencional en estufa.

```
### 1. Cálculos de cuba
df['est_kg_cuba'] = df['kg_cuba'] * df['est_cuba'] / 100
df['mg_kg_cuba'] = df['kg_cuba'] * df['mg_cuba'] / 100
df['mp_kg_cuba'] = df['kg_cuba'] * df['mp_cuba'] / 100
df['esm_kg_cuba'] = df['est_kg_cuba'] - df['mg_kg_cuba']

### 2. Cálculos de queso
### ejemplo de cómo poner fórmulas largas para que no desborden
### la línea
df['est_kg_queso'] = (
    df['peso_queso_total'] * df['est_salida_salmuera'] / 100
)
df['mg_kg_queso'] = (
    df['peso_queso_total'] * df['mg_salida_salmuera'] / 100
)

df['esm_kg_queso'] = df['est_kg_queso'] - df['mg_kg_queso']
df['sal_kg_queso'] = df['peso_queso_total'] * df['sal_queso'] / 100

### 3. Coeficientes de recuperación (expresado en porcentaje)
df['coef_recup_mg'] = df['mg_kg_queso'] / df['mg_kg_cuba'] * 100
df['coef_recup_mp'] = df['esm_kg_queso'] / df['mp_kg_cuba'] * 100
df['coef_recup_esm'] = df['esm_kg_queso'] / df['esm_kg_cuba'] * 100
```

```

print(
    f"Coeficiente de recuperación de materia grasa "
    f"(MG queso/MG leche en cuba): "
    f"{df['coef_recup_mg'].mean():.3f}"
)

print(
    f"Coeficiente de recuperación de proteína "
    f"(ESM queso/MP leche en cuba): "
    f"{df['coef_recup_mp'].mean():.3f}"
)

print(
    f"Coeficiente de recuperación de extracto seco magro "
    f"(ESM queso/ESM leche en cuba): "
    f"{df['coef_recup_esm'].mean():.3f}"
)

```

Coeficiente de recuperación de materia grasa (MG queso/MG leche en cuba): 91.694

Coeficiente de recuperación de proteína (ESM queso/MP leche en cuba): 94.457

Coeficiente de recuperación de extracto seco magro (ESM queso/ESM leche en cuba): 35.94

A partir de aquí, hacemos los cálculos teóricos del estándar técnico.

Utilizando el coeficiente de recuperación de la MP, podemos calcular la cantidad de ESM que vamos a recuperar. La mediana de la cantidad de proteína que tenemos en la leche es 32,10 g/L; sabemos que con 1 L de leche vamos a obtener en el queso

$$\frac{32.10 \text{ g proteina}}{1 \text{ L leche}} * \frac{0.94457 \text{ g ESM}}{1 \text{ g proteina}} = 30.32 \text{ g ESM en el queso por litro de leche}$$

Esta es la conversión clave de nuestra tecnología, y es conocida como **coeficiente G** en honor a su autor, Antoine M Guérault, que lo introdujo formalmente por primera vez por en su libro *La fromagerie devant les techniques nouvelles*, en 1966; desde entonces sigue utilizándose con plena validez. Originalmente, el coeficiente indica el porcentaje de ESM que hemos recuperado en el queso, dividiendo el ESM del queso entre el ESM de la leche puesta en fabricación, aunque es más habitual usar el coeficiente de recuperación de proteína en vez del de ESM, como hemos hecho arriba.

Como comentamos más atrás, para una tecnología concreta, normalmente la cantidad de sal incluida en el salado y las pérdidas de ESM en el proceso de fabricación deberían mantenerse en una horquilla más o menos estrecha, de acuerdo con la variabilidad de nuestro proceso y nuestra precisión analítica. Si



#### 11.4. CÁLCULO DE LA CANTIDAD DE LECHE NECESARIA PARA FABRICAR 1 KG DE QUESO 173

el valor del coeficiente se mantiene en estos límites, nos indicará que la transformación se ha realizado dentro de los valores normales de la tecnología; en caso contrario, si hemos tenido valores anormalmente altos o bajos, será indicador de incidencias tecnológicas que intentaremos explicar para poner en marcha acciones correctivas. En este capítulo veremos el cálculo de los índices; en un capítulo posterior abordaremos el análisis de las desviaciones.

Como sabemos la cantidad de ESM que vamos a obtener de 1 L de leche, podemos calcular los litros de leche que necesitamos para obtener 1 kg de queso mediante una conversión simple. A este valor lo llamaremos *litraje unitario*.

$$\text{Litraje unitario} = \frac{25.05 \text{ g ESM}}{100 \text{ g queso}} * \frac{1000 \text{ g queso}}{1 \text{ kg queso}} * \frac{1 \text{ L leche}}{30,32 \text{ g ESM}} = 8,261 \text{ L leche para hacer 1 kg de queso}$$

y el consumo de MP por kilo de queso es el que resulta de calcular

$$8,261 \text{ L leche} * \frac{32.10 \text{ g MP}}{1 \text{ L leche}} = 265,18 \text{ g MP por kilo de queso}$$

```
### 5. Litrajes unitarios: (g ESM en queso por kilo de queso) / (g MP leche en cuba por litro de
df['litraje_unitario'] = (df['esm_kg_queso']/df['peso_queso_total']) * 100 / ((df['mp_cuba'] / 1

### 6. Consumos de materia por kilo de queso (g / kg)
df['consumo_mg_kg'] = df['mg_kg_cuba']/df['peso_queso_total'] * 1000
df['consumo_mp_kg'] = df['mp_kg_cuba']/df['peso_queso_total'] * 1000

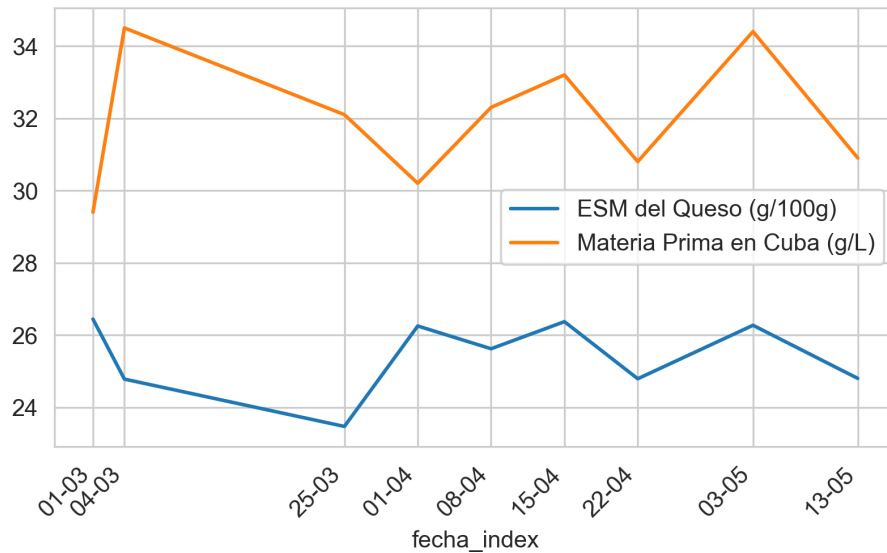
import matplotlib.dates as mdates

esm_queso = df['esm_kg_queso'] / df['peso_queso_total'] * 100
mp_cuba = df['mp_cuba'] * 10

# 1. Graficar y capturar los ejes
ax = esm_queso.plot(label='ESM del Queso (g/100g)')
mp_cuba.plot(label='Materia Prima en Cuba (g/L)', ax=ax)

# 2. Definir y aplicar el formateador
# Usamos "%d-%m" para mostrar Día-Mes sin el año.
ax.set_xticks(df.index)
date_form = mdates.DateFormatter("%d-%m")
ax.xaxis.set_major_formatter(date_form)

# 3. Ajustes de visualización
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout() # Ajusta automáticamente el gráfico para que las etiquetas no se corten
plt.show()
```



Es necesario insistir en que estos cálculos teóricos sólo son válidos en nuestras condiciones de trabajo precisas:

- con la composición en ESM de nuestro queso
- con la composición de proteína de nuestra leche
- con la tasa de recuperación de la proteína de la leche en el ESM del queso de nuestro específico proceso de fabricación.

Son valores teóricos que utilizamos como valor objetivo, y que pueden variar a medida que adaptemos o variemos nuestra tecnología. Para cualquier otro producto, otra composición de leche y/u otro proceso de fabricación, deberemos recalcular nuestro litraje por kilo y nuestro consumo de materia.

Sabemos que, en nuestras condiciones, 8,261 L de leche deben darnos 1 kg de queso. Si obtenemos más o menos queso querrá decir que nuestras hipótesis de partida no se han cumplido, y analizando los tres elementos básicos, sabremos la razón de la desviación:

- Nuestra leche tiene una composición diferente
- Nuestro queso tiene una composición diferente
- Nuestra recuperación de proteína ha sido diferente.

Si disponemos de la analítica correspondiente, podremos tomar las decisiones correctas; no es lo mismo actuar ante un problema de composición de leche que ante un problema de composición de queso o un problema de tecnología quesera (el extracto seco magro del queso es consecuencia de la tecnología quesera). Otros factores pueden influir directamente en el coeficiente de recuperación de

proteína, por ejemplo, la cantidad de proteína coagulable (caseína) frente a la proteína total analizada puede variar a lo largo del año, y el análisis de proteína total no nos da una indicación de esta ratio.

## 11.5. La materia grasa

Hasta aquí hemos hablado de proteína y ESM, pero ¿y la materia grasa?

En nuestro análisis de rendimiento hemos calculado también un % de recuperación de MG. Sin embargo, no utilizaremos este % para calcular el litraje, lo haremos tal como hemos visto, con la transformación de la proteína en ESM. La razón es que *la tecnología quesera nos permite actuar sobre el ESM del queso, pero la MG depende de la estandarización*, como veremos a continuación.

Como el cálculo que hemos realizado con el coeficiente de recuperación de la MP nos da los litros de leche que necesitamos para hacer 1 kg de queso, y tenemos una composición de nuestra leche, podemos saber la cantidad de MG que vamos a tener en esos litros. Por otra parte, sabemos la MG que hemos definido en nuestro queso de forma empírica, es decir, en la práctica: mediante un promedio de los diferentes análisis de nuestro producto que hemos ido obteniendo.

Hemos calculado nuestro litraje unitario: necesitamos 8,261 L leche para hacer 1 kg de queso.

Según nuestra composición de leche, 1 L de leche tiene 38,10 g de MG, luego los 8,261 L de leche tendrán

$$8,261 \text{ L leche} * \frac{38.10 \text{ g MG}}{1 \text{ L leche}} = 314,74 \text{ g MG}$$

Es decir, para fabricar 1 kg de queso necesitamos 8.261 L leche, que según su composición tienen 314,74 g MG

Sin embargo, la analítica de nuestro queso nos dice que en 1 kg de queso tenemos 29.80 % de MG, es decir 298.0 g de MG por kilo de queso.

¿A qué se debe esta diferencia? La respuesta es: *a las pérdidas de MG en el proceso*, es decir, *a nuestro porcentaje de recuperación de MG*.

Podemos calcular este porcentaje mediante la fracción

$$\frac{\text{MG en el queso}}{\text{MG en la leche de fabricacin}} = \frac{298,0 \text{ g MG/kg en el queso}}{314,74 \text{ g MG/kg en la leche de fabricacin}} = 0,9468$$

Es decir, nuestra recuperación teórica de MG es del 94.68 %

¿Por qué *recuperación teórica*? Porque es la que resulta de hacer los cálculos según nuestros datos de composición de leche, composición del queso y recuperación de proteína, no de los análisis realizados.

De la misma forma que en el caso de la recuperación de proteína, si nuestra recuperación de MG es diferente de la calculada teóricamente, debemos encontrar la explicación en los mismos factores:

- Composición real de la leche
- Composición real del queso
- Variaciones en la tecnología

Estas variaciones pueden implicar una mayor pérdida de MG, en el suero de quesería o en los finos de queso (o en ambos), y que en parte puede recuperarse mediante un desnatado del suero.

### 11.6. El estándar de fabricación o estándar técnico

Como hemos visto, la construcción del estándar técnico y el seguimiento de la tecnología quesera no se hacen siguiendo el consumo de *litros de leche*, sino de la cantidad de proteína y grasa utilizadas. Por esta razón, la construcción del estándar debe proporcionar las cantidades de MG y MP necesarias para obtener 1 kg de queso, y no sólo el litraje en cuba.

El razonamiento y cálculos que hemos ido realizando, se suelen recoger en lo que se llama un *estándar técnico de fabricación*. Este documento recoge los valores estándar de nuestra tecnología dada una composición media de nuestro queso obtenido (o un objetivo a obtener) y una composición de leche de partida. Al final del capítulo se puede descargar una hoja Excel con el estándar de fabricación que hemos ido construyendo para nuestro queso de vaca.

Llamamos **MG teórica en la cuba** a la cantidad de MG que calculamos que vamos a tener en la cuba, es decir, en nuestros 8,261 L de leche a fabricar. Esta MG puede provenir de la leche de partida o del resultado de estandarizar la leche en MG, añadiendo nata o bien leche desnatada, para conseguir la cantidad de MG que nos permita alcanzar los valores de MG deseada en el queso.

De la misma forma, llamamos **MP en cuba** a la cantidad de MP que hemos puesto en cuba, ya sea la de la leche de partida o del resultado de haber estandarizado la leche mediante el añadido de proteínas.

En el caso de nuestro queso de vaca, vemos un proceso de fabricación que *no* usa estandarización y que no está recuperando la materia grasa del suero; las diferencias entre la MG en cuba y la MG en el queso se consideran pérdidas de proceso y su reducción es un objetivo prioritario del proceso de mejora.

## 11.7. La importancia de la estandarización de la leche

La tecnología quesera se diseña para optimizar el coeficiente de recuperación de proteína. Aunque la retención de materia grasa (MG) también depende de factores tecnológicos como el corte, el calentamiento y el pH, no está directamente controlada por la tecnología, aunque sí influenciada por ella.

En una tecnología concreta, con parámetros de trabajo fijados, cuando la cantidad de MG en la leche es superior a la definida en el proceso, se producen dos efectos:

- Una parte del exceso de MG queda retenida en la matriz proteica, lo que incrementa la cantidad de grasa en el queso y, por tanto, modifica su composición.
- Otra parte de la MG se pierde en el suero, ya que las condiciones del proceso determinan una relación fija con la cantidad total de MG de partida. Es decir, al aumentar la MG inicial, también aumentan las pérdidas relativas en el suero.

Desgraciadamente, no es posible calcular teóricamente cuánta MG se retendrá en el queso y cuánta se perderá en el suero, ya que estas cantidades no responden a una función conocida. Lo que sí sabemos es que:

- Aumentarán las pérdidas.
- Se modificará la composición del queso, lo que puede alterar su textura y el valor nutricional declarado.
- Se alterará la expulsión de suero de la matriz proteica, lo que afectará la cantidad de lactosa retenida en la cuajada y, en consecuencia, las curvas de acidificación, el extracto seco, etc.

Cuando la MG es *inferior* a la definida, estos efectos se producen en sentido inverso, afectando tanto la composición del queso como su comportamiento tecnológico.

Para minimizar estas variaciones y mantener una composición constante del producto, es necesario:

- Trabajar con leche que tenga un contenido estable de materia grasa (MG) y materia proteica (MP). Dado que la composición de la leche varía estacionalmente, esto implica estandarizarla en MG y MP, **manteniendo constante la relación MG/MP**.
- Adaptar la tecnología de forma estacional, ya que la variación estacional no solo afecta la relación MG/MP, sino también la proporción de caseína coagulable dentro del total de proteínas.

Es fundamental recordar lo señalado al definir el estándar técnico: *la tecnología permite modificar el extracto seco magro (ESM) del queso, pero la MG depende principalmente de la cantidad presente en la leche de fabricación. No es posible modificar la cantidad de grasa en el queso mediante la tecnología quesera en la cuba*; es necesario preparar la leche con la cantidad de grasa adecuada para cubrir tanto la retención esperada como las pérdidas previstas.

En la medida en que no se puedan realizar estas adaptaciones, se deberá aceptar una mayor variabilidad en el producto, con dos consecuencias:

- Irregularidad en el producto final, que afecta la aceptación por parte del consumidor.
- Variación en el coste del producto, que puede dar lugar a pérdidas económicas no previstas.

Por estas razones, es imprescindible realizar un seguimiento detallado de las desviaciones tecnológicas, para comprender mejor el comportamiento del proceso y actuar eficazmente sobre sus causas. Las desviaciones observadas deben analizarse siempre apoyándose en el conocimiento tecnológico, para que los planes correctivos sean eficaces.

Conociendo el coste de la MG y de la MP pagada, es posible calcular el valor económico de cada desviación, lo que permite priorizar las acciones correctivas según su impacto financiero. El análisis de las desviaciones se hace en un capítulo posterior.

Hay una hoja Excel con todos los cálculos, que incluye también los estándares de fabricación que veremos en el capítulo siguiente para el resto de recetas.

Abrir hoja Excel con estándar técnico

Veamos las características de los quesos, excluyendo ya los ensayos de I+D

```
df['esm_salida_salmuera'] = df['est_salida_salmuera'] - df['mg_salida_salmuera']
df['hqd_salida_salmuera'] = (100 - df['est_salida_salmuera']) / (100 - df['mg_salida_salmuera'])
df['ges_salida_salmuera'] = df['mg_salida_salmuera'] / df['est_salida_salmuera']
```

```
# Filtrar receta 'i+D'
df_filtrado = df[df['receta'] != 'i+D'].reset_index(drop=True)

# Crear paleta de colores por receta
recetas = df_filtrado['receta'].unique()
palette = sns.color_palette('tab10', n_colors=len(recetas))
color_dict = dict(zip(recetas, palette))

sns.scatterplot(data=df[df['receta'] != 'i+D'],
                x='est_salida_salmuera',
                y='mg_salida_salmuera',
                hue='receta',
```

```

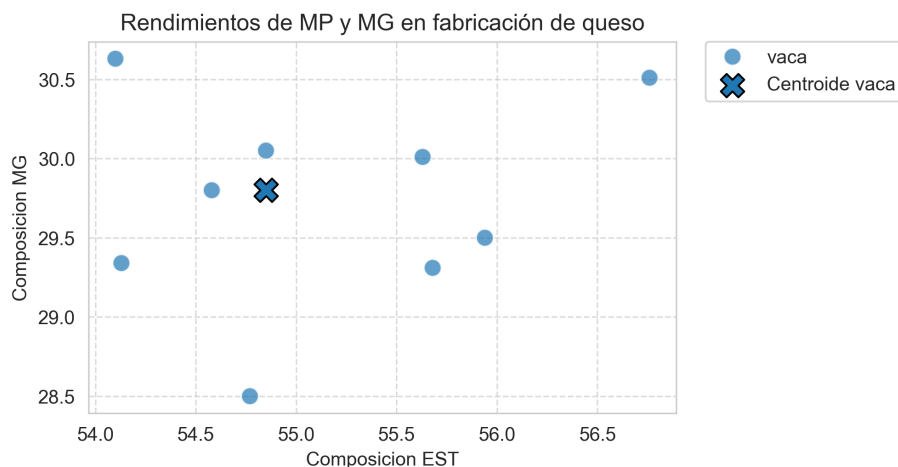
        palette = color_dict,
        s=80,
        # edgecolor = 'black',
        alpha=0.7)

# Calcular y dibujar centroides
centroides_est_mg = df.groupby('receta')[['est_salida_salmuera', 'mg_salida_salmuera']].median()

for receta in recetas:
    centro = centroides_est_mg.loc[receta]
    plt.scatter(
        centro['est_salida_salmuera'],
        centro['mg_salida_salmuera'],
        color=color_dict[receta],
        marker='X',
        s=150,
        edgecolor='black',
        label=f'Centroide {receta}')
)

plt.title('Rendimientos de MP y MG en fabricación de queso')
plt.xlabel('Composicion EST')
plt.ylabel('Composicion MG')
plt.legend(title='Receta') # Mostrar leyenda para la especie
# Mover la leyenda fuera del gráfico
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=0.)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```



```
print(centroides_est_mg)
```

```

          est_salida_salmuera  mg_salida_salmuera
receta
vaca                54.85                29.8
df.head()
```

|             | fecha      | receta | litros_past | est_past | mg_past | mp_past | kg_cuba  | est_c |
|-------------|------------|--------|-------------|----------|---------|---------|----------|-------|
| fecha_index |            |        |             |          |         |         |          |       |
| 2022-03-01  | 01/03/2022 | vaca   | 15350       | 12.542   | 4.200   | 3.076   | 15810.99 | 11.37 |
| 2022-03-04  | 04/03/2022 | vaca   | 14530       | 13.312   | 4.002   | 3.673   | 14989.74 | 13.25 |
| 2022-03-25  | 25/03/2022 | vaca   | 12633       | 12.856   | 3.889   | 3.372   | 13011.79 | 12.89 |
| 2022-04-01  | 01/04/2022 | vaca   | 15430       | 13.151   | 3.886   | 3.218   | 15946.80 | 11.88 |
| 2022-04-08  | 08/04/2022 | vaca   | 15930       | 13.103   | 4.022   | 3.396   | 16452.76 | 11.93 |

```

# Crear paleta de colores por receta
recetas = df['receta'].unique()
palette = sns.color_palette('tab10', n_colors=len(recetas))
color_dict = dict(zip(recetas, palette))

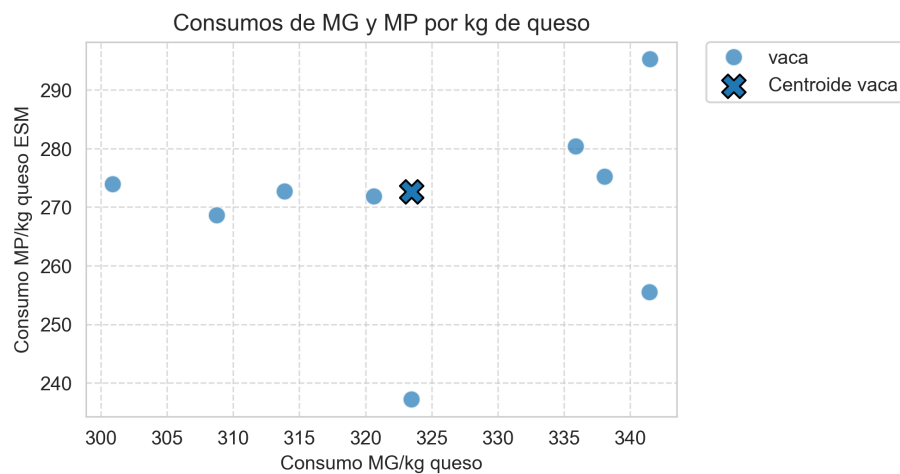
sns.scatterplot(data=df[df['receta'] != 'i+D'],
                x='consumo_mg_kg',
                y='consumo_mp_kg',
                hue='receta',
                palette = color_dict,
                s=80,
                # edgecolor = 'black',
                alpha=0.7)

# Calcular y dibujar centroides
centroides_consumos_materia = df.groupby('receta')[['consumo_mg_kg', 'consumo_mp_kg']]

for receta in recetas:
    centro = centroides_consumos_materia.loc[receta]
    plt.scatter(
        centro['consumo_mg_kg'],
        centro['consumo_mp_kg'],
        color=color_dict[receta],
        marker='X',
        s=150,
        edgecolor='black',
        label=f'Centroide {receta}'
    )
```



```
plt.title('Consumos de MG y MP por kg de queso')
plt.xlabel('Consumo MG/kg queso')
plt.ylabel('Consumo MP/kg queso ESM')
plt.legend(title='Receta') # Mostrar leyenda para la especie
# Mover la leyenda fuera del gráfico
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=0.)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```





## Capítulo 12

# La valoración económica del balance de materia y los rendimientos.

### 12.1. El pago de la leche por calidad en España

El pago por calidad de la leche de vaca en España se basa en parámetros físico-químicos y sanitarios como la materia grasa (MG), materia proteica (MP), células somáticas y bacterias. Estos factores determinan el precio final que recibe el ganadero por litro entregado.

#### 12.1.1. Conceptos clave del pago por calidad

El sistema de pago por calidad busca incentivar la producción de leche con mejores características nutricionales e higiénicas. Los principales criterios son:

##### 1. Composición química

- **Materia Grasa (MG):** Aporta valor energético. Cuanto mayor el porcentaje, mayor el precio.
- **Materia Proteica (MP):** Indicador de calidad nutricional. Muy valorada por la industria quesera.
- **Urea:** Indicador del equilibrio nutricional de la dieta del ganado.

##### 2. Calidad higiénico-sanitaria

- **Recuento de células somáticas (RCS):** Refleja la salud mamaria. Valores altos penalizan el precio.

- **Recuento de bacterias (RB):** Mide la higiene en el ordeño y conservación. Penaliza si supera ciertos umbrales.
- **Inhibidores:** Presencia de antibióticos u otras sustancias. Si se detectan, la leche se rechaza.

### 3. Otros factores

- **Punto crioscópico:** Detecta adulteraciones (agua añadida).
- **Letra Q:** Etiquetado voluntario que certifica calidad superior. Está controlado por la autoridades veterinarias de cada región.
- **Marcas de calidad:** DOP, IGP, ecológica, etc., pueden mejorar el precio.

#### 12.1.2. Aplicación práctica: ejemplo de cálculo

Supongamos que un ganadero entrega 10.000 litros de leche con la siguiente composición:

- **MG:** 4,10 %
- **MP:** 3,45 %
- **RCS:** 180.000 células/ml
- **RB:** 20.000 ufc/ml

La industria tiene una tabla de precios base de 0,490 €/litro, con bonificaciones:

| Parámetro     | Valor | Bonificación |
|---------------|-------|--------------|
| MG > 4,00 %   |       | +0,005 €/l   |
| MP > 3,40 %   |       | +0,004 €/l   |
| RCS < 200.000 |       | +0,003 €/l   |
| RB < 50.000   |       | +0,002 €/l   |

**Precio final por litro:**

$$0,490 + 0,005 + 0,004 + 0,003 + 0,002 = 0,504 \text{ €/l}$$

**Total a percibir:**

$$10.000 \text{ l} \cdot 0,504 \text{ €/l} = 5.040 \text{ €}$$

De esta manera se establece un precio por litro, que se aplica mensualmente tras el análisis de las muestras representativas. En las diferentes regiones, este proceso es gestionado por los laboratorios interprofesionales lecheros mediante protocolos de muestreo y análisis estandarizados. El Ministerio de Agricultura también ofrece ayudas específicas por calidad bajo marcas reconocidas o la Letra Q.

Es importante recordar que cada industria puede tener su propia tabla de bonificaciones y penalizaciones, es decir, la importancia de las bonificaciones y penalizaciones son acordadas por la negociación entre ganaderos (o centrales lecheras) y la industria. En el ejemplo actual, vemos que la bonificación por materia grasa es superior a la bonificación por proteína; este tipo de bonificación, aplicable en la industria lechera, no es recomendable en la industria quesera, que valora sobre todo la riqueza en proteína coagulable (caseína). Por esta razón, no siempre las características de la leche recogida por la industria lechera es idónea para la fabricación de queso. ### La conversión del precio por litro en los costes totales de materia prima, grasa y proteína. Una vez establecido el precio de la leche, a la industria quesera (y lechera) le interesa descomponer este precio en los ingredientes que van a ser objeto de transformación, la materia grasa y la proteína, para poder establecer con precisión los costes de materia en producción.

La transformación quesera está determinada por la eficiencia en la retención de estas materias primas en el queso, como veremos en el análisis de los rendimientos queseros. La materia prima más importante para el quesero es la proteína, que forma la matriz quesera; la materia grasa se necesita como elemento que confiere sabor y textura al queso.

Hasta hace unos años, el precio de la leche se determinaba por su riqueza en **materia seca útil (MSU)**, también llamada **extracto seco quesero (ESQ)**, que se calcula sumando el contenido de materia grasa más la proteína:

$$MSU = MG + MP$$

Este parámetro, que sigue siendo aplicado en el caso de la leche de oveja y la leche de cabra, no es, sin embargo, de uso recomendable en la industria quesera, porque el productor puede incrementar la cantidad de materia grasa en la leche con determinados tipos de alimentación animal que permiten subir el contenido en MG de la leche, pero no el de la proteína, que está determinado fundamentalmente por la genética del animal.

El efecto de un contenido en materia grasa demasiado elevado es una leche desequilibrada, con una cantidad de materia grasa que no puede ser utilizada en la producción de queso porque se rebasaría el contenido en grasa del queso, obligando al quesero a desnatar parte de la leche y vender la nata como excedente a bajo precio, impactando negativamente en sus resultados.

La forma de controlar la cantidad de materia grasa necesaria en la leche de aprovisionamiento (compra) consiste en el control del factor MG/MP, que debe ser sólo ligeramente superior al del producto final (queso), para cubrir las necesidades del queso más las pérdidas de producción.

Por esta razón, es muy frecuente que la industria quesera no valore de la misma manera la materia grasa y la proteína en la leche de compra, y establezca un criterio de valoración que bonifique la riqueza en proteína, por ejemplo, 40/60.

Si aplicamos este criterio a nuestro precio de leche calculado antes, suponiendo una composición media de la leche de 3,4 g MG/L y 3,1 g MP/L, calcularíamos el precio para el kg de MG y de MP como

$$\text{Precio de la MG: } \frac{0,504 \text{ euros}}{1 \text{ L leche}} \cdot \frac{1 \text{ L leche}}{0,0034 \text{ kg MG}} \cdot 0,40 = 139,73 \frac{\text{euros}}{\text{kg MG}}$$

De la misma manera, podemos establecer el precio de la proteína como sigue:

$$\text{Precio de la MP: } 0,504 \frac{\text{euros}}{\text{L leche}} \cdot \frac{1 \text{ L leche}}{1,030 \text{ kg leche}} \cdot \frac{1 \text{ L leche}}{1,030 \text{ kg leche}} \cdot \frac{1 \text{ L leche}}{0,0031 \text{ kg MG}} \cdot 0,60 = 139,73 \frac{\text{euros}}{\text{kg MG}}$$

|             | fecha_receta | litros_past | est_past | mg_past | mp_past | \     |
|-------------|--------------|-------------|----------|---------|---------|-------|
| fecha_index |              |             |          |         |         |       |
| 2022-03-01  | 01/03/2022   | vaca        | 15350    | 12.542  | 4.200   | 3.076 |
| 2022-03-04  | 04/03/2022   | vaca        | 14530    | 13.312  | 4.002   | 3.673 |
| 2022-03-25  | 25/03/2022   | vaca        | 12633    | 12.856  | 3.889   | 3.372 |
| 2022-04-01  | 01/04/2022   | vaca        | 15430    | 13.151  | 3.886   | 3.218 |
| 2022-04-08  | 08/04/2022   | vaca        | 15930    | 13.103  | 4.022   | 3.396 |

|             | kg_cuba  | est_cuba | mg_cuba | mp_cuba | formato | \ |
|-------------|----------|----------|---------|---------|---------|---|
| fecha_index |          |          |         |         |         |   |
| 2022-03-01  | 15810.99 | 11.37    | 4.01    | 2.94    | barra   |   |
| 2022-03-04  | 14989.74 | 13.25    | 3.79    | 3.45    | barra   |   |
| 2022-03-25  | 13011.79 | 12.89    | 3.69    | 3.21    | barra   |   |
| 2022-04-01  | 15946.80 | 11.88    | 3.71    | 3.02    | barra   |   |
| 2022-04-08  | 16452.76 | 11.93    | 3.81    | 3.23    | barra   |   |

|             | est_salida_salmuera | mg_salida_salmuera | ph_salida_salmuera | \ |
|-------------|---------------------|--------------------|--------------------|---|
| fecha_index |                     |                    |                    |   |
| 2022-03-01  | 55.94               | 29.50              | 5.41               |   |
| 2022-03-04  | 54.58               | 29.80              | 5.88               |   |
| 2022-03-25  | 54.10               | 30.63              | 5.48               |   |
| 2022-04-01  | 56.76               | 30.51              | 5.36               |   |
| 2022-04-08  | 55.63               | 30.01              | 5.31               |   |

|             | peso_queso_total | sal_queso |
|-------------|------------------|-----------|
| fecha_index |                  |           |
| 2022-03-01  | 1960.0           | 1.55      |
| 2022-03-04  | 1888.0           | 1.77      |
| 2022-03-25  | 1555.0           | 1.32      |
| 2022-04-01  | 1750.0           | 1.36      |
| 2022-04-08  | 1955.0           | 1.90      |

## Capítulo 13

# Práctica 2. Análisis de los rendimientos en fabricación de queso mediante Python

En esta práctica utilizaremos un conjunto de datos que provienen de una fábrica de queso tipo ibérico. Con los valores analíticos calcularemos los rendimientos queseros utilizando Python, y nos apoyaremos en diferentes tipos de gráficos para explicar los resultados que hemos obtenido. Utilizaremos los datos calculados aquí para hacer un balance de materia y calcular las pérdidas de nuestra fabricación.

Para ayudar a concentrarse en los resultados, el código *python* se presenta plegado, de manera que no interfiera demasiado en la presentación. Puede desplegarse para ver cómo se han generado cada tabla y cada gráfico.

Como siempre, es opcional descargar los datos de ejemplo y abrir el cuaderno en Colab para ejecutar el código a medida que se avanza en el estudio.

Abrir este cuaderno en Google Colab

Descargar los datos de ejemplo utilizados en este cuaderno (archivo `fab_queso_bm.csv`)

Empezamos el análisis inicializando las bibliotecas de **Python**, cargamos los datos del **CSV**, creamos el índice de fecha y mostramos los primeros registros del dataframe para comprobar que todo es correcto.

---

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```

sns.set_style("whitegrid")

# Python encadena de forma automática las líneas como una sola
url_datos = (
    'https://raw.githubusercontent.com/'
    'juanriera/master-queseria/master/datos/fab_queso_bm.csv'
)

try:
    df = pd.read_csv(url_datos,
                     decimal = ",",
                     sep=';',
                     encoding='ISO-8859-1')
except Exception as e:
    print(f"Error al cargar el archivo: {e}")

# crear index con fecha
df['fecha_index'] = pd.to_datetime(
    df['fecha'],
    format='%d/%m/%Y', # Formato Día/Mes/Año confirmado
    errors='coerce'     # convierte los valores que no puede leer a NaT
                       # (Not a Time)
)

# 2. Limpiar NaT y establecer el índice
df.dropna(subset=['fecha_index'], inplace=True)
df.set_index('fecha_index', inplace=True)
df.sort_index(inplace=True)

df.head()

```

|             | fecha      | receta  | litros_past | est_past | mg_past | mp_past | kg_cuba   | est |
|-------------|------------|---------|-------------|----------|---------|---------|-----------|-----|
| fecha_index |            |         |             |          |         |         |           |     |
| 2022-02-23  | 23/02/2022 | bajo mg | 12380       | 10.760   | 2.58    | 3.550   | 12860.510 | 10  |
| 2022-02-24  | 24/02/2022 | mezcla  | 7440        | 13.290   | 5.15    | 4.090   | 7704.462  | 12  |
| 2022-03-01  | 01/03/2022 | vaca    | 15350       | 12.542   | 4.20    | 3.076   | 15810.990 | 11  |
| 2022-03-01  | 01/03/2022 | bajo mg | 13080       | 11.090   | 2.59    | 3.600   | 13867.370 | 10  |
| 2022-03-04  | 04/03/2022 | mezcla  | 13420       | 14.220   | 5.45    | 4.310   | 14131.770 | 13  |

En total, tenemos 45 entradas que corresponden a fabricaciones diarias, desde el 23/02/2022 hasta el 30/5/2022.

El fichero de datos incluye diferentes datos analíticos:



| Columna                     | Descripción                                      |
|-----------------------------|--------------------------------------------------|
| <b>fecha</b>                | día de fabricación                               |
| <b>receta</b>               | tipo de queso fabricado                          |
| <b>litros_past</b>          | litros de leche enviados a pasteurización        |
| <b>kg_cuba</b>              | cantidad de leche recibida en cuba, en kilos     |
| <b>est_cuba</b>             | extracto seco de la leche en la cuba             |
| <b>mg_cuba</b>              | materia grasa de la leche en la cuba             |
| <b>mp_cuba</b>              | materia proteica de la leche en la cuba          |
| <b>lactosa_cuba</b>         | lactosa de la leche en la cuba                   |
| <b>ph_final_moldeo_cuba</b> | pH del queso al final del proceso de moldeo      |
| <b>formato</b>              | formato de moldeo                                |
| <b>est_salida_salmuera</b>  | extracto seco total del queso después del salado |
| <b>mg_salida_salmuera</b>   | materia grasa del queso después del salado       |
| <b>ph_salida_salmuera</b>   | pH del queso después del salado                  |
| <b>peso_queso_total</b>     | peso del queso después del salado                |
| <b>sal_queso</b>            | porcentaje de sal analizada en el queso          |

El queso se fabrica en varias recetas, y se envasa en diferentes formatos:

| Receta        | Formatos           | Comentario                     |
|---------------|--------------------|--------------------------------|
| i+D (ensayos) | 3 kg, 500 g, barra |                                |
| bajo mg       | 3 kg, 500 g, barra | Queso con bajo contenido en MG |
| mezcla        | 3 kg, 500 g, barra |                                |
| oveja         | barra              |                                |
| vaca          | barra              |                                |

El fichero de datos contiene varios ensayos del departamento de i+D que se han mantenido para mostrar que a veces las cosas no salen bien a la primera...

## 13.1. Exploración gráfica de los datos

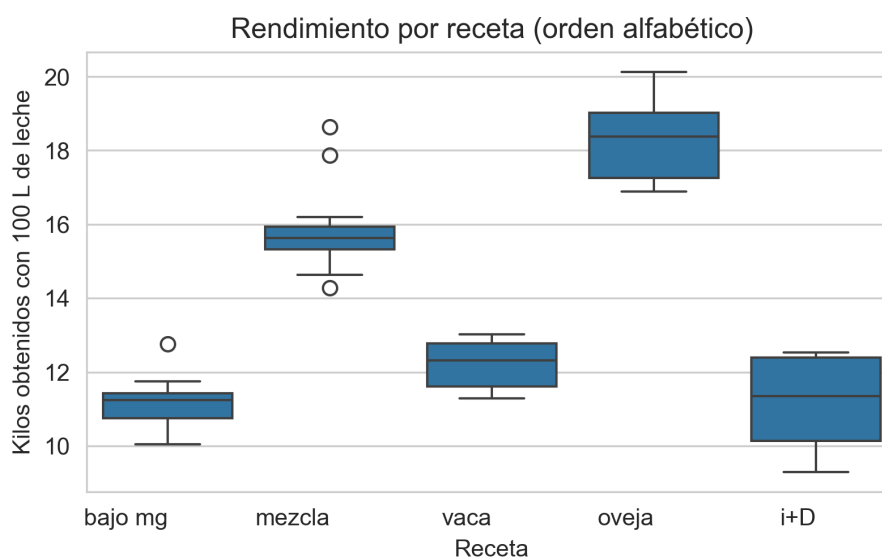
Vamos a explorar los datos antes de empezar el análisis, y los usaremos para visualizar las diferencias de rendimiento entre mezclas.

Podemos calcular el rendimiento simple como los kilos de queso que podemos obtener con 100 L de leche

```
df['rendimiento_simple'] = (
    df['peso_queso_total']/df['litros_past']*100
)
```

```
sns.boxplot(data=df, x='receta', y='rendimiento_simple')
plt.xticks(rotation=0, ha='right')
```

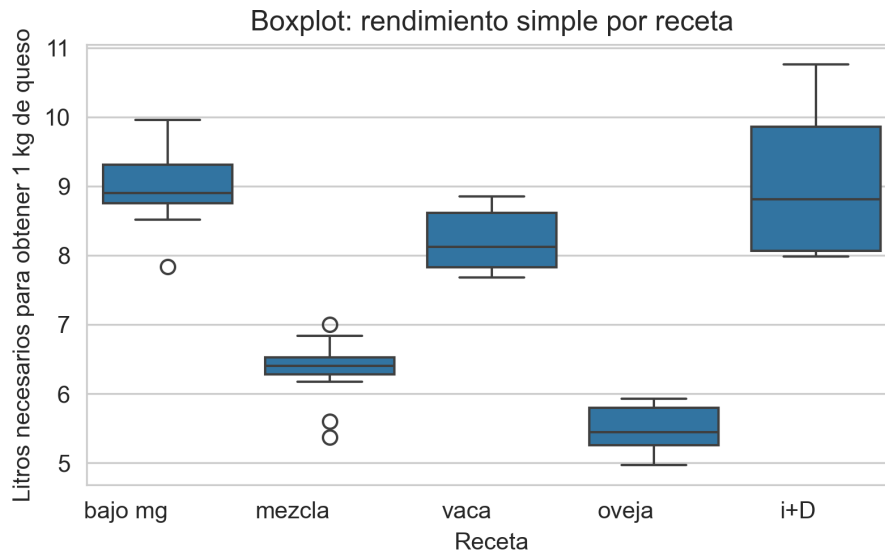
```
plt.title('Rendimiento por receta (orden alfabético)')
plt.xlabel('Receta')
plt.ylabel('Kilos obtenidos con 100 L de leche')
plt.tight_layout()
plt.show()
```



Aunque es más frecuente calcular el rendimiento simple como los litros de leche que necesitamos para hacer un kilo de queso

```
df['rendimiento_simple'] = df['litros_past']/df['peso_queso_total']
```

```
sns.boxplot(data=df, x='receta', y='rendimiento_simple')
plt.xticks(rotation=0, ha='right')
plt.title('Boxplot: rendimiento simple por receta')
plt.xlabel('Receta')
plt.ylabel('Litros necesarios para obtener 1 kg de queso')
plt.tight_layout()
plt.show()
```



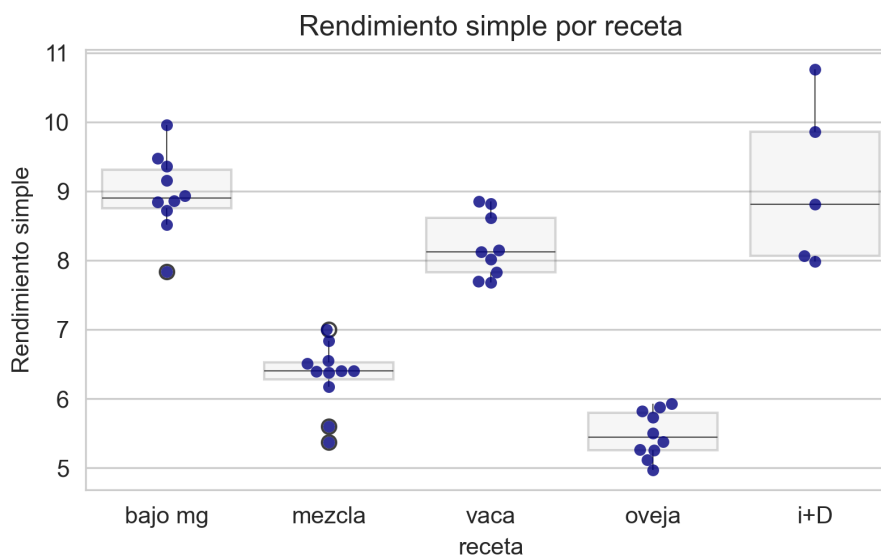
## 13.2. Análisis previo de los datos

Tenemos pocos datos por receta, por lo que vamos a utilizar el *beeswarm*, junto con el *boxplot*, para visualizar el *rendimiento simple* entre las diferentes recetas.

```
# Boxplot como fondo tenue
sns.boxplot(
    data=df,
    x='receta',
    y='rendimiento_simple',
    # boxplots en gris claro y muy transparente
    boxprops=dict(facecolor='lightgray', alpha=0.2),
    showcaps=False,
    whiskerprops={'linewidth': 0.5},
    medianprops={'linewidth': 0.5}
)

# Beeswarm destacado encima
sns.swarmplot(
    data=df,
    x='receta',
    y='rendimiento_simple',
    size=5,
    color='navy', # color fuerte y contrastante
    alpha=0.8 # casi opaco para destacar
)
```

```
# Estética general
plt.xticks(rotation=0)
plt.title('Rendimiento simple por receta')
plt.ylabel('Rendimiento simple')
plt.tight_layout()
plt.show()
```



Como estamos evaluando el rendimiento como los litros necesarios para obtener 1 kg de queso, resulta que la cantidad de agua que tenga el queso nos influye en el cálculo: cuanto más húmedo el queso, más favorable será el valor el rendimiento simple. Fijémonos en el queso de mezcla: hay un valor de rendimiento muy favorable, de poco más de 5 L/kg de queso, pero vemos que el queso ha tenido casi un 52 % de humedad. En cambio, otra fabricación de esa misma receta, que sólo tiene algo más de 40 % de humedad, ha necesitado 7 L/kg.

Así, dentro de cada receta, los mejores rendimientos en L/kg aparecen siempre en quesos más húmedos, y los peores, en quesos más secos. Veámoslo creando una columna adicional para los valores d humedad.

```
# Crear columna de humedad
df['humedad_salida_salmuera'] = 100 - df['est_salida_salmuera']

# crear dataframe para seaborn sin index para evitar errores
df_plot_sns = df.reset_index(drop=True)

# Crear gráfico con regresión lineal por receta
g = sns.lmplot()
```

```
data=df_plot_sns,
x='humedad_salida_salmuera',
y='rendimiento_simple',
hue='receta',
ci=None,
scatter_kws={'s': 80, 'alpha': 0.7},
line_kws={'linewidth': 2}
)

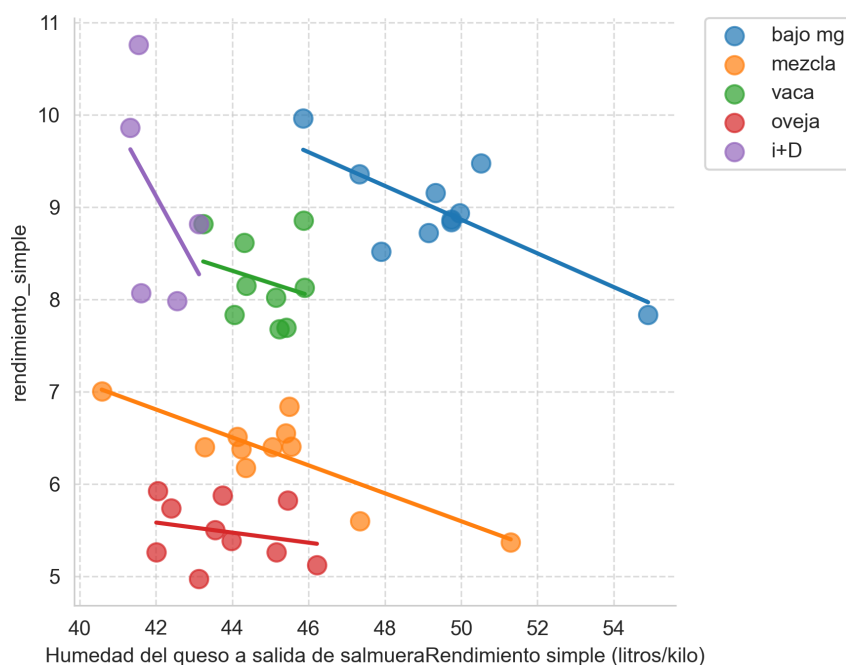
# Eliminar la leyenda generada por FacetGrid
g._legend.remove()

# Personalizar título y ejes
g.fig.suptitle('Influencia de la humedad del queso '
              'en la evaluación del rendimiento en litros/kilo',
              y=1.03)
g.set_axis_labels('Humedad del queso a salida de salmuera'
                  'Rendimiento simple (litros/kilo)', )
g.ax.grid(True, linestyle='--', alpha=0.7)

# Mover la leyenda fuera del gráfico
plt.legend(bbox_to_anchor=(1.05, 1),
           loc='upper left', borderaxespad=0.)

plt.tight_layout()
plt.show()
```

Influencia de la humedad del queso en la evaluación del rendimiento en litros/kilo



Claramente, el rendimiento en litros/kilo está afectado por la humedad del queso, ya que, para la misma tecnología, cuanto más humedad más peso tendremos en el queso, y en consecuencia, menos litros necesitaremos para hacer un kilo de queso.

Por esta razón, como hemos visto en el capítulo anterior, debemos calcular el rendimiento mediante parámetros que no se vean afectados por la humedad del queso (o lo que es equivalente, el extracto seco total), sino que reflejen con precisión la calidad del trabajo quesero. La mejor forma de hacerlo consiste en calcular el porcentaje de materia quesera que hemos sido capaces de retener en el queso. Dado que los elementos de valor de la leche, que son los que se utilizan en el pago por calidad, son la materia grasa y la proteína, calcularemos la recuperación de materia grasa, de materia proteica y de extracto seco magro.

Estos valores son propios de cada tecnología quesera, y cada uno sirve de referencia para comprender cuáles son los diferentes problemas en fabricación,

- Una mala retención de materia grasa
- Una mala retención de proteína
- Una mala retención de extracto seco magro

Dado que nuestros valores analíticos están expresados en porcentaje en peso, calculamos las cantidades de materia como se indica a continuación:

```

### 1. Cálculos de pasteurización
df['est_kg_past'] = df['litros_past'] * df['est_past'] / 100
df['mg_kg_past'] = df['litros_past'] * df['mg_past'] / 100
df['mp_kg_past'] = df['litros_past'] * df['mp_past'] / 100
df['esm_kg_past'] = df['est_kg_past'] - df['mg_kg_past']

### 1. Cálculos de cuba
df['est_kg_cuba'] = df['kg_cuba'] * df['est_cuba'] / 100
df['mg_kg_cuba'] = df['kg_cuba'] * df['mg_cuba'] / 100
df['mp_kg_cuba'] = df['kg_cuba'] * df['mp_cuba'] / 100
df['esm_kg_cuba'] = df['est_kg_cuba'] - df['mg_kg_cuba']

### 2. Cálculos de queso
df['est_kg_queso'] = (
    df['peso_queso_total'] * df['est_salida_salmuera'] / 100)
df['mg_kg_queso'] = (
    df['peso_queso_total'] * df['mg_salida_salmuera'] / 100)
df['esm_kg_queso'] = df['est_kg_queso'] - df['mg_kg_queso']
df['sal_kg_queso'] = df['peso_queso_total'] * df['sal_queso'] / 100

### 3. Coeficientes de recuperación (expresado en porcentaje)
df['coef_recup_mg'] = df['mg_kg_queso'] / df['mg_kg_cuba'] * 100
df['coef_recup_mp'] = df['esm_kg_queso'] / df['mp_kg_cuba'] * 100
df['coef_recup_esm'] = df['esm_kg_queso'] / df['esm_kg_cuba'] * 100

### 4. Consumos de materia por kilo de queso
# (expresado en gramos de materia por kilo de queso)
df['consumo_mg_kg'] = df['mg_kg_cuba'] / df['peso_queso_total'] * 1000
df['consumo_mp_kg'] = df['mp_kg_cuba'] / df['peso_queso_total'] * 1000

### 5. Otros cálculos: relación MG/MP en la leche de entrada
# y en el queso
df['ratio_mg_mp_leche'] = df['mg_kg_cuba'] / df['mp_kg_cuba']
df['ratio_mg_esm_leche'] = df['mg_kg_cuba'] / df['esm_kg_cuba']
df['ratio_mg_esm_queso'] = df['mg_kg_queso'] / df['esm_kg_queso']

### 6. Otros coeficientes: HQD y G/ES
df['hqd_salida_salmuera'] = (
    (100 - df['est_salida_salmuera']) /
    (100 - df['mg_salida_salmuera']) * 100
)

df['ges_salida_salmuera'] = (
    df['mg_salida_salmuera'] /
    df['est_salida_salmuera'] * 100
)

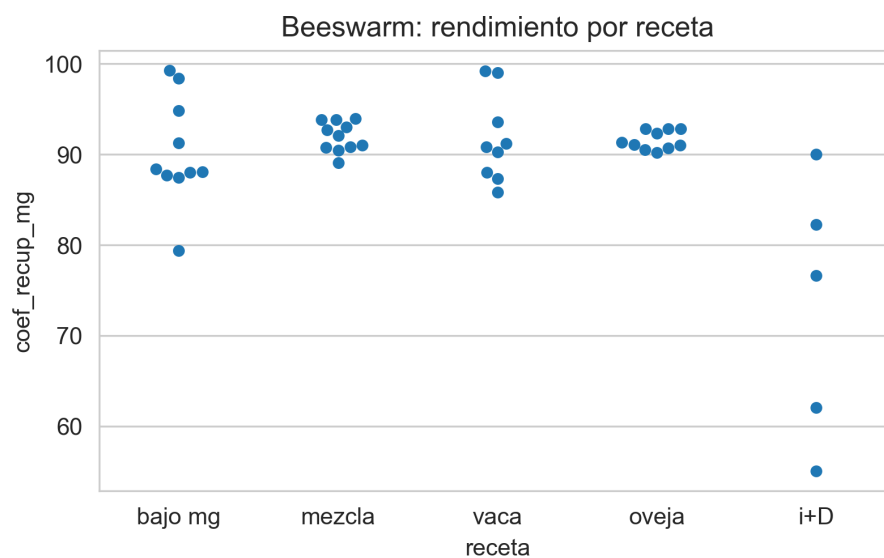
```

```
)
```

Comparemos ahora la recuperación de materia medida por estos coeficientes entre recetas.

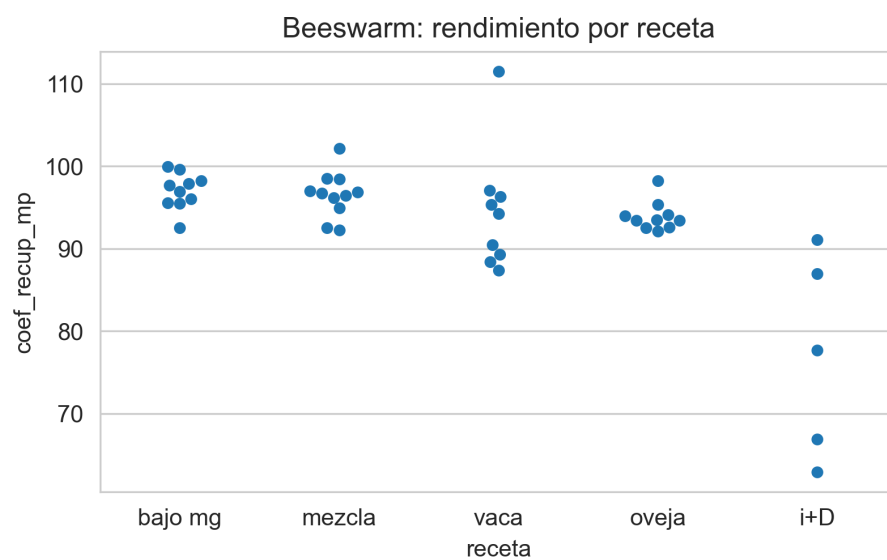
```
# crear dataframe para seaborn sin index para evitar errores
df_sns = df.reset_index(drop=True)

sns.swarmplot(data=df_sns, x='receta', y='coef_recup_mg', size=5)
plt.xticks(rotation=0)
plt.title('Beeswarm: rendimiento por receta')
plt.tight_layout()
plt.show()
```

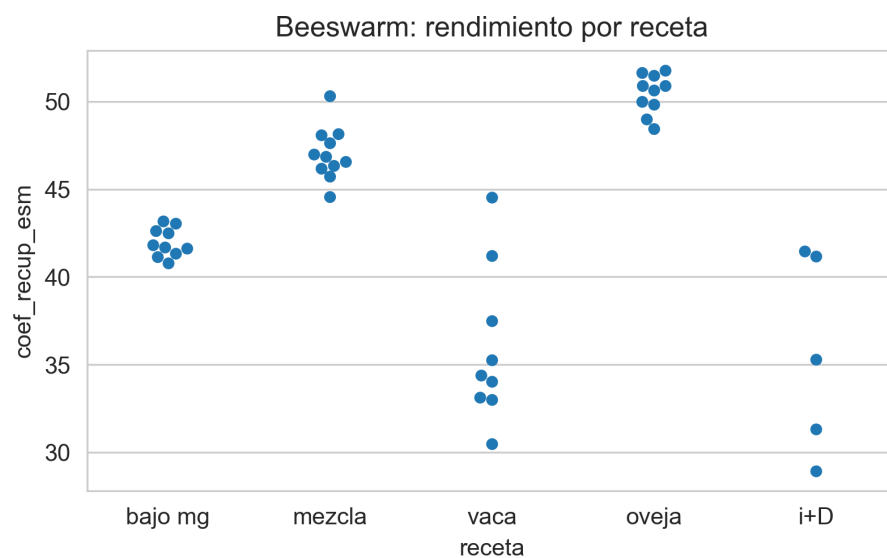


```
sns.swarmplot(data=df_sns, x='receta', y='coef_recup_mg', size=5)
plt.xticks(rotation=0)
plt.title('Beeswarm: rendimiento por receta')
plt.tight_layout()
plt.show()
```





```
sns.swarmplot(data=df_sns, x='receta', y='coef_recup_esm', size=5)
plt.xticks(rotation=0)
plt.title('Beeswarm: rendimiento por receta')
plt.tight_layout()
plt.show()
```

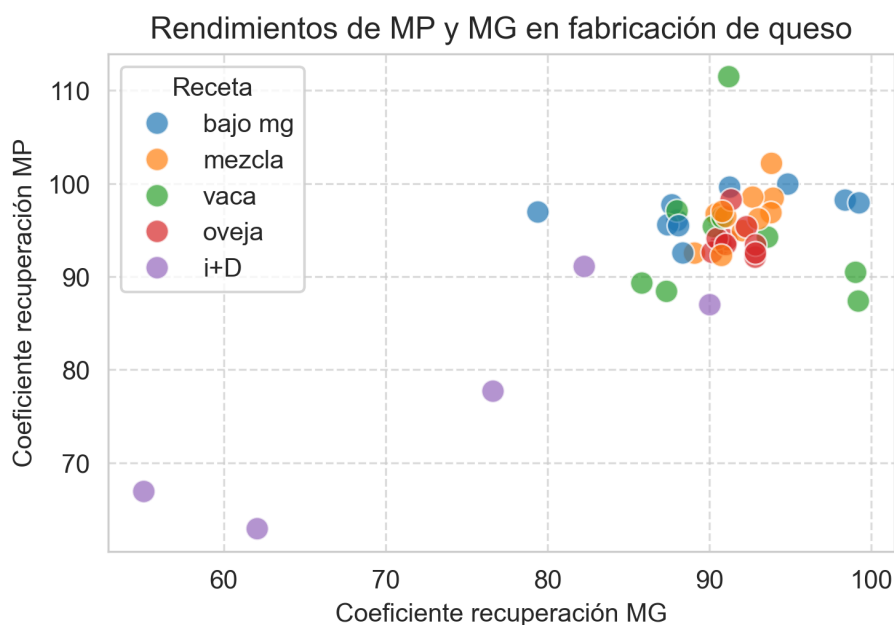


```
sns.scatterplot(data=df_sns,
                 x='coef_recup_mg',
```

```

        y='coef_recup_mp',
        hue='receta',
        s=80,
        alpha=0.7)
plt.title('Rendimientos de MP y MG en fabricación de queso')
plt.xlabel('Coeficiente recuperación MG')
plt.ylabel('Coeficiente recuperación MP')
plt.legend(title='Receta') # Mostrar leyenda para la especie
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```



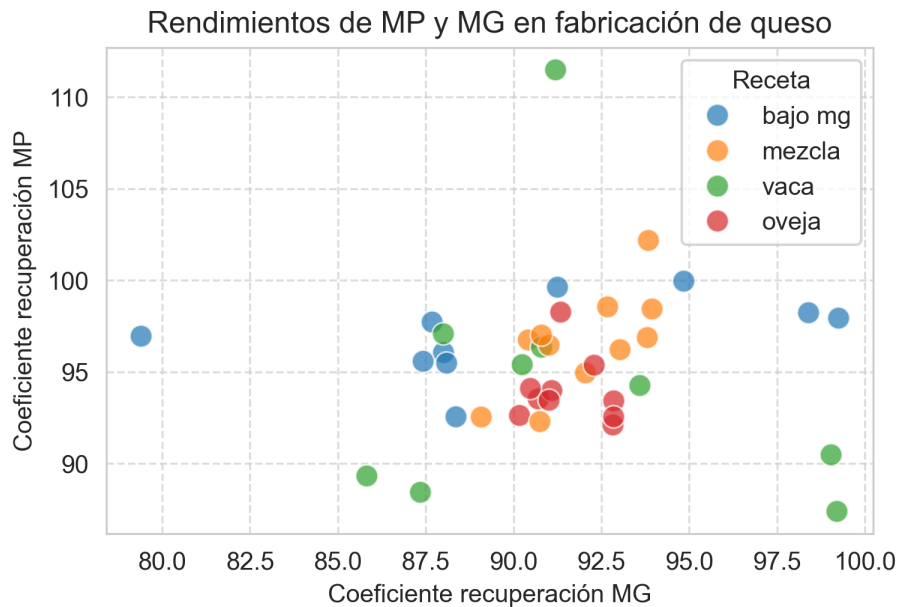
Filtremos los resultados de i+D que no son relevantes para nuestro análisis y están distorsionando la escala del gráfico.

```

sns.scatterplot(data=df[df['receta'] != 'i+D'],
                x='coef_recup_mg',
                y='coef_recup_mp',
                hue='receta',
                s=80,
                alpha=0.7)
plt.title('Rendimientos de MP y MG en fabricación de queso')
plt.xlabel('Coeficiente recuperación MG')
plt.ylabel('Coeficiente recuperación MP')
plt.legend(title='Receta') # Mostrar leyenda para la especie
plt.grid(True, linestyle='--', alpha=0.7)

```

```
plt.show()
```



Calculemos los centroides de cada grupo (receta) mediante la mediana.

```
# Filtrar receta 'i+D'
df_filtrado = df[df['receta'] != 'i+D'].reset_index(drop=True)

# Crear paleta de colores por receta
recetas = df_filtrado['receta'].unique()
palette = sns.color_palette('tab10', n_colors=len(recetas))
color_dict = dict(zip(recetas, palette))

sns.scatterplot(data=df[df['receta'] != 'i+D'],
                x='coef_recup_mg',
                y='coef_recup_mp',
                hue='receta',
                palette = color_dict,
                s=80,
                # edgecolor = 'black',
                alpha=0.7)

# Calcular y dibujar centroides
centroides_mg_mp = (
    df.groupby('receta')[['coef_recup_mg', 'coef_recup_mp']]
    .median()
```

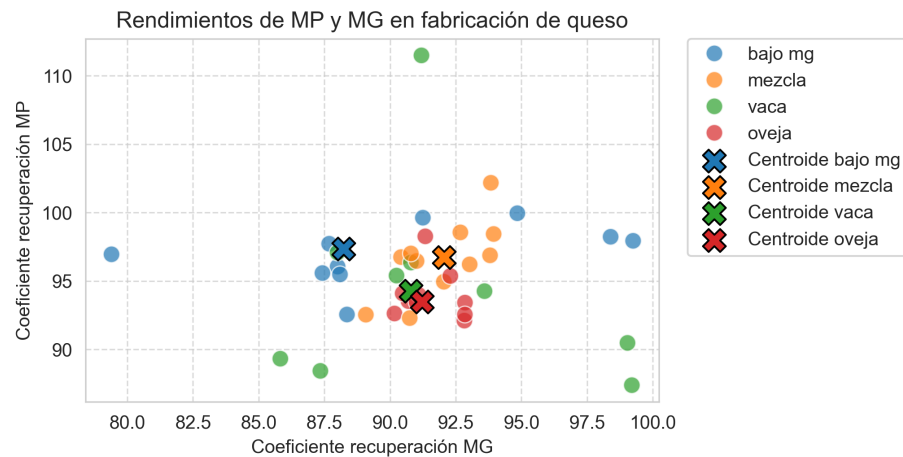
```

)

for receta in recetas:
    centro = centroides_mg_mp.loc[receta]
    plt.scatter(
        centro['coef_recup_mg'],
        centro['coef_recup_mp'],
        color=color_dict[receta],
        marker='X',
        s=150,
        edgecolor='black',
        label=f'Centroide {receta}'
    )

plt.title('Rendimientos de MP y MG en fabricación de queso')
plt.xlabel('Coeficiente recuperación MG')
plt.ylabel('Coeficiente recuperación MP')
plt.legend(title='Receta') # Mostrar leyenda para la especie
# Mover la leyenda fuera del gráfico
plt.legend(bbox_to_anchor=(1.05, 1),
           loc='upper left',
           borderaxespad=0.)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```



```
print(centroides_mg_mp)
```

```

      coef_recup_mg  coef_recup_mp
receta
bajo mg      88.229046      97.329485

```

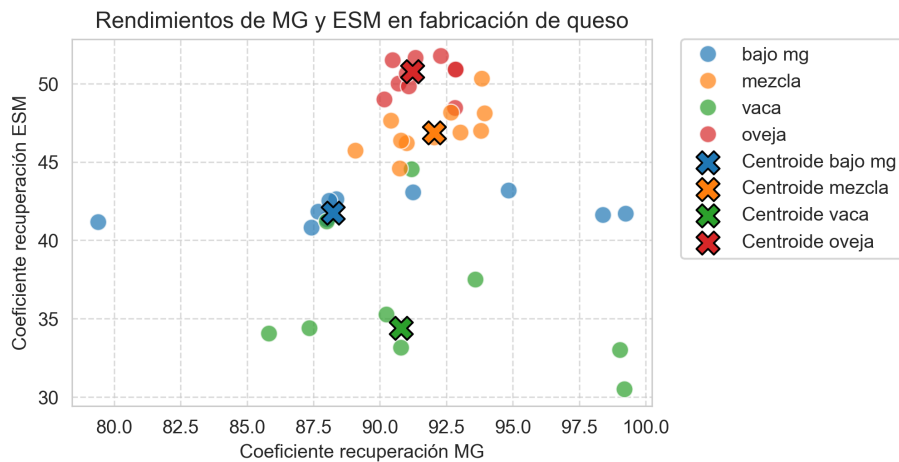
|        |           |           |
|--------|-----------|-----------|
| i+D    | 76.630772 | 77.697322 |
| mezcla | 92.047224 | 96.738573 |
| oveja  | 91.215043 | 93.482273 |
| vaca   | 90.794847 | 94.250704 |

```
sns.scatterplot(data=df[df['receta'] != 'i+D'],
                x='coef_recup_mg',
                y='coef_recup_esm',
                hue='receta',
                palette = color_dict,
                s=80,
                # edgecolor = 'black',
                alpha=0.7)

# Calcular y dibujar centroides
centroides_mg_esm = (
    df.groupby('receta')[['coef_recup_mg', 'coef_recup_esm']]
    .median()
)

for receta in recetas:
    centro = centroides_mg_esm.loc[receta]
    plt.scatter(
        centro['coef_recup_mg'],
        centro['coef_recup_esm'],
        color=color_dict[receta],
        marker='X',
        s=150,
        edgecolor='black',
        label=f'Centroide {receta}'
    )

plt.title('Rendimientos de MG y ESM en fabricación de queso')
plt.xlabel('Coeficiente recuperación MG')
plt.ylabel('Coeficiente recuperación ESM')
plt.legend(title='Receta') # Mostrar leyenda para la especie
# Mover la leyenda fuera del gráfico
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=0.)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()
```



```
print(centroides_mg_esm)
```

|         | coef_recup_mg | coef_recup_esm |
|---------|---------------|----------------|
| receta  |               |                |
| bajo mg | 88.229046     | 41.760190      |
| i+D     | 76.630772     | 35.290377      |
| mezcla  | 92.047224     | 46.876581      |
| oveja   | 91.215043     | 50.769829      |
| vaca    | 90.794847     | 34.385900      |

Finalmente, aprovechamos nuestros cálculos para crear un documento con los rendimientos y el balance de materia de la fabricación, que exportaremos a un 'csv' y que podemos analizar sea en *python* o en Excel mediante tablas dinámicas.

```

### 5. Balance de Materia de pasteurizacion (Salida - Entrada)
# La falta de producto queda como negativo al hacer Salida - Entrada
df['balance_est_past'] = df['est_kg_cuba'] - df['est_kg_past']
df['balance_mg_past'] = df['mg_kg_cuba'] - df['mg_kg_past']
df['balance_mp_past'] = df['mp_kg_cuba'] - df['mp_kg_past']
df['balance_esm_past'] = df['esm_kg_cuba'] - df['esm_kg_past']

### 6. Balance de Materia de cubas (Salida - Entrada)
# La falta de producto queda como negativo al hacer Salida - Entrada
df['balance_est_cuba'] = df['est_kg_queso'] - df['est_kg_cuba']
df['balance_mg_cuba'] = df['mg_kg_queso'] - df['mg_kg_cuba']
df['balance_esm_cuba'] = (
    df['esm_kg_queso'] - df['esm_kg_cuba'] - df['sal_kg_queso']
)

### 7. Preparación y Exportación del DataFrame de Resultados
# Columnas finales a incluir en el reporte

```

```

columnas_balance = [
    'fecha', 'receta', 'formato',
    'litros_past', 'peso_queso_total',
    'est_kg_past', 'est_kg_cuba', 'est_kg_queso',
    'mg_kg_past', 'mg_kg_cuba', 'mg_kg_queso',
    'mp_kg_past', 'mp_kg_cuba',
    'esm_kg_past', 'esm_kg_cuba', 'esm_kg_queso',
    'balance_est_past', 'balance_mg_past', 'balance_mp_past',
    'balance_esm_past',
    'balance_est_cuba', 'balance_mg_cuba', 'balance_esm_cuba',
    'coef_recup_mg', 'coef_recup_mp', 'coef_recup_esm',
    'consumo_mg_kg', 'consumo_mp_kg'
]

# Crear un nuevo DataFrame con las columnas de interés
df_balance = df[columnas_balance].copy()

# Guardar el DataFrame con punto decimal y separador ';'
nombre_archivo_salida = 'datos/informe_balance_materia.csv'

df_balance.to_csv(
    nombre_archivo_salida,
    sep=';',
    index=False,
    decimal=', '
)

```

Veamos las características de los quesos.

```

df['esm_salida_salmuera'] = (
    df['est_salida_salmuera'] - df['mg_salida_salmuera']
)
df['hqd_salida_salmuera'] = (
    (100 - df['est_salida_salmuera']) / (100 - df['mg_salida_salmuera']) * 100
)
df['ges_salida_salmuera'] = (
    df['mg_salida_salmuera'] / df['est_salida_salmuera']
)

# Filtrar receta 'i+D'
df_filtrado = df[df['receta'] != 'i+D'].reset_index(drop=True)

# Crear paleta de colores por receta
recetas = df_filtrado['receta'].unique()
palette = sns.color_palette('tab10', n_colors=len(recetas))
color_dict = dict(zip(recetas, palette))

```

```

sns.scatterplot(data=df[df['receta'] != 'i+D'],
                x='est_salida_salmuera',
                y='mg_salida_salmuera',
                hue='receta',
                palette = color_dict,
                s=80,
                # edgecolor = 'black',
                alpha=0.7)

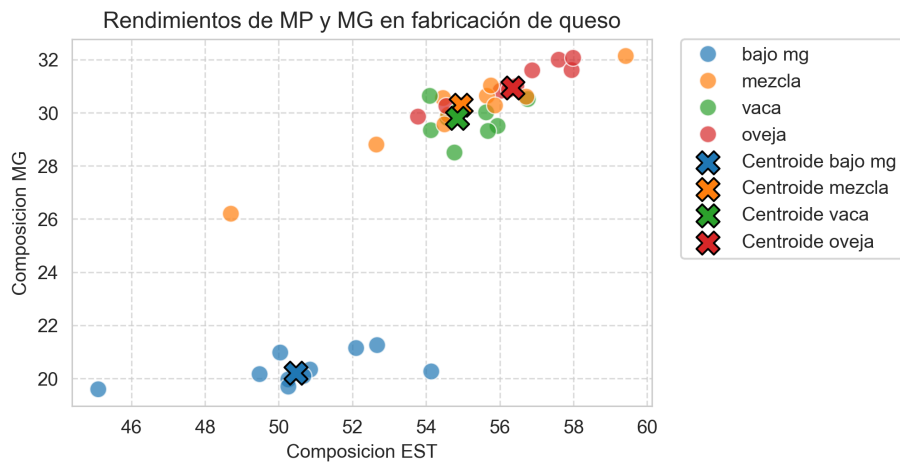
# Calcular y dibujar centroides
centroides_est_mg = (
    df.groupby('receta')[['est_salida_salmuera', 'mg_salida_salmuera']]
    .median()
)

for receta in recetas:
    centro = centroides_est_mg.loc[receta]
    plt.scatter(
        centro['est_salida_salmuera'],
        centro['mg_salida_salmuera'],
        color=color_dict[receta],
        marker='X',
        s=150,
        edgecolor='black',
        label=f'Centroide {receta}'
    )

plt.title('Rendimientos de MP y MG en fabricación de queso')
plt.xlabel('Composicion EST')
plt.ylabel('Composicion MG')
plt.legend(title='Receta') # Mostrar leyenda para la especie
# Mover la leyenda fuera del gráfico
plt.legend(bbox_to_anchor=(1.05, 1),
           loc='upper left',
           borderaxespad=0.)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```





```
print(centroides_est_mg)
```

|         | est_salida_salmuera | mg_salida_salmuera |
|---------|---------------------|--------------------|
| receta  |                     |                    |
| bajo mg | 50.465              | 20.220             |
| i+D     | 58.390              | 30.890             |
| mezcla  | 54.950              | 30.280             |
| oveja   | 56.350              | 30.935             |
| vaca    | 54.850              | 29.800             |

```
# Crear paleta de colores por receta
recetas = df['receta'].unique()
palette = sns.color_palette('tab10', n_colors=len(recetas))
color_dict = dict(zip(recetas, palette))

sns.scatterplot(data=df[df['receta'] != 'i+D'],
                x='consumo_mg_kg',
                y='consumo_mp_kg',
                hue='receta',
                palette = color_dict,
                s=80,
                # edgecolor = 'black',
                alpha=0.7)

# Calcular y dibujar centroides
centroides_consumos_materia = (
    df.groupby('receta')['consumo_mg_kg', 'consumo_mp_kg'].median()
)

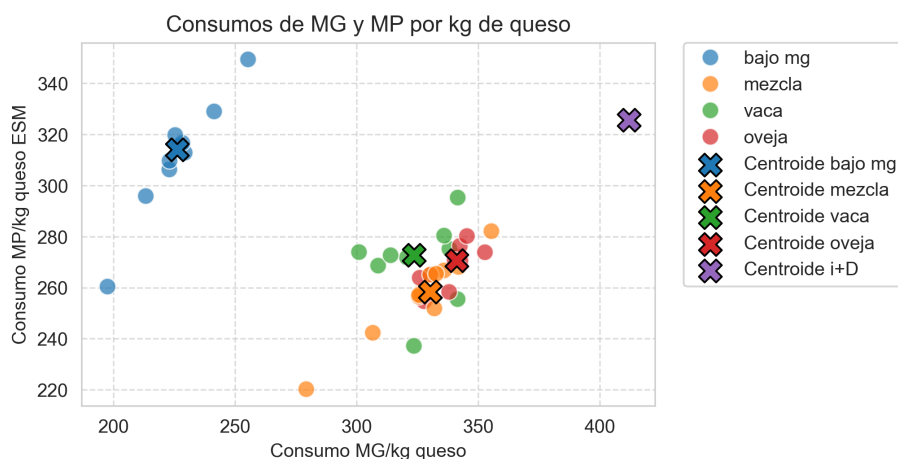
for receta in recetas:
    centro = centroides_consumos_materia.loc[receta]
```

```

plt.scatter(
    centro['consumo_mg_kg'],
    centro['consumo_mp_kg'],
    color=color_dict[receta],
    marker='X',
    s=150,
    edgecolor='black',
    label=f'Centroide {receta}')
)

plt.title('Consumos de MG y MP por kg de queso')
plt.xlabel('Consumo MG/kg queso')
plt.ylabel('Consumo MP/kg queso ESM')
plt.legend(title='Receta') # Mostrar leyenda para la especie
# Mover la leyenda fuera del gráfico
plt.legend(bbox_to_anchor=(1.05, 1),
           loc='upper left',
           borderaxespad=0.)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```



```
print(centroides_consumos_materia)
```

|         | consumo_mg_kg | consumo_mp_kg |
|---------|---------------|---------------|
| receta  |               |               |
| bajo mg | 226.119987    | 314.108816    |
| i+D     | 411.975490    | 325.622549    |
| mezcla  | 330.181811    | 258.288108    |
| oveja   | 341.012161    | 270.611497    |
| vaca    | 323.479948    | 272.676000    |

```

# Crear paleta de colores por receta
recetas = df['receta'].unique()
palette = sns.color_palette('tab10', n_colors=len(recetas))
color_dict = dict(zip(recetas, palette))

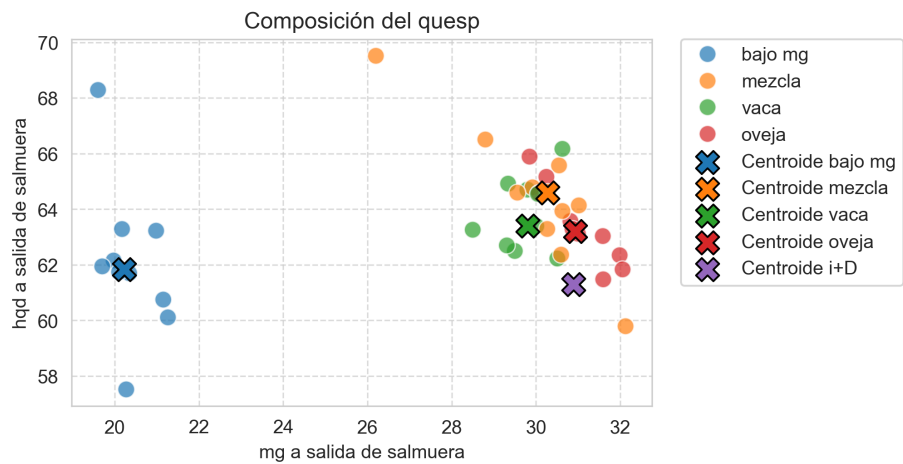
sns.scatterplot(data=df[df['receta'] != 'i+D'],
                x='mg_salida_salmuera',
                y='hqd_salida_salmuera',
                hue='receta',
                palette = color_dict,
                s=80,
                # edgecolor = 'black',
                alpha=0.7)

# Calcular y dibujar centroides
centroides_consumos_materia = (
    df.groupby('receta')[['mg_salida_salmuera', 'hqd_salida_salmuera']]
    .median()
)

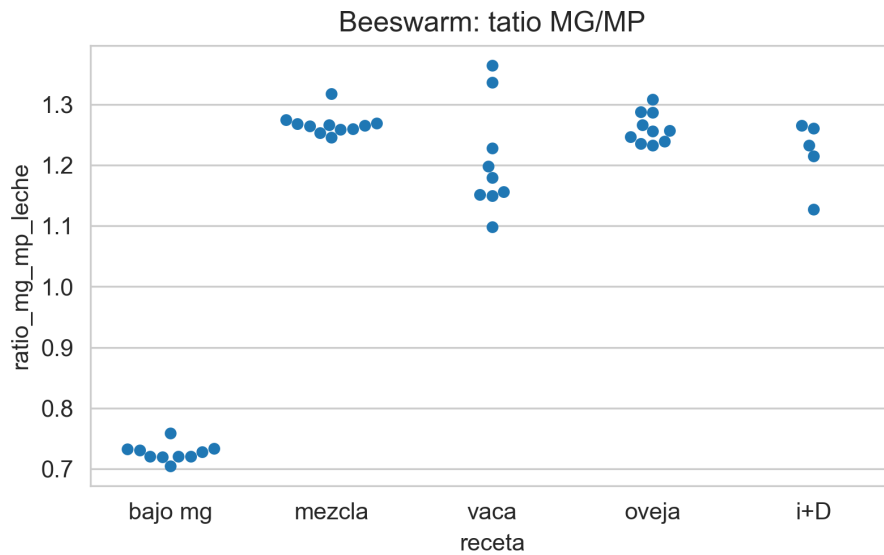
for receta in recetas:
    centro = centroides_consumos_materia.loc[receta]
    plt.scatter(
        centro['mg_salida_salmuera'],
        centro['hqd_salida_salmuera'],
        color=color_dict[receta],
        marker='X',
        s=150,
        edgecolor='black',
        label=f'Centroide {receta}'
    )

plt.title('Composición del quesp')
plt.xlabel('mg a salida de salmuera')
plt.ylabel('hqd a salida de salmuera')
plt.legend(title='Receta') # Mostrar leyenda para la especie
# Mover la leyenda fuera del gráfico
plt.legend(bbox_to_anchor=(1.05, 1),
           loc='upper left',
           borderaxespad=0.)
plt.grid(True, linestyle='--', alpha=0.7)
plt.show()

```



```
sns.swarmplot(data=df_sns, x='receta', y='ratio_mg_mp_leche', size=5)
plt.xticks(rotation=0)
plt.title('Beeswarm: tatio MG/MP')
plt.tight_layout()
plt.show()
```



```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Filtrar recetas excluidas
```

```

df_filtrado = df[df['receta'] != 'i+D'].copy()

# Crear paleta de colores por receta
recetas = df_filtrado['receta'].unique()
palette = sns.color_palette('tab10', n_colors=len(recetas))
color_dict = dict(zip(recetas, palette))

# Crear figura
plt.figure(figsize=(10, 6))

# Dibujar puntos y regresión por receta
for receta in recetas:
    datos = df_filtrado[df_filtrado['receta'] == receta]

    # Puntos
    sns.scatterplot(data=datos,
                    x='est_salida_salmuera',
                    y='hqd_salida_salmuera',
                    color=color_dict[receta],
                    s=80,
                    alpha=0.7,
                    label=receta)

    # Regresión
    sns.regplot(data=datos,
                x='est_salida_salmuera',
                y='hqd_salida_salmuera',
                scatter=False,
                ci=False,
                color=color_dict[receta],
                line_kws={'linewidth': 2})

# Calcular y dibujar centroides
centroides = (
    df_filtrado.groupby('receta')[['est_salida_salmuera', 'hqd_salida_salmuera']]
    .median()
)

for receta in recetas:
    centro = centroides.loc[receta]
    plt.scatter(centro['est_salida_salmuera'],
                centro['hqd_salida_salmuera'],
                color=color_dict[receta],
                marker='X',
                s=150,

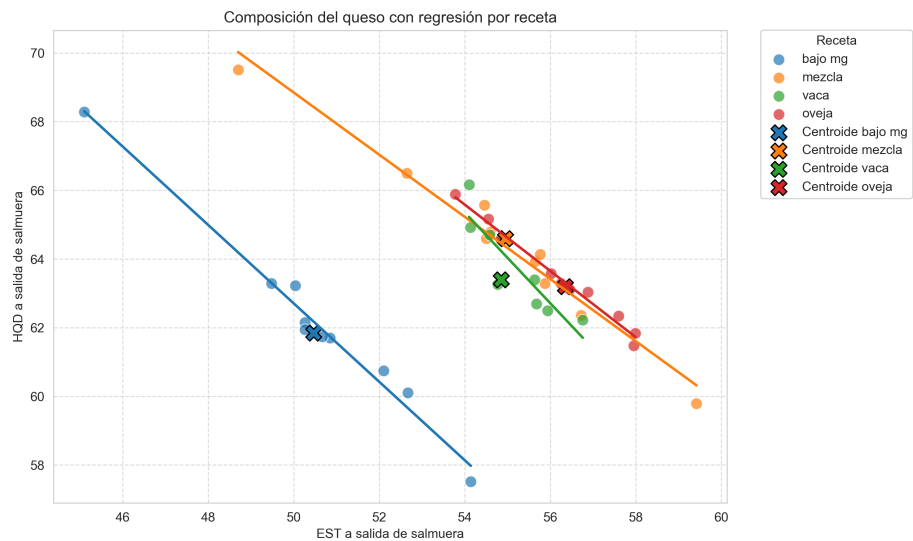
```

```

        edgecolor='black',
        label=f'Centroide {receta}')

# Etiquetas y estilo
plt.title('Composición del queso con regresión por receta')
plt.xlabel('EST a salida de salmuera')
plt.ylabel('HQD a salida de salmuera')
plt.legend(title='Receta', bbox_to_anchor=(1.05, 1), loc='upper left', borderaxespad=0)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()

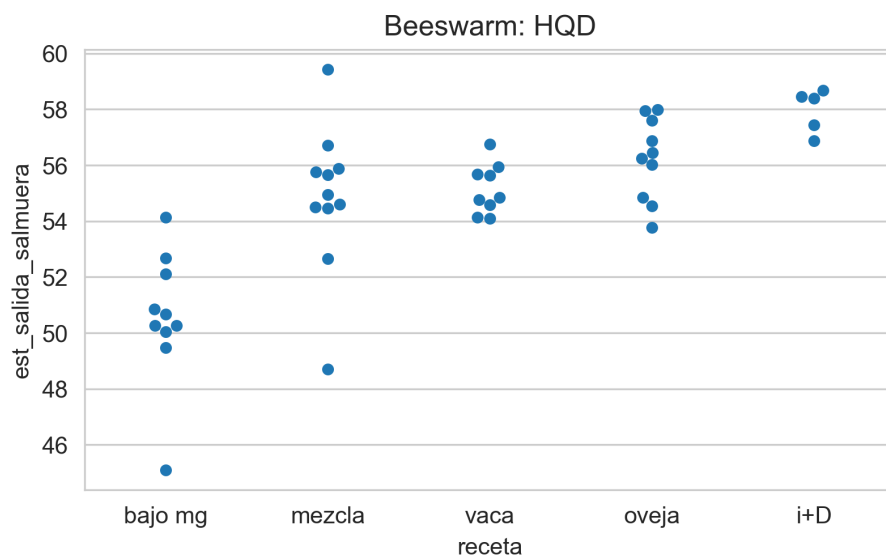
```



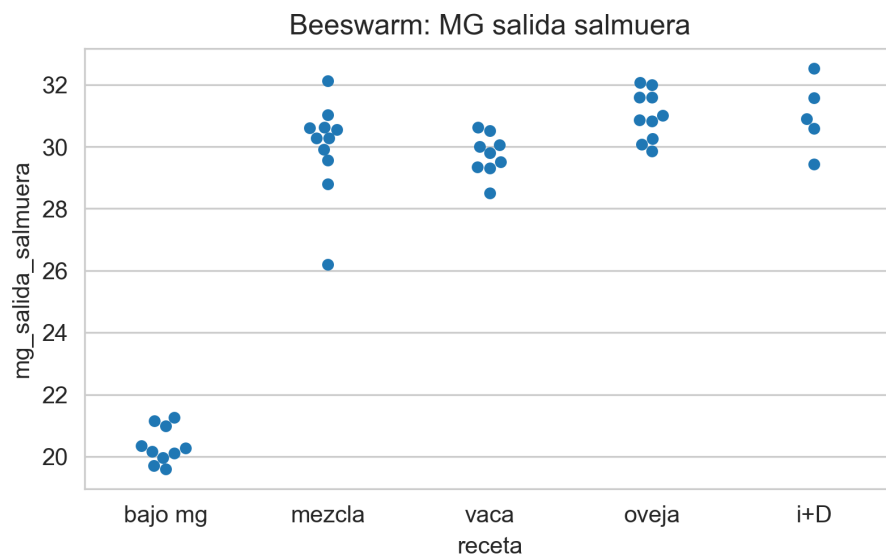
```

sns.swarmplot(data=df_sns, x='receta', y='est_salida_salmuera', size=5)
plt.xticks(rotation=0)
plt.title('Beeswarm: HQD')
plt.tight_layout()
plt.show()

```

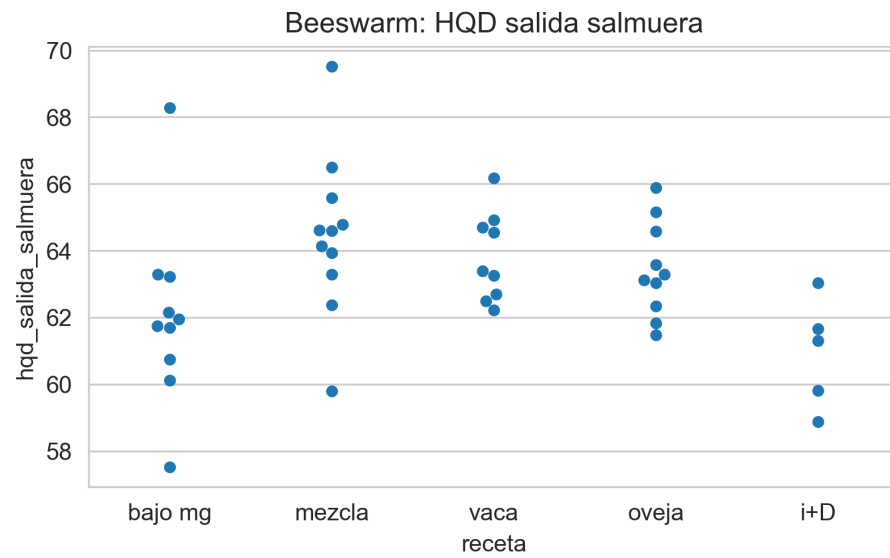


```
sns.swarmplot(data=df_sns, x='receta', y='mg_salida_salmuera', size=5)
plt.xticks(rotation=0)
plt.title('Beeswarm: MG salida salmuera')
plt.tight_layout()
plt.show()
```



```
sns.swarmplot(data=df_sns, x='receta', y='hqd_salida_salmuera', size=5)
plt.xticks(rotation=0)
```

```
plt.title('Beeswarm: HQD salida salmuera')  
plt.tight_layout()  
plt.show()
```





## Apéndice A

# La media aritmética como centro de gravedad de un grupo de datos

Para ver cómo la media aritmética equivale al punto de equilibrio o centro de gravedad de un grupo de datos, imaginaremos que tenemos un conjunto de datos cuyos valores se distribuyen en un eje longitudinal  $X$ . Podemos hacer equivalentes estos valores a un conjunto de pesos que se distribuyen a lo largo de una barra, y vamos a suponer que existe un punto a una distancia  $d_i$  del origen de la barra en el cual dicha barra está en equilibrio. Para encontrar ese valor, empezaremos considerando el principio o ley de la palanca.

El **Principio de la Palanca** o la **Ley de la Palanca** fue formulada por el científico y matemático griego Arquímedes. Este principio dice que, en equilibrio, el producto de la fuerza aplicada (potencia) por su distancia al punto de apoyo (brazo de la potencia) es igual al producto de la resistencia por su distancia al punto de apoyo (brazo de la resistencia). Matemáticamente, se expresa como:

$$P \cdot d_p = R \cdot d_r$$

Donde:

- $P$  es la potencia o fuerza aplicada.
- $d_p$  es la distancia desde el punto de apoyo hasta el punto donde se aplica la potencia.
- $R$  es la resistencia o carga.
- $d_r$  es la distancia desde el punto de apoyo hasta el punto donde se aplica la resistencia.

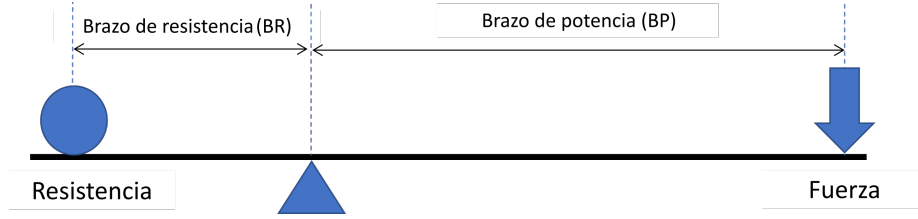


Figura A.1: Palanca de primer grado con el fulcro como punto de equilibrio

### A.0.1. Ejemplo

Si tienes una palanca con una longitud de 5 metros y aplicas una fuerza de 10 Newtons a 1 metro del punto de apoyo, para mantener el equilibrio, la fuerza de resistencia en el otro extremo a 4 metros del punto de apoyo debería ser:

$$P \cdot d_p = R \cdot d_r$$

$$10 \text{ N} \cdot 1 \text{ m} = R \cdot 4 \text{ m}$$

$$R = \frac{10 \cdot 1}{4} = 2,5 \text{ N}$$

Ahora vamos a considerar el punto de equilibrio de una barra de la que se cuelgan diferentes pesos a diferentes distancias de su origen. Calcular el punto de equilibrio de una barra de la que se cuelgan diferentes pesos a diferentes distancias de su origen es un problema clásico de física que se puede resolver usando el principio de momentos.

### A.0.2. Paso a Paso

1. **Identificar las fuerzas:** Supongamos que tienes varios pesos  $W_1, W_2, W_3, \dots, W_n$  colgados a distancias  $d_1, d_2, d_3, \dots, d_n$  del origen (punto de apoyo).
2. **Calcular los momentos:** El momento ( $M$ ) de un peso alrededor del punto de apoyo se calcula como el producto de la fuerza (peso) y la distancia al punto de apoyo:

$$M_i = W_i \times d_i$$

3. **Sumar los momentos:** Calcula la suma de todos los momentos:

$$M_{\text{total}} = W_1 \times d_1 + W_2 \times d_2 + W_3 \times d_3 + \dots + W_n \times d_n$$

4. **Calcular el peso total:** Suma todos los pesos:

$$W_{\text{total}} = W_1 + W_2 + W_3 + \dots + W_n$$

5. **Determinar el punto de equilibrio ( $x$ ):** El punto de equilibrio se encuentra dividiendo la suma de los momentos por el peso total:

$$x = \frac{\sum (W_i \times d_i)}{\sum W_i}$$

### A.0.3. Ejemplo Práctico

Supongamos que tenemos tres pesos de 4, 9 y 1 kg, situados, respectivamente, a 2, 1 y 3 m respectivamente del origen de la barra.

- $W_1 = 4 \text{ kg}$  a  $d_1 = 2 \text{ m}$
- $W_2 = 9 \text{ kg}$  a  $d_2 = 1 \text{ m}$
- $W_3 = 1 \text{ kg}$  a  $d_3 = 3 \text{ m}$

1. **Calcular los momentos:**

$$M_1 = 4 \times 2 = 8 \text{ kg} \cdot \text{m}$$

$$M_2 = 9 \times 1 = 9 \text{ kg} \cdot \text{m}$$

$$M_3 = 1 \times 3 = 3 \text{ kg} \cdot \text{m}$$

2. **Sumar los momentos:**

$$M_{\text{total}} = 8 + 9 + 3 = 20 \text{ kg} \cdot \text{m}$$

3. **Calcular el peso total:**

$$W_{\text{total}} = 4 + 9 + 1 = 14 \text{ kg}$$

4. **Determinar el punto de equilibrio:**

$$x = \frac{20}{14} \approx 1.43 \text{ m}$$

El punto de equilibrio se encuentra a aproximadamente 1.43 metros del origen.

### A.0.4. Media Ponderada

En muchas situaciones, el centro de gravedad de un sistema de masas puede interpretarse como una **media ponderada** de las posiciones de las masas.

**Media Aritmética**

Para un conjunto de números  $x_1, x_2, \dots, x_n$ , la media aritmética es:

$$\text{Media} = \frac{x_1 + x_2 + \dots + x_n}{n}$$

**Media Ponderada**

Para un conjunto de valores  $x_1, x_2, \dots, x_n$  con pesos asociados  $w_1, w_2, \dots, w_n$ , la media ponderada es:

$$\text{Media ponderada} = \frac{w_1 x_1 + w_2 x_2 + \dots + w_n x_n}{w_1 + w_2 + \dots + w_n}$$

**A.0.5. Centro de Gravedad como Media Ponderada**

Cuando calculamos el centro de gravedad ( $x_{cg}$ ) de un sistema de masas, estamos esencialmente calculando una media ponderada de las posiciones ( $x_i$ ) de esas masas ( $m_i$ ):

$$x_{cg} = \frac{\sum (m_i \cdot x_i)}{\sum m_i}$$

Aquí, las posiciones  $x_i$  son ponderadas por las masas  $m_i$ .

**A.0.6. Ejemplo Numérico**

En nuestro ejemplo anterior,

- Masa 1:  $m_1 = 4$  kg en posición  $x_1 = 2$  m
- Masa 2:  $m_2 = 9$  kg en posición  $x_2 = 1$  m
- Masa 3:  $m_3 = 1$  kg en posición  $x_3 = 3$  m

La media aritmética de las posiciones sería:

$$\text{Media} = \frac{2 + 1 + 3}{3} = 2 \text{ m}$$

El punto de equilibrio de la barra, o centro de gravedad, calculado como media ponderada, sería:

$$x_{cg} = \frac{(4 \cdot 2) + (9 \cdot 1) + (1 \cdot 3)}{4 + 9 + 1} = \frac{8 + 9 + 3}{14} = \frac{20}{14} \approx 1,43 \text{ m}$$

que es el mismo resultado que obteníamos formulando el cálculo de los momentos; la fórmula resulta ser idéntica.

### A.0.7. Media Aritmética y Centro de Gravedad con Masas Idénticas

En los ejemplos anteriores, calculábamos el punto de equilibrio para diferentes pesos colocados a lo largo de una barra. Si en vez de eso suponemos que la distancia al origen de la barra equivale a nuestro eje  $X$ , que recoge los valores de los que queremos calcular nuestra media aritmética, podemos eliminar el efecto de la masa suponiendo que todas las masas son iguales.

Si las masas de los diferentes objetos son idénticas, podemos decir que la media aritmética de las posiciones coincide con el centro de gravedad. Esto se debe a que, en este caso, cada masa tiene el mismo peso o influencia en el cálculo del centro de gravedad. Esta es la explicación detallada:

**Suposición:** Cada objeto tiene la misma masa  $m$ .

#### Media Aritmética

Para un conjunto de posiciones  $x_1, x_2, \dots, x_n$ , la media aritmética es:

$$\text{Media} = \frac{x_1 + x_2 + \dots + x_n}{n}$$

#### Centro de Gravedad

El centro de gravedad ( $x_{cg}$ ) para un conjunto de masas idénticas en posiciones  $x_1, x_2, \dots, x_n$  es:

$$x_{cg} = \frac{\sum(m \cdot x_i)}{\sum m}$$

Dado que las masas  $m$  son idénticas, el numerador se convierte en:

$$\sum(m \cdot x_i) = m \cdot (x_1 + x_2 + \dots + x_n)$$

Y el denominador se convierte en:

$$\sum m = n \cdot m$$

Al sustituir estos en la fórmula del centro de gravedad, obtenemos:

$$x_{cg} = \frac{m \cdot (x_1 + x_2 + \dots + x_n)}{n \cdot m}$$

Al simplificar, los términos  $m$  se cancelan, y nos queda:

$$x_{cg} = \frac{x_1 + x_2 + \dots + x_n}{n}$$

que es exactamente la fórmula de la media aritmética.

Si repetimos nuestro ejemplo anterior, suponiendo en este caso masas idénticas

- Masa 1:  $m_1 = 1$  kg en posición  $x_1 = 2$  m
- Masa 2:  $m_2 = 1$  kg en posición  $x_2 = 1$  m
- Masa 3:  $m_3 = 1$  kg en posición  $x_3 = 3$  m

La media aritmética de las posiciones sería:

$$\text{Media} = \frac{2 + 1 + 3}{3} = 2 \text{ m}$$

El punto de equilibrio de la barra, o centro de gravedad, calculado como media ponderada, sería:

$$x_{\text{cg}} = \frac{(1 \cdot 2) + (1 \cdot 1) + (1 \cdot 3)}{1 + 1 + 1} = \frac{2 + 1 + 3}{3} = \frac{6}{3} = 2 \text{ m}$$

que es el mismo resultado que obteníamos con la fórmula de la media aritmética de las distancias.

#### A.0.8. Conclusión

Cuando las masas son idénticas, la media aritmética de las posiciones coincide con el centro de gravedad. Esto nos permite afirmar que en el caso de una dimensión (por ejemplo, el peso) la media aritmética de un conjunto de valores coincide con el centro de gravedad o punto de equilibrio de ese conjunto de valores. Este es un resultado interesante que muestra la conexión entre conceptos estadísticos y físicos.

### A.1. Importancia del concepto de la media como centro de gravedad o punto de equilibrio de un conjunto de datos

Efecto de los *outliers*: desequilibrio, desplazamiento de la media, incremento de la varianza (gan diferencia), mientras que la mediana y el rango intercuartil no se ven afectados

# Bibliografía

- Hadley Wickham, Garret Grolemund. 2023. «R Para Ciencia de Datos». 2023. <https://es.r4ds.hadley.nz/>.
- Hadley Wickham, Garret Grolemund, Mine Çetinkaya-Rundel. 2023. *R for Data Science, 2nd ed.* 1005 Gravenstein Highway North, Sebastopol, CA95472: O'Reilly Media Inc. <https://r4ds.hadley.nz/>.
- World Economic Forum. 2025. «Future of Jobs Report, 2025». 91-93 route de la Capite, CH-1223 Cologny/Geneva, Switzerland: World Economic Forum. <https://www.weforum.org/publications/the-future-of-jobs-report-2025/>.





## Apéndice B

# Indice de DataFrames utilizados

### B.1. Análisis de producto terminado de una fabricación diaria de camembert

En formato CSV y Excel

|   | fecha      | fabricacion | est   | mg   | ph   | cloruros | coliformes |
|---|------------|-------------|-------|------|------|----------|------------|
| 0 | 11/01/2020 | 1           | 46.68 | 24.0 | 4.85 | 1.50     | 80.0       |
| 1 | 12/01/2020 | 1           | 48.09 | 25.0 | 4.67 | 1.61     | 0.0        |
| 2 | 13/01/2020 | 1           | 45.97 | 24.0 | 4.71 | 1.48     | 9200.0     |
| 3 | 14/01/2020 | 1           | 46.24 | 23.5 | 4.78 | 1.69     | 20.0       |
| 4 | 15/01/2020 | 1           | 45.81 | 23.0 | 4.77 | 1.56     | 150.0      |

### B.2. Datos de fabricación de diferentes recetas de queso ibérico para balance de materia

|   | fecha      | receta  | litros_past | est_past | mg_past | mp_past | kg_cuba  | est_cuba | mg_cuba | mp  |
|---|------------|---------|-------------|----------|---------|---------|----------|----------|---------|-----|
| 0 | 23/02/2022 | bajo mg | 12380       | 10.76    | 2.58    | 3.55    | 12860.51 | 10.28    | 2.46    | 3.3 |
| 1 | 01/03/2022 | bajo mg | 13080       | 11.09    | 2.59    | 3.60    | 13867.37 | 10.41    | 2.47    | 3.4 |
| 2 | 08/03/2022 | bajo mg | 10700       | 10.70    | 2.48    | 3.45    | 11638.92 | 9.98     | 2.36    | 3.2 |
| 3 | 29/03/2022 | bajo mg | 12680       | 11.27    | 2.56    | 3.50    | 13011.79 | 10.22    | 2.44    | 3.3 |
| 4 | 05/04/2022 | bajo mg | 14590       | 11.05    | 2.52    | 3.50    | 14508.35 | 10.26    | 2.40    | 3.3 |

### B.3. Lecturas de humedad relativa y temperatura en una cueva para maduración de queso

|   | fecha               | temp      | hrel      |
|---|---------------------|-----------|-----------|
| 0 | 2020-01-01 00:00:00 | 10.232205 | 87.796974 |
| 1 | 2020-01-01 00:05:00 | 10.091809 | 87.807218 |
| 2 | 2020-01-01 00:10:00 | 10.139651 | 87.801365 |
| 3 | 2020-01-01 00:15:00 | 10.030622 | 87.796916 |
| 4 | 2020-01-01 00:20:00 | 9.901193  | 87.811986 |

### B.4. Datos de análisis de quesos tipo ibérico a la salida de la salmuera

|   | fecha      | receta   | formato | maduracion | extractoseco | materiagrasa | sal  | ph   |
|---|------------|----------|---------|------------|--------------|--------------|------|------|
| 0 | 07/10/2012 | mezcla 5 | 3 kg    | tierno     | 57.330002    | 32.139999    | 1.04 | 5.27 |
| 1 | 07/10/2012 | mezcla 5 | 500 g   | tierno     | 58.549999    | 32.980000    | 0.99 | 5.45 |
| 2 | 07/10/2012 | mezcla 5 | 3 kg    | tierno     | 58.509998    | 33.160000    | 1.16 | 5.26 |
| 3 | 07/10/2012 | mezcla 5 | 500 g   | tierno     | 59.009998    | 33.070000    | 1.27 | 5.46 |
| 4 | 07/10/2012 | mezcla 5 | 500 g   | tierno     | 57.869999    | 31.940001    | 0.93 | 5.47 |

### B.5. Datos a salida de saladero de quesos ibéricos

|   | Fecha      | Num_Cuba | Lote  | FORMATO      | PH   | ES    | MG    | SAL  |
|---|------------|----------|-------|--------------|------|-------|-------|------|
| 0 | 05/11/2015 | 7        | 42015 | Barra 3 kg   | 5.51 | 58.31 | 32.36 | 1.45 |
| 1 | 13/03/2015 | 28       | 18315 | Barra 4,2 kg | 5.49 | 58.59 | 30.90 | 1.18 |
| 2 | 04/05/2016 | 8        | 23616 | Barra 3 kg   | 5.43 | 57.82 | 31.60 | 1.57 |
| 3 | 14/03/2016 | 9        | 18516 | Barra 3 kg   | 5.59 | 58.17 | 31.24 | 1.37 |
| 4 | 23/05/2016 | 6        | 25516 | 1 kg         | 5.46 | 59.71 | 31.92 | 0.92 |

### B.6. Datos de un proceso de envasado de queso en unidades de 250 g

|   | fecha               | peso   |
|---|---------------------|--------|
| 0 | 2020-05-16 09:00:00 | 254.55 |
| 1 | 2020-05-16 09:00:10 | 246.92 |

*B.7. DATOS DE ENTRADA DE LECHE EN UNA FÁBRICA DE QUESO IBÉRICO DE MEZCLA.223*

|   | fecha               | peso   |
|---|---------------------|--------|
| 2 | 2020-05-16 09:00:20 | 236.29 |
| 3 | 2020-05-16 09:00:30 | 241.60 |
| 4 | 2020-05-16 09:00:40 | 247.41 |

**B.7. Datos de entrada de leche en una fábrica de queso ibérico de mezcla.**

|   | codruta | peso  | tipoleche | fecha      | acidez | caseina | densidad | est   | lactosa | mg   | ph   | prot |
|---|---------|-------|-----------|------------|--------|---------|----------|-------|---------|------|------|------|
| 0 | 13      | 13380 | VACA      | 04/11/2015 | 15.0   | 2.31    | 1.030    | 12.45 | 4.71    | 3.71 | 6.84 | 3.34 |
| 1 | 13      | 8520  | VACA      | 04/11/2015 | 15.0   | 2.41    | 1.030    | 12.83 | 4.74    | 3.94 | 6.81 | 3.40 |
| 2 | 22      | 23900 | VACA      | 04/11/2015 | 15.0   | 2.29    | 1.030    | 12.47 | 4.85    | 3.66 | 6.82 | 3.22 |
| 3 | 41      | 4180  | CABRA     | 04/11/2015 | 14.0   | 2.88    | 1.032    | 14.88 | 4.70    | 5.50 | 6.86 | 3.95 |
| 4 | 24      | 14640 | VACA      | 04/11/2015 | 15.0   | 2.26    | 1.030    | 12.35 | 4.69    | 3.68 | 6.86 | 3.25 |

**B.8. Datos de análisis de producto terminado de diferentes productos, en una tabla Excel**

|   | FECHA | CLAVE | CODIGO | HUMEDAD | EXSECO | ESMAGRO | HQMAGRO | GRASA | GRASASE |
|---|-------|-------|--------|---------|--------|---------|---------|-------|---------|
| 0 | 7     | 660   | 10078  | 48.48   | 51.52  | 18.52   | 72.36   | 33.0  | 64.05   |
| 1 | 7     | 660   | 20078  | 47.35   | 52.65  | 17.65   | 72.85   | 35.0  | 66.48   |
| 2 | 7     | 660   | 30078  | 47.02   | 52.98  | 19.98   | 70.18   | 33.0  | 62.29   |
| 3 | 8     | 660   | 10088  | 47.02   | 52.98  | 18.98   | 71.24   | 34.0  | 64.18   |
| 4 | 8     | 660   | 20088  | 47.22   | 52.78  | 18.78   | 71.55   | 34.0  | 64.42   |

